

Arquitetura de Software Enterprise com IA

Formação completa — do discovery ao deploy assistido por agentes

Autor: AI Software Engineering Academy

Idioma: pt-BR

Edição: 1 — 2026

Introdução — O Papel do Arquiteto

Módulo 00 — Introdução: A Nova Engenharia de Software

****Antes de escrever uma linha de código.****

1. Como a IA mudou o desenvolvimento de software

A IA generativa não é apenas mais uma ferramenta no cinto do desenvolvedor. Ela muda fundamentalmente **quem** escreve código, **como** o código é escrito e **o que** significa ser um engenheiro de software.

O cenário antes da IA

- Programador escrevia 100% do código manualmente - Produtividade limitada pela velocidade de digitação - Conhecimento fragmentado em fóruns, documentação e livros - Cada erro era descoberto em tempo de execução ou compilação

O cenário com IA

- Programador **orquestra** agentes que escrevem código - Produtividade limitada pela qualidade das instruções (prompts) - Conhecimento integrado no contexto do LLM - Erros são prevenidos por revisão sistemática dos agentes

A virada de chave

"O desenvolvedor do futuro não será avaliado pela quantidade de código que escreve, mas pela qualidade das decisões que toma."

Isso porque o código em si se torna commodity — o diferencial está em:

1. **Saber o que construir** (Product Discovery)
2. **Saber como arquitetar** (Enterprise Architecture)
3. **Saber o que delegar** (Agent Orchestration)
4. **Saber validar o resultado** (Auditing & Review)

2. Desenvolvimento tradicional vs. com agentes

Dimensão	Tradicional	Com Agentes
Escrita de código	Manual, linha a linha	Gerada por agentes, revis
Erros	Descobertos em runtime	Prevenidos por checklists
Documentação	Esquecida ou desatualizad	Gerada automaticamente pe
Escopo do desenvolvedor	Full stack (código + infr	Orquestrador (agentes + r
Velocidade	Limitada pela digitação	Limitada pela clareza das
Consistência	Varia por desenvolvedor	Garantida por templates e
Curva de aprendizado	Anos para dominar stack	Dias para configurar agen
Testes	Muitas vezes negligenciad	Gerados e mantidos pelos

O paradoxo da produtividade

Dados da pesquisa Stack Overflow 2024 mostram:

- **63%** dos desenvolvedores reportam que código gerado por IA frequentemente contém erros inesperados - **40%** dos projetos com IA sem configuração adequada têm retrabalho - **~3%** de erro quando o time define **12+ regras** de projeto (AGENTS.md, .cursorrules, etc.)

Conclusão: IA não elimina a necessidade de engenharia de qualidade — ela a torna mais importante do que nunca.

3. O papel do arquiteto

O arquiteto de software no mundo dos agentes não desapareceu — ele se tornou **mais estratégico**.

Responsabilidades do Arquiteto

ARQUITETO DE SOFTWARE
Antes:
• Escolher tecnologia • Desenhar diagramas • Revisar código • Resolver problemas técnicos
Agora (+):
• Definir regras para os agentes • Criar templates de prompt • Configurar checklists de auditoria • Estruturar o workspace para IA • Decidir o que delegar vs. o que reter • Validar outputs dos agentes

Decisões que o arquiteto **não pode** delegar

1. **Visão arquitetural** — o "desenho grande" do sistema 2. **Trade-offs críticos** — custo vs. performance vs. prazo 3. **Padrões e convenções** — o "jeito certo" de fazer as coisas 4. **Revisão estratégica** — o que entra e o que fica para depois 5. **Contexto de negócio** — o "porquê" por trás das decisões técnicas

Decisões que o arquiteto **deve** delegar

1. **Implementação de CRUDs** — agentes geram com consistência 2. **Testes unitários** — agentes cobrem edge cases 3. **Documentação técnica** — agentes mantêm atualizada 4. **Validação de boas práticas** — checklists automáticos 5. **Refatoração de código legado** — agentes aplicam padrões

4. O papel do Product Owner

O Product Owner (PO) também evolui no contexto de times com agentes.

Responsabilidades do PO

Atividade	Tradicional	Com Agentes
Escrever histórias	Texto livre	Formato Gherkin (Given/Wh
Critérios de aceite	Descritivos vagos	Especificações executáveis
Priorização	Feeling	RICE score com dados dos
Validação	Manual, no final	Contínua, com protótipos
Comunicação com time	Reuniões longas	Documentação assíncrona g

O PO define o "o quê", não o "como"

Com agentes, o PO pode: 1. Descrever o problema em linguagem natural 2. Um agente de discovery transforma em user stories 3. Um agente de design gera wireframes 4. Um agente de arquitetura valida a viabilidade 5. O time implementa com agentes especializados

5. O papel da IA

A IA não é um substituto — é um **membro do time** com habilidades específicas.

Como a IA deve ser posicionada

NÃO: "A IA vai substituir os desenvolvedores"

SIM: "A IA é um desenvolvedor júnior infinitamente rápido e disposto,
que precisa de supervisão técnica e contexto de negócio"

O que a IA faz bem

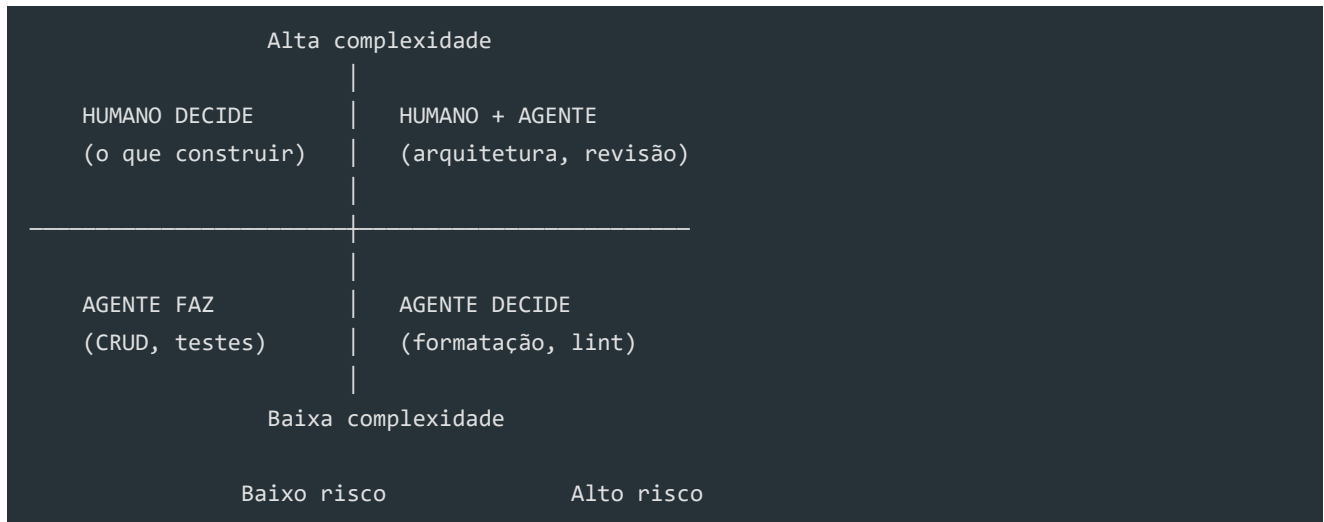
- Geração de código repetitivo e padronizado - Cobertura de edge cases em testes - Documentação técnica - Refatoração sistemática - Revisão de boas práticas - Sugestão de padrões e soluções conhecidas

Onde a IA falha

- Compreensão profunda de negócio - Decisões com contextos contraditórios - Inovação fora dos padrões conhecidos - Responsabilidade e ownership - Compreensão de implicações de longo prazo

6. Como dividir trabalho entre humanos e agentes

A Matriz de Delegação



Regras de ouro

1. **O humano decide o "quê" e o "porquê"** — o agente executa o "como"
2. **Sempre revisar o output do agente** — especialmente em produção
3. **Quanto maior o risco, mais supervisão humana**
4. **Crie checklists para os agentes seguirem** — eles seguem regras melhor que humanos
5. **Documente as regras** em AGENTS.md, .cursorsrules, opencode.json
6. **Itere** — ajuste as regras conforme os agentes aprendem o contexto

7. O ciclo de desenvolvimento com agentes

O ciclo completo que usaremos na formação:



8. O que esperar desta formação

O que você vai aprender

1. **Produto** — discovery, UX research, Design Thinking 2. **Design** — UI Design, Design System, wireframes 3. **Arquitetura** — Clean Arch, DDD, modelagem de dados 4. **Implementação** — Backend (NestJS), Frontend (Next.js), Prisma 5. **Qualidade** — Segurança, testes, performance 6. **Operações** — DevOps, observabilidade, governança 7. **Inovação** — Agentes de IA, automação, auditoria

O que você NÃO vai aprender

- Sintaxe básica de JavaScript/TypeScript (presume-se conhecimento) - Configuração de ambiente de desenvolvimento - Conceitos fundamentais de programação

O formato

- **Módulos práticos** com exercícios e projetos - **Agentes pré-configurados** para cada etapa - **Auditoria** como mecanismo de qualidade - **Projeto final** unificando todos os conceitos

Resumo

1. A IA mudou o papel do desenvolvedor de **escritor de código** para **orquestrador de agentes** 2. O arquiteto se torna ainda mais importante — define as regras que os agentes seguem 3. O PO ganha ferramentas para especificar com mais precisão 4. A divisão de trabalho segue a **Matriz de Delegação** (complexidade x risco) 5. O ciclo de desenvolvimento com agentes é mais iterativo e com mais pontos de validação 6. Código gerado por IA sem governança tem **40%** de erro — com regras cai para **~3%**

Mentalidade Enterprise

Módulo 01 — Mentalidade Enterprise

Como empresas de verdade desenvolvem software.

1. O que é software Enterprise?

Software Enterprise é aquele construído para **organizações**, não para indivíduos.

Características

Característica	Exemplo de problema se ig
Multi-usuário	Dois usuários editam o me
Multi-tenant	Cliente A vê dados do Cli
Segurança	Dado sensível exposto por
Auditabilidade	Não é possível rastrear q
Disponibilidade	Sistema fora do ar em hor
Performance	Relatório leva 5 minutos

Diferença entre software de consumo e Enterprise

Software de Consumo:

- Um usuário
- Dados isolados
- "Funcionou no meu PC"
- Atualização quando quiser
- Suporte via chat
- Compliance: nenhum

Software Enterprise:

- Centenas/milhares de usuários
- Dados compartilhados com permissões
- Funciona em múltiplos ambientes
- Atualização com rollback e migração
- SLA definido contratualmente
- Compliance: LGPD, SOC2, ISO 27001

2. Escalabilidade

Escalabilidade é a capacidade do sistema de **manter a performance** à medida que a demanda cresce.

Tipos de escala

Escala Vertical (Scale Up):

Aumentar recursos da máquina

CPU: 4 cores → 16 cores

RAM: 8GB → 64GB

Limite: hardware máximo

Escala Horizontal (Scale Out):

Adicionar mais máquinas

1 servidor → 10 servidores

Balanceador de carga na frente

Limite: gerenciamento de estado

O que escalar no sistema Enterprise

Usuários → Autenticação, session, rate limiting

Dados → Índices, partições, sharding

Funcionalidades → Módulos, microserviços, feature flags

Times → Código modular, documentação, padronização

Anti-padrões de escalabilidade

- **Banco como broker de mensagens** — usar fila dedicada (RabbitMQ, Redis) - **N+1 queries** — usar eager loading ou batch - **Session in-process** — usar Redis session store - **Arquivos no servidor** — usar S3/CDN - **Monólito sem modularização** — pelo menos módulos bem definidos

3. Governança

Governança é o conjunto de **regras e processos** que garantem consistência e qualidade no desenvolvimento.

Pilares da governança

GOVERNANÇA		
Código	Processo	Dados
TypeScript strict	Git Flow	Migrações
Lint/format	Code Review	Audit trail
Padrões	CI/CD	Backup
Arquitetura	ADRs	Retention
	Documentação	Compliance

Exemplos de regras de governança

```
## Regras de Governança

### Código
- TypeScript strict mode obrigatório
- Sem `any` – exceções revisadas em PR
- Lint e format automáticos (pre-commit hook)

### Processo
- Toda feature começa com ADR se mudar arquitetura
- PR aprovado por 1 reviewer + CI verde
- Commits semânticos: feat, fix, refactor, docs, test

### Dados
- Migrações são revisadas como código
- Soft delete para dados críticos
- Audit trail para todas as alterações
```

4. Manutenibilidade

Manutenibilidade é a facilidade de **entender, modificar e estender** o sistema.

O custo da falta de manutenibilidade

Código sem manutenibilidade:

Feature nova	2 semanas
Corrigir bug	3 dias
Onboarding dev novo	2 meses
Refatorar módulo	"melhor reescrever"

Código com manutenibilidade:

Feature nova	2 dias
Corrigir bug	2 horas
Onboarding dev novo	1 semana
Refatorar módulo	2 dias

Como garantir manutenibilidade

1. **Código limpo** — nomes descritivos, funções pequenas, sem duplicação
2. **Testes** — unitários + integração + E2E
3. **Documentação** — README, ADRs, Swagger
4. **Modularização** — baixo acoplamento, alta coesão
5. **Padronização** — mesmas convenções em todo o código

5. Observabilidade

Observabilidade é a capacidade de **entender o estado interno do sistema** a partir de seus outputs externos.

Os 3 pilares

LOGS	MÉTRICAS	TRACING
Eventos discretos "Usuário X fez Y"	Dados agregados "500 requisições/segundo"	Fluxo de requisições "Requisição passou por A→B→C"
Exemplos: - log.error("Falha no pgto") - log.info("Usuário logou") - log.warn("Rate limit")	Exemplos: - response_time_p95 - error_rate - cpu/memory usage	Exemplos: - span do endpoint - span do banco - span do cache

O que observar em um sistema Enterprise

Saúde da aplicação:

- Uptime, memory, CPU, conexões ativas
- Health check endpoints (/health, /ready)

Performance:

- Response time (p50, p95, p99)
- Throughput (req/s)
- Error rate

Negócio:

- Usuários ativos, signups, churn
- Funcionalidades mais usadas
- Funis de conversão

Segurança:

- Tentativas de login falhas
- Rate limiting acionado
- Acessos não autorizados

6. Segurança

Segurança em software Enterprise não é opcional — é **pré-requisito**.

Mindset de segurança

Não: "Vamos adicionar segurança depois"
Sim: "Segurança é parte da definição de "pronto""

Camadas de segurança

Camada 1: Código
→ Input validation, ORM (previne injection), prepared statements

Camada 2: Autenticação
→ JWT, OAuth2, MFA, senhas com hash

Camada 3: Autorização
→ RBAC, CASL abilities, verificação por recurso

Camada 4: Rede
→ HTTPS, CORS, CSP, firewall, VPN

Camada 5: Infraestrutura
→ Secrets management, network isolation, backup

Checklist Enterprise de segurança

- ☐ Nenhum segredo no código (.env, secrets manager) - ☐ HTTPS obrigatório (redirect HTTPHTTPS) - ☐ CSP header configurado - ☐ CORS com origens específicas (não `\*`) - ☐ Rate limiting em endpoints críticos - ☐ Validação de entrada em todos os endpoints - ☐ Autenticação em todas as rotas protegidas - ☐ Auditoria de ações sensíveis

7. Compliance

Compliance é a **conformidade com leis e regulamentações**.

Principais regulamentações

Regulamentação	Região	Foco
LGPD	Brasil	Dados pessoais
GDPR	Europa	Dados pessoais
PCI DSS	Global	Dados de cartão
HIPAA	EUA	Dados de saúde
SOC 2	Global	Controles internos

O que a LGPD exige

1. **Consentimento** — usuário autoriza coleta de dados 2. **Finalidade** — dados coletados para propósito específico 3. **Minimização** — colete apenas o necessário 4. **Transparência** — informe como os dados são usados 5. **Segurança** — proteja os dados armazenados 6. **Direitos do titular** — acesso, correção, exclusão 7. **Registro de tratamento** — documento o que faz com os dados

Implementando compliance no código

```
// Consentimento
interface Consentimento {
  usuarioId: string;
  finalidade: string; // "marketing", "analytics", "essencial"
  autorizado: boolean;
  data: DateTime;
}

// Direito de exclusão (right to erasure)
async function esquecerUsuario(usuarioId: string) {
  await prisma.usuario.update({
    where: { id: usuarioId },
    data: {
      nome: anonimizar(usuario.nome),
      email: anonimizar(usuario.email),
      deletedAt: new Date(),
    },
  });
};
await prisma.consentimento.deleteMany({
```

```
    where: { usuarioId },  
  });  
}
```

```
// Consentimento  
interface Consentimento {  
  usuarioId: string;  
  finalidade: string; // "marketing", "analytics", "essencial"  
  autorizado: boolean;  
  data: DateTime;  
}  
  
// Direito de exclusão (right to erasure)  
async function esquecerUsuario(usuarioId: string) {  
  await prisma.usuario.update({  
    where: { id: usuarioId },  
    data: {  
      nome: anonimizar(usuario.nome),  
      email: anonimizar(usuario.email),  
      deletedAt: new Date(),  
    },  
  });  
  await prisma.consentimento.deleteMany({
```


8. Multi-tenant

Multi-tenant é a capacidade de **atender múltiplos clientes** (tenants) com uma única instância do sistema.

Estratégias de isolamento

Database per Tenant:

Prós: isolamento máximo, backup individual
Contras: caro, complexo (migrations em N bancos)
Quando: dados sensíveis (saúde, finanças)
Custo: \$\$\$\$\$

Schema per Tenant:

Prós: bom isolamento, um banco
Contras: migrations complexas, conexões
Quando: dados moderadamente sensíveis
Custo: \$\$\$

Row-Level Security:

Prós: simples, barato, migrations únicas
Contras: risco de vazamento entre tenants
Quando: dados de baixa sensibilidade
Custo: \$

Implementação prática (RLS no Prisma)

```
model Tenant {
  id    String @id
  slug  String @unique
  name  String
}

model Usuario {
  id          String @id
  tenantId    String
  tenant      Tenant @relation(fields: [tenantId], references: [id])
  email       String
}
```

```
// Middleware que filtra por tenant
async function getUsuarios(tenantId: string) {
  return prisma.usuario.findMany({
    where: { tenantId },
  });
}
```

```
}
```

```
// Middleware que filtra por tenant
async function getUsuarios(tenantId: string) {
  return prisma.usuario.findMany({
    where: { tenantId },
  });
}
```


9. Alta disponibilidade

Alta disponibilidade (HA) é a capacidade do sistema de ****permanecer acessível**** mesmo com falhas.

Métricas de disponibilidade

Disponibilidade	Downtime/ano	Exemplo
99% (1 nove)	3.65 dias	Sistemas internos
99.9% (2 noves)	8.76 horas	SaaS padrão
99.99% (3 noves)	52.56 minutos	Enterprise crítico
99.999% (4 noves)	5.26 minutos	Missão crítica

Estratégias de HA

Sem ponto único de falha:

- Múltiplas instâncias do servidor
- Múltiplas réplicas do banco
- CDN para assets estáticos

Redundância:

- Load balancer (distribui tráfego)
- Database replica (leitura em réplicas)
- Cache (Redis cluster)

Recuperação:

- Health checks → reinício automático
- Backup automático + testado
- Disaster recovery plan

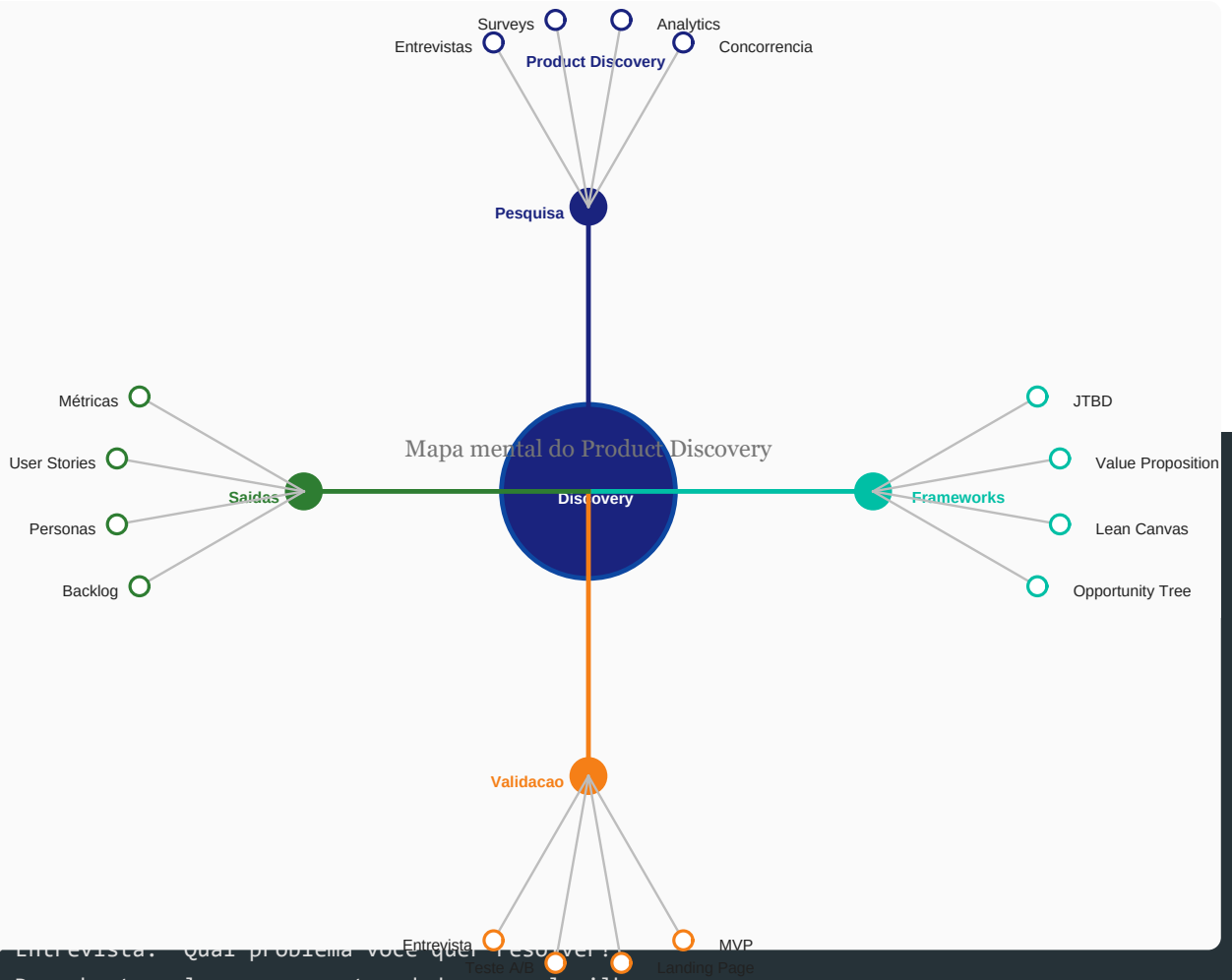
Resumo

1. **Software Enterprise** é construído para organizações — multi-usuário, seguro, auditável 2. **Escalabilidade** — horizontal > vertical; banco não é broker 3. **Governança** — código + processo + dados com regras claras 4. **Manutenibilidade** — código limpo, testado, documentado, modular 5. **Observabilidade** — logs + métricas + tracing = entender o sistema 6. **Segurança** — em camadas, desde o código até a infra 7. **Compliance** — LGPD/GDPR não são opcionais 8. **Multi-tenant** — escolha a estratégia de isolamento certa 9. **Alta disponibilidade** — sem ponto único de falha

Product Discovery

Módulo 02 — Product Discovery: Validando Ideias Antes de Construir

Descubra o que construir antes de construir.



- Entrevista: Qual problema você vai resolver?
- Descoberta: ele quer exportar dados pra planilha
- Solução: botão de exportar CSV em 1 dia
- Valida: ele testa, funciona, usa todo dia

Armadilha comum

Time: "Vamos construir um chat bot!"

Discovery: Nenhum

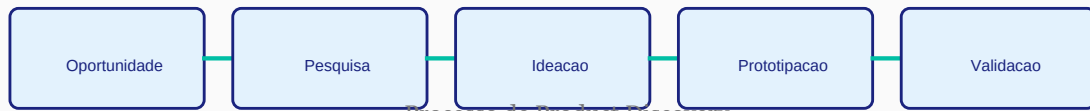
3 meses depois:

Ninguém usa o chat bot.

Por quê?

- Ninguém perguntou se o problema era real
- Ninguém validou se chat bot era a melhor solução
- Ninguém testou com usuários antes de construir

Processo de Product Discovery



2.1 Oportunidades

Toda feature request, bug report, reclamação de cliente ou insight de analytics é uma **oportunidade**. O desafio é priorizá-las.

Fontes de oportunidades: - Feedback de clientes (Zendesk, Intercom, surveys) - Dados de uso (amplitude, mixpanel, GA4) - Conversas com sales/suporte - Análise concorrencial - Visão do produto (OKRs, estratégia) - Próprio time (dívida técnica, bugs recorrentes)

2.2 Pesquisa

Antes de pensar em solução, **entenda o problema**. Use técnicas qualitativas e quantitativas (ver seção 3).

2.3 Ideação

Gere múltiplas opções de solução, não apenas uma. Técnicas: - **Crazy 8s** — 8 ideias em 8 minutos - **Brainstorming** — quantidade > qualidade, sem julgamento - **Design Sprint** — Google Ventures, 5 dias - **How Might We** — reformular problema como pergunta

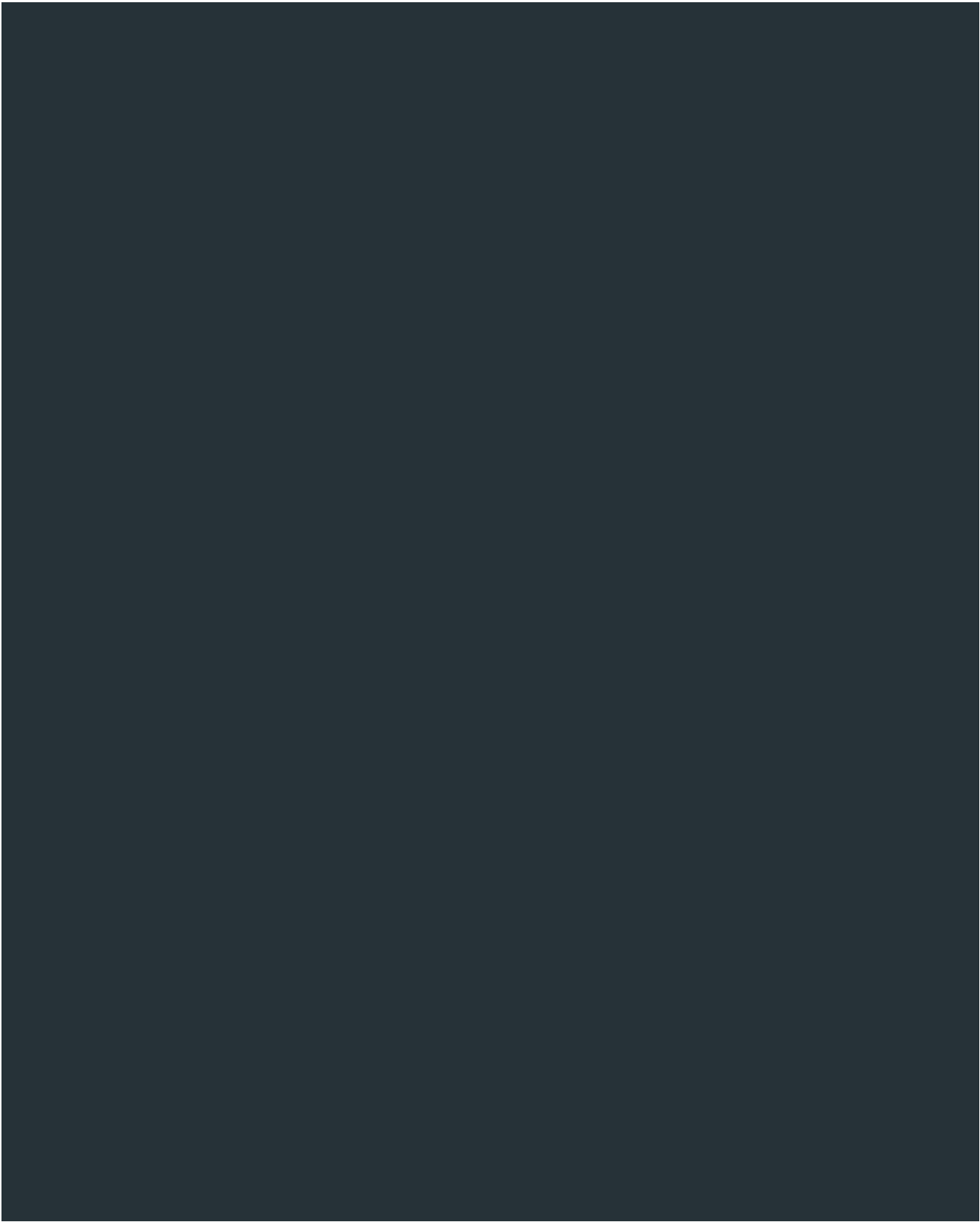
```
Problema: "Usuários não completam o cadastro"
HMW: "Como poderíamos reduzir o atrito no cadastro?"
  → "Como poderíamos deixar o cadastro opcional?"
  → "Como poderíamos usar login social?"
  → "Como poderíamos pré-preencher dados?"
```

2.4 Prototipação

Crie representações da solução com o mínimo esforço necessário para testar:

Nível	Esforço	O que testa
Sketch	Minutos	Fluxo macro
Wireframe	Horas	Layout, estrutura
Mockup	Dias	Visual, branding

Prototype	Dias	Interação, fluxo real
MVP	Semanas	Valor real no mercado



Nível	Esforço	O que testa
Sketch	Minutos	Fluxo macro
Wireframe	Horas	Layout, estrutura
Mockup	Dias	Visual, branding

Prototype	Dias	Interação, fluxo real
MVP	Semanas	Valor real no mercado

2.5 Validação

Cada protótipo precisa ser testado com usuários reais. Se a hipótese não for validada, volte para ideação.

3. Técnicas de Pesquisa

3.1 Entrevistas com usuários

A ferramenta mais poderosa de discovery.

****Estrutura de uma entrevista:****

1. Abertura (5min)
"Obrigado por participar. Não estamos testando você.
Queremos entender como você lida com [assunto].
Pode ser sincero – críticas nos ajudam."
2. Contexto (10min)
"Me conta como é seu dia a dia com [assunto]."
"O que você mais usa hoje?"
"O que te frustra?"
3. Profundidade (20min)
"Me conta da última vez que você passou por [situação]."
"O que você fez? O que aconteceu?"
"Se pudesse mudar uma coisa, o que seria?"
4. Fechamento (5min)
"Algo mais que gostaria de compartilhar?"
"Posso voltar a te procurar se tiver mais perguntas?"

****Regras de ouro:**** - Não faça perguntas indutoras ("Você não acha que X é melhor?") - Não pergunte "você usaria?" (todo mundo diz sim) - Pergunte sobre ****comportamento passado****, não intenção futura - Escute mais do que fala (ratio 80/20)

3.2 Surveys

Bom para validar achados qualitativos com escala.

"Você usaria uma funcionalidade de exportar relatórios?"
(todo mundo diz sim – viés de cortesia)

"Na última semana, quantas vezes você precisou exportar dados do sistema?"
(comportamento real, mensurável)

****Dicas de surveys:**** - Máximo 10 perguntas - Escalas consistentes (Likert 1-5 ou 1-7) - Sempre incluir pergunta aberta no final - Ferramentas: Typeform, Google Forms, SurveyMonkey

3.3 Análise Concorrencial

Concorrente A	Concorrente B	Concorrente C
Faz X bem	Faz Y bem	Faz Z bem
Não tem W	Tem W mas é confuso	Tem W excelente
Feedback: lento	Feedback: caro	Feedback: complexo
Oportunidade: X + W simples + preço acessível		

****O que analisar:**** - Proposta de valor - Fluxos principais - Pontos fortes e fracos - Reviews (G2, Capterra, Play Store, App Store) - Pricing e posicionamento

3.4 Analytics

Dados quantitativos não mentem — mas precisam de interpretação.

Métricas de Discovery:		
Métrica	O que indica	
-----	-----	
Page views	Interesse bruto	
Bounce rate	Não encontrou o que queria	
Drop-off por etapa	Atrito no fluxo	
Funil de conversão	Onde os usuários desistem	
Retenção D1/D7/D30	Valor entregue vs expectativa	
NPS / CSAT	Satisfação geral	
Search terms	O que usuários procuram (e não acham)	
Feature usage	O que realmente é usado	

4. Frameworks de Discovery

4.1 JTBD — Jobs to be Done

JTBD parte da premissa: **usuários não compram produtos, eles contratam serviços para fazer um trabalho**.

Exemplo:

Job: "Me manter informado sobre tecnologia"

Opção A: Assinar newsletter (R\$ 0, 5min/dia)
Opção B: Seguir influencers (R\$ 0, 15min/dia)
Opção C: Assinar portal de tech (R\$ 30/mês, 30min/dia)
Opção D: Podcasts (R\$ 0, 1h/dia no trânsito)

"Contratamos" a opção que melhor resolve o job
dado nosso contexto (tempo, dinheiro, momento)

Estrutura JTBD:

Quando [situação], eu quero [motivação] para [resultado esperado].

Exemplo:

"Quando estou começando um novo projeto, eu quero entender o
que outros times já tentaram antes, para não repetir erros."

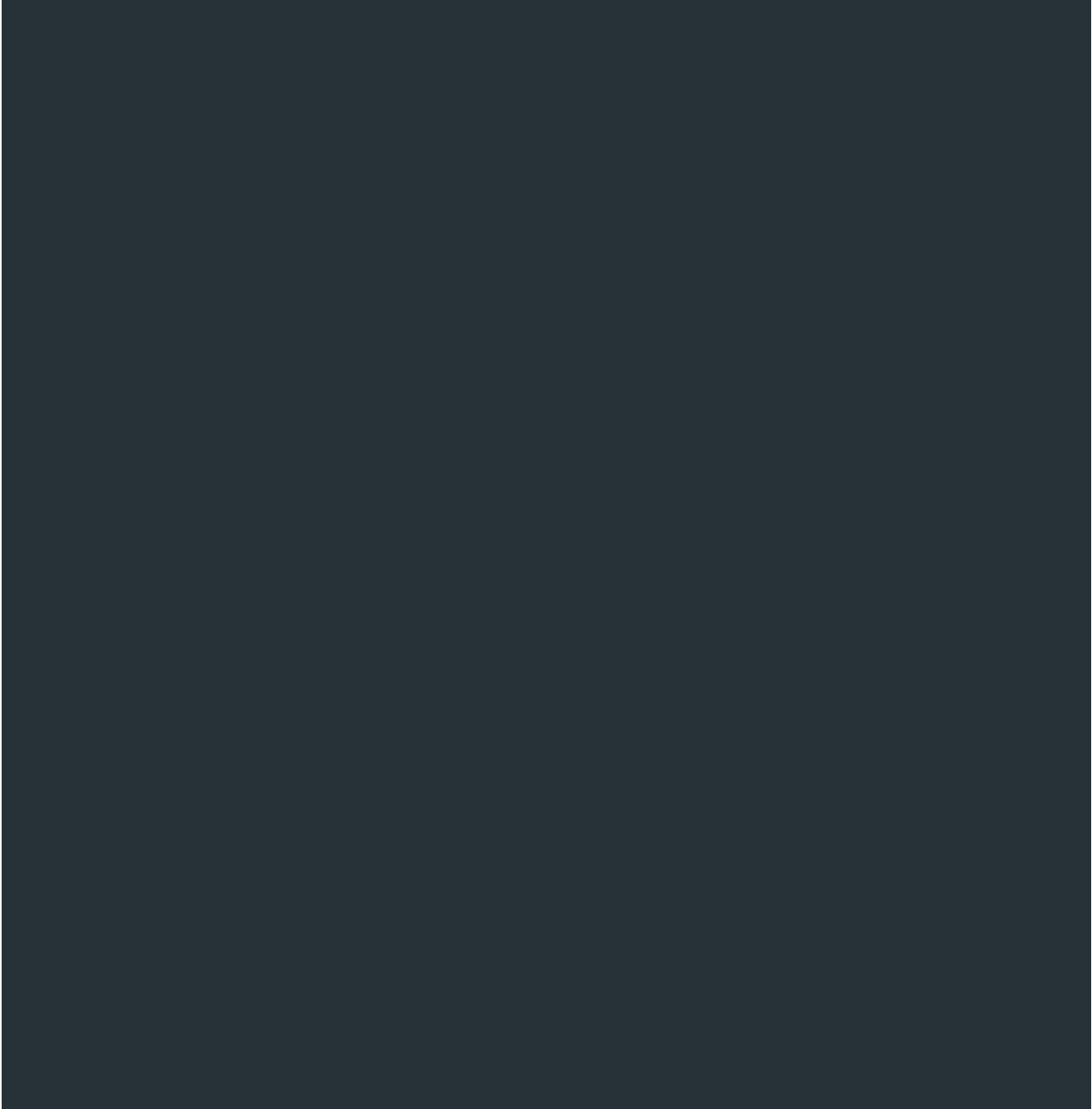
Jobs principais vs jobs funcionais: - **Functional:** "Organizar tarefas do time" -
Emocional: "Me sentir no controle do projeto" - **Social:** "Parecer competente para meu chefe"

4.2 Lean Canvas

Uma página que resume o modelo de negócio. Ideal para early stage.

PROBLEMA	SOLUÇÃO	PROPOSTA DE	VANTAGEM	SEGMENTO
Top 3 problemas	Top 3 soluções	VALOR	COMPETITIVA	DE CLIENTES
		Única	Não copiável	
		Mensagem clara	facilmente	
MÉTRICAS		CANAIS		
CHAVE				
O que medir		Como chegar		

para saber se		no cliente			
está dando certo					
ESTRUTURA DE CUSTOS			RECEITAS		
Fixos, variáveis, custo de aquisição			Como ganha dinheiro		



PROBLEMA	SOLUÇÃO	PROPOSTA DE	VANTAGEM	SEGMENTO
Top 3 problemas	Top 3 soluções	VALOR	COMPETITIVA	DE CLIENTES
		Única	Não copiável	
		Mensagem clara	facilmente	
MÉTRICAS		CANAIS		
CHAVE				
O que medir		Como chegar		

para saber se está dando certo	no cliente			
ESTRUTURA DE CUSTOS		RECEITAS		
Fixos, variáveis, custo de aquisição		Como ganha dinheiro		

4.3 Value Proposition Canvas

Detalha o encaixe entre o cliente e o produto.



5. Mapeamento

5.1 Opportunity Solution Tree (Teresa Torres)

Árvore que conecta **oportunidades** **soluções** **experimentos**.

```
RESULTADO ESPERADO
(reduzir churn em 20%)
|
├── OPORTUNIDADE: Usuários não entendem o valor do produto
|   ├── SOLUÇÃO: Onboarding guiado
|   │   └── EXPERIMENTO: Teste A/B com novo onboarding vs atual
|   └── SOLUÇÃO: Vídeo de apresentação no primeiro login
|       └── EXPERIMENTO: Medir % que completa o vídeo
|
├── OPORTUNIDADE: Usuários não conseguem exportar dados
|   ├── SOLUÇÃO: Botão de exportar CSV
|   │   └── EXPERIMENTO: Protótipo clicável com 5 usuários
|   └── SOLUÇÃO: Integração com Google Sheets
|       └── EXPERIMENTO: Survey de interesse com clientes
|
└── OPORTUNIDADE: Produto é caro para PMEs
    ├── SOLUÇÃO: Plano "Starter" com funcionalidades limitadas
    └── EXPERIMENTO: Landing page com pricing e botão de compra
```

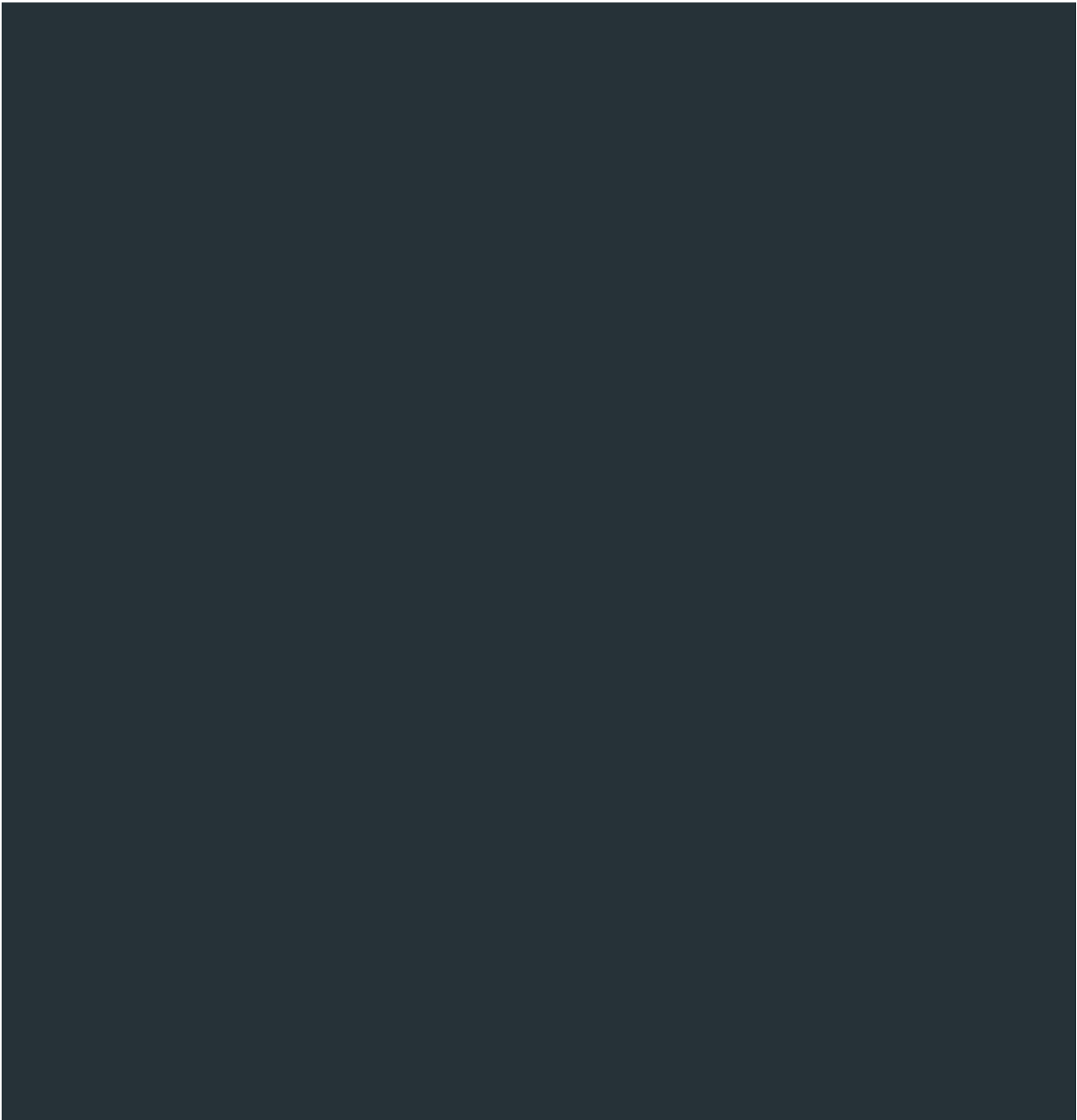
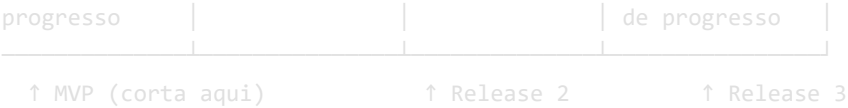
Como construir: 1. Defina o **outcome** (resultado esperado, ex: "aumentar ativação em 30%") 2. Liste **oportunidades** (problemas/dores que impedem o outcome) 3. Para cada oportunidade, gere **soluções** 4. Para cada solução, defina um **experimento** para testar

5.2 User Story Mapping

Técnica de Jeff Patton que organiza o backlog em **atividades do usuário** de forma visual, ajudando o time a enxergar o produto como um fluxo.

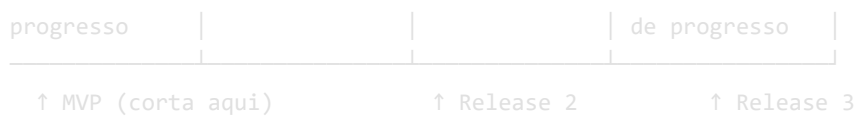
Jornada: "Gerenciar projetos"

	Semana 1	Semana 2	Semana 3
Criar projeto	Criar nome	Convidar membros	
Organizar tarefas	Criar colunas	Adicionar cards	Arrastar cards
Acompanhar			Dashboard



Jornada: "Gerenciar projetos"

	Semana 1	Semana 2	Semana 3
Criar projeto	Criar nome	Convidar membros	
Organizar tarefas	Criar colunas	Adicionar cards	Arrastar cards
Acompanhar			Dashboard



****Por que usar:**** - Mostra o produto como um ****fluxo****, não uma lista - Ajuda a identificar ****gaps**** no fluxo - Facilita decisões de ****MVP**** (o corte vertical) - Alinha time e stakeholders visualmente

6. Validação de Hipóteses

Estrutura de hipótese

HIPÓTESE:

Acreditamos que [solução] para [público] resultará em [outcome].

Saberemos que estamos certos quando [métrica/meta].

Exemplo:

"Acreditamos que um onboarding guiado para novos usuários resultará em 30% mais ativação no D7.

Saberemos que estamos certos quando a taxa de ativação subir de 40% para 52% no teste A/B."

Tipos de experimento

Experimento	Esforço	Confiança	Quando usar
Entrevista	Baixo	Média	Entender problema
Landing page fake	Baixo	Alta	Testar interesse real
Prototype test	Médio	Média	Validar usabilidade
Teste A/B	Médio	Alta	Comparar soluções
Concierge MVP	Alto	Alta	Validar valor real
Wizard of Oz MVP	Médio	Alta	Validar viabilidade
Single-feature MVP	Médio	Alta	Testar feature isolada
Piecemeal MVP	Baixo	Média	Validar sem construir

Exemplo: Landing page fake

Ideia: "App que agenda reuniões automaticamente"

1. Criar landing page:

"Agende reuniões sem trocar emails"

2. Botão: "Começar grátis →" (leva a página de "em breve")

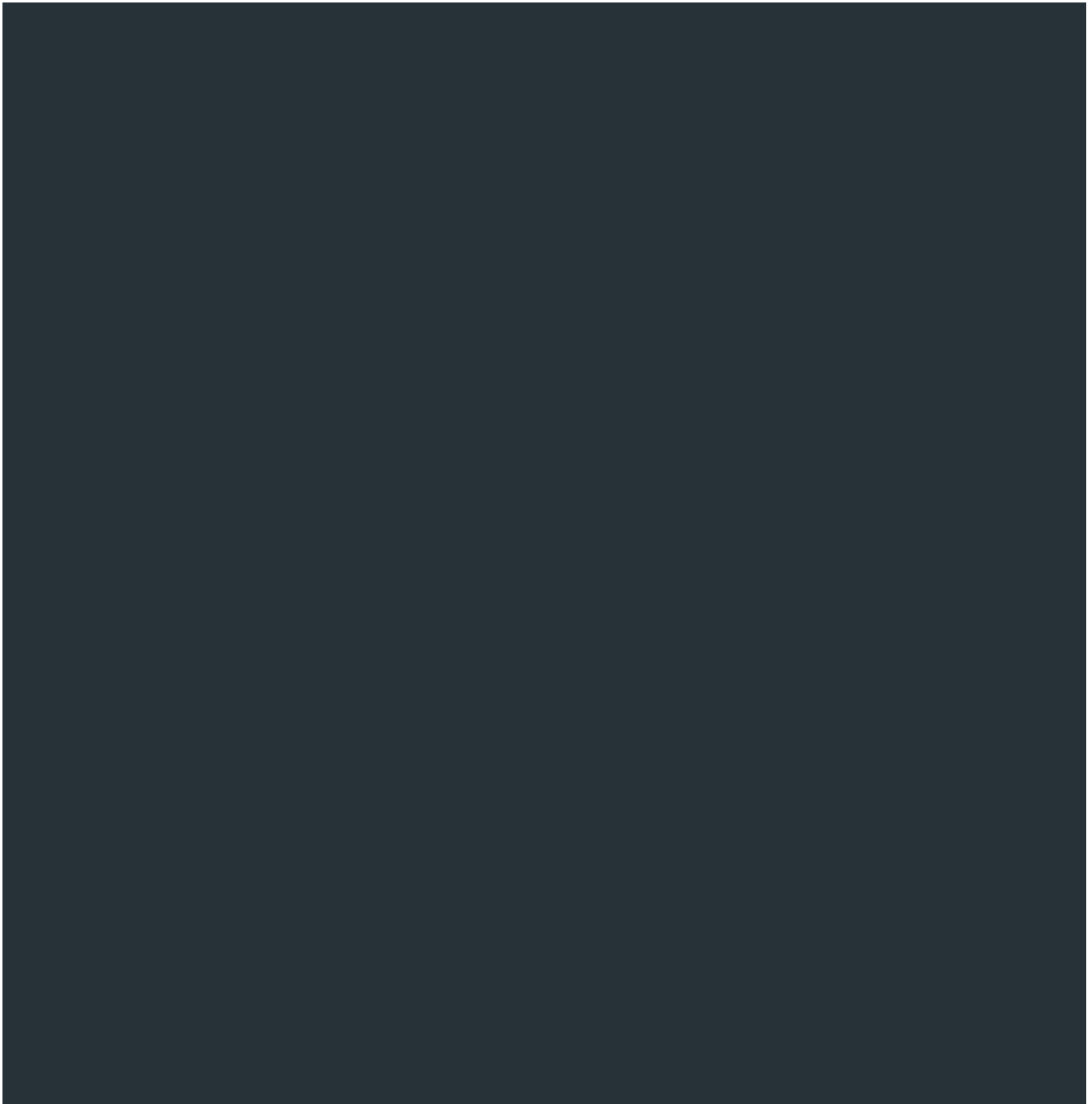
3. Métricas:

- Visitantes que chegam na página
- % que clica no CTA ← indicador de interesse real
- % que deixa o email ← lead qualificado

4. Resultado:

Se < 5% clica → interesse baixo, repense a proposta

Se > 15% clica → interesse real, continue



Ideia: "App que agenda reuniões automaticamente"

1. Criar landing page:

"Agende reuniões sem trocar emails"

2. Botão: "Começar grátis →" (leva a página de "em breve")

3. Métricas:

- Visitantes que chegam na página
- % que clica no CTA ← indicador de interesse real
- % que deixa o email ← lead qualificado

4. Resultado:

Se < 5% clica → interesse baixo, repense a proposta

Se > 15% clica → interesse real, continue

MVP não é produto mínimo

MITO: MVP é a versão mais simples do produto final

REAL: MVP é o menor experimento que valida ou invalida uma hipótese

MVP serve para APRENDER, não para entregar valor.

Se você já sabe que funciona, não precisa de MVP.

7. Product Discovery no Contexto Enterprise

Discovery em empresas grandes tem desafios específicos.

Desafios Enterprise

Desafio	Impacto no Discovery
Muitos stakeholders	Múltiplas visões do que é "prioridade"
Processos burocráticos	Discovery lento, pouca iteração
Métrica de sucesso errada	Time mede output, não outcome
Times isolados	Descobertas não são compartilhadas
Gerenciamento de risco	Medo de errar → pouca experimentação
Orçamento anual fixo	Discovery não é orçado → não existe

Como fazer discovery em Enterprise

1. Alinhe com OKRs

OKR da empresa: "Aumentar receita recorrente em 25%"

- Opportunity: churn de 15% ao mês
- Discovery: por que os clientes estão cancelando?
- Outcome: reduzir churn pela metade
- Backlog: funcionalidades que endereçam as causas

2. Discovery Sprints

Reserve ciclos fixos para discovery (ex: 2 semanas a cada quarter).

Sprint Discovery (2 semanas):

Semana 1: Pesquisa (entrevistas, analytics, concorrência)

Semana 2: Ideação, prototipação, testes com usuários

Saída: Backlog priorizado para as próximas 6 semanas de delivery

3. Envolver stakeholders cedo

Stakeholder	O que quer saber	Quando envolver
Diretor	"Isso entrega o OKR?"	Kickoff, review
Produto	"Isso resolve o problema certo?"	Discovery inteiro
Engenharia	"Isso é viável tecnicamente?"	Prototipação
Design	"Isso é usável?"	Prototipação, teste
Comercial	"O cliente pagaria por isso?"	Pesquisa, validação
Sucesso do cl.	"Isso reduz chamados?"	Pesquisa

****4. Discovery contínuo vs Discovery por projeto****

Discovery Contínuo (recomendado):

- Discovery e Delivery rodam em paralelo
- Time sempre tem 20-30% do tempo para discovery
- Pipeline de oportunidades sempre abastecido

Discovery por Projeto (menos eficaz):

- Discovery só acontece no início do projeto
- Depois que o delivery começa, não se questiona mais
- Se descobrir algo errado no meio, é tarde demais

Cultura de Experimentação

Empresas maduras em discovery têm:

Amazon:	"É mais fácil pedir desculpas do que permissão"
Netflix:	Testa tudo, aprende rápido, falha barato
Spotify:	Squads têm autonomia para discovery
Google:	Data beats opinions
Airbnb:	Design ops com discovery contínuo

Cultura que NÃO funciona:

- "Quem decide é o VP"
- "A gente sabe o que o cliente quer"
- "Se deu trabalho, vamos lançar mesmo assim"

8. Saídas do Discovery

O discovery não termina sem artefatos. As saídas principais são:

8.1 Backlog Refinado

Um backlog que não é só lista de desejos — é priorizado por evidência.

Antes do Discovery:

- [] Tela de relatórios (pedido pelo stakeholder)
- [] Exportar para PDF (ideia do PO)
- [] Modo escuro (sugestão do dev)
- [] Integração com Slack (pedido de 1 cliente)

Depois do Discovery:

- [] Exportar CSV (validado: 78% dos usuários precisam)
- [] Relatório de vendas (validado: resolve job #2)
- [] Tema escuro (despriorizado: 12% pediram)
- [] Integração Slack (despriorizado: 3 clientes, custo alto)

8.2 Proto-personas

Persona provisória baseada em hipóteses, não em pesquisa extensa.

Proto-persona: Analista de Dados

Nome fictício: Carla, 32 anos
Cargo: Analista de BI Sênior
Contexto: Trabalha em empresa de médio porte, time pequeno
Objetivo: Extrair insights rápido para tomar decisões
Dores:
1. Dados espalhados (planilhas, BI, ERP)
2. Demora dias para gerar um relatório
3. Não confia na qualidade dos dados
Comportamento: Usa Excel, Tableau, SQL (básico)
Job to be done: "Quando preciso responder uma pergunta de negócio, quero encontrar os dados certos em minutos, para não perder credibilidade com o diretor."

8.3 User Stories

Histórias baseadas em evidência, não em suposição.

Sem Discovery:

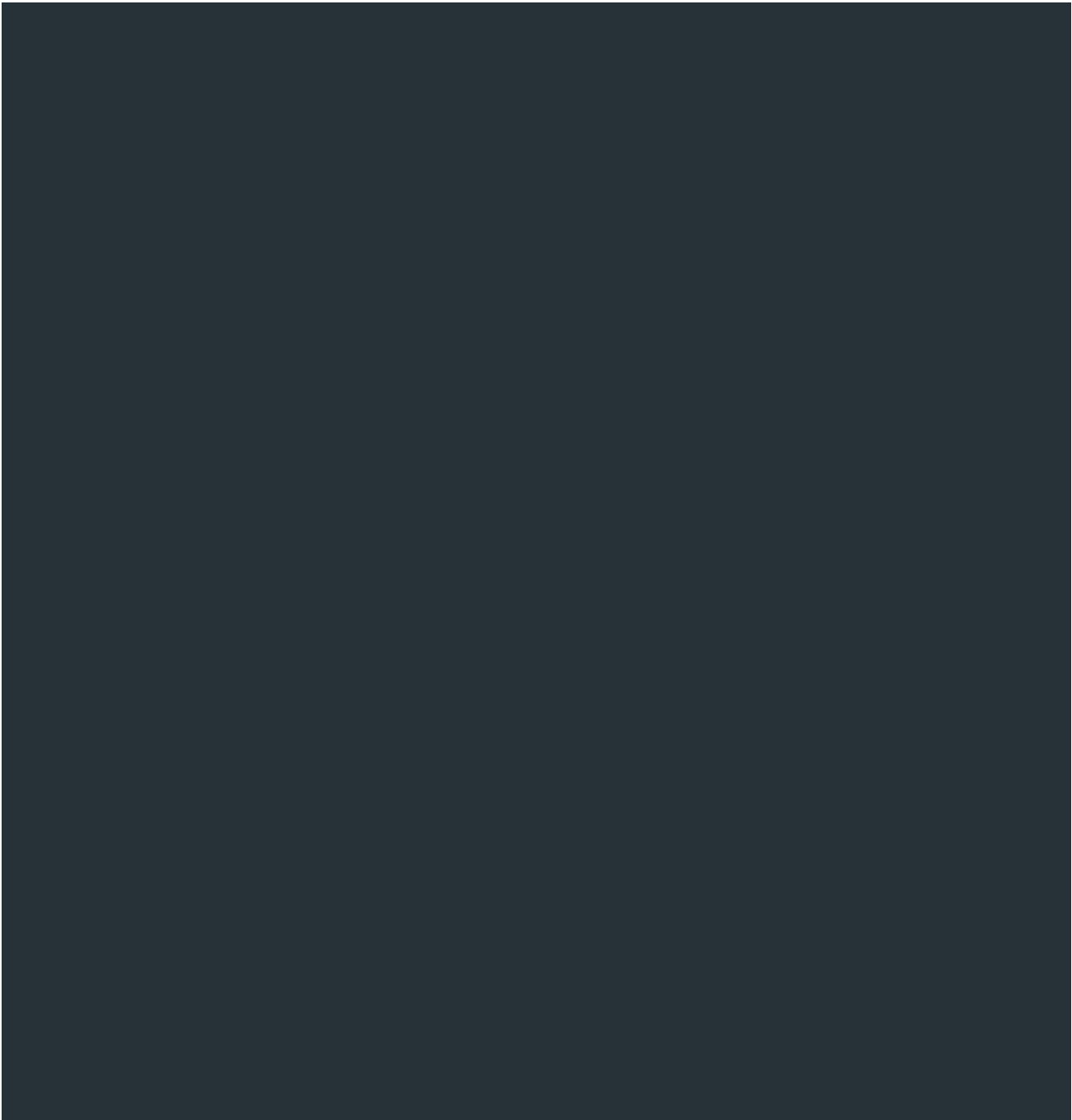
"Como usuário, quero uma tela de relatórios."

Com Discovery:

"Como analista de BI, quero exportar dados em CSV para poder analisar no Excel, porque meu time não tem acesso direto ao banco."

Critérios de aceite:

- Botão de exportar na tela de search
- Formato CSV com encoding UTF-8
- Arquivo baixa em até 10s (para < 100k linhas)
- Colunas traduzidas para pt-BR
- Nome do arquivo: `export-{tipo}-{data}.csv`



Sem Discovery:

"Como usuário, quero uma tela de relatórios."

Com Discovery:

"Como analista de BI, quero exportar dados em CSV para poder analisar no Excel, porque meu time não tem acesso direto ao banco."

Critérios de aceite:

- Botão de exportar na tela de search
- Formato CSV com encoding UTF-8
- Arquivo baixa em até 10s (para < 100k linhas)
- Colunas traduzidas para pt-BR
- Nome do arquivo: `export-{tipo}-{data}.csv`

8.4 Opportunity Backlog

Um backlog de oportunidades (não de soluções).

ID	Oportunidade	Evidência	Tamanho	Prioridade
001	Usuários não encontram dados no sistema	45% dos chamados	Grande	P0
002	Demora para gerar relatórios	Survey: 78% citaram	Médio	P0
003	Não confiabilidade dos dados	NPS: 4.2 (crítico)	Grande	P1
004	Integração com ferramentas externas	3 clientes enterprise	Médio	P2

Resumo

1. **Product Discovery** é o processo de validar problemas e soluções antes de construir 2. **Discovery vs Delivery**: uma decide o que construir, a outra constrói 3. **Ciclo**: Oportunidade Pesquisa Ideação Prototipação Validação 4. **Pesquisa**: entrevistas, surveys, análise concorrencial, analytics 5. **Frameworks**: JTBD, Lean Canvas, Value Proposition Canvas 6. **Mapeamento**: Opportunity Solution Tree, User Story Mapping 7. **Hipóteses**: estruture, meça, aprenda (MVP = experimento, não produto) 8. **Enterprise**: alinhe com OKRs, envolva stakeholders, cultura de experimentação 9. **Saídas**: backlog refinado, proto-personas, user stories, opportunity backlog

Design Thinking

Módulo 03 — Design Thinking: Inovação Centrada no Usuário

Uma abordagem human-centered para resolver problemas complexos.

Evolucao do Design Thinking



Mindset do Design Thinking

1969

Herbert Simon publica "Th

MINDSET DT

- Centrado no ser humano
- Colaborativo e multidisciplinar
- Orientado à ação (aprender fazendo)
- Tolerante ao erro (falhe rápido, aprenda logo)
- Otimista (toda solução é possível)
- Iterativo (nunca está pronto)

Abordagem tradicional vs Design Thinking

TRADICIONAL:

Problema → Análise → Solução → Entrega
→ Teste

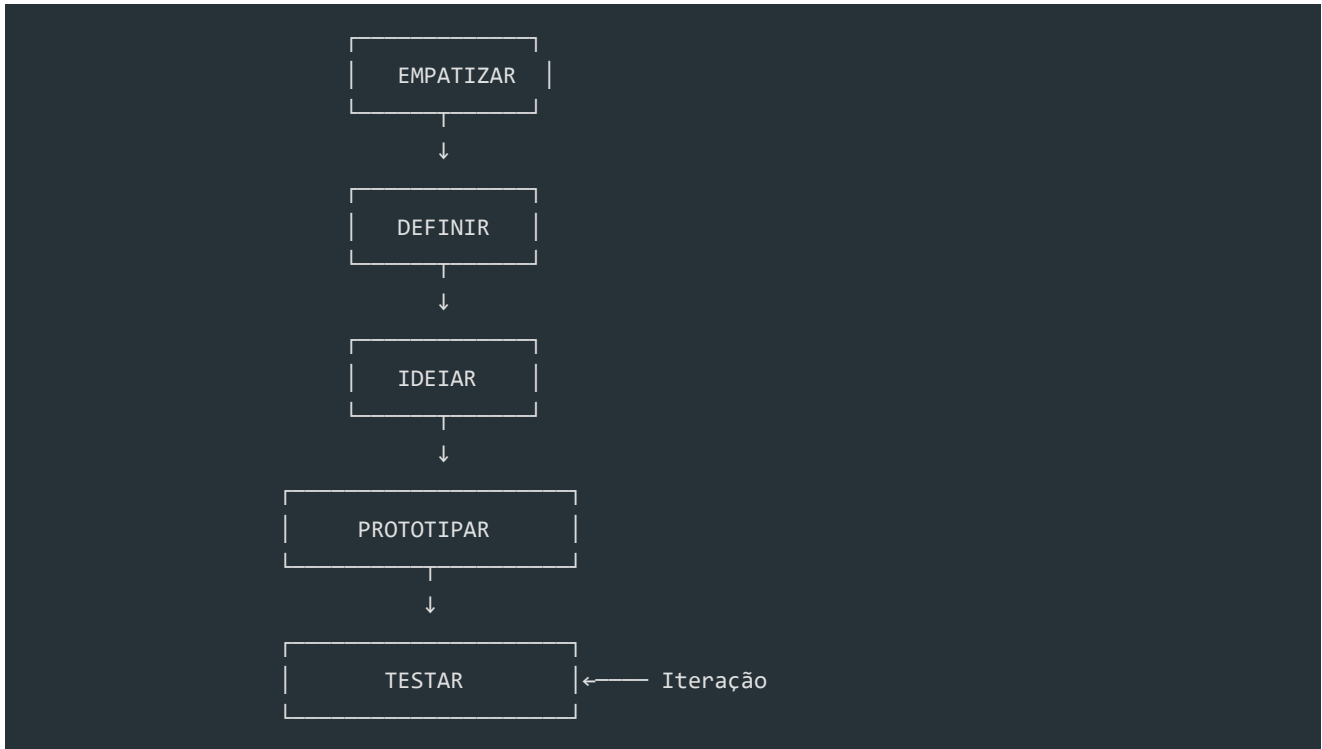
DESIGN THINKING:

Problema → Empatia → Definir → Ideias → Protótipo

Enquanto a abordagem tradicional busca a ****solução certa**** de primeira, o Design Thinking busca ****entender o problema certo**** antes de solucionar, iterando quantas vezes for necessário.

2. As 5 Fases do Design Thinking

O processo é dividido em 5 fases ****não-lineares**** — você pode (e deve) voltar a fases anteriores conforme aprende.



Cada fase responde a uma pergunta central:

Fase	Pergunta
Empatizar	**O quê** o usuário sente
Definir	**Qual** é o problema real
Ideiar	**Quantas** soluções pode
Prototipar	**Como** tornar a solução
Testar	**Funciona** na prática c

3. Fase 1: Empatizar

Por que empatizar?

Sem empatia, você constrói soluções baseadas em **suposições**. Com empatia, você constrói baseado em **fatos** sobre o que o usuário realmente vive.

Técnicas de empatia

Roteiro de entrevista – Exemplo

Abertura (5 min)

- "Conte um pouco sobre seu trabalho/dia a dia"
- "Como você lida com [tópico] atualmente?"

Exploração (15 min)

- "Me conte a última vez que você precisei fazer [ação]"
- "O que foi mais frustrante nesse processo?"
- "O que você fez para contornar?"

Aprofundamento (10 min)

- "Por que isso é importante para você?"
- "O que aconteceria se você não conseguisse fazer isso?"
- "Como você descreveria a solução ideal?"

Fechamento (5 min)

- "Mais alguma coisa que gostaria de compartilhar?"
- "Posso voltar a falar com você se surgir mais dúvidas?"

Regras de ouro para entrevistas:

Faça:

- Perguntas abertas ("Me conte sobre...")
- Escute mais do que fala (proporção 80/20)
- Pergunte "por quê?" repetidamente (técnica dos 5 porquês)
- Observe linguagem corporal e tom de voz
- Registre com autorização (áudio/anotações)

Não faça:

- Perguntas direcionadas ("Você não acha que...")
- Perguntas fechadas ("Você usa X? Sim ou não?")
- Interromper o usuário
- Defender ideias ou justificar o sistema atual
- Buscar validação para sua solução

A observação revela o que as pessoas **realmente fazem** (vs. o que dizem fazer).

```
// Framing da observação
interface Observacao {
  usuario: string;
  contexto: string;
  tarefa: string;
  acoes: Array<{
    timestamp: string;
    acao: string;
    tempo: number;      // segundos
    frustracao: 1 | 2 | 3 | 4 | 5;
    observacao: string;
  }>;
  workarounds: string[];
  insights: string[];
}
```

Exemplo de observação:

```
Usuário: Maria (Analista de BI)
Tarefa: Gerar relatório mensal de vendas

09:01 → Abre sistema, digita login – 12s
09:02 → Navega por 4 telas até encontrar "Relatórios" – 45s
09:03 → Seleciona filtros (mês, região, produto) – 30s
09:05 → Sistema trava ao carregar 3 meses de dados – frustração: 4/5
09:07 → Chama o suporte, enquanto isso abre Excel e começa a fazer manual
```

Insight: A usuária prefere fazer manual no Excel (30 min) do que esperar o sistema travar repetidamente.

A imersão coloca o time **na pele do usuário**. Você experimenta o problema em primeira pessoa.

- **Imersão direta:** Use o sistema como se fosse o usuário - **Imersão indireta:** Acompanhe o usuário por um dia (shadowing) - **Auto-imersão:** Passe um dia sem a solução atual e documente as dificuldades

Mapa de Empatia

MAPA DE EMPATIA	
O QUE ELE FALA?	O QUE ELE FAZ?
"Isso é muito	• Abre Excel direto

complicado"	• Pede ajuda no WhatsApp
"Perco muito tempo nisso"	• Tenta 3x antes de desistir
O QUE ELE OUVI?	O QUE ELE PENSA E SENTE?
Gerente: "Preciso do relatório"	"Deve ter um jeito mais fácil"
Colega: "Sistema é horrível"	"Sou burro por não conseguir?"
	"Isso me estressa"
	"Se eu aprender Python resolvo"
DORES (Frustrações)	GANHOS (Desejos)
• Sistema lento	• Relatório em 1 clique
• Curva de aprendizado alta	• Automação de tarefas
• Falta de suporte humano	• Interface intuitiva
• Retrabalho constante	• Reconhecimento do chefe



MAPA DE EMPATIA	
O QUE ELE FALA?	O QUE ELE FAZ?
"Isso é muito	• Abre Excel direto

4. Fase 2: Definir

Do problema amplo ao ponto de vista

Na fase de Definir, você sintetiza tudo que aprendeu na empatia para criar um ****ponto de vista**** (POV) claro.

Problem Statement

Uma boa declaração de problema segue esta estrutura:

```
[USUÁRIO] precisa de [NECESSIDADE] porque [INSIGHT]
```

****Exemplos:****

Ruim: "O sistema de relatórios é lento"
(focado na solução, não no usuário)

Bom: "Maria, analista de BI, precisa gerar relatórios semanais sem depender do time de TI porque cada solicitação leva em média 3 dias para ser atendida"
(focado no usuário, necessidade e motivo real)

Enterprise: "João, gerente de operações, precisa consolidar dados de 5 fontes diferentes em tempo real porque as decisões baseadas em dados de ontem já não são competitivas"

How Might We (HMW)

As perguntas ****How Might We**** transformam o problem statement em oportunidades de solução.

HMW = How Might We (Como Poderíamos)

How → Assume que é possível (mente aberta)

Might → Permite tentativa e erro (não precisa acertar)

We → É colaborativo (não é individual)

****Técnica:**** Para cada problem statement, gere 5-10 HMWs em diferentes direções:

Problem Statement: Maria precisa gerar relatórios sem o time de TI

HMWs:

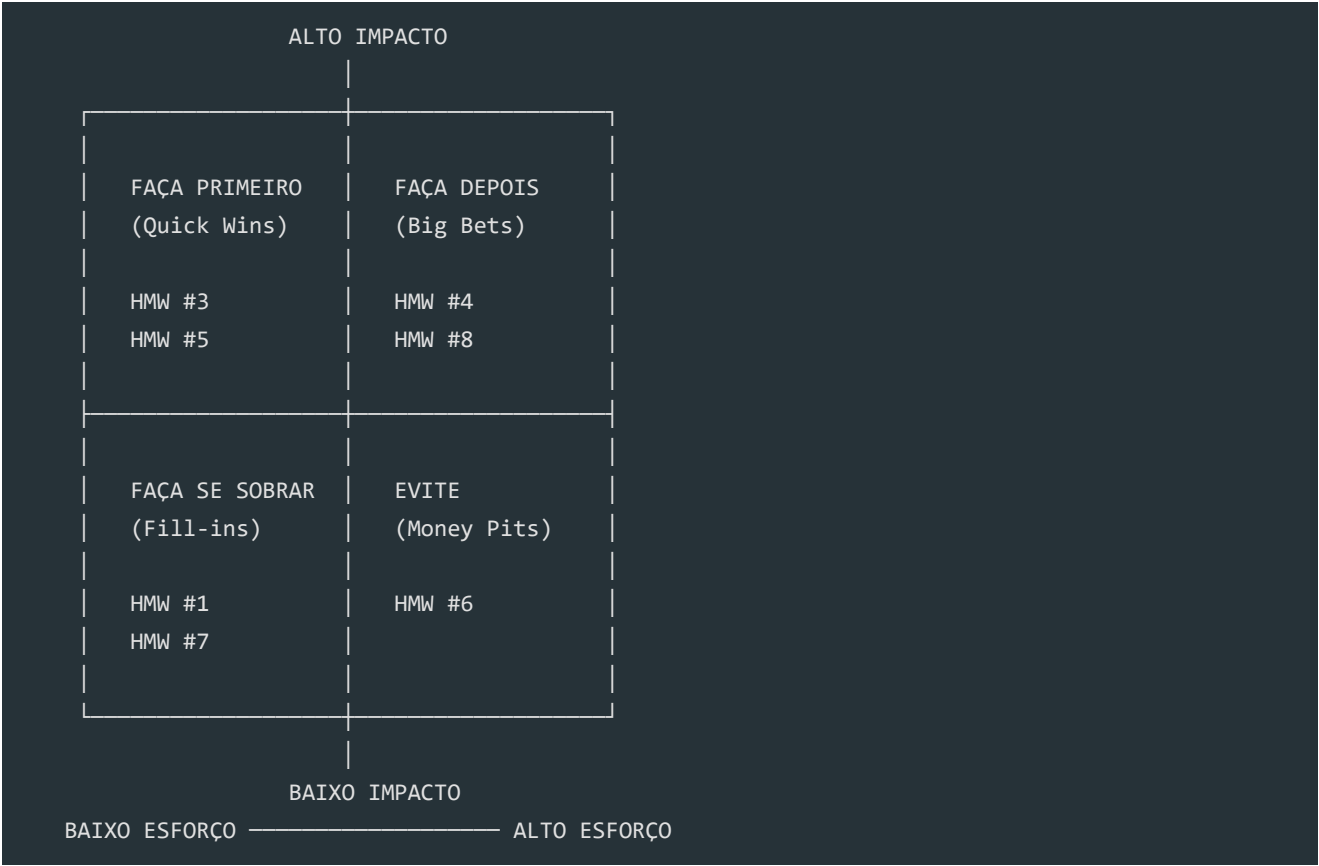
1. HMW tornar a criação de relatórios tão fácil quanto escrever um email?
2. HMW permitir que Maria combine dados sem saber SQL?
3. HMW reduzir o tempo de relatório de 3 dias para 5 minutos?
4. HMW usar IA para sugerir relatórios que Maria nem sabia que precisava?
5. HMW fazer o time de TI responder em minutos em vez de dias?
6. HMW eliminar a necessidade de relatórios manuais completamente?
7. HMW transformar Maria em power-user que ajuda outros colegas?
8. HMW integrar as 5 fontes em um único dashboard em tempo real?

Problem Statement: Maria precisa gerar relatórios sem o time de TI

HMWs:

1. HMW tornar a criação de relatórios tão fácil quanto escrever um email?
2. HMW permitir que Maria combine dados sem saber SQL?
3. HMW reduzir o tempo de relatório de 3 dias para 5 minutos?
4. HMW usar IA para sugerir relatórios que Maria nem sabia que precisava?
5. HMW fazer o time de TI responder em minutos em vez de dias?
6. HMW eliminar a necessidade de relatórios manuais completamente?
7. HMW transformar Maria em power-user que ajuda outros colegas?
8. HMW integrar as 5 fontes em um único dashboard em tempo real?

Matriz de Priorização



5. Fase 3: Ideiar

Princípios do brainstorming

REGRAS DO BRAINSTORMING:

1. Quantidade gera qualidade – quanto mais ideias, melhor
2. Não critique – julgamento só depois
3. Construa sobre ideias alheias ("Sim, e...")
4. Busque ideias selvagens – as mais loucas viram as melhores
5. Seja visual – desenhe, rabisque, use post-its
6. Tempo curto – 15-30 minutos no máximo
7. Um tópico por vez

Crazy 8

Crazy 8 é uma técnica de **divergência rápida**: cada pessoa dobra uma folha A4 em 8 partes e tem 8 minutos para preencher **8 ideias diferentes** (1 minuto por ideia).

Ideia 1	Ideia 2	Ideia 3	Ideia 4
(esboço)	(esboço)	(esboço)	(esboço)
Ideia 5	Ideia 6	Ideia 7	Ideia 8
(esboço)	(esboço)	(esboço)	(esboço)

Por que funciona: A pressão do tempo impede o perfeccionismo e força o cérebro a criar conexões inesperadas.

Matriz Impacto x Esforço

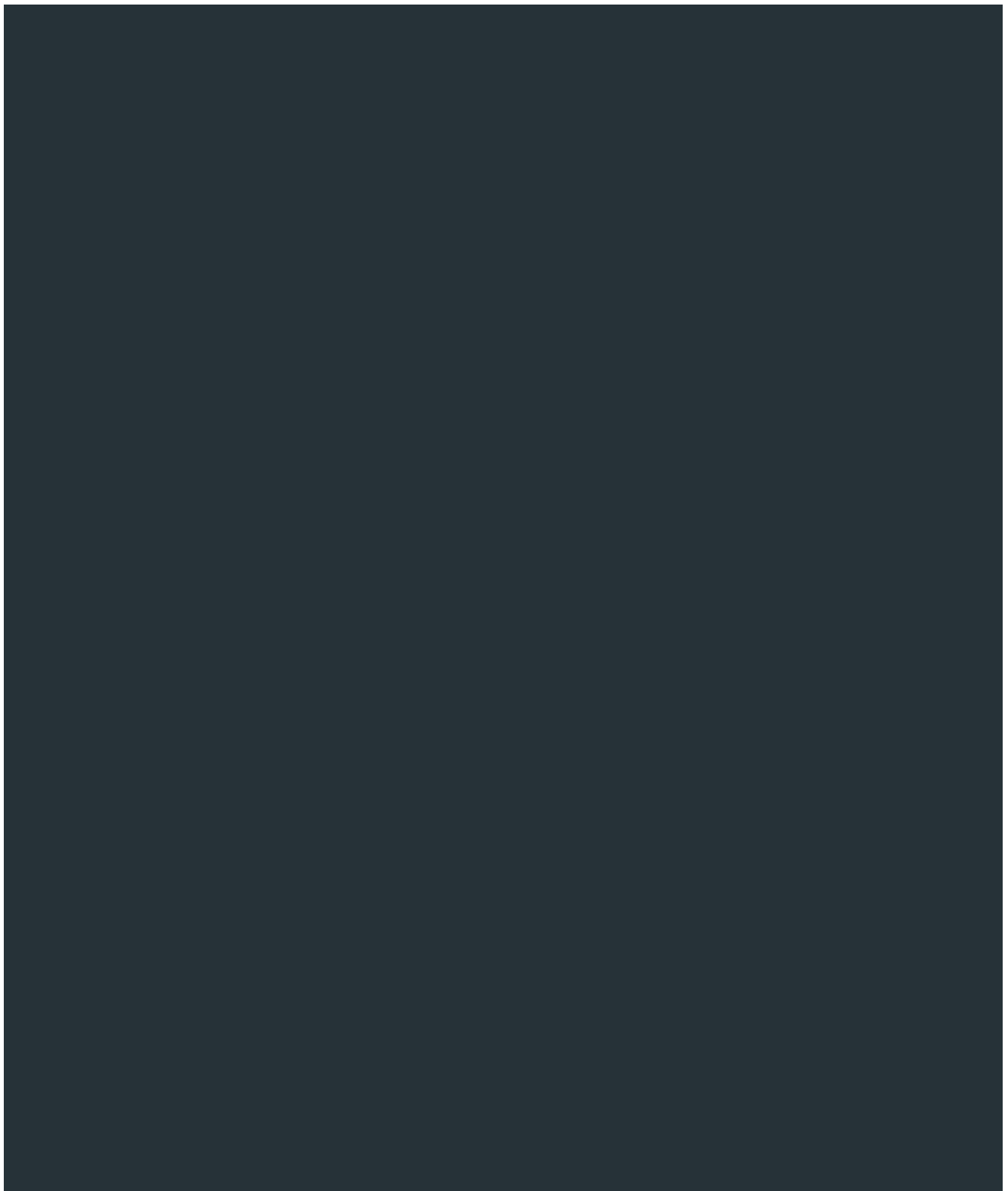
Após o brainstorming, organize as ideias para priorizar:

Esforço ↓ / Impacto →	Baixo Impacto	Alto Impacto
Baixo Esforço	Quick Wins menores	Implemente agora
Alto Esforço	Evite	Big Bets (planeje)

Exemplo de priorização para um sistema de onboarding:

Ideia	Impacto	Esforço	Prioridade
-----	-----	-----	-----

Tutorial interativo na primeira tela	Alto	Baixo	1º	
Integração com LinkedIn para preencher	Baixo	Alto	4º	
Chat ao vivo com suporte	Alto	Alto	2º (Big Bet)	
Remover campos obrigatórios desnecessários	Alto	Baixo	1º	
Gamificação com badges	Baixo	Médio	3º	



Exemplo de priorização para um sistema de onboarding:

Ideia	Impacto	Esforço	Prioridade	
-----	-----	-----	-----	

Tutorial interativo na primeira tela	Alto	Baixo	1º	
Integração com LinkedIn para preencher	Baixo	Alto	4º	
Chat ao vivo com suporte	Alto	Alto	2º (Big Bet)	
Remover campos obrigatórios desnecessários	Alto	Baixo	1º	
Gamificação com badges	Baixo	Médio	3º	

Outras técnicas de ideação

Técnica	Como funciona	Quando usar
Brainwriting	Cada pessoa escreve ideia	Grupos grandes ou times t
SCAMPER	Substitute, Combine, Adap	Melhorar solução existent
Analogias	"Como a Netflix resolveri	Buscar perspectivas difer
Storyboarding	Desenhar cenas do usuário	Validar fluxo de uso

6. Fase 4: Prototipar

Por que prototipar?

"Um protótipo vale mais que mil reuniões."

Prototipar transforma ideias abstratas em algo **tangível** que pode ser testado, discutido e melhorado.

Níveis de fidelidade

BAIXA FIDELIDADE ALTA FIDELIDADE

|-----|

Papel → Wireframe → Mockup → Protótipo clicável → MVP

Feitos com papel, post-its, ou ferramentas simples. **Rápidos e descartáveis.**

Vantagens:

- Leva minutos para criar
- Qualquer um pode fazer
- Ninguém se apega (fácil de descartar)
- Foco no conceito, não na estética
- Baixíssimo custo

Materiais: Papel, caneta, post-it, tesoura, celular para filmar

```
interface WireframeElement {  
  tipo: 'header' | 'footer' | 'card' | 'form' | 'button' | 'modal';  
  posicao: { x: number; y: number; w: number; h: number };  
  conteudo: string;  
  estado?: 'normal' | 'hover' | 'error' | 'loading' | 'empty' | 'success';  
}
```

Ferramentas: Figma, Balsamiq, Whimsical, Miro.

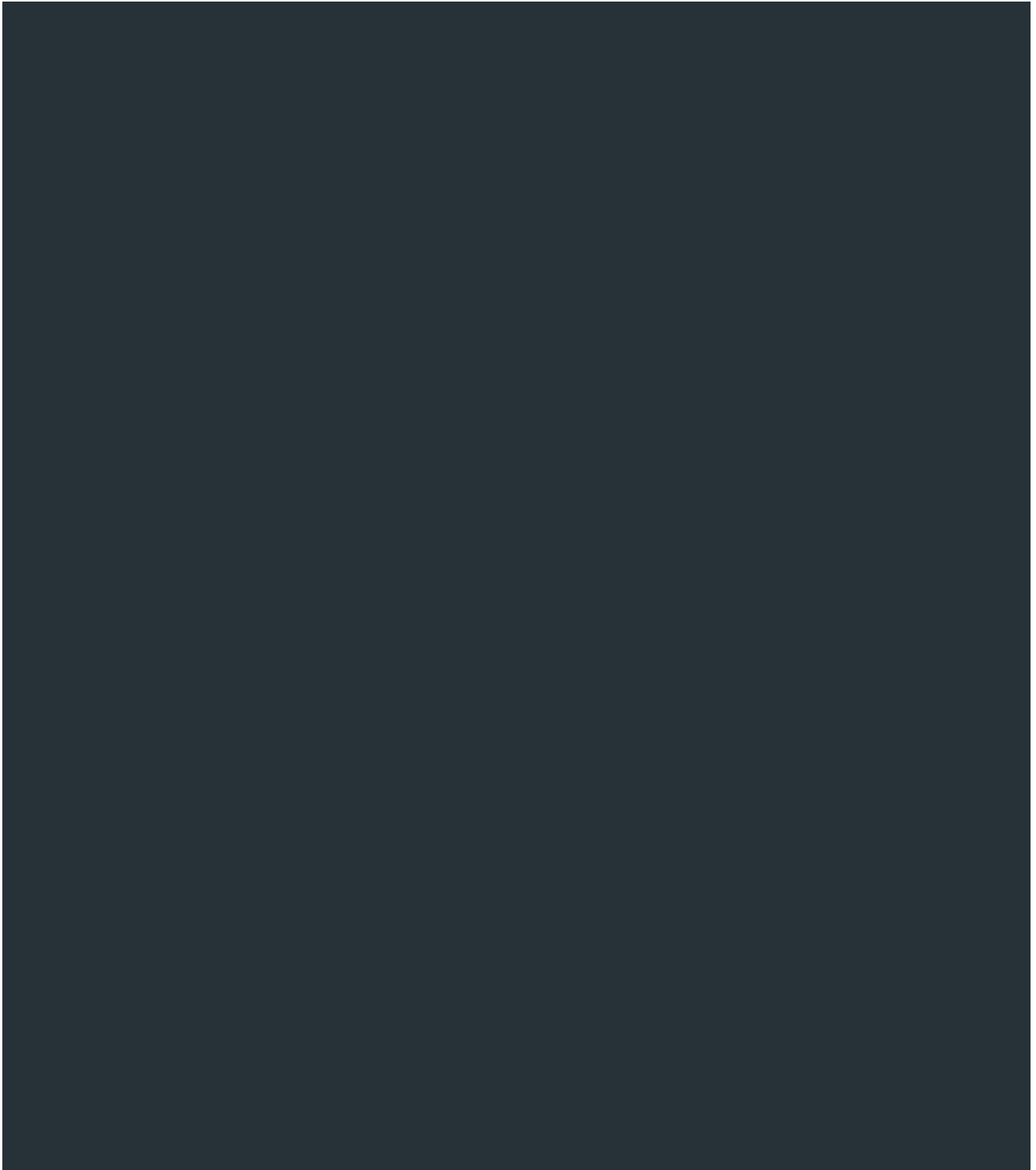
Interativos, simulam a experiência real. Podem ser confundidos com o produto final.

Ferramentas:

- Figma (com prototipagem interativa)
- Axure RP
- Framer
- ProtoPie
- Código real (HTML/CSS/React)

Use quando:

- Precisa testar interações complexas
- Stakeholders precisam visualizar o produto final
- Vai apresentar para clientes



Ferramentas:

- Figma (com prototipagem interativa)
- Axure RP
- Framer
- ProtoPie
- Código real (HTML/CSS/React)

Use quando:

- Precisa testar interações complexas
- Stakeholders precisam visualizar o produto final
- Vai apresentar para clientes

Exemplo: Protótipo de baixa fidelidade para app mobile



Dicas para prototipar

FAÇA:

- Prototipe apenas o essencial para testar a hipótese
- Use papel primeiro (5 min vs 5h no Figma)
- Dê nomes às telas (facilita discussão)
- Mostre estados: loading, empty, error, success
- Teste com usuários reais o mais cedo possível

NÃO FAÇA:

- Prototipar o sistema inteiro de uma vez
- Gastar horas em detalhes visuais antes de validar
- Apresentar protótipo como "quase pronto"
- Pular etapas (papel → direto para código)

7. Fase 5: Testar

O ciclo de teste

PROTÓTIPO → TESTAR → APRENDER → ITERAR → PROTÓTIPO
(e recomeça)

Tipos de teste

Tipo	O que testa	Participantes	Formato
Teste de usabilidade	Navegação, compreensão	5 usuários	Presencial/remoto
Teste A/B	Qual versão performa melh	Centenas/milhares	Online
Teste de conceito	A ideia faz sentido?	10-20 pessoas	Entrevista
Teste de wizard of Oz	Backend simulado por huma	3-5 usuários	Controlado
Teste de protótipo	Fluxo e interações	5 usuários	Moderado

Conduzindo um teste de usabilidade

Setup

1. Defina as tarefas que o usuário deve executar
2. Prepare o protótipo (papel, figma, código)
3. Configure gravação (tela + áudio + câmera)
4. Prepare o roteiro de moderação

Roteiro (20-30 min)

1. Aquecimento (3 min):
 - "Conte um pouco sobre você"
 - "O que você entende que esse sistema faz?"
2. Tarefas (15 min):
 - "Você quer comprar um presente para sua mãe. Como faria?"
 - "Você percebeu que o endereço está errado. Como corrige?"
 - "Quanto custou seu último pedido?"
3. Exploração livre (5 min):
 - "Navegue à vontade e me diga o que está pensando"

4. Fechamento (5 min):

- "O que mais gostou? O que menos gostou?"
- "Se pudesse mudar uma coisa, o que seria?"

O que observar

O usuário conseguiu completar a tarefa?
Quanto tempo levou?
Onde ele hesitou?
Onde ele errou?
O que ele verbalizou? (pensar em voz alta)
Expressões faciais e linguagem corporal

Erro comum: explicar o protótipo

Moderador: "Aqui você clica nesse botão e abre um modal..."
Moderador: "O que você faria agora?"



Setup

1. Defina as tarefas que o usuário deve executar
2. Prepare o protótipo (papel, figma, código)
3. Configure gravação (tela + áudio + câmera)
4. Prepare o roteiro de moderação

Roteiro (20-30 min)

1. Aquecimento (3 min):
 - "Conte um pouco sobre você"
 - "O que você entende que esse sistema faz?"
2. Tarefas (15 min):
 - "Você quer comprar um presente para sua mãe. Como faria?"
 - "Você percebeu que o endereço está errado. Como corrige?"
 - "Quanto custou seu último pedido?"
3. Exploração livre (5 min):
 - "Navegue à vontade e me diga o que está pensando"

4. Fechamento (5 min):
- "O que mais gostou? O que menos gostou?"
 - "Se pudesse mudar uma coisa, o que seria?"

O que observar

O usuário conseguiu completar a tarefa?
Quanto tempo levou?
Onde ele hesitou?
Onde ele errou?
O que ele verbalizou? (pensar em voz alta)
Expressões faciais e linguagem corporal

Erro comum: explicar o protótipo

Moderador: "Aqui você clica nesse botão e abre um modal..."
Moderador: "O que você faria agora?"

Feedback Loop

COLETA	SÍNTESE	AÇÃO
Gravações	Agrupar padrões	Definir o que
Anotações	Priorizar problemas	mudar no protótipo
Métricas (tempo, taxa de sucesso)	Identificar insights	Iterar e testar novamente

Documente os achados com:

Relatório de teste – Sprint 3

#	Problema	Gravidade	Frequência	Solução proposta
1	Usuário não encontra o botão "Finalizar"	Alta	4/5	Mover para o topo da página
2	Confunde "Salvar" com "Enviar"	Média	3/5	Renomear botões
3	Campos de data aceitam formato errado	Baixa	5/5	Adicionar máscara e validação

8. Design Thinking + Ágil

Integração com Scrum e Kanban

Design Thinking e Métodos Ágeis são ****complementares****, não concorrentes.

DESIGN THINKING		SCRUM
Emponder (descobrir)	————→	Sprint 0 / Discovery Sprint
Definir (problema)	————→	Product Backlog (PBI bem definidos)
Ideiar (soluções)	————→	Sprint Planning (discutir abordagens)
Prototipar (testar)	————→	Sprint (desenvolvimento)
Testar (validar)	————→	Sprint Review (feedback do usuário)

Antes de começar a codificar, dedique 1-2 semanas para as fases de Empatizar + Definir + Ideiar.

Sprint 0 / Discovery Sprint (2 semanas):

Semana 1 – Empatizar + Definir

Seg: Planejamento da pesquisa, recrutamento de usuários

Ter-Qua: Entrevistas com 5-8 usuários

Qui: Sessão de síntese, mapas de empatia

Sex: Definição do problem statement e HMWs

Semana 2 – Ideiar + Prototipar

Seg: Sessão de brainstorming + Crazy 8

Ter: Priorização (matriz impacto x esforço)

Qua: Prototipação de baixa fidelidade

Qui: Teste do protótipo com 3-5 usuários

Sex: Iteração + apresentação para stakeholders

Adicione colunas de Discovery no Kanban:

BACKLOG (Ideias)	DISCOVERY (Empatia)	IDEATION (Definir)	PROTOT. (Testar)	DEV (Sprint)	DONE
Item A Item G	Item B HMW #2	Item C HMW #1	Item D Wirefr.	Item E Dev #3	Item F Valid.

Ritmo: Discovery + Delivery

Empresas maduras separam em dois tracks paralelos:

TRACK 1 – DISCOVERY (Design Thinking)

|— Pesquisa com usuários

|— Prototipação e testes

|— Validação de hipóteses

|

▼ (hipóteses validadas viram PBIs)

|

TRACK 2 – DELIVERY (Ágil)

|— Sprint Planning

|— Desenvolvimento

|— Sprint Review

9. Design Thinking em Enterprise

Desafios de escala

Em empresas de grande porte, Design Thinking enfrenta desafios específicos:

Desafio	Impacto	Como mitigar
Stakeholders demais	Decisões lentas, conflito	Mapear influenciadores, s
Processos engessados	Dificuldade de iterar rápido	Criar espaços protegidos
Usuários internos complexos	Múltiplos perfis com necessidades	Segmentar personas, design
Regulamentação	Restrições legais para testes	Envolver compliance desde o início
Escala global	Diferenças culturais	Pesquisas localizadas, de contexto
ROI difícil de medir	Dificuldade de justificar	Métricas de aprendizado e validação

Escalando a abordagem

Assim como DevOps escala engenharia, ****DesignOps**** escala design:

DesignOps – O que faz:

- Cria processos padronizados de pesquisa
- Mantém repositório de insights e personas
- Define métricas de sucesso de design
- Treina times em Design Thinking
- Gerencia ferramentas e assets compartilhados
- Facilita comunicação entre design e negócio

Workshops são a principal ferramenta de adoção em Enterprise.

****Kit do workshop facilitador:****

Checklist para facilitar workshops:

ANTES (1-2 semanas):

- [] Definir objetivo e outcomes esperados
- [] Recrutar participantes diversos (devs, PO, UX, negócios)
- [] Preparar materiais: post-its, canetas, flipchart, timer
- [] Preparar templates (mapa de empatia, matriz, HMW)
- [] Reservar sala com paredes livres e projetor
- [] Enviar briefing prévio para participantes

DURANTE:

- [] Check-in inicial (5 min)
- [] Contexto e objetivo (10 min)
- [] Atividade 1: Empatizar (30 min)

- ☐ Atividade 2: Definir + HMW (30 min)
- ☐ Pausa (10 min)
- ☐ Atividade 3: Crazy 8 (15 min)
- ☐ Atividade 4: Priorização (20 min)
- ☐ Atividade 5: Prototipação em papel (30 min)
- ☐ Apresentação e feedback (20 min)
- ☐ Próximos passos e check-out (10 min)

DEPOIS:

- ☐ Digitalizar e documentar resultados
- ☐ Compartilhar com stakeholders ausentes
- ☐ Agendar follow-up para validação
- ☐ Medir impacto (o que mudou depois do workshop?)



Checklist para facilitar workshops:

ANTES (1-2 semanas):

- ☐ Definir objetivo e outcomes esperados
- ☐ Recrutar participantes diversos (devs, PO, UX, negócios)
- ☐ Preparar materiais: post-its, canetas, flipchart, timer
- ☐ Preparar templates (mapa de empatia, matriz, HMW)
- ☐ Reservar sala com paredes livres e projetor
- ☐ Enviar briefing prévio para participantes

DURANTE:

- ☐ Check-in inicial (5 min)
- ☐ Contexto e objetivo (10 min)
- ☐ Atividade 1: Empatizar (30 min)

- [] Atividade 2: Definir + HMW (30 min)
- [] Pausa (10 min)
- [] Atividade 3: Crazy 8 (15 min)
- [] Atividade 4: Priorização (20 min)
- [] Atividade 5: Prototipação em papel (30 min)
- [] Apresentação e feedback (20 min)
- [] Próximos passos e check-out (10 min)

DEPOIS:

- [] Digitalizar e documentar resultados
- [] Compartilhar com stakeholders ausentes
- [] Agendar follow-up para validação
- [] Medir impacto (o que mudou depois do workshop?)

A IBM criou uma adaptação própria chamada ****Enterprise Design Thinking****, que adiciona:

Princípios IBM:

1. Foco no resultado do usuário (não nas funcionalidades)
2. Times multidisciplinares (não silos)
3. Iteração contínua (não entregas gigantes)

Hills (Metas):

- Diferente de épicos/user stories, Hills descrevem uma mudança de comportamento do usuário em linguagem humana

Exemplo:

"Um gerente de operações consegue identificar gargalos logísticos em tempo real e tomar ações corretivas antes que impactem o cliente final"

10. Anti-padrões em Design Thinking

Pular a fase de empatia

"Já conhecemos nossos usuários" – Não, você não conhece.

Consequência: Solução para o problema errado.

Brainstorming sem regras

Sem moderação, líderes dominam e ideias tímidas morrem.

Consequência: Mesmas soluções de sempre.

Protótipo fotorrealista antes da hora

Gastar dias no Figma antes de validar a ideia com papel.

Consequência: Apego à solução, resistência a mudanças.

Testar com amigos e familiares

Eles querem te agradar, não vão criticar.

Consequência: Feedback enviesado, falsa validação.

Design Thinking como caixa preta

"Vamos fazer um workshop de DT e sai uma solução mágica"

Consequência: Falta de engajamento, resultados superficiais.

Tratar DT como processo linear

"Já testamos, passamos para a próxima fase"

Consequência: Perde-se o principal benefício: a iteração.

Resumo

1. **Design Thinking** é uma abordagem human-centered para resolver problemas complexos 2. **5 fases não-lineares:** Empatizar Definir Ideiar Prototipar Testar 3. **Empatia** é a base — entrevistas, observação e imersão revelam necessidades reais 4. **Definir** sintetiza aprendizados em problem statement e perguntas HMW 5. **Ideiar** prioriza quantidade com brainstorming, Crazy 8 e matriz impacto x esforço 6. **Prototipar** vai do papel ao código, do rápido ao refinado 7. **Testar** valida com usuários reais e alimenta o ciclo de iteração 8. **Design Thinking + Ágil** funciona com Discovery Sprints e Kanban com discovery track 9. **Em Enterprise**, DesignOps e workshops estruturados escalam a prática 10. **Anti-padrões** incluem pular empatia, prototipar cedo demais e testar com amigos

UX Research & Design

Módulo 04 — UX: Experiência do Usuário

Design centrado no usuário não é opcional — é o que separa produtos que vendem de produtos que acumulam poeira.

1. O que é UX?

UX (User Experience) é a **percepção geral que uma pessoa tem ao interagir com um produto, sistema ou serviço**. Não se trata apenas de telas bonitas — envolve emoções, eficiência, acessibilidade e satisfação.

UX vs UI

UX	UI
Experiência completa	Superfície visual
"Como o usuário se sente?"	"Como o produto se parece?"
Pesquisa, arquitetura, fluxo	Cores, tipografia, ícones
Funcionalidade e usabilidade	Estética e identidade
Ciência + Design	Design + Arte

UI sem UX é como um carro bonito sem motor. UX sem UI é como um motor potente sem carroceria.

Os 5 Planos de Jesse James Garrett

O modelo mais clássico para estruturar UX, de baixo (abstrato) para cima (concreto):

5. SUPERFÍCIE	Visual Design	← UI, cores, ícones
4. ESQUELETO	Interface / Navegação	← Layout, botões, inputs
3. ESTRUTURA	Interação / Info Arch	← Fluxos, jornadas
2. ESCOPO	Funcionalidades	← Features, conteúdo
1. ESTRATÉGIA	Necessidades do negócio	← Objetivos, dores

Plano	Pergunta central	Entregável típico
Estratégia	"Por que estamos fazendo	Visão do produto, OKRs
Escopo	"O que vamos construir?"	Backlog, requisitos
Estrutura	"Como as coisas se relaci	Fluxogramas, IA
Esqueleto	"Onde cada coisa aparece?"	Wireframes, protótipos
Superfície	"Qual a aparência final?"	Design system, UI

Para devs: entender os 5 planos significa saber que um bug de frontend pode estar no plano da estrutura (fluxo errado) e não no da superfície (CSS feio).

2. Pesquisa com Usuários

Pesquisa não é opcional — é o que impede você de construir a feature errada.

Entrevistas

```
"Você usaria um botão de exportar CSV?"  
"Me conta como você faz para gerar relatórios hoje."
```

****Estrutura de uma entrevista:****

1. Abertura (5min) – "Não estamos testando você, queremos aprender"
2. Contexto (10min) – "Me conta seu dia a dia com..."
3. Tarefa (15min) – "Você pode tentar fazer X enquanto pensa em voz alta?"
4. Exploração (10min) – "Por que você fez isso? O que esperava que acontecesse?"
5. Fechamento (5min) – "Algo mais que não perguntei?"

****Bons hábitos:**** - Pergunte sobre ****comportamento passado**** (não intenção futura) - Fale 20% do tempo, ouça 80% - Não faça perguntas indutoras ("Você não achou confuso?") - Grave com permissão

Testes de Usabilidade

Peça para o usuário ****realizar uma tarefa**** enquanto observa:

```
// Exemplo de roteiro de teste  
const tasks = [  
  {  
    id: 'cadastro',  
    instrucao: 'Você é um novo usuário. Crie uma conta.',  
    sucesso: 'consegue finalizar o cadastro em < 3 min',  
  },  
  {  
    id: 'pedido',  
    instrucao: 'Faça um pedido do produto X.',  
    sucesso: 'conclui a compra sem ajuda',  
  },  
  {  
    id: 'suporte',  
    instrucao: 'Você precisa cancelar seu pedido. Onde procuraria?',  
    sucesso: 'encontra a opção em < 2 cliques',  
  },  
];
```

****Métricas de usabilidade:**** | Métrica | O que mede | |-----|-----| | Success Rate | % de

usuários que completam a tarefa | | Time on Task | Tempo médio para completar | | Error Rate |
Quantidade de erros cometidos | | SUS Score | Percepção subjetiva de usabilidade | | NPS |
Probabilidade de recomendação |

Card Sorting

Técnica para **entender como usuários organizam informações**:

- **Aberto:** usuários criam suas próprias categorias - **Fechado:** usuários classificam itens em categorias pré-definidas - **Híbrido:** pode sugerir novas categorias

Use card sorting quando for definir a navegação do seu produto. O resultado pode destruir (ou validar) a arquitetura que seu time inventou.

3. Arquitetura da Informação (IA)

Arquitetura da Informação é a **organização estrutural do conteúdo** — como as informações são categorizadas, rotuladas e navegadas.

Os 4 componentes da IA

1. ORGANIZAÇÃO — Como o conteúdo é agrupado
 - └ Hierárquica (ex: categorias e subcategorias)
 - └ Sequencial (ex: wizard de cadastro)
 - └ Matricial (ex: tags, filtros)
2. NAVEGAÇÃO — Como o usuário se movimenta
 - └ Global (menu principal)
 - └ Local (links internos de uma seção)
 - └ Contextual (links no corpo do texto)
 - └ Facetada (filtros dinâmicos)
3. LABELING — Nomes e rótulos
 - └ "Minha Conta" vs "Perfil"
 - └ "Gestão de Usuários" vs "Team"
 - └ Consistência é mais importante que criatividade
4. BUSCA — Sistema de busca interna
 - └ Autocomplete
 - └ Filtros
 - └ Resultados relevantes

Princípios de IA

Princípio	Descrição
Revelância	Mostre o suficiente para
Exemplos	Mostre exemplos do que es
Portas de entrada	Permita chegar ao mesmo c
Classificação múltipla	Ofereça diferentes formas
Navegação focada	Evite misturar navegação

****Para devs:**** AO definir rotas de API e URLs, pense na IA: `typescript // IA fraca, URLs confusas /products/list?type=all /user/info`

`// IA clara, URLs previsíveis /produtos /produtos/:id /minha-conta/dados /minha-conta/pedidos ````

4. Jornada do Usuário

A jornada mapeia **cada passo que o usuário dá** ao interagir com o produto, incluindo emoções, canais e pontos de dor.

User Journey Map

FASE	AÇÕES	EMOÇÃO	OPORTUNIDADE
DESCOBERTA	"Pesquisei 'SaaS CRM'"	Confuso	SEO melhor
	"Li review no Google"	Curioso	Comparativo claro
	"Acessei landing page"	Neutro	CTAs mais claros
AVALIAÇÃO	"Preenchi formulário"	Irritado	Formulário menor
	"Agendei demo"	Ok	Auto-agendamento
	"Testei 14 dias"	Animado	Onboarding guiado
ATIVACÃO	"Convidei time"	Ansioso	Convite em massa
	"Configurei dados"	Frust.	Importação fácil
	"Primeiro relatório"	Feliz	Template pronto
RETENÇÃO	"Uso semanal"	Satisf.	Notificações
	"Preciso de ajuda"	Irrit.	Chat + FAQ
	"Renovei contrato"	Leal	Programa de fidelidade

Service Blueprint

Vai além do User Journey Map: **inclui a visão do backstage** (o que o sistema e o time fazem).

CLIENTE	Ações visíveis ao usuário
	↓
FRONTSTAGE	Interações com o sistema (UI, API calls)
	↓
BACKSTAGE	Ações internas invisíveis ao usuário
	↓
PROCESSOS	Sistemas, jobs, integrações

Exemplo (simplificado de uma compra).

Fase	Cliente	Frontstage	Backstage	Processos
Carrinho	Adiciona item	UI atualiza	Calcula frete	API de frete
Pagamento	Preenche dados	Formulário	Valida antifraude	Gateway
Confirmação	Recebe email	Tela de sucesso	Gera nota fiscal	ERP
Entrega	Rastreia	Tracking page	Logística	Transportadora

****Para devs:**** Service Blueprint mostra que seu código é apenas uma camada. Falhas na integração com ERP, no job de email ou no gateway de pagamento são ****falhas de UX**** também.

5. Personas

Personas são **personagens fictícios baseados em dados reais** que representam segmentos de usuários.

Proto-personas vs Data-driven

Proto-persona	Data-driven persona
Baseada em hipóteses do time	Baseada em dados reais
Rápida (1 workshop)	Leva semanas
"Eu acho que o usuário..."	"Os dados mostram que..."
Valida: "Erramos, vamos pesquisar"	Valida: "Confirmamos nossas hipóteses"

Estrutura de uma persona

```
nome: Dr. Carlos
idade: 42
cargo: Diretor de TI em empresa de médio porte
stack: Legado (Java 8, Oracle, mainframe)

Objetivos:
- Modernizar infra sem parar produção
- Justificar ROI para o CFO
- Reduzir em 30% os incidentes críticos

Dores:
- "Toda mudança é um risco"
- "Não consigo atrair devs bons por causa do legado"
- "Pressão da diretoria para inovar"

Frustrações:
- Ferramentas "modernas" não integram com o legacy
- Fornecedores não entendem o contexto enterprise
- Falta tempo para aprender novas stacks

Comportamento digital:
- Pesquisa no Google antes de comprar
- Lê relatórios do Gartner e Forrester
- Participa de webinar, não de evento presencial
```

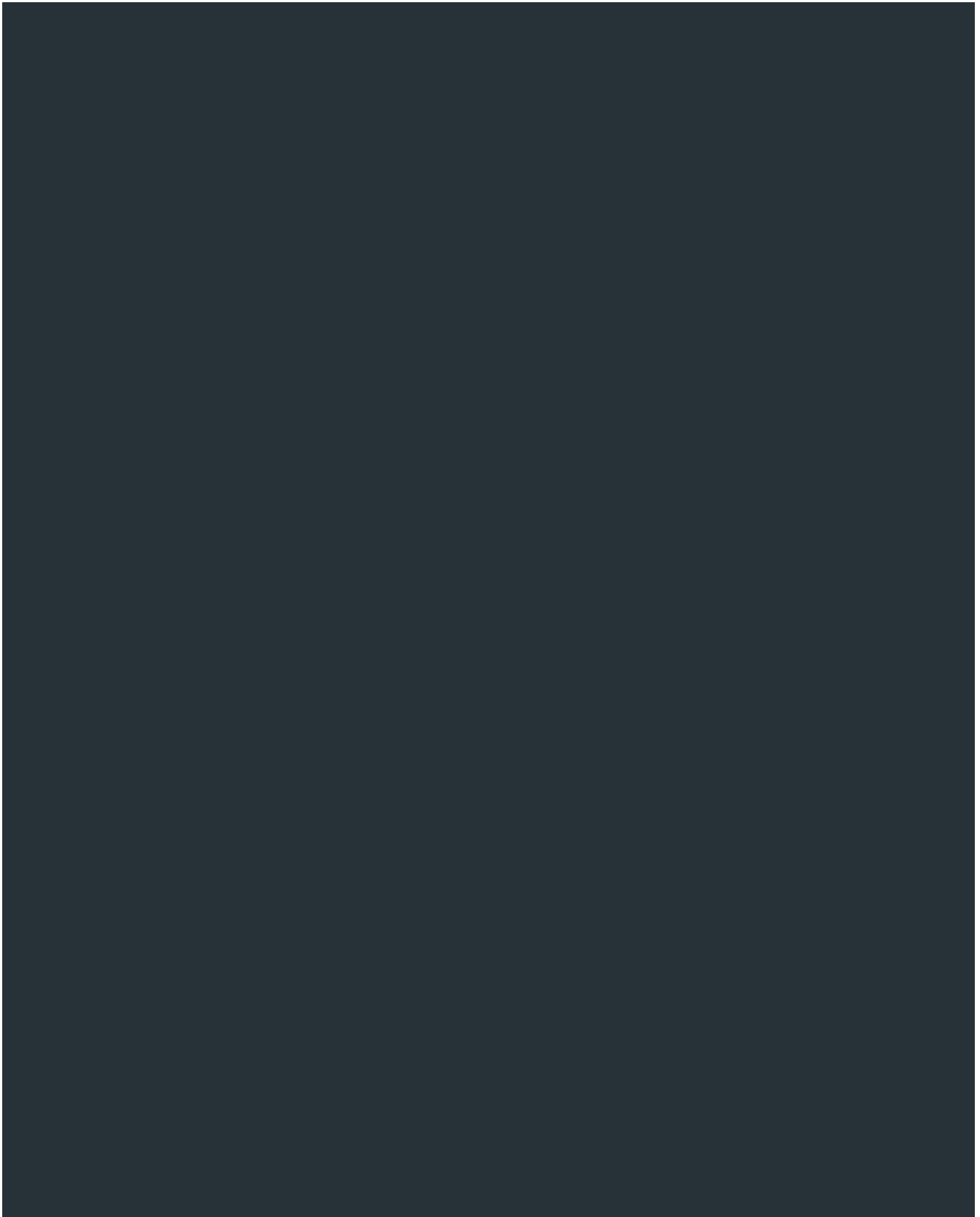
Anti-personas

Usuários que **não são seu público-alvo**. Importante para não desperdiçar esforço.

Anti-persona: "Usuário Free"

- Usa só o plano gratuito
- Abre 3 tickets de suporte por mês
- Dá nota 2 no NPS porque "falta recurso premium"
- Gera custo operacional sem retorno financeiro

O que NÃO fazer: não projetar para ele.



Anti-persona: "Usuário Free"

- Usa só o plano gratuito
- Abre 3 tickets de suporte por mês
- Dá nota 2 no NPS porque "falta recurso premium"
- Gera custo operacional sem retorno financeiro

O que NÃO fazer: não projetar para ele.

Para devs: personas no código

```
// Mapeamento de personas para feature flags
type Persona = 'admin' | 'dev_individual' | 'gestor_enterprise' | 'suporte';

interface FeatureFlag {
  persona: Persona[];
  rollout: number; // % de usuários
}

const flags: Record<string, FeatureFlag> = {
  'export-csv': {
    persona: ['gestor_enterprise', 'admin'],
    rollout: 100,
  },
  'ai-suggestions': {
    persona: ['dev_individual', 'gestor_enterprise'],
    rollout: 10, // Lançamento gradual
  },
};
```


6. Acessibilidade (a11y)

Acessibilidade não é um plus — é **requisito legal e moral**. No Brasil, a **Lei Brasileira de Inclusão (LBI)** exige que sites e apps sejam acessíveis.

WCAG (Web Content Accessibility Guidelines)

As diretrizes se organizam em 4 princípios:

```
P
├─ Perceptível      - O conteúdo deve ser perceptível
│  ├─ Alternativas textuais para imagens
│  ├─ Legendas para vídeos
│  └─ Contraste suficiente
│
O
├─ Operável         - A interface deve ser operável
│  ├─ Navegação por teclado
│  ├─ Tempo suficiente para ler
│  └─ Não causar convulsões (flash)
│
U
├─ Compreensível    - A interface deve ser compreensível
│  ├─ Idioma identificado
│  ├─ Comportamento previsível
│  └─ Tratamento de erros claro
│
R
├─ Robusto          - Deve funcionar com tecnologias assistivas
│  ├─ HTML semântico
│  ├─ ARIA labels
│  └─ Funciona com screen readers
```

Níveis de conformidade

Nível	Descrição	Exemplo
A	Mínimo obrigatório	Alt text em imagens
AA	Recomendado (padrão legal)	Contraste 4.5:1
AAA	Avançado	Linguagem de sinais em ví

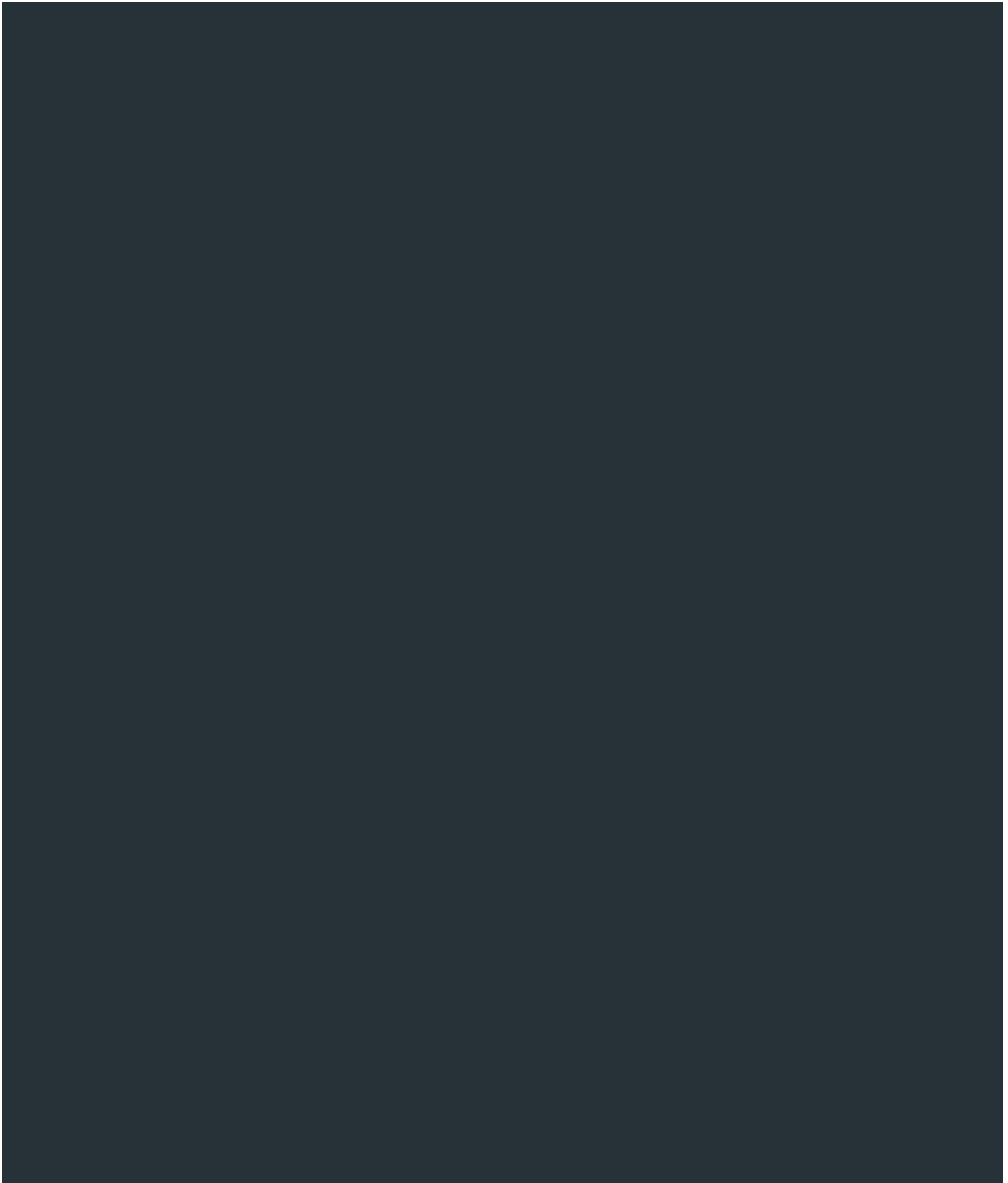
Contraste

Bom contraste (AA):

#333333 sobre #FFFFFF – texto normal
#000000 sobre #FFFFFF – texto grande
#FFFFFF sobre #0055CC – botões

Contraste insuficiente:

#CCCCC sobre #FFFFFF – texto cinza claro
#999999 sobre #EEEEEE – links desativados
#FFCC00 sobre #FFFFFF – alertas amarelos



Bom contraste (AA):

#333333 sobre #FFFFFF – texto normal
#000000 sobre #FFFFFF – texto grande
#FFFFFF sobre #0055CC – botões

Contraste insuficiente:

#CCCCC sobre #FFFFFF – texto cinza claro
#999999 sobre #EEEEEE – links desativados
#FFCC00 sobre #FFFFFF – alertas amarelos

Navegação por teclado

```
// Botão que não recebe foco
<div onClick={handleClick}>Salvar</div>

// Botão nativo com foco
<button onClick={handleClick}>Salvar</button>

// Se precisar de div, adicione ARIA
<div
  role="button"
  tabIndex={0}
  onClick={handleClick}
  onKeyDown={(e) => e.key === 'Enter' && handleClick()}
>
  Salvar
</div>
```

Screen Readers

```
<!-- Imagem decorativa sem alt -->


<!-- Imagem funcional com alt descritivo -->


<!-- Ícone sem contexto -->
<button><i class="icon-cog"></i></button>

<!-- Ícone com aria-label -->
<button aria-label="Configurações"><i class="icon-cog"></i></button>

<!-- Tabela sem escopo -->
<td>Nome</td>

<!-- Tabela com scopo -->
<th scope="col">Nome</th>
```

Para devs: checklist de acessibilidade no código

```
interface A11yChecklist {  
  semanticHtml: boolean;    // Usa tags nativas (<nav>, <main>, <button>)  
  keyboardNav: boolean;     // Tudo operável por teclado  
  focusVisible: boolean;   // Focus ring visível  
  ariaLabels: boolean;     // Labels para ações não textuais  
  colorContrast: boolean;  // 4.5:1 mínimo (AA)  
  reducedMotion: boolean;  // Respeita prefers-reduced-motion  
  zoomTested: boolean;     // Funciona com zoom até 200%  
}  
  
const checkA11y: A11yChecklist = {  
  semanticHtml: false, // div onclick no lugar de button  
  keyboardNav: false,  // dropdown não abre com Enter  
  // ...  
};
```

7. UX Writing

UX Writing é a **criação de textos para interfaces** que guiam o usuário de forma clara e humana.

Microtexto

```
// Texto focado no sistema
"Erro 0x87E50007: Falha na autenticação do certificado"

// Texto focado no usuário
"Seu login expirou. Faça login novamente para continuar."

// Técnico e genérico
"Requisição inválida"

// Claro e acionável
"Preencha todos os campos obrigatórios antes de continuar."
```

Tom de voz

Situação	Tom	Exemplo
Sucesso	Positivo	"Conta criada com sucesso"
Erro	Empático	"Algo deu errado. Não se"
Alerta	Direto	"Sua sessão vai expirar e"
Confirmação	Neutro	"Tem certeza que deseja e"

Mensagens de erro

```
"Falha ao processar requisição"
"Ocorreu um erro inesperado"
"Campos inválidos"

"E-mail ou senha incorretos. Tente novamente."
"O servidor não respondeu. Seu rascunho foi salvo automaticamente."
"O campo 'CNPJ' precisa ter 14 dígitos."
```

Para devs: UX Writing no frontend

```
// Mensagens de erro hardcoded e genéricas
const errorMessagesOld = {
```

```
400: 'Bad Request',
401: 'Unauthorized',
500: 'Internal Server Error',
};
```

```
// Mensagens humanas centralizadas
```

```
const errorMessages = {
  400: 'Verifique os dados enviados e tente novamente.',
  401: 'Sua sessão expirou. Faça login novamente.',
  403: 'Você não tem permissão para essa ação.',
  404: 'O recurso solicitado não foi encontrado.',
  409: 'Já existe um registro com esses dados.',
  422: 'Corrija os campos destacados e tente novamente.',
  429: 'Muitas requisições. Aguarde alguns segundos.',
  500: 'Erro interno. Nossa equipe foi notificada.',
  502: 'Serviço temporariamente indisponível. Tente novamente.',
  503: 'Manutenção programada. Volte em instantes.',
};
```

```
// Mensagens de erro hardcoded e genéricas
const errorMessagesOld = {
```

```
400: 'Bad Request',
401: 'Unauthorized',
500: 'Internal Server Error',
};

// Mensagens humanas centralizadas
const errorMessages = {
  400: 'Verifique os dados enviados e tente novamente.',
  401: 'Sua sessão expirou. Faça login novamente.',
  403: 'Você não tem permissão para essa ação.',
  404: 'O recurso solicitado não foi encontrado.',
  409: 'Já existe um registro com esses dados.',
  422: 'Corrija os campos destacados e tente novamente.',
  429: 'Muitas requisições. Aguarde alguns segundos.',
  500: 'Erro interno. Nossa equipe foi notificada.',
  502: 'Serviço temporariamente indisponível. Tente novamente.',
  503: 'Manutenção programada. Volte em instantes.',
};
```

Princípios de UX Writing

CLARO	→ "Salvar" (não "Persistir alterações no repositório")
CONCISO	→ "E-mail inválido" (não "O endereço de e-mail digitado não é válido")
ÚTIL	→ "Digite seu e-mail corporativo" (orienta, não só rotula)
CONSISTENTE	→ Sempre "Excluir", não "Excluir"/"Remover"/"Deletar" aleatoriamente
HUMANO	→ "Algo deu errado, mas já estamos cuidando disso"

8. Heurísticas de Nielsen

As 10 heurísticas são ****critérios de usabilidade**** que funcionam como checklist para avaliar qualquer interface.

As 10 Heurísticas

1. VISIBILIDADE DO STATUS DO SISTEMA
 - "O sistema deve sempre informar o que está acontecendo."
 - Clica em "Salvar" e nada acontece por 10s
 - Spinner + "Salvando..." + "Salvo!" com timestamp
2. CORRESPONDÊNCIA COM O MUNDO REAL
 - "Use linguagem do usuário, não do sistema."
 - "Diretório raiz do repositório"
 - "Pasta principal do projeto"
3. CONTROLE E LIBERDADE DO USUÁRIO
 - "O usuário deve poder desfazer ações."
 - Excluiu sem confirmação, sem undo
 - "Excluir → Confirmar → Desfazer (5s)"
4. CONSISTÊNCIA E PADRÕES
 - "Mesma palavra = mesma ação em todo o sistema."
 - "Excluir" em um lugar, "Apagar" em outro
 - Sempre "Excluir" + ícone de lixeira
5. PREVENÇÃO DE ERROS
 - "Melhor que uma boa mensagem de erro é nenhum erro."
 - Data digitada livremente e depois valida
 - Datepicker que já impede data inválida
6. RECONHECIMENTO EM VEZ DE RECORDAÇÃO
 - "Mostre opções, não force o usuário a lembrar."
 - Campo "Categoria" em branco
 - Dropdown com as categorias existentes
7. FLEXIBILIDADE E EFICIÊNCIA DE USO
 - "Atenda novatos e experts."
 - Mesmo fluxo para todos
 - Atalhos de teclado para experts, wizard para novatos
8. DESIGN ESTÉTICO E MINIMALISTA
 - "Cada informação extra compete com a informação relevante."
 - Dashboard com 20 gráficos
 - Top 5 métricas que importam

9. AJUDE USUÁRIOS A RECONHECER, DIAGNOSTICAR E RECUPERAR DE ERROS

"Mensagens de erro claras e acionáveis."

"Erro 500"

"Serviço indisponível. Tente novamente em alguns minutos."

10. AJUDA E DOCUMENTAÇÃO

"Se o usuário precisa de ajuda, ela deve estar disponível."

Documentação escondida

"?" contextual + FAQ + chat

1. VISIBILIDADE DO STATUS DO SISTEMA

"O sistema deve sempre informar o que está acontecendo."

Clica em "Salvar" e nada acontece por 10s

Spinner + "Salvando..." + "Salvo!" com timestamp

2. CORRESPONDÊNCIA COM O MUNDO REAL

"Use linguagem do usuário, não do sistema."

"Diretório raiz do repositório"

"Pasta principal do projeto"

3. CONTROLE E LIBERDADE DO USUÁRIO

"O usuário deve poder desfazer ações."

Excluiu sem confirmação, sem undo

"Excluir → Confirmar → Desfazer (5s)"

4. CONSISTÊNCIA E PADRÕES

"Mesma palavra = mesma ação em todo o sistema."

"Excluir" em um lugar, "Apagar" em outro

Sempre "Excluir" + ícone de lixeira

5. PREVENÇÃO DE ERROS

"Melhor que uma boa mensagem de erro é nenhum erro."

Data digitada livremente e depois valida

Datepicker que já impede data inválida

6. RECONHECIMENTO EM VEZ DE RECORDAÇÃO

"Mostre opções, não force o usuário a lembrar."

Campo "Categoria" em branco

Dropdown com as categorias existentes

7. FLEXIBILIDADE E EFICIÊNCIA DE USO

"Atenda novatos e experts."

Mesmo fluxo para todos

Atalhos de teclado para experts, wizard para novatos

8. DESIGN ESTÉTICO E MINIMALISTA

"Cada informação extra compete com a informação relevante."

Dashboard com 20 gráficos

Top 5 métricas que importam

9. AJUDE USUÁRIOS A RECONHECER, DIAGNOSTICAR E RECUPERAR DE ERROS

"Mensagens de erro claras e acionáveis."

"Erro 500"

"Serviço indisponível. Tente novamente em alguns minutos."

10. AJUDA E DOCUMENTAÇÃO

"Se o usuário precisa de ajuda, ela deve estar disponível."

Documentação escondida

"?" contextual + FAQ + chat

Como avaliar com as heurísticas

```
type HeuristicSeverity = 0 | 1 | 2 | 3 | 4;

interface HeuristicEvaluation {
  heuristic: string;
  severity: HeuristicSeverity;
  finding: string;
  suggestion: string;
}

// Severidade:
// 0 = não é problema
// 1 = problema cosmético
// 2 = problema menor
// 3 = problema grave (deve ser corrigido)
// 4 = catástrofe (uso impossível)

const evaluation: HeuristicEvaluation[] = [
  {
    heuristic: '1. Visibilidade do status',
    severity: 3,
    finding: 'Botão "Exportar" não mostra progresso',
    suggestion: 'Adicionar barra de progresso + notificação ao finalizar',
  },
  {
    heuristic: '9. Mensagens de erro',
    severity: 4,
    finding: 'Erro 500 sem mensagem ao usuário',
    suggestion: 'Mensagem amigável + log no backend',
  },
];
```

9. UX para Devs

Mentalidade centrada no usuário

ANTES	DEPOIS
"O usuário vai aprender"	"Como tornar isso intuitivo?"
"Está no requisito"	"O usuário realmente precisa disso?"
"Funciona no meu ambiente"	"Funciona para o usuário real?"
"O design está errado"	"Qual problema estamos resolvendo?"

Como colaborar com designers

Situação	Faça	Não faça
Recebeu um design	Pergunte "qual o fluxo do	Comece a codificar sem en
Conflito de implementação	Mostre dados (tempo, perf	Fale "não dá pra fazer" s
Review de design	Aponte violações de acess	Critique cor ou fonte
Entrega de funcionalidade	Valide com teste rápido d	Considere pronto só porqu
Refinamento	Leve dados de analytics/f	Apenas "o PO pediu"

Checklist para devs

```
## Antes de codificar
- [ ] Entendi o problema do usuário (não só a solução)
- [ ] Verifiquei se existem dados/entrevistas que embasam a demanda
- [ ] Questionei: "essa é a melhor forma de resolver?"

## Durante o código
- [ ] Componentes são acessíveis (teclado, screen reader)
- [ ] Mensagens de erro são humanas e acionáveis
- [ ] Estados vazios, loading e erro estão tratados
- [ ] Contraste mínimo 4.5:1
- [ ] Microinterações informam o status do sistema

## Antes do deploy
- [ ] Testei com teclado apenas (sem mouse)
- [ ] Testei com zoom de 200%
- [ ] Textos estão consistentes (mesmo label para mesma ação)
- [ ] Performance: < 3s para carregar (mobile)
```

Exemplo prático: refatorando uma feature com mentalidade UX

```
// Antes: feature-centrica
function TelaRelatorios() {
  const [relatorios] = useQuery(GET_RELATORIOS);
  return <Table data={relatorios} />;
  // Usuário: "e agora? o que eu faço com isso?"
}

// Depois: centrada no usuário
function TelaRelatorios() {
  const [relatorios, { loading, error }] = useQuery(GET_RELATORIOS);
  const [search, setSearch] = useState('');
  const filtered = relatorios.filter(r => r.name.includes(search));

  if (loading) return <Skeleton lines={5} aria-label="Carregando relatórios" />;
  if (error) return <ErrorAlert message="Não foi possível carregar. Tente novamente." />;
  if (filtered.length === 0) return <EmptyState message="Nenhum relatório encontrado. Crie o primeiro." />;

  return (
    <Page>
      <SearchInput
        value={search}
        onChange={setSearch}
        placeholder="Buscar relatório por nome..."
        aria-label="Buscar relatórios"
      />
      <Table
        data={filtered}
        emptyMessage="Nenhum resultado para essa busca"
      />
    </Page>
  );
}
```

```

// Antes: feature-centrica
function TelaRelatorios() {
  const [relatorios] = useQuery(GET_RELATORIOS);
  return <Table data={relatorios} />;
  // Usuário: "e agora? o que eu faço com isso?"
}

// Depois: centrada no usuário
function TelaRelatorios() {
  const [relatorios, { loading, error }] = useQuery(GET_RELATORIOS);
  const [search, setSearch] = useState('');
  const filtered = relatorios.filter(r => r.name.includes(search));

  if (loading) return <Skeleton lines={5} aria-label="Carregando relatórios" />;
  if (error) return <ErrorAlert message="Não foi possível carregar. Tente novamente." />;
  if (filtered.length === 0) return <EmptyState message="Nenhum relatório encontrado. Crie o primeiro." />;

  return (
    <Page>
      <SearchInput
        value={search}
        onChange={setSearch}
        placeholder="Buscar relatório por nome..."
        aria-label="Buscar relatórios"
      />
      <Table
        data={filtered}
        emptyMessage="Nenhum resultado para essa busca"
      />
    </Page>
  );
}

```

Perguntas que todo dev deveria fazer

1. "Qual o objetivo do usuário nesta tela?"
2. "O que acontece se der erro?"
3. "Como um usuário novato se sentiria aqui?"
4. "Isso funciona sem JavaScript?"
5. "O usuário consegue desfazer essa ação?"
6. "Onde o usuário vai procurar essa funcionalidade?"
7. "Em quanto tempo isso carrega no 3G?"

Resumo

Tópico	Principal aprendizado
UX vs UI	UX é a experiência comple
5 Planos	Estratégia Escopo Est
Pesquisa	Entreviste para entender,
IA	Organize conteúdo como o
Jornada	Mapeie emoções, não só cl
Personas	Baseie em dados, não em a
Acessibilidade	WCAG AA é o mínimo legal
UX Writing	Mensagens claras, concisa
Heurísticas	10 critérios para avaliar
Dev + UX	Pergunte "por quê" antes

Wireframes e Prototipação

Módulo 05 — Wireframes

****Esboçar telas antes de codar. Validar antes de investir.****

1. O que são Wireframes

Wireframe é o **esqueleto visual** de uma tela. É a representação estrutural da interface, focando em **layout, hierarquia, conteúdo e funcionalidade** — sem nenhum polimento visual (cores, fontes, imagens).

Propósito

Wireframe serve para:

- Validar fluxo e navegação antes do design
- Alinhar time sobre a estrutura da tela
- Identificar inconsistências cedo
- Servir de contrato entre produto e dev
- Acelerar o ciclo de iteração

Níveis de Fidelidade

Fidelidade	Aparência	Quando usar	Prós	Contras
Baixa	Esboço à mão, traços simples	Brainstorming, ideação	Rápido, barato, encoraja	Pouco realista, difícil c
Média	Grey box, tons de cinza	Validação de fluxo, teste	Clara distinção de hierar	Ainda não testa apelo vis
Alta	Quase real, com grid de	Handoff, aprovação de	Mínima ambiguidade, l	Leva mais tempo, pode ser

```
// Representação de níveis de fidelidade
type Fidelity = 'low' | 'medium' | 'high';

interface WireframeSpec {
  fidelity: Fidelity;
  elements: WireframeElement[];
  interactions?: Interaction[];
}

interface WireframeElement {
  type: 'header' | 'hero' | 'card' | 'form' | 'button' | 'footer';
  boundingBox: { x: number; y: number; w: number; h: number };
  placeholder: string; // "imagem do produto", "título"
}
```

2. Wireframe vs Mockup vs Protótipo

É comum confundir os três. A diferença está no **nível de detalhe** e no **propósito**.

DETALHE VISUAL		
Wireframe	Mockup	Protótipo
Estrutura	Visual	Interação
Caixas	Cores	Clica
Hierarquia	Fontes	Navega
Fluxo	Ícones	Anima
Sem estilo	Imagens	Testa

Comparação

Aspecto	Wireframe	Mockup	Protótipo
O que é	Esqueleto estrutural	Representação estática fi	Simulação interativa
Fidelidade	Baixa/média	Alta	Alta
Interativo?	Não	Não	Sim
Cores?	Não (tons de cinza)	Sim (design final)	Sim
Tempo de criação	Minutos	Horas	Dias
Objetivo	Alinhar estrutura	Aprovar visual	Testar usabilidade
Quem usa	Produto + Dev + Design	Stakeholders	Time + Usuários

Quando usar cada um

```
interface DesignStage {
  phase: string;
  artifact: 'wireframe' | 'mockup' | 'prototype';
  goal: string;
  participants: string[];
  estimatedTime: string;
}

const stages: DesignStage[] = [
  {
    phase: 'Descoberta',
    artifact: 'wireframe',
    goal: 'Validar fluxo e estrutura com o time',
    participants: ['PO', 'Designer', 'Dev'],
    estimatedTime: '30 min - 2h',
  },
  {
```



```
    phase: 'Definição',
    artifact: 'mockup',
    goal: 'Aprovar estilo visual com stakeholder',
    participants: ['Designer', 'PO', 'Cliente'],
    estimatedTime: '1 - 3 dias',
  },
  {
    phase: 'Validação',
    artifact: 'prototype',
    goal: 'Testar usabilidade com usuários reais',
    participants: ['Designer', 'Dev', 'Usuários'],
    estimatedTime: '2 - 5 dias',
  },
];
```

```
interface DesignStage {
  phase: string;
  artifact: 'wireframe' | 'mockup' | 'prototype';
  goal: string;
  participants: string[];
  estimatedTime: string;
}

const stages: DesignStage[] = [
  {
    phase: 'Descoberta',
    artifact: 'wireframe',
    goal: 'Validar fluxo e estrutura com o time',
    participants: ['PO', 'Designer', 'Dev'],
    estimatedTime: '30 min - 2h',
  },
  {
```


3. Princípios de Layout

Wireframe eficiente segue princípios de design visual, mesmo sem cores.

Grid

O grid é a **espinha dorsal** do layout. Ele garante alinhamento, consistência e ritmo.

```
interface Grid {
  columns: number;      // ex: 12
  gutter: number;       // ex: 16px (espaço entre colunas)
  margin: number;       // ex: 24px (margem lateral)
  breakpoints: Record<string, number>;
}

const desktopGrid: Grid = {
  columns: 12,
  gutter: 16,
  margin: 24,
  breakpoints: { sm: 640, md: 768, lg: 1024, xl: 1280 },
};

// Utilitário para calcular largura de coluna
function colWidth(cols: number, grid: Grid): number {
  const contentWidth = 1200 - grid.margin * 2;
  const totalGutter = (grid.columns - 1) * grid.gutter;
  const colSize = (contentWidth - totalGutter) / grid.columns;
  return colSize * cols + (cols - 1) * grid.gutter;
}
```

Wireframe com grid de 12 colunas:

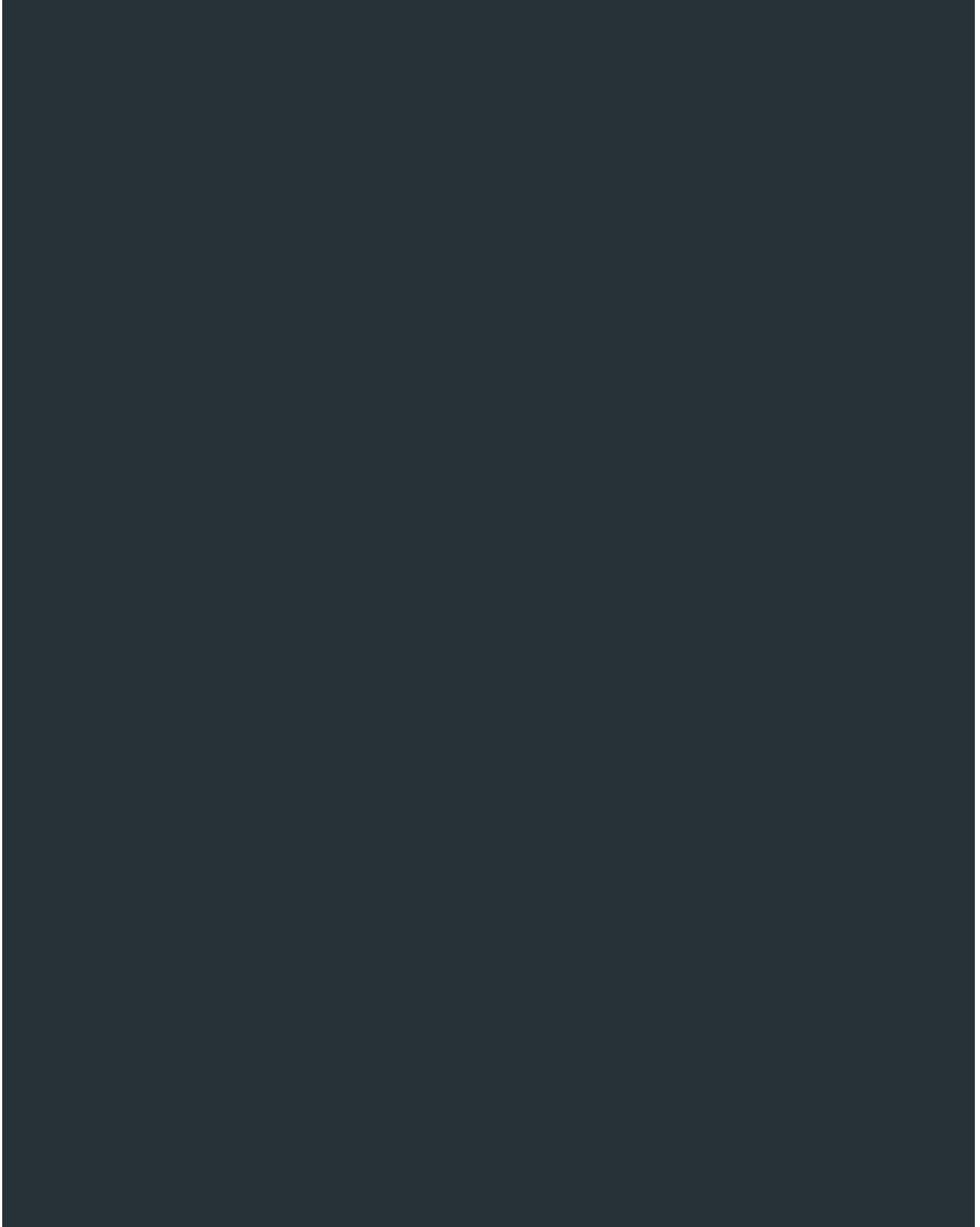
Header (12 col)											
3col	3col	3col	3col								
Content (8 col)								Sidebar (4 col)			
Footer (12 col)											

Hierarquia Visual

Organize os elementos por **importância**. O olho do usuário deve saber para onde olhar primeiro.

Hierarquia no wireframe:

- | | |
|----------------------------|---------------------------------|
| 1. Hero / Título principal | (maior box, posição central) |
| 2. Chamada para ação (CTA) | (destacado, contraste de forma) |
| 3. Seções de conteúdo | (blocos médios, organizados) |
| 4. Navegação secundária | (menor, no topo ou sidebar) |
| 5. Footer | (menor destaque, no final) |



Hierarquia no wireframe:

1. Hero / Título principal (maior box, posição central)
2. Chamada para ação (CTA) (destacado, contraste de forma)
3. Seções de conteúdo (blocos médios, organizados)
4. Navegação secundária (menor, no topo ou sidebar)
5. Footer (menor destaque, no final)

Espaçamento

```
// Sistema de espaçamento (8px grid)
const space = {
  xxs: 4,    // 4px  - ícones, badges
  xs: 8,     // 8px  - padding interno compacto
  sm: 16,    // 16px - entre elementos relacionados
  md: 24,    // 24px - entre seções
  lg: 32,    // 32px - entre blocos maiores
  xl: 48,    // 48px - entre seções principais
  xxl: 64,   // 64px - margens de página
};

// Regra: elementos relacionados ficam mais próximos (8-16px)
// Seções diferentes ficam mais distantes (32-48px)
```

Proporção

Use proporções familiares para criar harmonia visual:

```
|| 16:9  - Hero/Banner (widescreen) ||
|| 4:3   - Cards de conteúdo         ||
|| 1:1   - Avatares, thumbnails      ||
|| 3:2   - Imagens de destaque       ||
|| 2:1   - Painéis e dashboards      ||
```

4. Ferramentas

Comparação

Ferramenta	Tipo	Fidelidade	Curva	Colaboração	Preço
Papel e caneta	Físico	Baixa	Zero	Presencial	Grátis
Excalidraw	Web	Baixa	5 min	Tempo real (link)	Grátis
Balsamiq	Desktop/Web	Baixa-média	30 min	Compartilhamento	Pago (~\$12/mês)
Figma	Web	Média-alta	2h	Tempo real + comentários	Grátis (início)
Whimsical	Web	Baixa-média	15 min	Link compartilhável	Grátis (limitado)
Miro	Web	Baixa	10 min	Quadro infinito + templates	Grátis (limitado)

Quando usar qual

Papel e caneta

- Brainstorming individual ou em dupla
- Sprint de design (Google Design Sprint)

Excalidraw / Whimsical

- Esboço remoto rápido
- Wireframes de baixa fidelidade colaborativos

Balsamiq

- Wireframes de média fidelidade
- Prototipação rápida sem distração visual

Figma

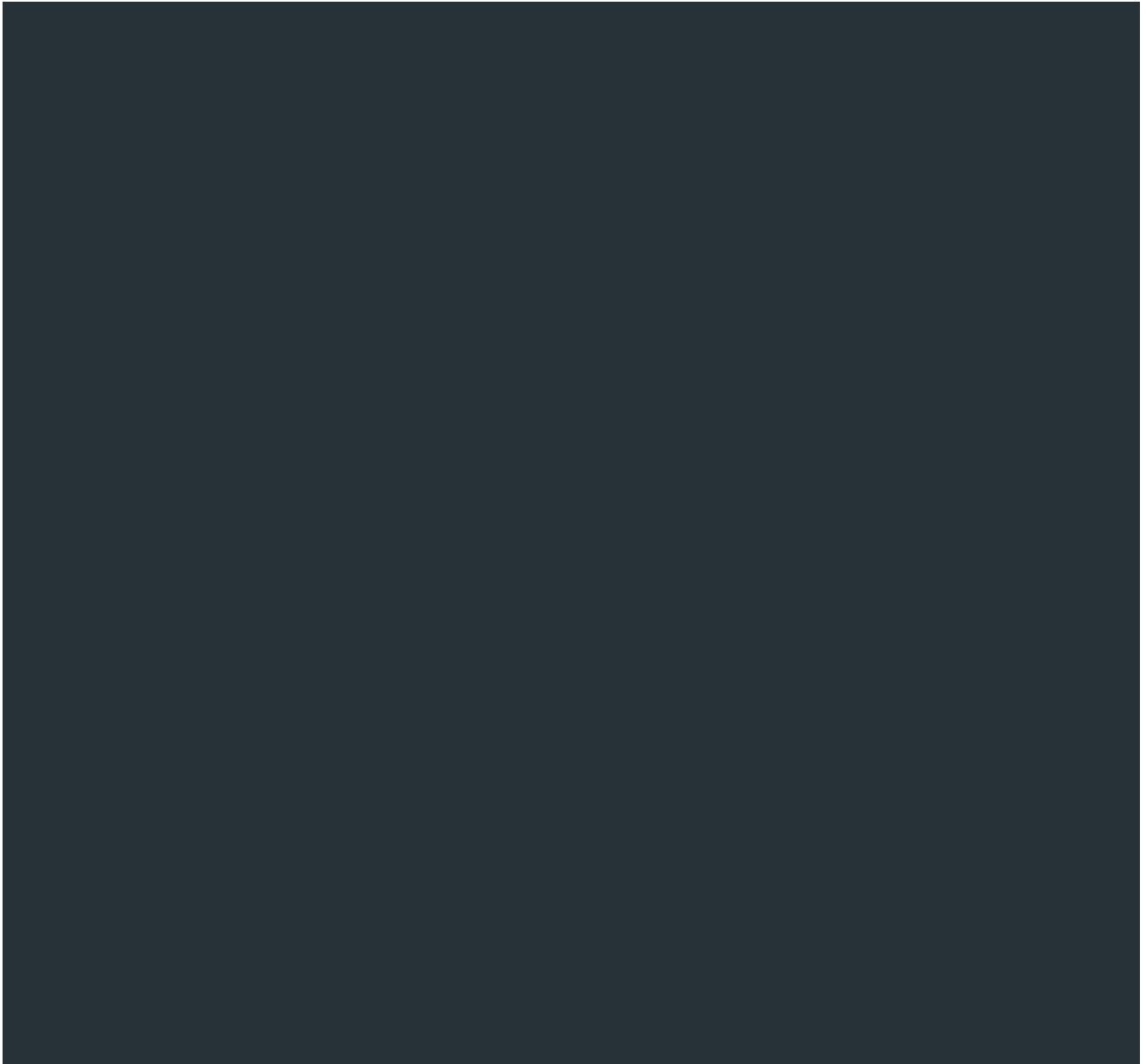
- Wireframes de média/alta fidelidade
- Componentes reutilizáveis
- Handoff para devs
- Protótipos interativos

Excalidraw — Exemplo rápido

```
// Representação de um wireframe no Excalidraw
interface ExcalidrawElement {
  type: 'rectangle' | 'text' | 'arrow' | 'diamond';
  x: number;
  y: number;
  width: number;
  height: number;
  backgroundColor: 'transparent' | '#ffffff' | '#cccccc';
  strokeStyle: 'solid' | 'dashed' | 'dotted';
```

```
}
```

```
const loginWireframe: ExcalidrawElement[] = [  
  { type: 'rectangle', x: 0, y: 0, width: 400, height: 60, backgroundColor: '#cccccc',  
    strokeStyle: 'solid' },  
  { type: 'text', x: 10, y: 20, width: 200, height: 20, backgroundColor: 'transparent',  
    strokeStyle: 'solid' },  
  // ...mais elementos  
];
```



```
// Representação de um wireframe no Excalidraw  
interface ExcalidrawElement {  
  type: 'rectangle' | 'text' | 'arrow' | 'diamond';  
  x: number;  
  y: number;  
  width: number;  
  height: number;  
  backgroundColor: 'transparent' | '#ffffff' | '#cccccc';  
  strokeStyle: 'solid' | 'dashed' | 'dotted';  
}
```

```
}  
  
const loginWireframe: ExcalidrawElement[] = [  
  { type: 'rectangle', x: 0, y: 0, width: 400, height: 60, backgroundColor: '#cccccc',  
    strokeStyle: 'solid' },  
  { type: 'text', x: 10, y: 20, width: 200, height: 20, backgroundColor: 'transparent',  
    strokeStyle: 'solid' },  
  // ...mais elementos  
];
```

****Dica Enterprise**:** Figma é o padrão da indústria. Invista em aprender componentes, auto-layout e variants. Balsamiq é ótimo para documentação regulatória (foco em estrutura, não em visual).

5. Técnicas de Wireframing

Esboço Rápido (Crazy 8s)

Técnica de **divergência criativa**: dobre uma folha em 8 partes, desenhe 8 variações diferentes de uma mesma tela em **8 minutos** (1 minuto cada).

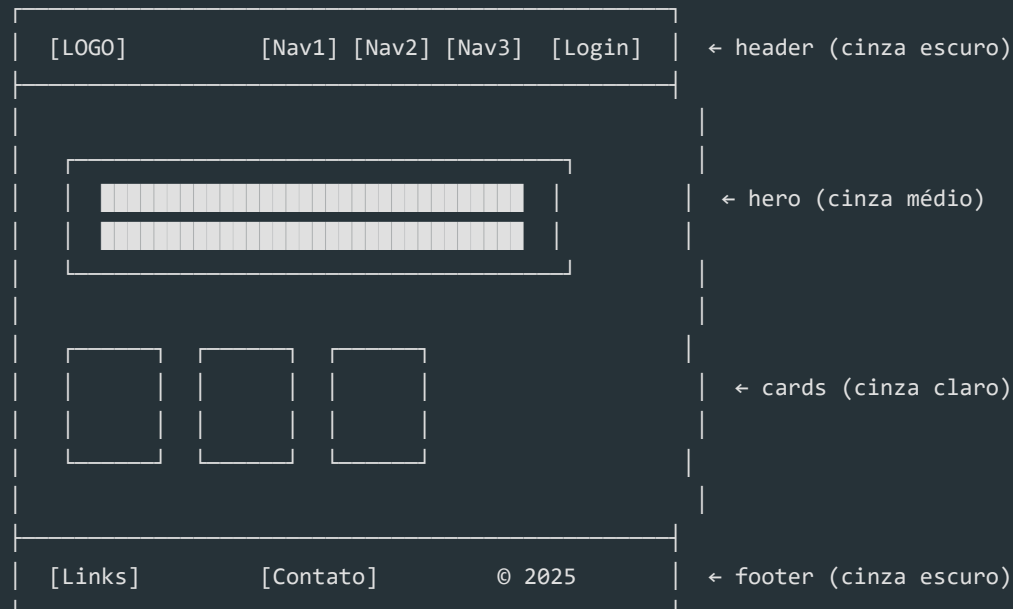
Como fazer:

1. Defina o problema: "Tela de dashboard pós-login"
2. Configure timer de 8 minutos
3. Desenhe 8 versões diferentes (sem repetir)
4. Ao final, vote na melhor ideia
5. Refine a vencedora em um wireframe único

Grey Box

Técnica de wireframing que usa **apenas caixas cinzas** para representar os elementos. Foco total na estrutura, sem distrações.

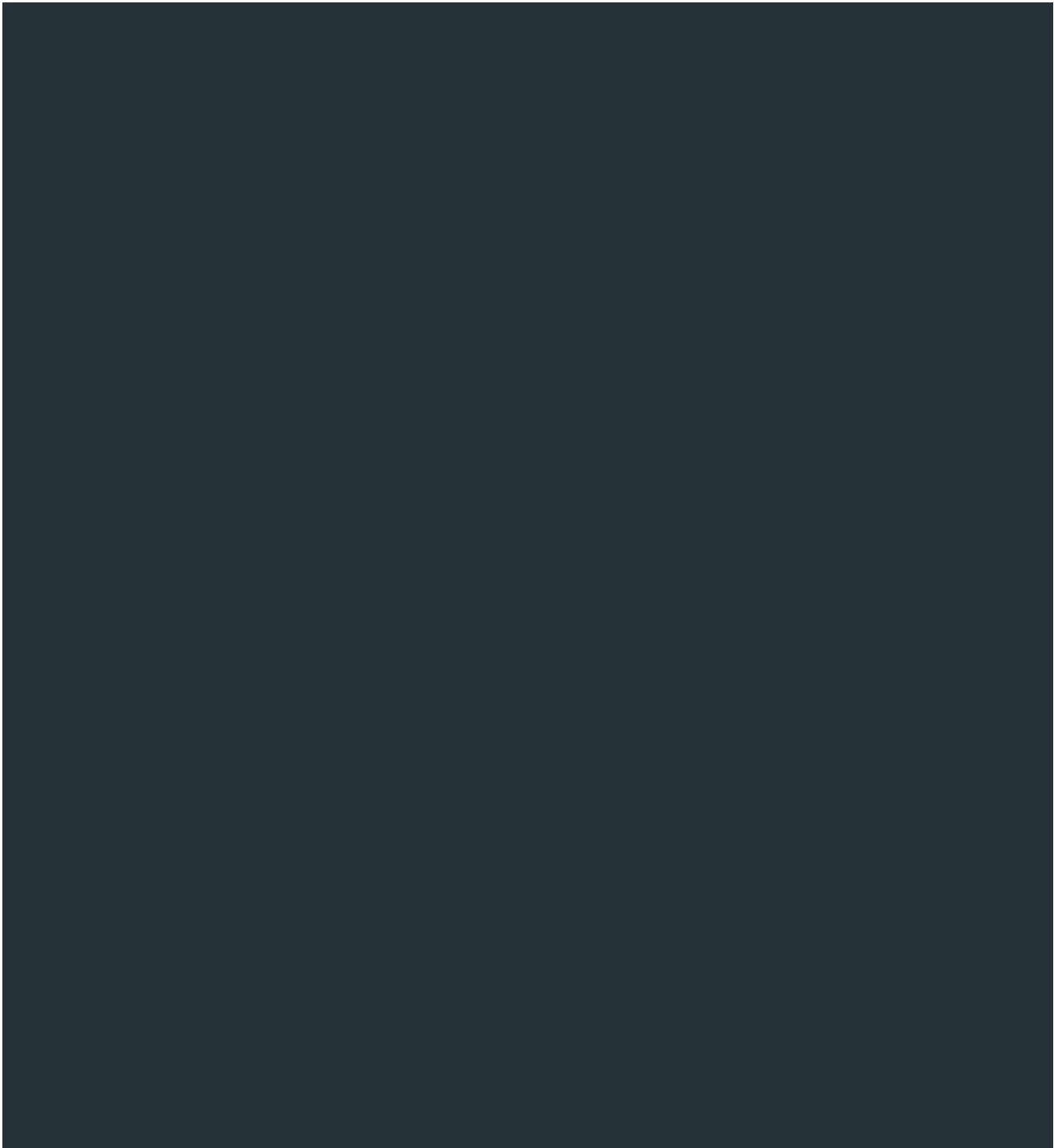
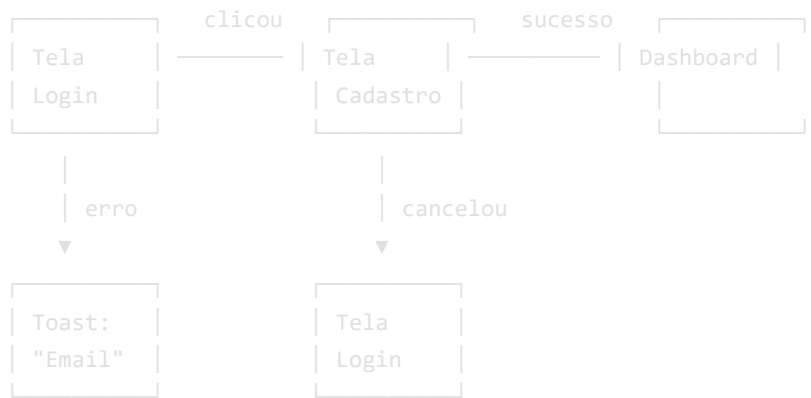
Exemplo de Grey Box:



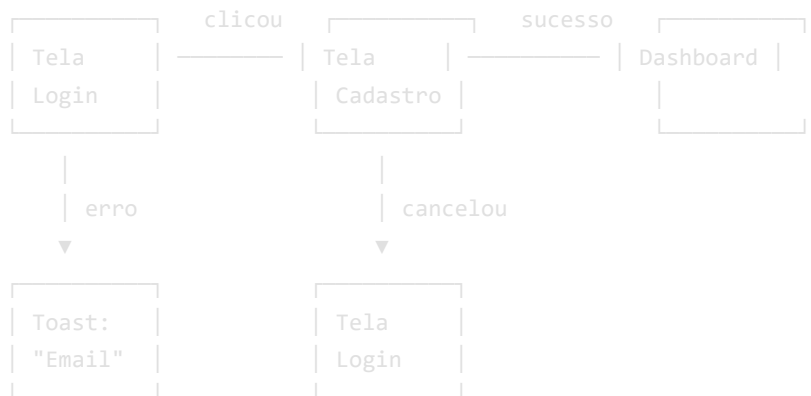
Wireflow

Combinação de **wireframe + fluxograma**. Cada tela é conectada por setas que mostram a navegação.

Wireflow de cadastro:



Wireflow de cadastro:



```
interface Wireflow {
  screens: WireframeSpec[];
  transitions: Transition[];
}

interface Transition {
  from: string;          // id da tela de origem
  to: string;            // id da tela de destino
  trigger: 'click' | 'submit' | 'error' | 'success' | 'timeout';
  element?: string;      // elemento que dispara (ex: "btn-login")
  condition?: string;    // condição (ex: "campos válidos")
}
```

Sketching (Desenho à mão)

Desenhar à mão força o **foco na estrutura** e elimina a tentação de polir visualmente. Além disso:

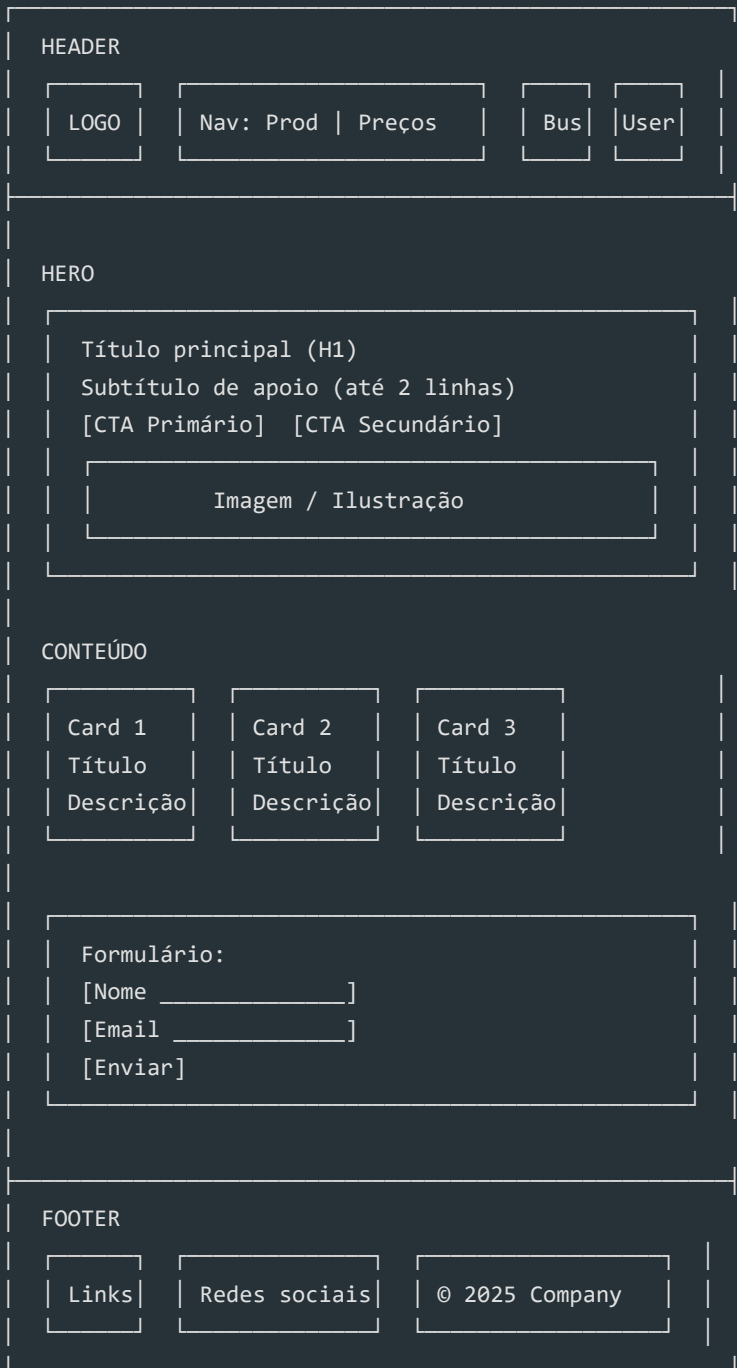
Vantagens do sketching:

- Velocidade: 10x mais rápido que digital
- Baixo comprometimento: fácil descartar e recomeçar
- Qualquer um participa: não precisa saber ferramenta
- Memorável: estudos mostram que esboços manuais geram mais feedback honesto que protótipos polidos

6. Anatomia de uma Tela

Toda tela segue uma estrutura comum. Conhecer a anatomia ajuda a criar wireframes consistentes.

Anatomia padrão de uma página:



Detalhamento dos elementos

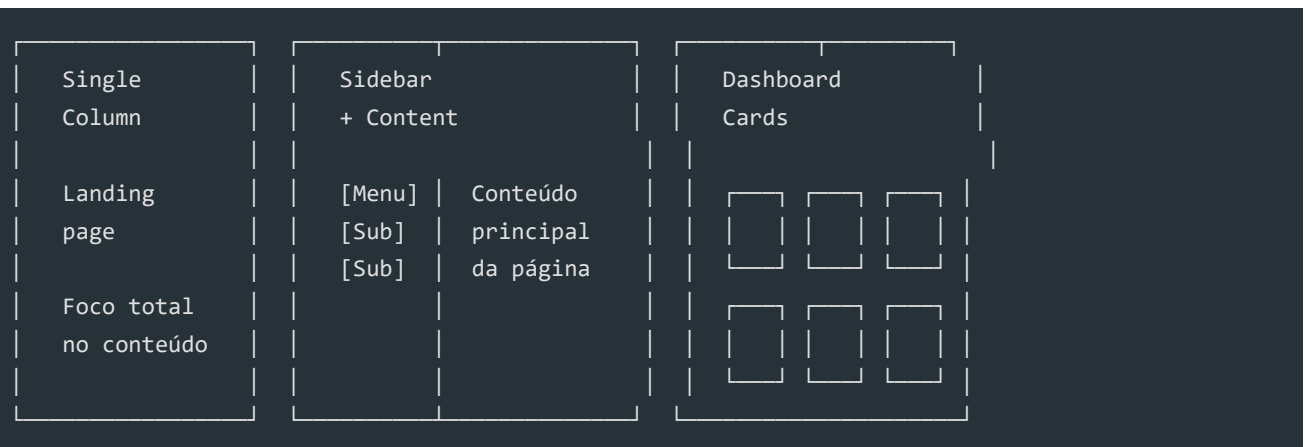
Elemento	Função	Wireframe
----------	--------	-----------

Header	Identidade + navegação gl	Box superior com logo à e
Hero	Primeira impressão, valor	Box grande com título, su
Conteúdo	Informação principal	Grid de cards, listas, fo
CTA	Ação desejada (primário/s	Botão destacado (maior, p
Navegação	Explorar o produto	Menu horizontal, sidebar,
Footer	Links secundários, legais	Box inferior com informaç

```
// Definição programática de uma tela
interface ScreenAnatomy {
  header: {
    logo: string;
    navItems: NavItem[];
    ctaButton?: { label: string; action: string };
  };
  hero: {
    headline: string;
    subheadline?: string;
    primaryCta: { label: string; action: string };
    secondaryCta?: { label: string; action: string };
    mediaType: 'image' | 'video' | 'illustration';
  };
  content: ContentSection[];
  footer: {
    links: { label: string; url: string }[];
    legal: string;
  };
}

interface NavItem {
  label: string;
  active: boolean;
  children?: NavItem[];
}
```

Padrões de layout comuns



7. Interação — Estados e Transições

Wireframes de média/alta fidelidade devem representar **estados da interface**.

Estados de Componentes

Cada componente pode estar em um dos seguintes estados:

```
type ComponentState = 'loading' | 'empty' | 'error' | 'success' | 'disabled' | 'default' | 'hover' | 'active';

interface StatefulComponent {
  name: string;
  states: Record<ComponentState, WireframeElement>;
}
```

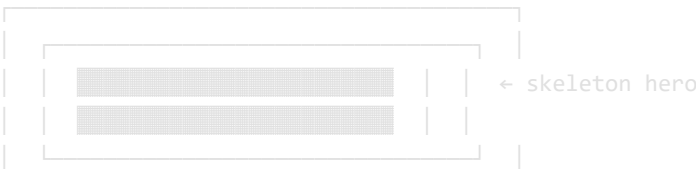
Como representar cada estado no wireframe

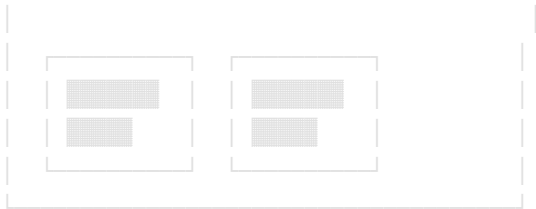
Estado	Representação visual	O que mostra
Default	Elemento normal, tons de	Estado neutro
Loading	Skeleton boxes (retângulo	Dados estão carregando
Empty	Box vazio com texto "Nenh	Não há dados
Error	Box com borda tracejada/v	Algo deu errado
Success	Confirmação visual (check	Ação concluída
Disabled	Elemento opaco, sem contr	Ação indisponível

Skeleton Loading

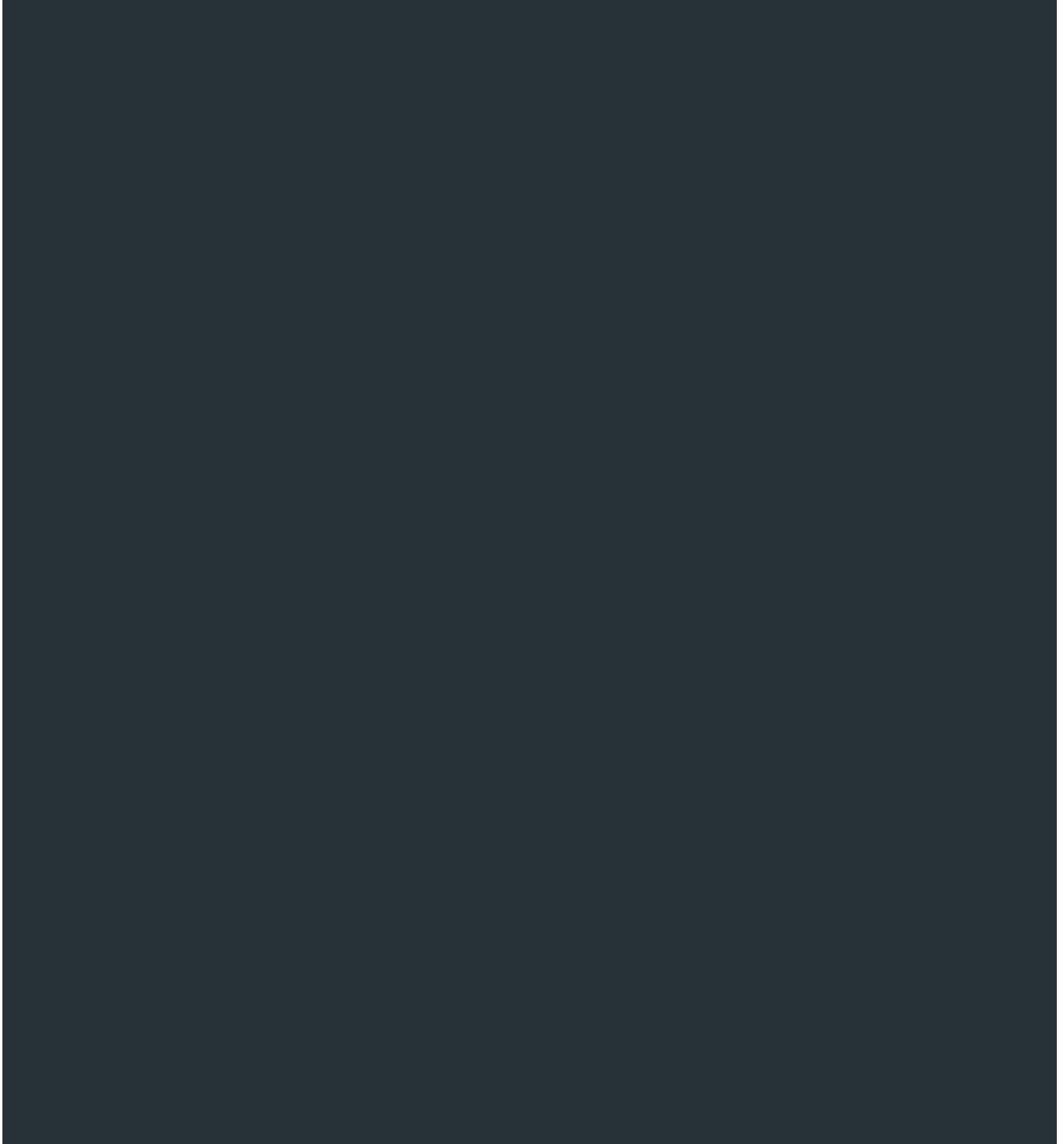
```
// Componente de skeleton para wireframe
interface SkeletonBox {
  width: number | string; // '100%' | '300px'
  height: number | string;
  borderRadius: number;
  lines?: number; // para texto simulado
}
```

Wireframe de estado loading (skeleton):

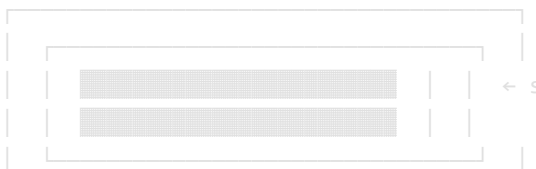




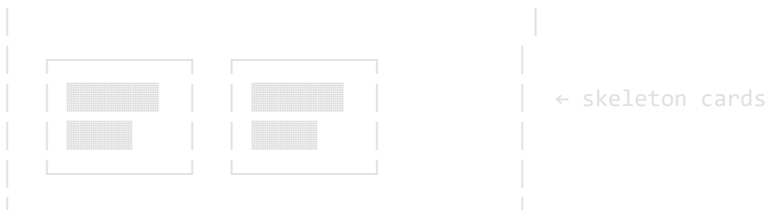
← skeleton cards



Wireframe de estado loading (skeleton):

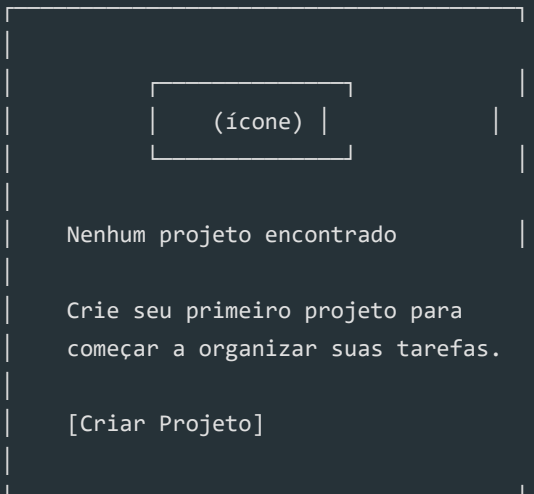


← skeleton hero



Estado Empty

Wireframe de estado vazio (empty):

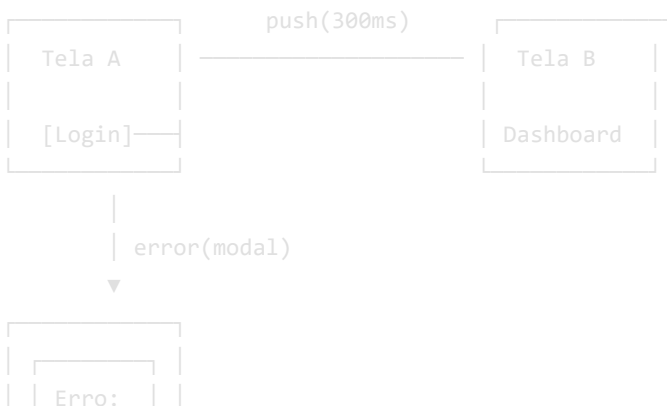


Transições

Wireframes podem (e devem) indicar transições entre telas:

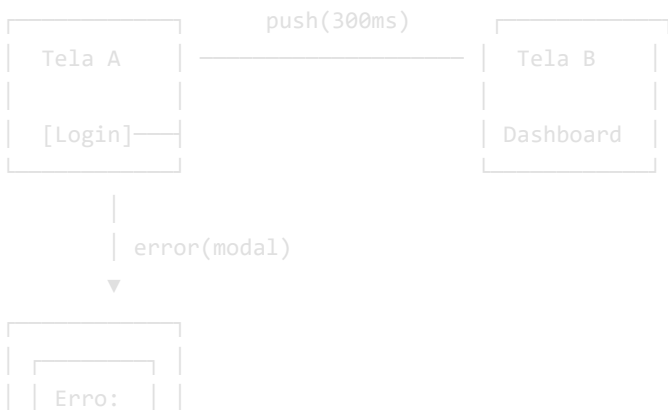
```
interface Transition {  
  type: 'push' | 'modal' | 'toast' | 'slide' | 'fade';  
  duration: number; // ms  
  trigger: string; // ação do usuário  
}
```

Representação de transição no wireframe:





Representação de transição no wireframe:



8. Validação de Wireframes

Wireframe não é o destino — é um **meio para validar**.

Clickable Wireframes

Transforme wireframes estáticos em protótipos clicáveis para testar fluxo:

```
interface ClickableWireframe {
  screens: WireframeSpec[];
  hotspots: Hotspot[];
}

interface Hotspot {
  screenId: string;
  area: { x: number; y: number; w: number; h: number };
  action: 'navigate' | 'showModal' | 'showToast';
  target: string; // id da tela ou ação
}
```

Ferramentas para criar clickable wireframes: - **Figma** — Prototyping mode (conecta frames com setas) - **Balsamiq** — Link entre telas - **Excalidraw** — Setas + links - **Miro** — Conecta boards com setas

Teste de wireframe com usuários

Roteiro de teste de wireframe:

1. Contexto

"Aqui está um esboço de uma tela. Não está finalizado.
Queremos entender se o fluxo faz sentido."

2. Tarefa

"Mostre como você faria para criar um novo projeto."

3. Observe

- Onde o usuário clica primeiro?
- Ele hesita em algum lugar?
- Ele encontra o CTA?

4. Pergunte

- "O que você acha que esse elemento faz?"
- "O que você esperaria ao clicar aqui?"
- "Faltou alguma informação?"

Iteração

O ciclo ideal: ``typescript Esboçar Validar Aprender Refinar (repetir) ``

```
interface IterationCycle {
  version: number;
  changes: string[];
  validatedBy: string[];
  feedbackThemes: string[];
  nextSteps: string[];
}

const cycle: IterationCycle = {
  version: 3,
  changes: [
    'Moveu CTA para o centro da tela',
    'Adicionou breadcrumb na navegação',
    'Reduziu formulário de 8 para 4 campos',
  ],
  validatedBy: ['PO', '2 usuários'],
  feedbackThemes: [
    'CTA não era visível',
    'Formulário muito longo',
    'Falta indicador de volta',
  ],
  nextSteps: ['Testar versão 4 com 5 usuários'],
};
```

9. Wireframes em Enterprise

Em contexto enterprise, wireframes vão além do esboço — eles se tornam **documentos de especificação**.

Documentação

Cada wireframe deve incluir metadados:

```
interface WireframeDocumentation {
  // Metadados
  id: string;           // "WF-001"
  title: string;        // "Tela de Login"
  module: string;       // "Autenticação"
  version: string;      // "1.3"
  author: string;       // "Maria Silva"
  createdAt: string;    // ISO date
  updatedAt: string;    // ISO date

  // Rastreabilidade
  userStoryId: string;  // "US-042"
  epicId: string;      // "EPIC-07"

  // Conteúdo
  elements: WireframeElement[];
  states: StatefulComponent[];
  interactions: Interaction[];
  notes: string;        // Observações para dev

  // Aprovações
  approvals: Approval[];
}

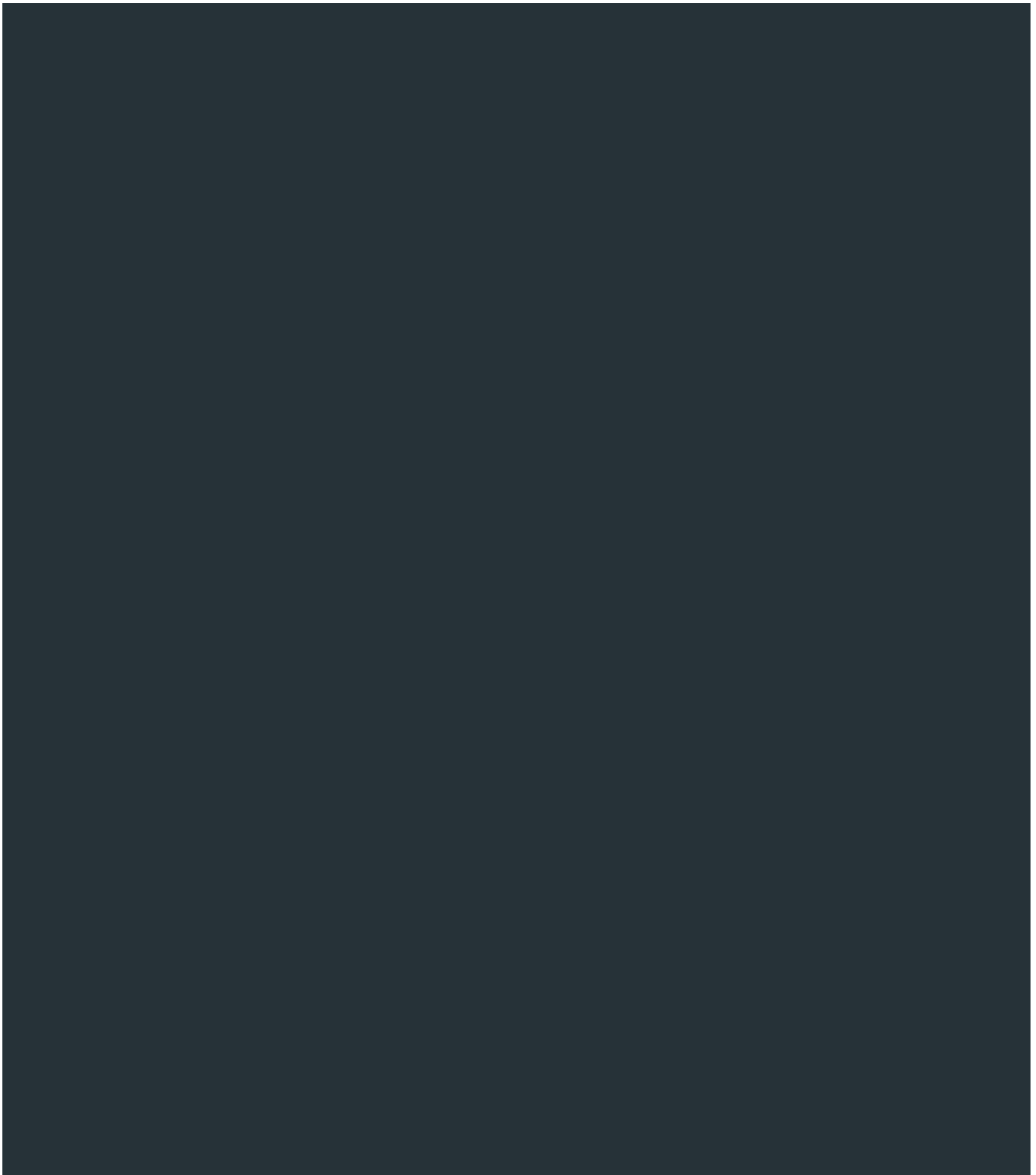
interface Approval {
  role: 'PO' | 'DesignLead' | 'TechLead';
  name: string;
  date: string;
  status: 'pending' | 'approved' | 'changes_requested';
  comments?: string;
}
```

Handoff para Devs

O handoff é o momento em que o wireframe vira código. Para que seja eficiente:

Checklist de handoff (wireframe → dev):

- [] Grid definido (colunas, gutters, margens)
- [] Medidas exatas dos elementos (largura, altura)
- [] Espaçamentos documentados (padding, margin)
- [] Estados mapeados (loading, empty, error, success)
- [] Comportamento em breakpoints (responsivo)
- [] Fluxo de navegação diagramado (wireflow)
- [] Anotações em elementos não óbvios
- [] Nomes de componentes consistentes com o design system



Checklist de handoff (wireframe → dev):

- [] Grid definido (colunas, gutters, margens)
- [] Medidas exatas dos elementos (largura, altura)
- [] Espaçamentos documentados (padding, margin)
- [] Estados mapeados (loading, empty, error, success)
- [] Comportamento em breakpoints (responsivo)
- [] Fluxo de navegação diagramado (wireflow)
- [] Anotações em elementos não óbvios
- [] Nomes de componentes consistentes com o design system

```
// Especificação de handoff
interface HandoffSpec {
  screen: WireframeDocumentation;
  responsiveBreakpoints: {
    mobile: WireframeSpec;
    tablet: WireframeSpec;
    desktop: WireframeSpec;
  };
  componentReferences: {
    [componentName: string]: string; // "Button" → "ds-button"
  };
  apiContracts: {
    endpoint: string;
    method: string;
    requestFields: string[];
    responseFields: string[];
  }[];
}
```

Versionamento

Wireframes em enterprise precisam de versionamento igual a código.

Estratégia de versionamento:

Figma:

- Branching: crie branches para variações
- Version history: Figma salva automaticamente
- Nomeie versões: "v1.0 - Aprovado PO", "v1.1 - Revisão dev"

Arquivos:

```
wireframe-tela-login-v1.0.fig
wireframe-tela-login-v1.1.fig
wireframe-tela-login-v2.0.fig
```

Convenção: {componente}-{tela}-v{major}.{minor}.{tipo}

Boas práticas:

- |— Congele versões aprovadas (não mexa)
- |— Versão major = mudança estrutural
- |— Versão minor = ajuste de layout
- |— Mantenha changelog por versão



Estratégia de versionamento:

Figma:

- Branching: crie branches para variações
- Version history: Figma salva automaticamente
- Nomeie versões: "v1.0 - Aprovado PO", "v1.1 - Revisão dev"

Arquivos:

wireframe-tela-login-v1.0.fig
wireframe-tela-login-v1.1.fig
wireframe-tela-login-v2.0.fig

Convenção: {componente}-{tela}-v{major}.{minor}.{tipo}

Boas práticas:

- └─ Congele versões aprovadas (não mexa)
- └─ Versão major = mudança estrutural
- └─ Versão minor = ajuste de layout
- └─ Mantenha changelog por versão

Exemplo de Changelog de Wireframe

```
interface WireframeChangelog {  
  version: string;  
  date: string;  
  author: string;  
  changes: string[];  
  reason: string;  
}
```

```
const changelog: WireframeChangelog[] = [  
  {  
    version: '1.0',  
    date: '2025-06-01',  
    author: 'Maria Silva',  
    changes: ['Criação inicial do wireframe'],  
    reason: 'Primeira versão para alinhamento com PO',  
  },  
  {  
    version: '1.1',  
    date: '2025-06-03',  
    author: 'Maria Silva',  
    changes: [  
      'Adicionado campo de busca no header',  
      'Reduzido hero de 800px para 600px',  
    ],  
    reason: 'Feedback do PO: buscar é prioridade',  
  },  
  {  
    version: '2.0',  
    date: '2025-06-10',  
    author: 'Carlos Lima',  
    changes: [  
      'Reestruturado layout para sidebar + content',  
      'Adicionada seção de relatórios',  
      'Removido carrossel do hero',  
    ],  
    reason: 'Mudança de escopo: nova feature de relatórios',  
  },  
];
```

```
# UI Design
```

```
# Módulo 06 – UI Design
```

```
**Transformar wireframes em interfaces funcionais, acessíveis e visualmente consistentes.**
```

```
---
```

```
interface WireframeChangelog {  
  version: string;  
  date: string;  
  author: string;  
  changes: string[];  
  reason: string;  
}
```

```
const changelog: WireframeChangelog[] = [  
  {  
    version: '1.0',  
    date: '2025-06-01',  
    author: 'Maria Silva',  
    changes: ['Criação inicial do wireframe'],  
    reason: 'Primeira versão para alinhamento com PO',  
  },  
  {  
    version: '1.1',  
    date: '2025-06-03',  
    author: 'Maria Silva',  
    changes: [  
      'Adicionado campo de busca no header',  
      'Reduzido hero de 800px para 600px',  
    ],  
    reason: 'Feedback do PO: buscar é prioridade',  
  },  
  {  
    version: '2.0',  
    date: '2025-06-10',  
    author: 'Carlos Lima',  
    changes: [  
      'Reestruturado layout para sidebar + content',  
      'Adicionada seção de relatórios',  
      'Removido carrossel do hero',  
    ],  
    reason: 'Mudança de escopo: nova feature de relatórios',  
  },  
];
```

```
# UI Design
```

```
# Módulo 06 – UI Design
```

```
**Transformar wireframes em interfaces funcionais, acessíveis e visualmente consistentes.**
```

```
---
```

```
## 1. O que é UI Design
```

UI (User Interface) Design é a disciplina responsável pela **aparência e comportamento visual** de um produto digital. Enquanto UX define a estrutura e a experiência, UI traduz essa estrutura em pixels – cores, tipografia, espaçamento, componentes e animações.

```
### UI ≠ UX
```

UI (User Interface)	UX (User Experience)
_____	_____
	Aparência visual
tipografia, ícones	Pesquisa, jornada, AI Layout e componentes
Microinterações	Emoção e usabilidade "Como vê?"
	Experiência completa Cores, Fluxo e arquitetura "Como se sente?" ``

UI é a **camada de superfície** dos 5 Planos de Garrett. É o que o usuário vê e com o que interage diretamente.

A relação UI + UX

```
UX sem UI: ideia sem forma, impossível de usar
UI sem UX: bonito mas inútil, frustrante
UI + UX: útil, usável e desejável
```

O papel do dev é implementar a UI com **fidelidade ao design**, respeitando decisões de UX embutidas nos componentes.

2. Princípios de Design Visual — CRAP

CRAP é um acrônimo para os 4 princípios fundamentais do design visual.

Contraste

Elementos diferentes devem **parecer diferentes**. Use contraste para destacar o que é importante.

```
// Botão primário vs secundário quase iguais
const bad = { primary: '#4A90D9', secondary: '#5BA0E9' };

// Contraste claro entre ações
const good = { primary: '#1A73E8', secondary: '#E8F0FE' };
```

- Contraste de **cor**: fundo vs texto, botão primário vs secundário - Contraste de **tamanho**: título vs corpo - Contraste de **peso**: bold vs regular - Contraste de **forma**: botão preenchido vs outline

Repetição

Repita estilos visuais para criar consistência.

```
Mesma cor de link      → usuário reconhece que é clicável
Mesmo padding nos cards → ritmo visual previsível
Mesmo border-radius     → identidade visual consistente
```

Alinhamento

Nada deve ser colocado arbitrariamente. Cada elemento tem conexão visual com outro.

```
// Alinhamento solto
const misaligned = {
  title: { marginLeft: 16 },
  body: { marginLeft: 20 },
  action: { marginLeft: 12 },
};

// Alinhamento consistente
const aligned = {
  title: { marginLeft: 16 },
  body: { marginLeft: 16 },
  action: { marginLeft: 16 },
};
```

```
};
```

```
// Alinhamento solto
const misaligned = {
  title: { marginLeft: 16 },
  body: { marginLeft: 20 },
  action: { marginLeft: 12 },
};
```

```
// Alinhamento consistente
const aligned = {
  title: { marginLeft: 16 },
  body: { marginLeft: 16 },
  action: { marginLeft: 16 },
};
```

```
};
```

Proximidade

Itens relacionados devem ficar ****próximos**** visualmente. Itens não relacionados, separados.

Agrupamento confuso:

```
[Nome] [Email] [Telefone]  
[Senha] [Confirmar Senha]
```

Agrupamento lógico por proximidade:

```
[Nome] [Email]           ← Dados pessoais  
[Telefone]              ← Contato  
[Senha] [Confirmar]     ← Segurança
```

3. Cor

Teoria das Cores

Cores comunicam significado e emoção. No contexto enterprise, a escolha deve ser funcional, não apenas estética.

Cor	Significado comum	Uso em UI enterprise
Azul	Confiança, segurança	Primary action, links
Verde	Sucesso, conclusão	Confirmação, status OK
Vermelho	Erro, perigo	Erro, ação destrutiva
Amarelo	Atenção, aviso	Warning, alerta
Cinza	Neutro, secundário	Background, disabled

Paletas de cor no Design System

```
interface ColorPalette {
  primary: string;
  primaryHover: string;
  primaryLight: string;
  secondary: string;
  success: string;
  warning: string;
  error: string;
  info: string;
  neutral: {
    50: string; // background mais claro
    100: string;
    200: string;
    300: string;
    400: string;
    500: string; // cor neutra base
    600: string;
    700: string;
    800: string;
    900: string; // texto mais escuro
  };
}

const enterprisePalette: ColorPalette = {
  primary: '#1A73E8',
  primaryHover: '#1557B0',
  primaryLight: '#E8F0FE',
  secondary: '#5F6368',
```

```
success: '#34A853',
warning: '#FBBC04',
error: '#EA4335',
info: '#4285F4',
neutral: {
  50: '#F8F9FA',
  100: '#F1F3F4',
  200: '#E8EAED',
  300: '#DADCE0',
  400: '#BDC1C6',
  500: '#9AA0A6',
  600: '#80868B',
  700: '#5F6368',
  800: '#3C4043',
  900: '#202124',
},
};
```

```
interface ColorPalette {
  primary: string;
  primaryHover: string;
  primaryLight: string;
  secondary: string;
  success: string;
  warning: string;
  error: string;
  info: string;
  neutral: {
    50: string; // background mais claro
    100: string;
    200: string;
    300: string;
    400: string;
    500: string; // cor neutra base
    600: string;
    700: string;
    800: string;
    900: string; // texto mais escuro
  };
}
```

```
const enterprisePalette: ColorPalette = {
  primary: '#1A73E8',
  primaryHover: '#1557B0',
  primaryLight: '#E8F0FE',
  secondary: '#5F6368',
```



```

success: '#34A853',
warning: '#FBBC04',
error: '#EA4335',
info: '#4285F4',
neutral: {
  50: '#F8F9FA',
  100: '#F1F3F4',
  200: '#E8EAED',
  300: '#DADCE0',
  400: '#BDC1C6',
  500: '#9AA0A6',
  600: '#80868B',
  700: '#5F6368',
  800: '#3C4043',
  900: '#202124',
},
};

```

Acessibilidade e Contraste

WCAG 2.2 define ratios mínimos de contraste:

Tipo	Ratio mínimo (AA)	Ratio mínimo (AAA)
Texto normal (< 18px)	4.5:1	7:1
Texto grande (>= 18px bol	3:1	4.5:1
Componentes ativos (ícone	3:1	—

```

// Utilitário de contraste
function isAccessible(foreground: string, background: string, level: 'AA' | 'AAA' = 'AA'):
boolean {
  const ratio = getContrastRatio(foreground, background); // implementação do módulo 04
  const threshold = level === 'AA' ? 4.5 : 7;
  return ratio >= threshold;
}

// Uso na validação do tema
const theme = {
  textPrimary: '#202124',
  bgPrimary: '#FFFFFF',
};

console.log(isAccessible(theme.textPrimary, theme.bgPrimary)); // true

```

Cor e Branding

No contexto enterprise, a paleta primária reflete a marca, mas a paleta **funcional** (success, error, warning, info) deve seguir padrões culturais. Nunca use verde para erros ou vermelho para

sucesso.

```
interface BrandTokens {
  colors: ColorPalette;
  usage: {
    primaryAction: string;
    dangerAction: string;
    link: string;
    border: string;
    surface: {
      page: string;
      card: string;
      modal: string;
      sidebar: string;
    };
    text: {
      primary: string;
      secondary: string;
      disabled: string;
      inverse: string;
    };
  };
};
```

4. Tipografia

Hierarquia Tipográfica

A hierarquia guia o olho do usuário: o que é mais importante deve ser visualmente mais proeminente.

```
interface TypographyScale {  
  display: { size: number; lineHeight: number; weight: number };  
  heading1: { size: number; lineHeight: number; weight: number };  
  heading2: { size: number; lineHeight: number; weight: number };  
  heading3: { size: number; lineHeight: number; weight: number };  
  body: { size: number; lineHeight: number; weight: number };  
  bodySmall: { size: number; lineHeight: number; weight: number };  
  caption: { size: number; lineHeight: number; weight: number };  
}  
  
const enterpriseTypography: TypographyScale = {  
  display: { size: 32, lineHeight: 40, weight: 700 },  
  heading1: { size: 24, lineHeight: 32, weight: 700 },  
  heading2: { size: 20, lineHeight: 28, weight: 600 },  
  heading3: { size: 16, lineHeight: 24, weight: 600 },  
  body: { size: 14, lineHeight: 20, weight: 400 },  
  bodySmall: { size: 12, lineHeight: 16, weight: 400 },  
  caption: { size: 11, lineHeight: 16, weight: 400 },  
};
```

Escalas Modulares

Use uma escala modular (ex: 1.25 ou 1.333) para garantir proporções harmônicas.

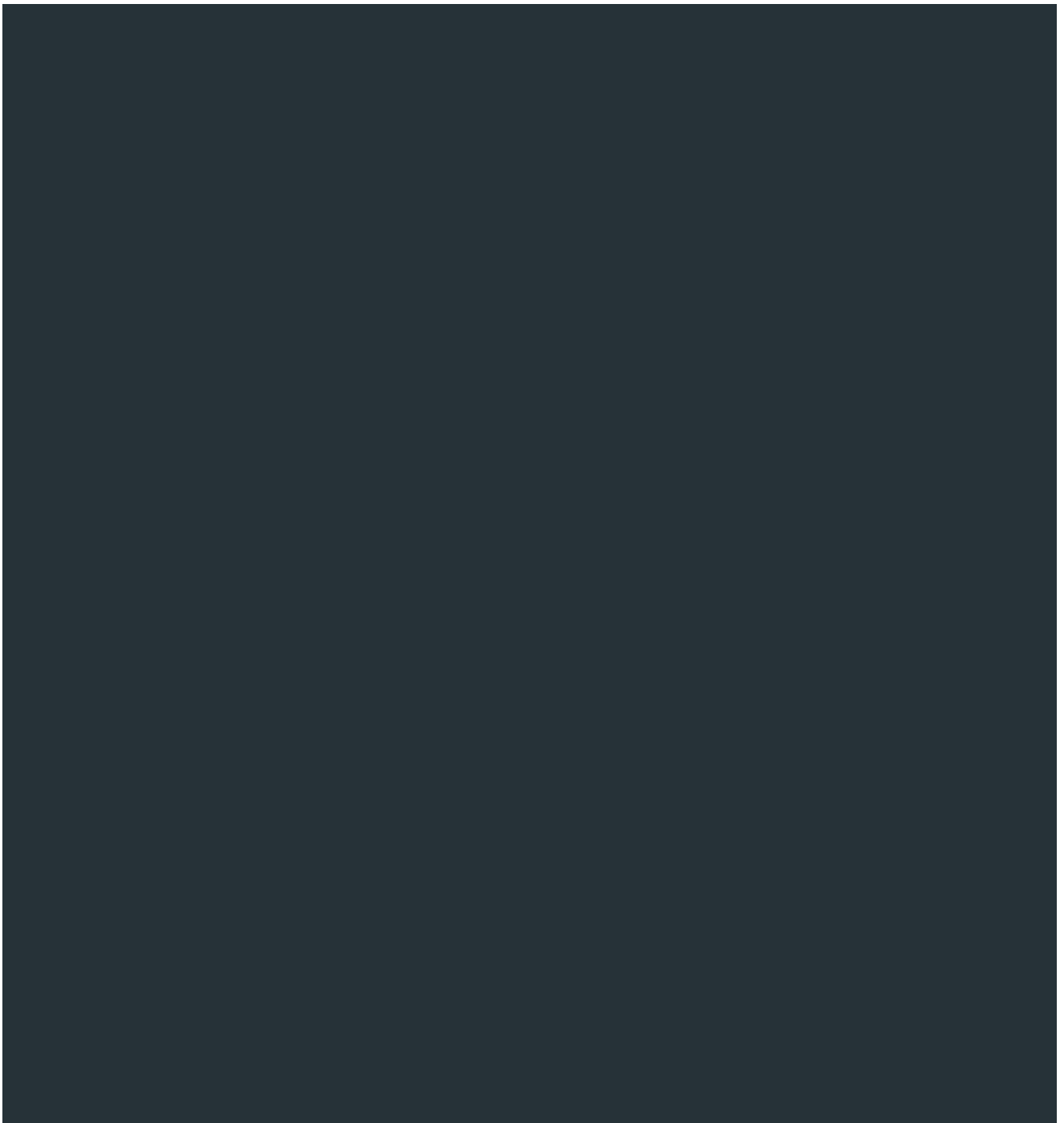
```
Escala 1.25 (Major Second):  
11 → 14 → 16 → 20 → 24 → 32 → 40  
  
Escala 1.333 (Major Third):  
12 → 16 → 20 → 24 → 32 → 40 → 48
```

Legibilidade

- **Largura de linha**: 45–75 caracteres por linha (ideal ~66) - **Line-height**: 1.4–1.6 para body text - **Font-weight**: 400 para body, 600+ para headings - **Font-family**: system-ui ou fonte carregada via CDN

```
// Stack de fontes para enterprise
const fontStack = {
  sans: `'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif`,
  mono: `'JetBrains Mono', 'Fira Code', 'Cascadia Code', Consolas, monospace`,
};

// Uso no CSS-in-JS
const bodyStyle = {
  fontFamily: fontStack.sans,
  fontSize: 14,
  lineHeight: 1.5,
  maxWidth: '66ch', // controle de largura
};
```



5. Espaçamento e Grid

Box Model e Spacing Consistente

Use uma escala de espaçamento, não valores arbitrários.

```
const spacing = {  
  xxs: 2,    // 2px - divider finíssimo  
  xs:  4,    // 4px - ícone pequeno, gap mínimo  
  sm:  8,    // 8px - padding interno de inputs  
  md:  12,   // 12px - gap entre label e input  
  lg:  16,   // 16px - padding de cards, seções  
  xl:  24,   // 24px - margem entre seções  
  xxl: 32,   // 32px - padding de página  
  xxxl: 48,  // 48px - seções grandes no layout  
};
```

```
// Espaçamento arbitrário  
const badCard = { padding: 17, gap: 11 };  
  
// Espaçamento da escala  
const goodCard = { padding: spacing.lg, gap: spacing.md };
```

Grid Systems

Grid	Colunas	Gutters	Uso
Mobile	4	16px	Telas < 768px
Tablet	8	24px	768px–1024px
Desktop	12	24px	> 1024px

```
interface GridConfig {  
  columns: number;  
  gutter: number;  
  margin: number;  
  maxWidth: number;  
}  
  
const grid: Record<string, GridConfig> = {  
  mobile: { columns: 4, gutter: 16, margin: 16, maxWidth: 480 },  
  tablet: { columns: 8, gutter: 24, margin: 24, maxWidth: 960 },  
  desktop: { columns: 12, gutter: 24, margin: 0, maxWidth: 1200 },  
};  
  
// Utility: cálculo de coluna  
function colWidth(columns: number, totalColumns: number, gutter: number): string {
```

```
const fraction = columns / totalColumns;
return `calc(${fraction * 100}% - ${gutter}px)`;
}
```

```
interface GridConfig {
  columns: number;
  gutter: number;
  margin: number;
  maxWidth: number;
}

const grid: Record<string, GridConfig> = {
  mobile: { columns: 4, gutter: 16, margin: 16, maxWidth: 480 },
  tablet: { columns: 8, gutter: 24, margin: 24, maxWidth: 960 },
  desktop: { columns: 12, gutter: 24, margin: 0, maxWidth: 1200 },
};

// Utility: cálculo de columna
function colWidth(columns: number, totalColumns: number, gutter: number): string {
```

```
const fraction = columns / totalColumns;  
return `calc(${fraction * 100}% - ${gutter}px)`;  
}
```

Layout Patterns

Dashboard:

Sidebar	Main	Panel
240px	1fr	320px

Single Column (formulários):

Content (max 720px)

Split Screen:

List (1fr)	Detail (1fr)
------------	--------------

6. Componentes de UI

Botões

```
type ButtonVariant = 'primary' | 'secondary' | 'outline' | 'ghost' | 'danger';
type ButtonSize = 'sm' | 'md' | 'lg';

interface ButtonProps {
  variant: ButtonVariant;
  size: ButtonSize;
  disabled?: boolean;
  loading?: boolean;
  icon?: React.ReactNode;
  children: React.ReactNode;
}

// Mapa de cores por variante
const buttonStyles: Record<ButtonVariant, { bg: string; color: string; border: string }> = {
  primary: { bg: '#1A73E8', color: 'FFFFFF', border: 'transparent' },
  secondary: { bg: '#E8F0FE', color: '#1A73E8', border: 'transparent' },
  outline: { bg: 'transparent', color: '#1A73E8', border: '#1A73E8' },
  ghost: { bg: 'transparent', color: '#5F6368', border: 'transparent' },
  danger: { bg: '#EA4335', color: 'FFFFFF', border: 'transparent' },
};
```

Inputs e Formulários

```
interface InputProps {
  label: string;
  placeholder?: string;
  error?: string;
  hint?: string;
  disabled?: boolean;
  required?: boolean;
  leftIcon?: React.ReactNode;
}

// Estados do input
const inputStates = {
  default: { border: '#DADCE0', bg: 'FFFFFF' },
  hover: { border: '#9AA0A6', bg: 'FFFFFF' },
  focus: { border: '#1A73E8', bg: 'FFFFFF', boxShadow: '0 0 0 3px #E8F0FE' },
  error: { border: '#EA4335', bg: 'FFFFFF' },
  disabled: { border: '#E8EAED', bg: 'F1F3F4', color: '#9AA0A6' },
};
```

Cards

```
interface CardProps {
  padding?: keyof typeof spacing;
  variant?: 'default' | 'elevated' | 'outlined';
  onClick?: () => void;
}

const cardVariants = {
  default: { bg: 'FFFFFF', border: '1px solid #E8EAED', boxShadow: 'none' },
  elevated: { bg: 'FFFFFF', border: 'none', boxShadow: '0 1px 3px rgba(0,0,0,0.1), 0 1px 2px rgba(0,0,0,0.06)' },
  outlined: { bg: 'FFFFFF', border: '1px solid #DADCE0', boxShadow: 'none' },
};
```

Modais

```
interface ModalProps {
  isOpen: boolean;
  onClose: () => void;
  title: string;
  size: 'sm' | 'md' | 'lg';
  children: React.ReactNode;
  footer?: React.ReactNode;
}

const modalSizes = {
  sm: { width: 400 },
  md: { width: 560 },
  lg: { width: 720 },
};

// Overlay deve fechar o modal
const overlayStyles = {
  position: 'fixed',
  inset: 0,
  bg: 'rgba(0, 0, 0, 0.5)',
  display: 'flex',
  alignItems: 'center',
  justifyContent: 'center',
  zIndex: 1000,
};
```

Tabelas

```
interface Column<T> {
  key: keyof T;
```

```
header: string;
width?: number;
align?: 'left' | 'center' | 'right';
render?: (value: T[keyof T], row: T) => React.ReactNode;
sortable?: boolean;
}
```

```
interface TableProps<T> {
  columns: Column<T>[];
  data: T[];
  loading?: boolean;
  emptyMessage?: string;
  onClick?: (row: T) => void;
}
```

```
interface Column<T> {
  key: keyof T;
```

```
header: string;
width?: number;
align?: 'left' | 'center' | 'right';
render?: (value: T[keyof T], row: T) => React.ReactNode;
sortable?: boolean;
}
```

```
interface TableProps<T> {
  columns: Column<T>[];
  data: T[];
  loading?: boolean;
  emptyMessage?: string;
  onClick?: (row: T) => void;
}
```

Dropdowns e Selects

```
interface Option {
  value: string;
  label: string;
  disabled?: boolean;
  group?: string;
}

interface SelectProps {
  options: Option[];
  value: string;
  onChange: (value: string) => void;
  placeholder?: string;
  searchable?: boolean;
  clearable?: boolean;
}
```

7. Design Patterns

Navegação

Tipo	Quando usar	Exemplo
Sidebar	Muitas seções, hierarquia	Dashboards
Top nav	Poucas seções, conteúdo h	Sites institucionais
Tabs	Seções relacionadas, mesm	Detalhes de registro
Breadcrumb	Hierarquia profunda, orie	E-commerce, docs
Stepper	Fluxo sequencial	Wizard, checkout

```
interface BreadcrumbItem {
  label: string;
  href?: string;
  isCurrent?: boolean;
}

function Breadcrumb({ items }: { items: BreadcrumbItem[] }) {
  return (
    <nav aria-label="Breadcrumb">
      {items.map((item, i) => (
        <span key={i}>
          {i > 0 && <ChevronRightIcon />}
          {item.isCurrent
            ? <span aria-current="page">{item.label}</span>
            : <a href={item.href}>{item.label}</a>
          }
        </span>
      ))}
    </nav>
  );
}
```

Formulários

Patterns que melhoram a experiência de formulários:

```
// Label visível e associado
<label htmlFor="email">Email</label>
<input id="email" type="email" aria-describedby="email-hint" />
<span id="email-hint">Usaremos seu email para login</span>

// Validação inline
function validateField(value: string): string | null {
```

```
if (!value) return 'Campo obrigatório';
if (value.length < 3) return 'Mínimo de 3 caracteres';
return null;
}
```

```
// Máscara de formatação
function formatPhone(value: string): string {
  const digits = value.replace(/\D/g, '').slice(0, 11);
  if (digits.length <= 2) return `(${digits}`;
  if (digits.length <= 7) return `(${digits.slice(0, 2)}) ${digits.slice(2)}`;
  return `(${digits.slice(0, 2)}) ${digits.slice(2, 7)}-${digits.slice(7)}`;
}
```

```
// Label visível e associado
<label htmlFor="email">Email</label>
<input id="email" type="email" aria-describedby="email-hint" />
<span id="email-hint">Usaremos seu email para login</span>

// Validação inline
function validateField(value: string): string | null {
```

```

    if (!value) return 'Campo obrigatório';
    if (value.length < 3) return 'Mínimo de 3 caracteres';
    return null;
}

// Máscara de formatação
function formatPhone(value: string): string {
    const digits = value.replace(/\D/g, '').slice(0, 11);
    if (digits.length <= 2) return `(${digits}`;
    if (digits.length <= 7) return `(${digits.slice(0, 2)}) ${digits.slice(2)}`;
    return `(${digits.slice(0, 2)}) ${digits.slice(2, 7)}-${digits.slice(7)}`;
}

```

Feedback

Situação	Componente	Exemplo
Sucesso	Toast/snackbar	"Projeto salvo"
Erro	Alert + inline error	"Email inválido"
Carregando	Skeleton / Spinner	Skeleton de 3 linhas
Vazio	Empty state	"Nenhum projeto ainda"
Confirmação	Modal/dialog	"Tem certeza?"

Onboarding

```

interface OnboardingStep {
    target: string;          // seletor do elemento alvo
    title: string;
    content: string;
    position: 'top' | 'bottom' | 'left' | 'right';
}

const onboarding: OnboardingStep[] = [
    {
        target: '#create-project-btn',
        title: 'Criar projeto',
        content: 'Clique aqui para criar seu primeiro projeto',
        position: 'bottom',
    },
    {
        target: '#sidebar',
        title: 'Navegação',
        content: 'Use o menu lateral para acessar todas as seções',
        position: 'right',
    },
];

```

Empty States

Nunca mostre uma tela em branco. Todo empty state deve ter:

1. **Ilustração** ou ícone representativo
2. **Título** claro do que falta
3. **Descrição** do que fazer
4. **CTA** para a ação principal

```
const emptyState = {  
  title: 'Nenhum projeto encontrado',  
  description: 'Crie seu primeiro projeto para começar a organizar suas tarefas.',  
  action: { label: 'Criar projeto', href: '/projects/new' },  
};
```


8. Microinterações

Microinterações são **pequenos momentos de feedback** que comunicam o resultado de uma ação.

Estados Visuais

```
/* Botão – todos os estados */
.button {
  background: #1A73E8;
  transition: background 0.15s ease, box-shadow 0.15s ease;
  cursor: pointer;
}

.button:hover {
  background: #1557B0; /* escurece 10-15% */
}

.button:active {
  background: #0D47A1; /* escurece mais */
  transform: scale(0.98);
}

.button:focus-visible {
  outline: none;
  box-shadow: 0 0 0 3px #E8F0FE;
}

.button:disabled {
  background: #E8EAED;
  color: #9AA0A6;
  cursor: not-allowed;
  transform: none;
}
```

Animações

```
interface AnimationTokens {
  duration: {
    instant: '50ms';
    fast:    '150ms';
    normal:  '200ms';
    slow:    '300ms';
  };
  easing: {
```

```

    default: 'ease';
    enter: 'cubic-bezier(0.0, 0.0, 0.2, 1)';
    exit: 'cubic-bezier(0.4, 0.0, 1, 1)';
    spring: 'cubic-bezier(0.34, 1.56, 0.64, 1)';
  };
}

// Modal enter/exit
const modalAnimation = {
  enter: {
    opacity: [0, 1],
    transform: ['translateY(8px)', 'translateY(0)'],
    duration: 200,
    easing: 'cubic-bezier(0.0, 0.0, 0.2, 1)',
  },
  exit: {
    opacity: [1, 0],
    transform: ['translateY(0)', 'translateY(4px)'],
    duration: 150,
    easing: 'cubic-bezier(0.4, 0.0, 1, 1)',
  },
};

```



```

interface AnimationTokens {
  duration: {
    instant: '50ms';
    fast: '150ms';
    normal: '200ms';
    slow: '300ms';
  };
  easing: {

```

```

    default: 'ease';
    enter: 'cubic-bezier(0.0, 0.0, 0.2, 1)';
    exit: 'cubic-bezier(0.4, 0.0, 1, 1)';
    spring: 'cubic-bezier(0.34, 1.56, 0.64, 1)';
  };
}

// Modal enter/exit
const modalAnimation = {
  enter: {
    opacity: [0, 1],
    transform: ['translateY(8px)', 'translateY(0)'],
    duration: 200,
    easing: 'cubic-bezier(0.0, 0.0, 0.2, 1)',
  },
  exit: {
    opacity: [1, 0],
    transform: ['translateY(0)', 'translateY(4px)'],
    duration: 150,
    easing: 'cubic-bezier(0.4, 0.0, 1, 1)',
  },
};

```

Micointerações Essenciais

Elemento	Interação	Feedback
Botão	Click	Scale 0.98 + cor
Toggle	Toggle	Animação do thumb + bg
Input	Focus	Borda azul + box-shadow
Dropdown	Open	Fade in + slide
Toast	Appear	Slide in da direita
Skeleton	Load	Pulse shimmer
Link	Hover	Underline aparece
Checkbox	Toggle	Check animado

9. Dark Mode

Estratégia de Implementação

Dark mode não é apenas inverter cores — é um **tema alternativo** com paleta própria.

```
type ThemeMode = 'light' | 'dark';
```

```
interface Theme {  
  mode: ThemeMode;  
  colors: {  
    background: {  
      primary: string;  
      secondary: string;  
      elevated: string;  
      modal: string;  
    };  
    text: {  
      primary: string;  
      secondary: string;  
      disabled: string;  
      inverse: string;  
    };  
    border: {  
      default: string;  
      hover: string;  
      focus: string;  
    };  
    action: {  
      primary: string;  
      primaryHover: string;  
      secondary: string;  
    };  
  };  
}
```

```
const lightTheme: Theme = {  
  mode: 'light',  
  colors: {  
    background: {  
      primary: 'FFFFFF',  
      secondary: 'F8F9FA',  
      elevated: 'FFFFFF',  
      modal: 'FFFFFF',  
    },  
    text: {  
      primary: '#202124',
```

```

    secondary: '#5F6368',
    disabled: '#9AA0A6',
    inverse: '#FFFFFF',
  },
  border: {
    default: '#DADCE0',
    hover: '#9AA0A6',
    focus: '#1A73E8',
  },

```

```
type ThemeMode = 'light' | 'dark';
```

```

interface Theme {
  mode: ThemeMode;
  colors: {
    background: {
      primary: string;
      secondary: string;
      elevated: string;
      modal: string;
    };
    text: {
      primary: string;
      secondary: string;
      disabled: string;
      inverse: string;
    };
    border: {
      default: string;
      hover: string;
      focus: string;
    };
    action: {
      primary: string;
      primaryHover: string;
      secondary: string;
    };
  };
}

```

```

const lightTheme: Theme = {
  mode: 'light',
  colors: {
    background: {
      primary: '#FFFFFF',
      secondary: '#F8F9FA',
      elevated: '#FFFFFF',
      modal: '#FFFFFF',
    },
    text: {
      primary: '#202124',

```

```

        secondary: '#5F6368',
        disabled: '#9AA0A6',
        inverse: '#FFFFFF',
    },
    border: {
        default: '#DADCE0',
        hover: '#9AA0A6',
        focus: '#1A73E8',
    },
    action: {
        primary: '#1A73E8',
        primaryHover: '#1557B0',
        secondary: '#E8F0FE',
    },
},
};

```

```

const darkTheme: Theme = {
    mode: 'dark',
    colors: {
        background: {
            primary: '#1F1F1F',
            secondary: '#2C2C2C',
            elevated: '#333333',
            modal: '#2C2C2C',
        },
        text: {
            primary: '#E8EAED',
            secondary: '#9AA0A6',
            disabled: '#5F6368',
            inverse: '#202124',
        },
        border: {
            default: '#3C4043',
            hover: '#5F6368',
            focus: '#8AB4F8',
        },
        action: {
            primary: '#8AB4F8',
            primaryHover: '#A8C7FA',
            secondary: '#3C4043',
        },
    },
};

```

Contraste no Dark Mode

O contraste absoluto não muda, mas a **percepção** sim. Ajustes necessários:

- Evite pretos puros (`#000000`) — use `#1F1F1F` - Prefira branco suave (`#E8EAED`) em vez de branco puro (`#FFFFFF`) - Sombras viram luz (elevation vira tom mais claro que o fundo) - Cores vibrantes (primary) devem ser dessaturadas para evitar fadiga

Light mode:

#FFFFFF
bg
#202124
text
#1A73E8
primary

Dark mode:

#1F1F1F
bg
#E8EAED
text
#8AB4F8
primary

Implementação com Context

```
const ThemeContext = React.createContext<{
  theme: Theme;
  toggleTheme: () => void;
}>({
  theme: lightTheme,
  toggleTheme: () => {},
});

function ThemeProvider({ children }: { children: React.ReactNode }) {
  const [mode, setMode] = useState<ThemeMode>('light');

  const toggleTheme = () => {
    setMode(prev => prev === 'light' ? 'dark' : 'light');
  };

  const theme = mode === 'light' ? lightTheme : darkTheme;

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      <div data-theme={mode} style={{ background: theme.colors.background.primary }}>
        {children}
      </div>
    </ThemeContext.Provider>
  );
}
```

10. UI para Devs

Inspeção no Figma

Ao inspecionar um layout no Figma, extraia:

Elemento	O que inspecionar
Botão	width, height, padding, border-radius, font-size, bg
Input	padding, border, border-radius, font-size, label position
Card	padding, border-radius, box-shadow, gap entre filhos
Text	font-family, font-size, line-height, letter-spacing, color
Spacing	margin, gap, top/left/bottom/right values

Medidas e Densidade

```
// Prefira valores da escala de espaçamento
const badImplementation = {
  padding: '13px 22px', // arbitrário
};

const faithfulImplementation = {
  padding: 12, // da escala: spacing.sm
  gap: 8,     // da escala: spacing.xs
};
```

Implementação Fiel

Checklist para implementar um componente com fidelidade:

```
interface UIInspectionChecklist {
  layout: {
    dimensions: boolean; // width, height, min/max
    padding: boolean;    // internal spacing
    margin: boolean;     // external spacing
    alignment: boolean;  // flex, grid, absolute
  };
  typography: {
    fontFamily: boolean;
    fontSize: boolean;
    fontWeight: boolean;
    lineHeight: boolean;
    letterSpacing: boolean;
    textAlign: boolean;
  };
};
```



```

};
visuals: {
  backgroundColor: boolean;
  border: boolean;          // width, style, color, radius
  boxShadow: boolean;
  opacity: boolean;
};
states: {
  hover: boolean;
  active: boolean;
  focus: boolean;
  disabled: boolean;
  error: boolean;
};
responsive: {
  mobile: boolean;
  tablet: boolean;
  desktop: boolean;
};
accessibility: {
  contrast: boolean;        // WCAG AA
  focusVisible: boolean;
  arialabels: boolean;
  keyboardNav: boolean;
};
}

function checkFidelity(inspection: UIInspectionChecklist): number {
  const items = Object.values(inspection).flatMap(category =>
    Object.values(category)
  );
  const total = items.length;
  const passed = items.filter(Boolean).length;
  return Math.round((passed / total) * 100);
}

```

```

interface UIInspectionChecklist {
  layout: {
    dimensions: boolean;    // width, height, min/max
    padding: boolean;       // internal spacing
    margin: boolean;        // external spacing
    alignment: boolean;     // flex, grid, absolute
  };
  typography: {
    fontFamily: boolean;
    fontSize: boolean;
    fontWeight: boolean;
    lineHeight: boolean;
    letterSpacing: boolean;
    textAlign: boolean;
  };
}

```

```

};
visuals: {
  backgroundColor: boolean;
  border: boolean;          // width, style, color, radius
  boxShadow: boolean;
  opacity: boolean;
};
states: {
  hover: boolean;
  active: boolean;
  focus: boolean;
  disabled: boolean;
  error: boolean;
};
responsive: {
  mobile: boolean;
  tablet: boolean;
  desktop: boolean;
};
accessibility: {
  contrast: boolean;        // WCAG AA
  focusVisible: boolean;
  arialabels: boolean;
  keyboardNav: boolean;
};
}

function checkFidelity(inspection: UIInspectionChecklist): number {
  const items = Object.values(inspection).flatMap(category =>
    Object.values(category)
  );
  const total = items.length;
  const passed = items.filter(Boolean).length;
  return Math.round((passed / total) * 100);
}

```

Densidade de Informação

Produtos enterprise geralmente precisam de mais densidade que produtos B2C.

```

// B2C – mais espaçado
const consumerTable = {
  padding: 16,
  fontSize: 14,
  rowHeight: 48,
};

// Enterprise – mais denso
const enterpriseTable = {

```

```
padding: 12,
fontSize: 13,
rowHeight: 40,
};

// Tools/dashboard – alta densidade
const denseTable = {
padding: 8,
fontSize: 12,
rowHeight: 32,
};
```

Design System

Módulo 07 – Design System: Consistência em Escala

****Um Design System não é um projeto. É um produto que serve outros produtos.****

1. O que é Design System

Design System é um ****conjunto integrado de padrões, componentes, diretrizes e ferramentas**** que orientam a criação de interfaces digitais de forma consistente e escalável.

Design System vs Style Guide vs Component Library

```
// B2C – mais espaçado
const consumerTable = {
padding: 16,
fontSize: 14,
rowHeight: 48,
};

// Enterprise – mais denso
const enterpriseTable = {
```

```
padding: 12,
fontSize: 13,
rowHeight: 40,
};

// Tools/dashboard – alta densidade
const denseTable = {
padding: 8,
fontSize: 12,
rowHeight: 32,
};
```

Design System

Módulo 07 – Design System: Consistência em Escala

****Um Design System não é um projeto. É um produto que serve outros produtos.****

1. O que é Design System

Design System é um ****conjunto integrado de padrões, componentes, diretrizes e ferramentas**** que orientam a criação de interfaces digitais de forma consistente e escalável.

Design System vs Style Guide vs Component Library

Style Guide:	Component Library:	Design System: "Guia de estilos"
"Biblioteca de componentes"	"Sistema integrado"	
		_____ Cores, tipografia,
Botões, inputs, modais,	Tudo acima + grid, regras de uso	tabelas, cards
processos, governance,		documentação, — Estático —
Dinâmico	tooling, princípios — PDF/doc	— npm package —
Ecosistema vivo — "O que usar"	— "O que implementar"	— "Como pensar"
Exemplo:	Exemplo:	Exemplo: Manual de marca
Shopify Polaris ```		Material UI

Propósito

```
interface DesignSystemPurpose {
consistencia: 'Mesma aparência e comportamento em todo o ecossistema';
eficiencia: 'Devs e designers produzem mais rápido, sem recriar o óbvio';
escalabilidade: 'Novos produtos/heranças adotam o sistema sem retrabalho';
}
```

```
qualidade:      'Componentes testados, acessíveis, documentados';  
colaboracao:    'Linguagem comum entre devs, designers, PMs, QA';  
}
```

```
interface DesignSystemPurpose {  
  consistencia:  'Mesma aparência e comportamento em todo o ecossistema';  
  eficiencia:    'Devs e designers produzem mais rápido, sem recriar o óbvio';  
  escalabilidade: 'Novos produtos/heranças adotam o sistema sem retrabalho';  
}
```

```
qualidade:      'Componentes testados, acessíveis, documentados';  
colaboracao:    'Linguagem comum entre devs, designers, PMs, QA';  
}
```

"Design Systems are how we scale design decisions across an organization." — Nathan Curtis

2. Atomic Design

Criado por **Brad Frost**, o Atomic Design é uma metodologia que divide interfaces em **níveis hierárquicos** de complexidade.

Os 5 níveis

Átomos	→	Moléculas	→	Organismos	→	Templates	→	Páginas
		Combina		Junta		Define		Aplica
		átomos		moléculas		layout		conteúdo
▼		▼		▼		▼		▼
Botão		Input +		Formulário		Página de		Página de
Ícone		Label +		+ Header +		cadastro		cadastro
Label		Error =		Sidebar +		(vazia,		(com dados
Tag		Campo de		Footer =		estrutura)		reais)
Cor		texto		Página				
Tipografia								
Espaçamento								

```
// Átomo: Label
interface LabelProps {
  text: string;
  variant: 'default' | 'required' | 'optional';
  size: 'sm' | 'md';
}

function Label({ text, variant, size }: LabelProps) {
  const variants = {
    default: 'text-gray-700',
    required: 'text-red-600 after:content-["*"] after:ml-0.5',
    optional: 'text-gray-400',
  };
  return (
    <span className={` ${variants[variant]} text-${size === 'sm' ? 'xs' : 'sm'} font-medium`} >
      {text}
    </span>
  );
}
```

```
// Molécula: Campo de texto (Label + Input + ErrorMessage)
interface TextFieldProps {
  label: string;
  name: string;
  value: string;
  onChange: (value: string) => void;
  error?: string;
```

```

    required?: boolean;
  }

function TextField({ label, name, value, onChange, error, required }: TextFieldProps) {
  return (
    <div className="flex flex-col gap-1.5">
      <Label text={label} variant={required ? 'required' : 'default'} size="md" />
      <Input
        name={name}
        value={value}
        onChange={onChange}
        hasError={!!error}
      />
      {error && <ErrorMessage text={error} />}
    </div>
  );
}

```

```

// Molécula: Campo de texto (Label + Input + ErrorMessage)
interface TextFieldProps {
  label: string;
  name: string;
  value: string;
  onChange: (value: string) => void;
  error?: string;
}

```



```

    required?: boolean;
  }

function TextField({ label, name, value, onChange, error, required }: TextFieldProps) {
  return (
    <div className="flex flex-col gap-1.5">
      <Label text={label} variant={required ? 'required' : 'default'} size="md" />
      <Input
        name={name}
        value={value}
        onChange={onChange}
        hasError={!!error}
      />
      {error && <ErrorMessage text={error} />}
    </div>
  );
}

```

```

// Organismo: Formulário de cadastro
interface SignupFormProps {
  onSubmit: (data: SignupData) => Promise<void>;
}

function SignupForm({ onSubmit }: SignupFormProps) {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');

  return (
    <form onSubmit={(e) => { e.preventDefault(); onSubmit({ email, password }); }}
      className="flex flex-col gap-4 max-w-md">
      <TextField label="Email" name="email" value={email} onChange={setEmail} required />
      <TextField label="Senha" name="password" value={password} onChange={setPassword} required
    />
      <Button type="submit" variant="primary" fullWidth>Cadastrar</Button>
    </form>
  );
}

```

```

// Template: estrutura da página (sem dados reais)
function AuthTemplate({ children }: { children: React.ReactNode }) {
  return (
    <div className="min-h-screen flex items-center justify-center bg-gray-50">
      <div className="w-full max-w-md p-8 bg-white rounded-xl shadow-sm">
        <Logo />
        {children}
      </div>
    </div>
  );
}

```

```

// Página: template + dados reais

```

```
function SignupPage() {
  return (
    <AuthTemplate>
      <h1 className="text-2xl font-bold mb-6">Crie sua conta</h1>
      <SignupForm onSubmit={handleSignup} />
    </AuthTemplate>
  );
}
```



```
// Template: estrutura da página (sem dados reais)
function AuthTemplate({ children }: { children: React.ReactNode }) {
  return (
    <div className="min-h-screen flex items-center justify-center bg-gray-50">
      <div className="w-full max-w-md p-8 bg-white rounded-xl shadow-sm">
        <Logo />
        {children}
      </div>
    </div>
  );
}

// Página: template + dados reais
```

```
function SignupPage() {  
  return (  
    <AuthTemplate>  
      <h1 className="text-2xl font-bold mb-6">Crie sua conta</h1>  
      <SignupForm onSubmit={handleSignup} />  
    </AuthTemplate>  
  );  
}
```

Por que Atomic Design no Enterprise?

Sem Atomic Design:

"Onde está o botão primary?"

"Este input tem o mesmo estilo
que aquele?"

"Este formulário quebrou porque
mudei o Input em 10 lugares"

Com Atomic Design:

"Button: variant=primary"

"Input segue o Field token"
(sempre consistente)

"Mudei o Input, todos os
formulários atualizam"

3. Design Tokens

Design Tokens são as ****variáveis visuais**** do sistema. Eles abstraem decisões de design em valores únicos e reutilizáveis.

Definição em TypeScript

```
// tokens/colors.ts
export const colors = {
  brand: {
    primary: { 50: '#eff6ff', 100: '#dbeafe', 200: '#bfdbfe',
                300: '#93c5fd', 400: '#60a5fa', 500: '#3b82f6',
                600: '#2563eb', 700: '#1d4ed8', 800: '#1e40af',
                900: '#1e3a8a', 950: '#172554' },
    secondary: { 50: '#faf5ff', 100: '#f3e8ff', 500: '#8b5cf6',
                 600: '#7c3aed', 700: '#6d28d9' },
  },
  semantic: {
    success: '#10b981',
    warning: '#f59e0b',
    error: '#ef4444',
    info: '#3b82f6',
  },
  neutral: {
    white: '#ffffff',
    50: '#f9fafb',
    100: '#f3f4f6',
    200: '#e5e7eb',
    300: '#d1d5db',
    400: '#9ca3af',
    500: '#6b7280',
    600: '#4b5563',
    700: '#374151',
    800: '#1f2937',
    900: '#111827',
    black: '#000000',
  },
} as const;

export type ColorToken = keyof typeof colors.semantic;
```

Tipografia

```
// tokens/typography.ts
export const typography = {
```

```

fontFamily: {
  sans: "'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif",
  mono: "'JetBrains Mono', 'Fira Code', monospace",
},
scale: {
  'display-xl': { fontSize: '4.5rem', lineHeight: '5rem', fontWeight: 700, letterSpacing:
'-0.02em' },
  'display-lg': { fontSize: '3.75rem', lineHeight: '4.25rem', fontWeight: 700, letterSpacing:
'-0.02em' },
  'display': { fontSize: '3rem', lineHeight: '3.5rem', fontWeight: 700, letterSpacing:
'-0.01em' },
  'heading-xl': { fontSize: '2.25rem', lineHeight: '2.75rem', fontWeight: 600, letterSpacing:
'-0.01em' },
  'heading-lg': { fontSize: '1.875rem', lineHeight: '2.375rem', fontWeight: 600 },
  'heading': { fontSize: '1.5rem', lineHeight: '2rem', fontWeight: 600 },
  'heading-sm': { fontSize: '1.25rem', lineHeight: '1.75rem', fontWeight: 600 },
  'body-lg': { fontSize: '1.125rem', lineHeight: '1.75rem', fontWeight: 400 },
  'body': { fontSize: '1rem', lineHeight: '1.5rem', fontWeight: 400 },
  'body-sm': { fontSize: '0.875rem', lineHeight: '1.25rem', fontWeight: 400 },
  'caption': { fontSize: '0.75rem', lineHeight: '1rem', fontWeight: 400 },
  'overline': { fontSize: '0.75rem', lineHeight: '1rem', fontWeight: 500, textTransform:
'uppercase', letterSpacing: '0.08em' },
},
} as const;

```



```

// tokens/typography.ts
export const typography = {

```

```

fontFamily: {
  sans: "'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif",
  mono: "'JetBrains Mono', 'Fira Code', monospace",
},
scale: {
  'display-xl': { fontSize: '4.5rem', lineHeight: '5rem', fontWeight: 700, letterSpacing:
'-0.02em' },
  'display-lg': { fontSize: '3.75rem', lineHeight: '4.25rem', fontWeight: 700, letterSpacing:
'-0.02em' },
  'display': { fontSize: '3rem', lineHeight: '3.5rem', fontWeight: 700, letterSpacing:
'-0.01em' },
  'heading-xl': { fontSize: '2.25rem', lineHeight: '2.75rem', fontWeight: 600, letterSpacing:
'-0.01em' },
  'heading-lg': { fontSize: '1.875rem', lineHeight: '2.375rem', fontWeight: 600 },
  'heading': { fontSize: '1.5rem', lineHeight: '2rem', fontWeight: 600 },
  'heading-sm': { fontSize: '1.25rem', lineHeight: '1.75rem', fontWeight: 600 },
  'body-lg': { fontSize: '1.125rem', lineHeight: '1.75rem', fontWeight: 400 },
  'body': { fontSize: '1rem', lineHeight: '1.5rem', fontWeight: 400 },
  'body-sm': { fontSize: '0.875rem', lineHeight: '1.25rem', fontWeight: 400 },
  'caption': { fontSize: '0.75rem', lineHeight: '1rem', fontWeight: 400 },
  'overline': { fontSize: '0.75rem', lineHeight: '1rem', fontWeight: 500, textTransform:
'uppercase', letterSpacing: '0.08em' },
},
} as const;

```

Spacing

```

// tokens/spacing.ts
export const spacing = {
  0: '0px',
  0.5: '2px',
  1: '4px',
  2: '8px',
  3: '12px',
  4: '16px',
  5: '20px',
  6: '24px',
  8: '32px',
  10: '40px',
  12: '48px',
  16: '64px',
  20: '80px',
  24: '96px',
} as const;

export type SpacingToken = keyof typeof spacing;

```

Shadows, Border Radius e Breakpoints

```
// tokens/effects.ts
export const shadows = {
  none: 'none',
  sm: '0 1px 2px 0 rgba(0, 0, 0, 0.05)',
  md: '0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px -2px rgba(0, 0, 0, 0.1)',
  lg: '0 10px 15px -3px rgba(0, 0, 0, 0.1), 0 4px 6px -4px rgba(0, 0, 0, 0.1)',
  xl: '0 20px 25px -5px rgba(0, 0, 0, 0.1), 0 8px 10px -6px rgba(0, 0, 0, 0.1)',
} as const;

export const borderRadius = {
  none: '0px',
  sm: '4px',
  md: '8px',
  lg: '12px',
  xl: '16px',
  full: '9999px',
} as const;

// tokens/breakpoints.ts
export const breakpoints = {
  sm: 640,
  md: 768,
  lg: 1024,
  xl: 1280,
  '2xl': 1536,
} as const;

// Uso: breakpoints como media queries
export const mediaQueries = {
  sm: `@media (min-width: ${breakpoints.sm}px)`,
  md: `@media (min-width: ${breakpoints.md}px)`,
  lg: `@media (min-width: ${breakpoints.lg}px)`,
  xl: `@media (min-width: ${breakpoints.xl}px)`,
  '2xl': `@media (min-width: ${breakpoints['2xl']}px)`,
} as const;
```

```
// tokens/effects.ts
export const shadows = {
  none: 'none',
  sm: '0 1px 2px 0 rgba(0, 0, 0, 0.05)',
  md: '0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px -2px rgba(0, 0, 0, 0.1)',
  lg: '0 10px 15px -3px rgba(0, 0, 0, 0.1), 0 4px 6px -4px rgba(0, 0, 0, 0.1)',
  xl: '0 20px 25px -5px rgba(0, 0, 0, 0.1), 0 8px 10px -6px rgba(0, 0, 0, 0.1)',
} as const;

export const borderRadius = {
  none: '0px',
  sm: '4px',
  md: '8px',
  lg: '12px',
  xl: '16px',
  full: '9999px',
} as const;

// tokens/breakpoints.ts
export const breakpoints = {
  sm: 640,
  md: 768,
  lg: 1024,
  xl: 1280,
  '2xl': 1536,
} as const;

// Uso: breakpoints como media queries
export const mediaQueries = {
  sm: `@media (min-width: ${breakpoints.sm}px)`,
  md: `@media (min-width: ${breakpoints.md}px)`,
  lg: `@media (min-width: ${breakpoints.lg}px)`,
  xl: `@media (min-width: ${breakpoints.xl}px)`,
  '2xl': `@media (min-width: ${breakpoints['2xl']}px)`,
} as const;
```

CSS Custom Properties a partir dos Tokens

```
// tokens/toCSS.ts
import { colors } from './colors';
import { spacing } from './spacing';
import { shadows, borderRadius } from './effects';

export function generateCSSVariables(): string {
  let css = `:root {\n`;

  // Colors
  for (const [brand, shades] of Object.entries(colors.brand)) {
    for (const [shade, value] of Object.entries(shades)) {
```



```

        css += `  --color-${brand}-${shade}: ${value};\n`;
      }
    }

    // Spacing
    for (const [key, value] of Object.entries(spacing)) {
      css += `  --spacing-${key}: ${value};\n`;
    }

    // Shadows
    for (const [key, value] of Object.entries(shadows)) {
      css += `  --shadow-${key}: ${value};\n`;
    }

    // Border radius
    for (const [key, value] of Object.entries(borderRadius)) {
      css += `  --radius-${key}: ${value};\n`;
    }

    css += '}\n';
    return css;
  }
}

```

```

// Saída gerada:
// :root {
//   --color-primary-50: #eff6ff;
//   --spacing-4: 16px;
//   --shadow-md: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
//   --radius-md: 8px;
// }

```



```

// tokens/toCSS.ts
import { colors } from './colors';
import { spacing } from './spacing';
import { shadows, borderRadius } from './effects';

export function generateCSSVariables(): string {
  let css = ':root {\n';

  // Colors
  for (const [brand, shades] of Object.entries(colors.brand)) {
    for (const [shade, value] of Object.entries(shades)) {

```


4. Componentes

Componentes são a **implementação concreta** dos tokens. Cada componente deve ser: - **Padronizado** — props consistentes entre componentes - **Documentado** — spec, variantes, exemplos - **Acessível** — ARIA, teclado, contraste - **Testado** — unitário, visual, acessibilidade

Botão

```
// components/Button/Button.tsx
import { ButtonHTMLAttributes, forwardRef } from 'react';

type ButtonVariant = 'primary' | 'secondary' | 'outline' | 'ghost' | 'danger';
type ButtonSize = 'sm' | 'md' | 'lg';

interface ButtonProps extends ButtonHTMLAttributes<HTMLButtonElement> {
  variant?: ButtonVariant;
  size?: ButtonSize;
  loading?: boolean;
  fullWidth?: boolean;
  leftIcon?: React.ReactNode;
  rightIcon?: React.ReactNode;
}

const variantStyles: Record<ButtonVariant, string> = {
  primary: 'bg-[var(--color-primary-600)] text-white hover:bg-[var(--color-primary-700)] focus:ring-[var(--color-primary-500)]',
  secondary: 'bg-gray-100 text-gray-900 hover:bg-gray-200 focus:ring-gray-500',
  outline: 'border border-gray-300 text-gray-700 hover:bg-gray-50 focus:ring-[var(--color-primary-500)]',
  ghost: 'text-gray-600 hover:bg-gray-100 focus:ring-gray-500',
  danger: 'bg-[var(--color-semantic-error)] text-white hover:bg-red-700 focus:ring-red-500',
};

const sizeStyles: Record<ButtonSize, string> = {
  sm: 'px-3 py-1.5 text-sm gap-1.5',
  md: 'px-4 py-2 text-sm gap-2',
  lg: 'px-6 py-3 text-base gap-2',
};

export const Button = forwardRef<HTMLButtonElement, ButtonProps>(
  ({ variant = 'primary', size = 'md', loading, fullWidth, leftIcon, rightIcon, children, disabled, className, ...props }, ref) => {
    return (
      <button
        ref={ref}
        disabled={disabled || loading}

```

```

    className={`
      inline-flex items-center justify-center font-medium rounded-[var(--radius-md)]
      transition-all duration-150
      focus:outline-none focus:ring-2 focus:ring-offset-2
      disabled:opacity-50 disabled:cursor-not-allowed
      ${variantStyles[variant]}
      ${sizeStyles[size]}
      ${fullWidth ? 'w-full' : ''}
      ${className ?? ''}
    `.trim()}
    {...props}
  >
    {loading ? <Spinner /> : leftIcon}

```

```
// components/Button/Button.tsx
```

```
import { ButtonHTMLAttributes, forwardRef } from 'react';
```

```
type ButtonVariant = 'primary' | 'secondary' | 'outline' | 'ghost' | 'danger';
```

```
type ButtonSize = 'sm' | 'md' | 'lg';
```

```
interface ButtonProps extends ButtonHTMLAttributes<HTMLButtonElement> {
```

```
  variant?: ButtonVariant;
```

```
  size?: ButtonSize;
```

```
  loading?: boolean;
```

```
  fullWidth?: boolean;
```

```
  leftIcon?: React.ReactNode;
```

```
  rightIcon?: React.ReactNode;
```

```
}
```

```
const variantStyles: Record<ButtonVariant, string> = {
```

```
  primary: 'bg-[var(--color-primary-600)] text-white hover:bg-[var(--color-primary-700)]
```

```
  focus:ring-[var(--color-primary-500)]',
```

```
  secondary: 'bg-gray-100 text-gray-900 hover:bg-gray-200 focus:ring-gray-500',
```

```
  outline: 'border border-gray-300 text-gray-700 hover:bg-gray-50
```

```
  focus:ring-[var(--color-primary-500)]',
```

```
  ghost: 'text-gray-600 hover:bg-gray-100 focus:ring-gray-500',
```

```
  danger: 'bg-[var(--color-semantic-error)] text-white hover:bg-red-700 focus:ring-red-500',
```

```
};
```

```
const sizeStyles: Record<ButtonSize, string> = {
```

```
  sm: 'px-3 py-1.5 text-sm gap-1.5',
```

```
  md: 'px-4 py-2 text-sm gap-2',
```

```
  lg: 'px-6 py-3 text-base gap-2',
```

```
};
```

```
export const Button = forwardRef<HTMLButtonElement, ButtonProps>(  
  ({ variant = 'primary', size = 'md', loading, fullWidth, leftIcon, rightIcon, children,  
  disabled, className, ...props }, ref) => {  
    return (  
      <button  
        ref={ref}  
        disabled={disabled || loading}
```

```

        className={`
          inline-flex items-center justify-center font-medium rounded-[var(--radius-md)]
          transition-all duration-150
          focus:outline-none focus:ring-2 focus:ring-offset-2
          disabled:opacity-50 disabled:cursor-not-allowed
          ${variantStyles[variant]}
          ${sizeStyles[size]}
          ${fullWidth ? 'w-full' : ''}
          ${className ?? ''}
        `.trim()}
        {...props}
      >
        {loading ? <Spinner /> : leftIcon}
        {children}
        {rightIcon}
      </button>
    );
  }
);

```

Input

```

// components/Input/Input.tsx
import { InputHTMLAttributes, forwardRef } from 'react';

interface InputProps extends InputHTMLAttributes<HTMLInputElement> {
  label: string;
  error?: string;
  hint?: string;
  leftElement?: React.ReactNode;
  rightElement?: React.ReactNode;
}

export const Input = forwardRef<HTMLInputElement, InputProps>(
  ({ label, error, hint, leftElement, rightElement, id, className, ...props }, ref) => {
    const inputId = id ?? `input-${label.toLowerCase().replace(/\s+/g, '-')}`;

    return (
      <div className="flex flex-col gap-1.5">
        <label htmlFor={inputId} className="text-sm font-medium text-gray-700">
          {label}
          {props.required && <span className="text-red-500 ml-0.5">*</span>}
        </label>
        <div
          className={`
            flex items-center border rounded-[var(--radius-md)] transition-all
            ${error
              ? 'border-red-500 focus-within:ring-red-500'
              : 'border-gray-300 focus-within:ring-[var(--color-primary-500)]'}
          `
        >

```

```

    }
    ${props.disabled ? 'bg-gray-50 cursor-not-allowed' : 'bg-white'}
    focus-within:ring-2
  `.trim()})
  >
  {leftElement && <span className="pl-3 text-gray-400">{leftElement}</span>}
  <input
    ref={ref}
    id={inputId}
    className={`
      w-full px-3 py-2 text-sm bg-transparent outline-none
      ${props.disabled ? 'text-gray-400' : 'text-gray-900'}
      placeholder:text-gray-400
      `.trim()})
    aria-invalid={!error}
    aria-describedby={error ? `${inputId}-error` : hint ? `${inputId}-hint` : undefined}
    {...props}
  />
  {rightElement && <span className="pr-3 text-gray-400">{rightElement}</span>}
</div>
{error && <span id={` ${inputId}-error`} className="text-xs text-red-500">{error}</span>}
  {hint && !error && <span id={` ${inputId}-hint`} className="text-xs
text-gray-500">{hint}</span>}

```

```
// components/Input/Input.tsx
```

```
import { InputHTMLAttributes, forwardRef } from 'react';
```

```
interface InputProps extends InputHTMLAttributes<HTMLInputElement> {
  label: string;
  error?: string;
  hint?: string;
  leftElement?: React.ReactNode;
  rightElement?: React.ReactNode;
}
```

```
export const Input = forwardRef<HTMLInputElement, InputProps>(
  ({ label, error, hint, leftElement, rightElement, id, className, ...props }, ref) => {
    const inputId = id ?? `input-${label.toLowerCase().replace(/\s+/g, '-')}`;

    return (
      <div className="flex flex-col gap-1.5">
        <label htmlFor={inputId} className="text-sm font-medium text-gray-700">
          {label}
          {props.required && <span className="text-red-500 ml-0.5">*</span>}
        </label>
        <div
          className={`
            flex items-center border rounded-[var(--radius-md)] transition-all
            ${error
              ? 'border-red-500 focus-within:ring-red-500'
              : 'border-gray-300 focus-within:ring-[var(--color-primary-500)]'
            }
          `
        >

```

```

    }
    ${props.disabled ? 'bg-gray-50 cursor-not-allowed' : 'bg-white'}
    focus-within:ring-2
  `.trim()}
}
>
{leftElement && <span className="pl-3 text-gray-400">{leftElement}</span>}
<input
  ref={ref}
  id={inputId}
  className={`
    w-full px-3 py-2 text-sm bg-transparent outline-none
    ${props.disabled ? 'text-gray-400' : 'text-gray-900'}
    placeholder:text-gray-400
  `.trim()}
  aria-invalid={!!error}
  aria-describedby={error ? `${inputId}-error` : hint ? `${inputId}-hint` : undefined}
  {...props}
/>
{rightElement && <span className="pr-3 text-gray-400">{rightElement}</span>}
</div>
{error && <span id={`${inputId}-error`} className="text-xs text-red-500">{error}</span>}
  {hint && !error && <span id={`${inputId}-hint`} className="text-xs
text-gray-500">{hint}</span>}
</div>
);
}
);

```

Modal

```

// components/Modal/Modal.tsx
import { useEffect, useRef } from 'react';

interface ModalProps {
  open: boolean;
  onClose: () => void;
  title: string;
  size?: 'sm' | 'md' | 'lg' | 'xl';
  children: React.ReactNode;
  footer?: React.ReactNode;
}

const sizeStyles = {
  sm: 'max-w-sm',
  md: 'max-w-lg',
  lg: 'max-w-2xl',
  xl: 'max-w-4xl',
};

```

```

export function Modal({ open, onClose, title, size = 'md', children, footer }: ModalProps) {
  const overlayRef = useRef<HTMLDivElement>(null);

  useEffect(() => {
    if (!open) return;
    const handleEsc = (e: KeyboardEvent) => {
      if (e.key === 'Escape') onClose();
    };
    document.addEventListener('keydown', handleEsc);
    document.body.style.overflow = 'hidden';
    return () => {
      document.removeEventListener('keydown', handleEsc);
      document.body.style.overflow = '';
    };
  }, [open, onClose]);

  if (!open) return null;

  return (
    <div
      ref={overlayRef}
      className="fixed inset-0 z-50 flex items-center justify-center p-4"
      onClick={(e) => { if (e.target === overlayRef.current) onClose(); }}
      role="dialog"
      aria-modal="true"
      aria-labelledby="modal-title"
    >
      <div className="fixed inset-0 bg-black/50 backdrop-blur-sm" aria-hidden="true" />
      <div className={`relative bg-white rounded-xl shadow-xl w-full ${sizeStyles[size]} p-6
        animate-in fade-in zoom-in duration-200`} >
        <div className="flex items-center justify-between mb-4">

```

```

// components/Modal/Modal.tsx
import { useEffect, useRef } from 'react';

```

```

interface ModalProps {
  open: boolean;
  onClose: () => void;
  title: string;
  size?: 'sm' | 'md' | 'lg' | 'xl';
  children: React.ReactNode;
  footer?: React.ReactNode;
}

```

```

const sizeStyles = {
  sm: 'max-w-sm',
  md: 'max-w-lg',
  lg: 'max-w-2xl',
  xl: 'max-w-4xl',

```

```

};

```


Tabela

```
// components/Table/Table.tsx
interface Column<T> {
  key: string;
  header: string;
  render: (item: T) => React.ReactNode;
  sortable?: boolean;
  width?: string;
  align?: 'left' | 'center' | 'right';
}

interface TableProps<T> {
  columns: Column<T>[];
  data: T[];
  loading?: boolean;
  emptyMessage?: string;
  onClick?: (item: T) => void;
  sortable?: boolean;
}

export function Table<T extends Record<string, unknown>>({
  columns, data, loading, emptyMessage = 'Nenhum registro encontrado',
  onClick, sortable,
}: TableProps<T>) {
  const [sortKey, setSortKey] = useState<string | null>(null);
  const [sortDir, setSortDir] = useState<'asc' | 'desc'>('asc');

  const sorted = useMemo(() => {
    if (!sortKey) return data;
    return [...data].sort((a, b) => {
      const aVal = a[sortKey];
      const bVal = b[sortKey];
      if (aVal < bVal) return sortDir === 'asc' ? -1 : 1;
      if (aVal > bVal) return sortDir === 'asc' ? 1 : -1;
      return 0;
    });
  }, [data, sortKey, sortDir]);

  const handleSort = (key: string) => {
    if (sortKey === key) {
      setSortDir(d => d === 'asc' ? 'desc' : 'asc');
    } else {
      setSortKey(key);
      setSortDir('asc');
    }
  };

  return (
    <div className="overflow-x-auto border border-gray-200 rounded-[var(--radius-1g)]">
```



```
    )))
  </tr>
</table>

```

```
// components/Table/Table.tsx
interface Column<T> {
  key: string;
  header: string;
  render: (item: T) => React.ReactNode;
  sortable?: boolean;
  width?: string;
  align?: 'left' | 'center' | 'right';
}

interface TableProps<T> {
  columns: Column<T>[];
  data: T[];
  loading?: boolean;
  emptyMessage?: string;
  onClick?: (item: T) => void;
  sortable?: boolean;
}

export function Table<T extends Record<string, unknown>>({
  columns, data, loading, emptyMessage = 'Nenhum registro encontrado',
  onClick, sortable,
}: TableProps<T>) {
  const [sortKey, setSortKey] = useState<string | null>(null);
  const [sortDir, setSortDir] = useState<'asc' | 'desc'>('asc');

  const sorted = useMemo(() => {
    if (!sortKey) return data;
    return [...data].sort((a, b) => {
      const aVal = a[sortKey];
      const bVal = b[sortKey];
      if (aVal < bVal) return sortDir === 'asc' ? -1 : 1;
      if (aVal > bVal) return sortDir === 'asc' ? 1 : -1;
      return 0;
    });
  }, [data, sortKey, sortDir]);

  const handleSort = (key: string) => {
    if (sortKey === key) {
      setSortDir(d => d === 'asc' ? 'desc' : 'asc');
    } else {
      setSortKey(key);
      setSortDir('asc');
    }
  };

  return (
    <div className="overflow-x-auto border border-gray-200 rounded-[var(--radius-lg)]">

```



```

        )))
      </tr>
    ))
  })
</tbody>
</table>
</div>
);
}

```

Select

```

// components/Select/Select.tsx
interface SelectOption {
  label: string;
  value: string;
  disabled?: boolean;
}

interface SelectProps {
  label: string;
  options: SelectOption[];
  value: string;
  onChange: (value: string) => void;
  placeholder?: string;
  error?: string;
  required?: boolean;
  disabled?: boolean;
}

export function Select({ label, options, value, onChange, placeholder, error, required, disabled
}: SelectProps) {
  const selectId = `select-${label.toLowerCase().replace(/\s+/g, '-')}`;

  return (
    <div className="flex flex-col gap-1.5">
      <label htmlFor={selectId} className="text-sm font-medium text-gray-700">
        {label}
        {required && <span className="text-red-500 ml-0.5">*</span>}
      </label>
      <select
        id={selectId}
        value={value}
        onChange={(e) => onChange(e.target.value)}
        disabled={disabled}
        className={`
          w-full px-3 py-2 text-sm bg-white border rounded-[var(--radius-md)] transition-all
          ${error ? 'border-red-500' : 'border-gray-300'}
          ${disabled ? 'bg-gray-50 text-gray-400 cursor-not-allowed' : 'text-gray-900'}
        `
      >

```

```

        focus:outline-none focus:ring-2 ${error ? 'focus:ring-red-500' :
'focus:ring-[var(--color-primary-500)]'}
        `.trim())
        required={required}
        aria-invalid={!!error}
    >
        {placeholder && <option value="" disabled>{placeholder}</option>}
        {options.map(opt => (
            <option key={opt.value} value={opt.value} disabled={opt.disabled}>
                {opt.label}
            </option>
        ))}
    </select>

```

```

        focus:ring-2 focus:ring-[var(--color-primary-500)] focus:ring-offset-2

```

```

// components/Select/Select.tsx

```

```

interface SelectOption {

```

```

    label: string;

```

```

    value: string;

```

```

    disabled?: boolean;

```

```

}

```

```

interface SelectProps {

```

```

    label: string;

```

```

    options: SelectOption[];

```

```

    value: string;

```

```

    onChange: (value: string) => void;

```

```

    placeholder?: string;

```

```

    error?: string;

```

```

    required?: boolean;

```

```

    disabled?: boolean;

```

```

}

```

```

export function Select({ label, options, value, onChange, placeholder, error, required, disabled
}: SelectProps) {

```

```

    const selectId = `select-${label.toLowerCase().replace(/\s+/g, '-')}`;

```

```

    return (

```

```

        <div className="flex flex-col gap-1.5">

```

```

            <label htmlFor={selectId} className="text-sm font-medium text-gray-700">

```

```

                {label}

```

```

                {required && <span className="text-red-500 ml-0.5">*</span>}

```

```

            </label>

```

```

            <select

```

```

                id={selectId}

```

```

                value={value}

```

```

                onChange={(e) => onChange(e.target.value)}

```

```

                disabled={disabled}

```

```

                className={`

```

```

                    w-full px-3 py-2 text-sm bg-white border rounded-[var(--radius-md)] transition-all

```

```

                    ${error ? 'border-red-500' : 'border-gray-300'}

```

```

                    ${disabled ? 'bg-gray-50 text-gray-400 cursor-not-allowed' : 'text-gray-900'}

```


5. Documentação

Documentação é o que ****transforma uma biblioteca de componentes em um Design System****.

Storybook

Storybook é a ferramenta mais popular para documentar, testar e explorar componentes visualmente.

```
// components/Button/Button.stories.ts
import type { Meta, StoryObj } from '@storybook/react';
import { Button } from './Button';

const meta: Meta<typeof Button> = {
  title: 'Components/Button',
  component: Button,
  argTypes: {
    variant: {
      control: 'select',
      options: ['primary', 'secondary', 'outline', 'ghost', 'danger'],
    },
    size: { control: 'select', options: ['sm', 'md', 'lg'] },
    loading: { control: 'boolean' },
    disabled: { control: 'boolean' },
    fullWidth: { control: 'boolean' },
    children: { control: 'text' },
  },
  tags: ['autodocs'],
};

export default meta;
type Story = StoryObj<typeof Button>;

export const Primary: Story = {
  args: {
    variant: 'primary',
    children: 'Salvar',
    size: 'md',
  },
};

export const Secondary: Story = {
  args: {
    variant: 'secondary',
    children: 'Cancelar',
  },
};
```

```
export const Danger: Story = {
  args: {
    variant: 'danger',
    children: 'Excluir',
  },
};
```

```
export const Loading: Story = {
  args: {
    variant: 'primary',
    children: 'Salvando...',
  },
};
```

```
// components/Button/Button.stories.ts
import type { Meta, StoryObj } from '@storybook/react';
import { Button } from './Button';

const meta: Meta<typeof Button> = {
  title: 'Components/Button',
  component: Button,
  argTypes: {
    variant: {
      control: 'select',
      options: ['primary', 'secondary', 'outline', 'ghost', 'danger'],
    },
    size: { control: 'select', options: ['sm', 'md', 'lg'] },
    loading: { control: 'boolean' },
    disabled: { control: 'boolean' },
    fullWidth: { control: 'boolean' },
    children: { control: 'text' },
  },
  tags: ['autodocs'],
};

export default meta;
type Story = StoryObj<typeof Button>;

export const Primary: Story = {
  args: {
    variant: 'primary',
    children: 'Salvar',
    size: 'md',
  },
};

export const Secondary: Story = {
  args: {
    variant: 'secondary',
    children: 'Cancelar',
  },
};
```

```
export const Danger: Story = {
  args: {
    variant: 'danger',
    children: 'Excluir',
  },
};

export const Loading: Story = {
  args: {
    variant: 'primary',
    children: 'Salvando...',
    loading: true,
  },
};

export const Disabled: Story = {
  args: {
    variant: 'primary',
    children: 'Salvar',
    disabled: true,
  },
};

export const WithIcons: Story = {
  args: {
    variant: 'primary',
    children: 'Próximo',
    rightIcon: <span>→</span>,
  },
};

export const FullWidth: Story = {
  args: {
    variant: 'primary',
    children: 'Criar conta',
    fullWidth: true,
  },
};
```

Specs e Guidelines

Cada componente deve ter uma **especificação técnica** (spec) e **diretrizes de uso** (guidelines).

```
<!-- Button/Button.spec.md -->
```

```
## Button — Especificação Técnica
```

Props

Prop	Type	Default	Description
variant	`'primary'` \ `'secondary'` \ `'outline'` \ `'ghost'` \ `'danger'`	`'primary'`	Estilo visual do botão
size	`'sm'` \ `'md'` \ `'lg'`	`'md'`	Tamanho do botão
loading	`boolean`	`false`	Exibe spinner e desabilita
disabled	`boolean`	`false`	Desabilita interação
fullWidth	`boolean`	`false`	Ocupa 100% do container
leftIcon	`ReactNode`	–	Ícone à esquerda
rightIcon	`ReactNode`	–	Ícone à direita

Estados

State	Visual	Acessibilidade
Default	Cor sólida	Focus visible
Hover	1 shade mais escuro	Cursor: pointer
Focus	Ring 2px + offset 2px	Outline visível
Active	2 shades mais escuro	–
Disabled	Opacity 50%	aria-disabled
Loading	Spinner no lugar do ícone	aria-busy="true"

Comportamento

- **Teclado**: Enter/Space aciona o click
- **Loading**: Botão não responde a cliques
- **Full width**: Útil em mobile, raro em desktop



<!-- Button/Button.spec.md -->

Button – Especificação Técnica

Props

Prop	Type	Default	Description
variant	`'primary'` \ 'secondary' \ 'outline' \ 'ghost' \ 'danger'`	`'primary'`	Estilo visual do botão
size	`'sm'` \ 'md' \ 'lg'`	`'md'`	Tamanho do botão
loading	`boolean`	`false`	Exibe spinner e desabilita
disabled	`boolean`	`false`	Desabilita interação
fullWidth	`boolean`	`false`	Ocupa 100% do container
leftIcon	`ReactNode`	–	Ícone à esquerda
rightIcon	`ReactNode`	–	Ícone à direita

Estados

State	Visual	Acessibilidade
Default	Cor sólida	Focus visible
Hover	1 shade mais escuro	Cursor: pointer
Focus	Ring 2px + offset 2px	Outline visível
Active	2 shades mais escuro	–
Disabled	Opacity 50%	aria-disabled
Loading	Spinner no lugar do ícone	aria-busy="true"

Comportamento

- **Teclado**: Enter/Space aciona o click
- **Loading**: Botão não responde a cliques
- **Full width**: Útil em mobile, raro em desktop

Playground Interativo

```
// No Storybook, o próprio docs gera um playground automático:  
// controls interativos para testar todas as props.  
//  
// Exemplo de addons:  
// - @storybook/addon-controls: manipular props  
// - @storybook/addon-a11y: auditoria de acessibilidade  
// - @storybook/addon-interactions: testar interações  
// - storybook-addon-designs: embed do Figma
```

6. Versionamento

Design Systems evoluem. Versionamento garante que **mudanças sejam previsíveis e seguras**.

SemVer (Semantic Versioning)

MAJOR.MINOR.PATCH

MAJOR (1.x.x → 2.0.0):

Breaking changes – componente renomeado, prop removida, token alterado

Ex: Button: `variant="primary"` → `variant="filled"`

MINOR (x.1.x → x.2.0):

Novas funcionalidades – novo componente, nova variante, nova prop

Ex: Novo componente `Tooltip`, prop `size` no Modal

PATCH (x.x.1 → x.x.2):

Correções – bug fix, acessibilidade, performance

Ex: Corrigir contraste do Button danger, ARIA label no Modal

Breaking Changes

```
// BREAKING: renomear prop sem depreciação
interface ButtonProps {
  appearance?: 'primary' | 'secondary'; // antes era 'variant'
}

// CORRETO: depreciação gradual
interface ButtonProps {
  /** @deprecated Use `variant` instead */
  appearance?: 'primary' | 'secondary';
  variant?: 'primary' | 'secondary' | 'outline';
}

// Log de depreciação em runtime
if (appearance && !variant) {
  console.warn('[DS] Button: `appearance` is deprecated. Use `variant` instead.');
```

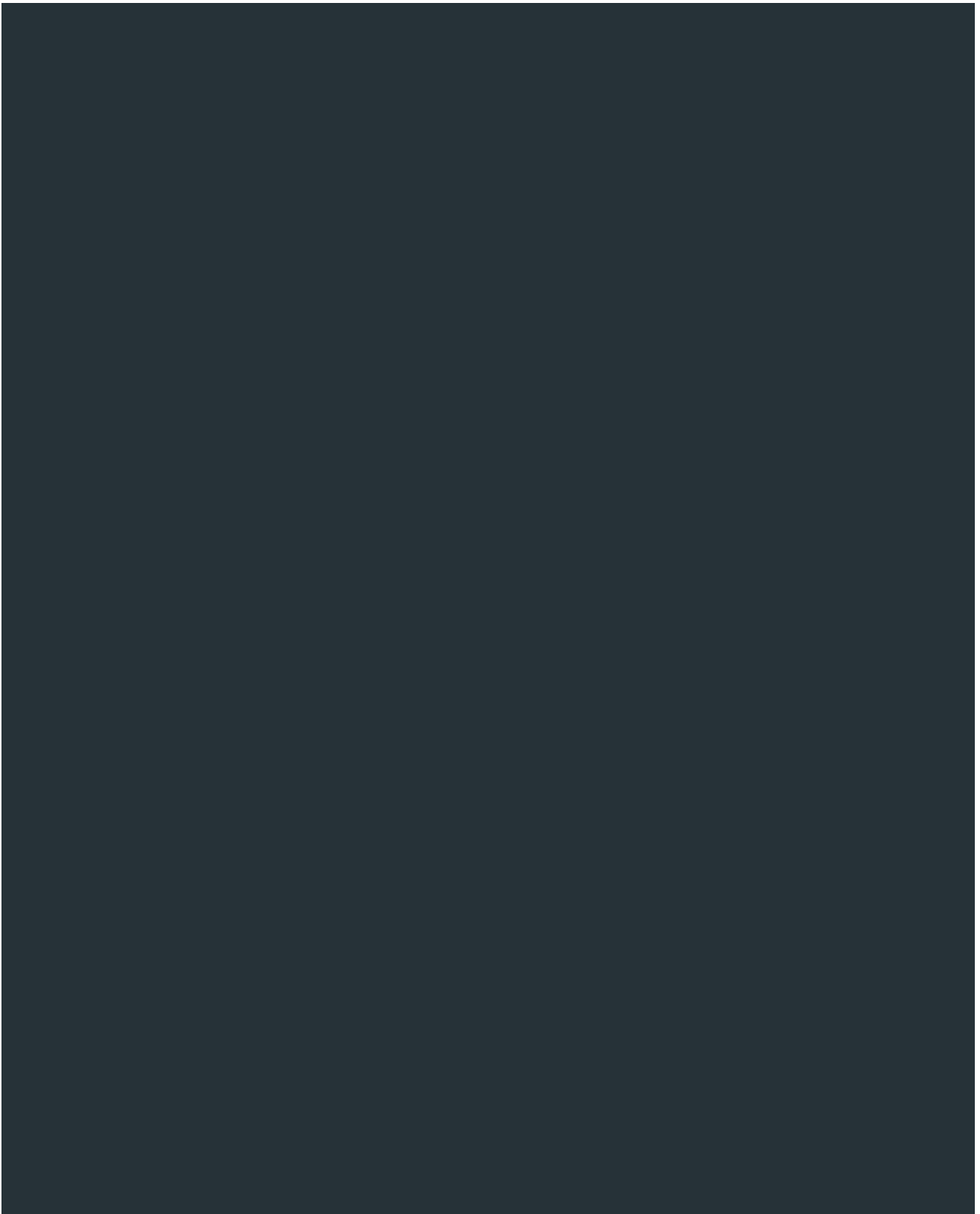
Migration Guides

Migration Guide: v1 → v2

Breaking Changes

Button: `appearance` → `variant`

****Antes:****



Migration Guide: v1 → v2

Breaking Changes

Button: `appearance` → `variant`

****Antes:****

```
<Button appearance="primary" /> ```tsx
```

****Depois:**** ``` <Button variant="primary" /> ```bash

****Codemod:**** ``` npx @acme/ds-codemod button-appearance-to-variant ```text

Modal: sizes renomeados

v1	v2
`size="small"`	`size="sm"`
`size="medium"`	`size="md"`
`size="large"`	`size="lg"`

Changelog

Changelog

[2.1.0] — 2026-06-15

Added

- Novo componente: `Tooltip` - Button: nova prop `leftIcon` e `rightIcon` - Modal: nova variante `size="xl"`

Changed

- Input: cor de foco agora usa token primary ao invés de blue-500 fixo - Table: sortable agora é opcional (default: true)

Fixed

- Modal: fechamento com Escape não funcionava em alguns browsers
- Select: placeholder não aparecia quando value era vazio
- Button: contraste do variant danger estava abaixo de WCAG AA

[2.0.0] — 2026-05-20

Breaking

- Button: ``appearance`` ``variant``
- Modal: ``size="small"|"medium"|"large"```
``size="sm"|"md"|"lg"```
- Tokens: ``--color-blue-*`` ``--color-primary-*`` `` ```

7. Consumo em Projetos

Publicação como npm Package

```
// package.json (do Design System)
{
  "name": "@empresa/design-system",
  "version": "2.1.0",
  "main": "dist/cjs/index.js",
  "module": "dist/esm/index.js",
  "types": "dist/types/index.d.ts",
  "exports": {
    ".": {
      "import": "./dist/esm/index.js",
      "require": "./dist/cjs/index.js",
      "types": "./dist/types/index.d.ts"
    },
    "./styles.css": "./dist/styles.css",
    "./tokens": "./dist/tokens/index.js"
  },
  "sideEffects": false,
  "files": ["dist", "README.md", "LICENSE"]
}
```

Tree-shaking

Para que o **tree-shaking** funcione, os componentes devem ser exportados individualmente:

```
// index.ts – barrel export
export { Button } from './components/Button/Button';
export { Input } from './components/Input/Input';
export { Modal } from './components/Modal/Modal';
export { Table } from './components/Table/Table';
export { Select } from './components/Select/Select';

// Tokens
export { colors } from './tokens/colors';
export { typography } from './tokens/typography';
export { spacing } from './tokens/spacing';
export { shadows, borderRadius } from './tokens/effects';
```

```
// Consumo com tree-shaking
import { Button } from '@empresa/design-system';
// Apenas Button entra no bundle (desde que sideEffects: false)
```

Theming

```
// components/ThemeProvider.tsx
import { createContext, useContext } from 'react';

interface Theme {
  colors: {
    primary: typeof import('../tokens/colors').colors.brand.primary;
    semantic: typeof import('../tokens/colors').colors.semantic;
  };
  spacing: typeof import('../tokens/spacing').spacing;
  borderRadius: typeof import('../tokens/effects').borderRadius;
}

const defaultTheme: Theme = {
  colors: {
    primary: { 500: '#3b82f6', 600: '#2563eb', /* ... */ },
    semantic: { success: '#10b981', error: '#ef4444', /* ... */ },
  },
  spacing: { 4: '16px', /* ... */ },
  borderRadius: { md: '8px', /* ... */ },
};

const ThemeContext = createContext<Theme>(defaultTheme);

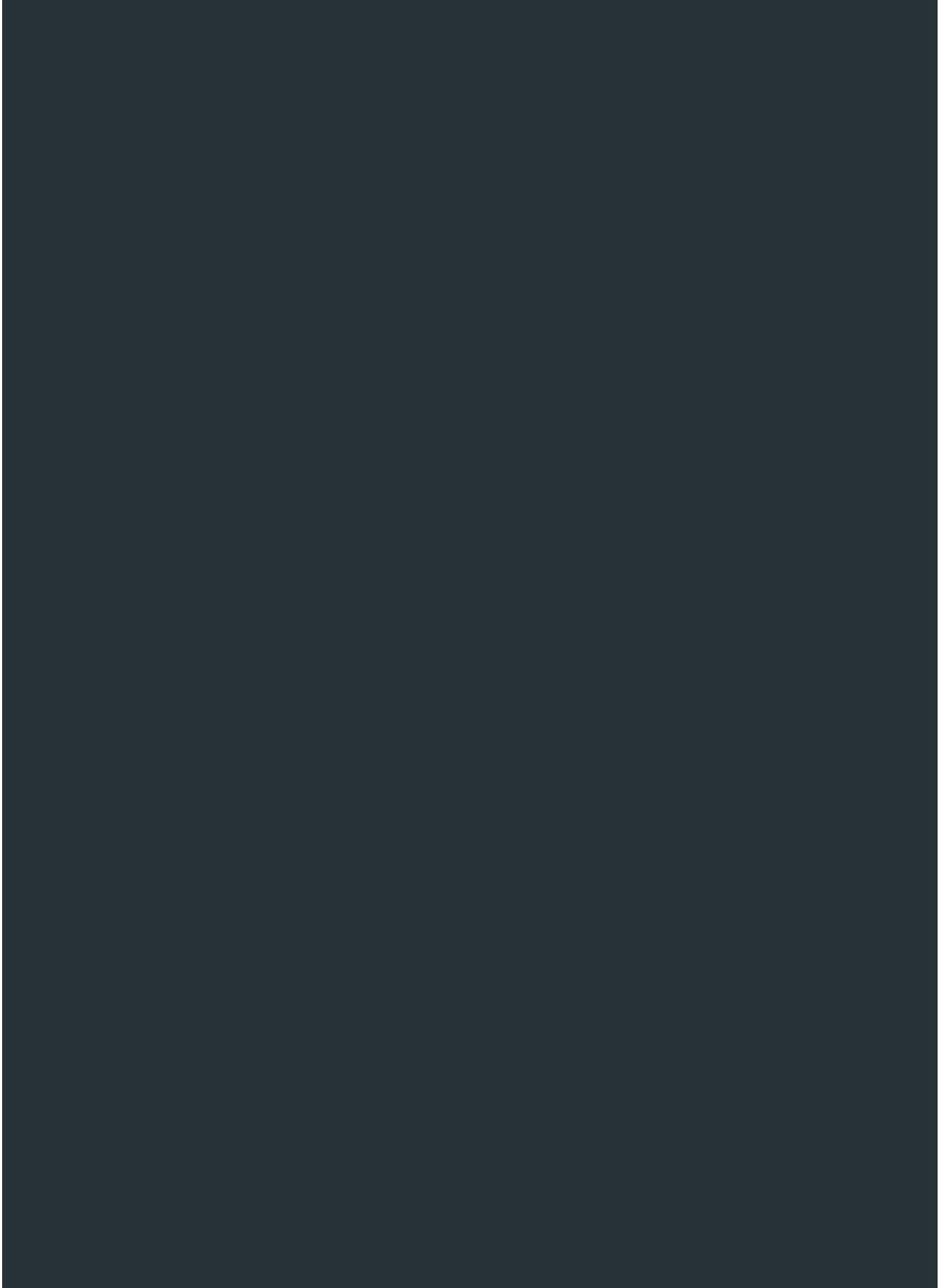
export function ThemeProvider({ theme, children }: { theme?: Partial<Theme>; children: React.ReactNode }) {
  const merged = { ...defaultTheme, ...theme };
  return (
    <ThemeContext.Provider value={merged}>
      {children}
    </ThemeContext.Provider>
  );
}

export function useTheme(): Theme {
  return useContext(ThemeContext);
}
```

Customização (Override via CSS Custom Properties)

```
/* Projeto: overrides.css */
:root {
  --color-primary-600: #7c3aed; /* roxo ao invés de azul */
  --color-primary-700: #6d28d9;
  --radius-md: 4px;
  --font-family-sans: 'Roboto', sans-serif;
}
```

```
// Uso
import '@empresa/design-system/styles.css';
import './overrides.css';
```



8. Manutenção

Governance

Equipe de Design System (DS Squad):

DS SQUAD	
Core Team (dedicado)	Contributors (de cada squad)
Design Lead	Designers de cada produto
Engineering Lead	Devs de frontend dos squads
Developer	QA engineers
Design Ops	Accessibility specialists

Rituais:

- Weekly sync (DS Squad)
- Monthly showcase (novos componentes)
- Quarterly review (ROI, métricas)
- Voting on RFCs (propostas de mudança)

Contribuição

CONTRIBUTING.md (extrato)

Como contribuir com um novo componente

- RFC**: Abra uma RFC no repositório do DS
 - Problema que resolve
 - Alternativas consideradas
 - Mockup / protótipo
- Review**: DS Squad revisa a RFC em até 1 semana
- Implementação**:
 - Componente + stories + specs + testes
 - Tokens se necessário
 - Documentação em PT-BR e EN
- Code Review**:
 - 2 approvals da DS Squad
 - all review obrigatório
- Release**: DS Squad faz o release

Critérios de aceite

- [] Componente segue os princípios do DS
- [] Suporta todos os estados (default, hover, focus, disabled, error, loading)
- [] Acessível (ARIA, teclado, contraste WCAG AA)
- [] Testado (unitário + visual + a11y)
- [] Documentado (Storybook + spec.md)
- [] Exemplo de uso real em projeto consumidor



CONTRIBUTING.md (extrato)

Como contribuir com um novo componente

1. **RFC**: Abra uma RFC no repositório do DS
 - Problema que resolve
 - Alternativas consideradas
 - Mockup / protótipo
2. **Review**: DS Squad revisa a RFC em até 1 semana
3. **Implementação**:
 - Componente + stories + specs + testes
 - Tokens se necessário
 - Documentação em PT-BR e EN
4. **Code Review**:
 - 2 approvals da DS Squad
 - a11y review obrigatório
5. **Release**: DS Squad faz o release

Critérios de aceite

- [] Componente segue os princípios do DS
- [] Suporta todos os estados (default, hover, focus, disabled, error, loading)
- [] Acessível (ARIA, teclado, contraste WCAG AA)
- [] Testado (unitário + visual + a11y)
- [] Documentado (Storybook + spec.md)
- [] Exemplo de uso real em projeto consumidor

Code Review de Design

```
// Checklist de review de design no PR

interface DesignReviewChecklist {
  tokens: [
    'Usa tokens de cor (não hex fixo)',
    'Usa tokens de spacing (não px fixo)',
    'Usa type scale (não font-size arbitrário)',
    'Usa shadow tokens (não box-shadow manual)',
  ];
  acessibilidade: [
    'Contraste WCAG AA (4.5:1 texto normal)',
    'Focus visível em todos os elementos interativos',
    'ARIA labels em elementos sem texto',
    'Suporte a navegação por teclado',
    'Testado com leitor de tela (NVDA/VoiceOver)',
  ];
  responsividade: [
    'Funciona em mobile (320px) até desktop (1920px)',
    'Touch targets >= 44px',
    'Texto não cortado em nenhum breakpoint',
  ];
  performance: [
    'Nenhum re-render desnecessário',
    'Imagens com lazy loading se aplicável',
    'Bundle size < 5KB (gzip) para componente simples',
  ];
}
```

Changelog Automatizado

```
// .github/release.yml

changelog:
  categories:
    - title: ' Breaking Changes'
      labels: ['breaking']
    - title: ' Novos Componentes'
      labels: ['new-component']
    - title: ' Novas Funcionalidades'
```



```
    labels: ['enhancement']
- title: ' Bug Fixes'
  labels: ['bug']
- title: ' Acessibilidade'
  labels: ['a11y']
- title: ' Documentação'
  labels: ['documentation']
- title: ' Manutenção'
  labels: ['chore', 'dependencies']
```

```
// .github/release.yml
```

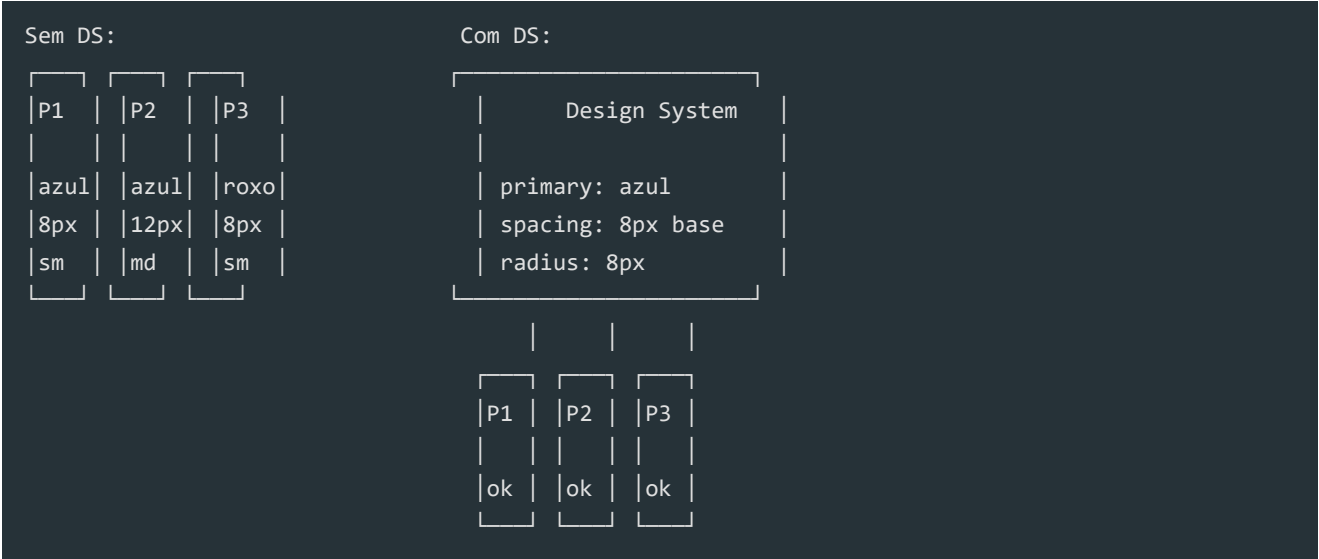
```
changelog:
```

```
  categories:
```

```
    - title: ' Breaking Changes'
      labels: ['breaking']
    - title: ' Novos Componentes'
      labels: ['new-component']
    - title: ' Novas Funcionalidades'
```


9. Design System no Enterprise

Escalabilidade



Consistência

```
// Antes do DS: cada squad tinha seu próprio "jeito"
const SquadA = () => <button className="btn-save">Salvar</button>;
const SquadB = () => <button className="submit-button">Salvar</button>;
const SquadC = () => <div class="action-btn" onclick={save}>Salvar</div>;

// Depois do DS: todos usam o mesmo componente
const SquadA = () => <Button variant="primary">Salvar</Button>;
const SquadB = () => <Button variant="primary">Salvar</Button>;
const SquadC = () => <Button variant="primary">Salvar</Button>;
```

Eficiência e ROI

Métricas de ROI de Design System:

Antes do DS	Depois do DS
Tela simples: 3 dias	Tela simples: 1 dia
Tela complexa: 5d	Tela complexa: 2.5d
Onboarding: 2 meses	Onboarding: 2 semanas
Design review: 3h	Design review: 30min
Bugs de UI: 15/mês	Bugs de UI: 3/mês

Cálculo de ROI:

5 squads × 4 devs × 1 tela/semana × R\$ 10.000/dia

→ Cada tela custa R\$ 30.000 (3 dias)

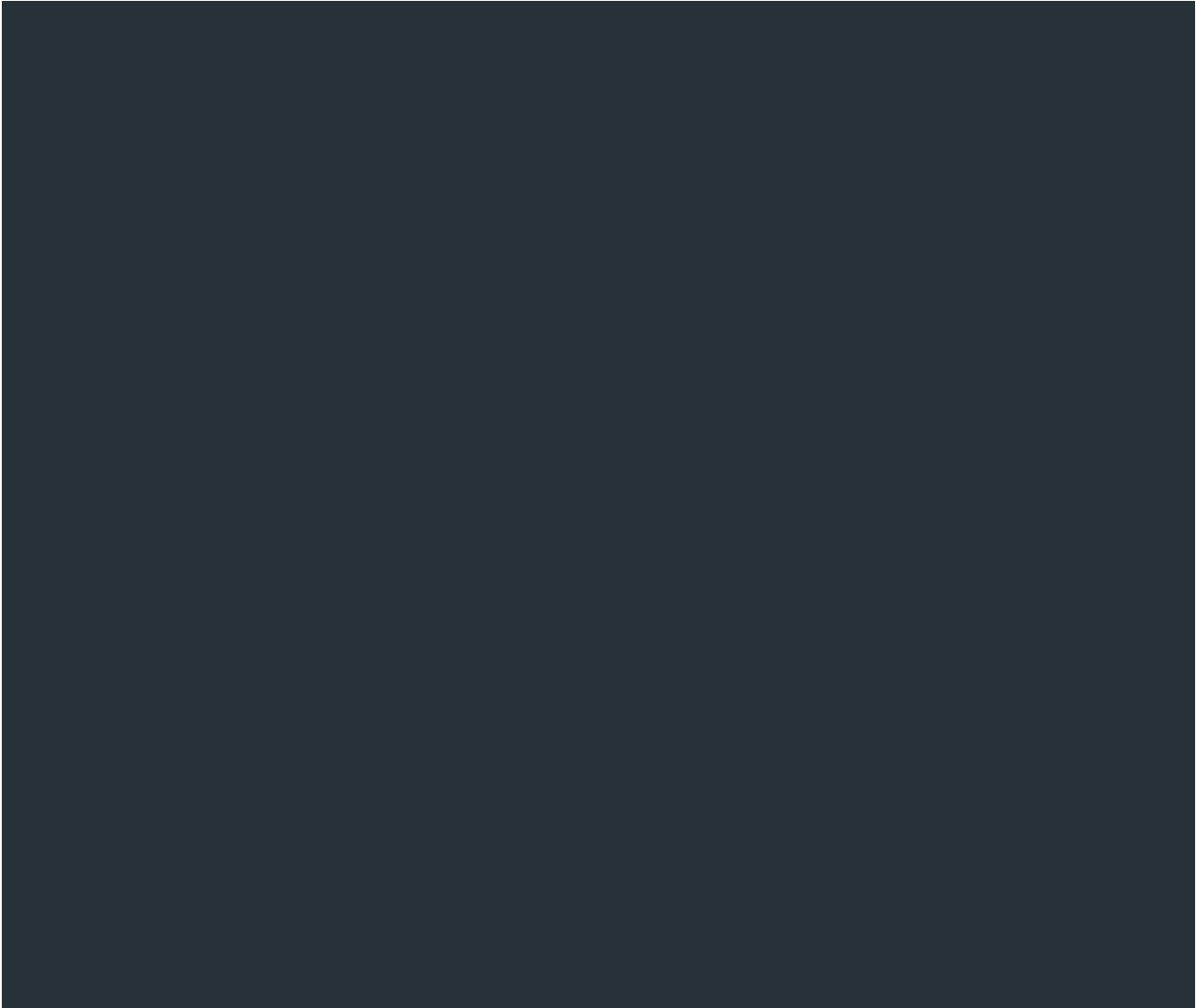
→ Com DS: R\$ 10.000 (1 dia)

→ Economia: R\$ 20.000/tela × 20 telas/mês × 12 meses

→ ROI: R\$ 4.8M/ano

Custo do DS Squad: 4 pessoas × R\$ 250K = R\$ 1M/ano

Net: R\$ 3.8M/ano (fonte: dados hipotéticos para ilustração)



Métricas de ROI de Design System:

Antes do DS	Depois do DS
Tela simples: 3 dias	Tela simples: 1 dia
Tela complexa: 5d	Tela complexa: 2.5d
Onboarding: 2 meses	Onboarding: 2 semanas
Design review: 3h	Design review: 30min
Bugs de UI: 15/mês	Bugs de UI: 3/mês

10. Implementação Prática

Vamos configurar um Design System básico com **React + TypeScript + Storybook + tokens**.

Setup inicial

```
# 1. Criar monorepo
mkdir my-design-system && cd my-design-system
pnpm init

# 2. Instalar dependências
pnpm add react react-dom
pnpm add -D typescript @types/react @types/react-dom
pnpm add -D vite @vitejs/plugin-react
pnpm add -D storybook @storybook/react @storybook/react-vite @storybook/addon-essentials

# 3. Iniciar Storybook
pnpm dlx storybook@latest init --builder=vite

# 4. Instalar tokens via CSS
pnpm add -D @tokens-studio/sd-transforms style-dictionary
```

Estrutura de diretórios

```
my-design-system/
├── tokens/
│   ├── colors.ts
│   ├── typography.ts
│   ├── spacing.ts
│   └── effects.ts
├── src/
│   ├── components/
│   │   ├── Button/
│   │   │   ├── Button.tsx
│   │   │   ├── Button.stories.ts
│   │   │   └── Button.spec.md
│   │   ├── Input/
│   │   ├── Modal/
│   │   ├── Table/
│   │   └── Select/
│   ├── index.ts
│   └── styles.css
├── .storybook/
│   ├── main.ts
│   └── preview.ts
└── package.json
```

└─ tsconfig.json

```
my-design-system/
├─ tokens/
│  ├─ colors.ts
│  ├─ typography.ts
│  ├─ spacing.ts
│  └─ effects.ts
├─ src/
│  ├─ components/
│  │  ├─ Button/
│  │  │  ├─ Button.tsx
│  │  │  ├─ Button.stories.ts
│  │  │  └─ Button.spec.md
│  │  ├─ Input/
│  │  ├─ Modal/
│  │  ├─ Table/
│  │  └─ Select/
│  ├─ index.ts
│  └─ styles.css
├─ .storybook/
│  ├─ main.ts
│  └─ preview.ts
└─ package.json
```

└─ tsconfig.json

Configuração do Vite para build

```
// vite.config.ts
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
import { resolve } from 'path';

export default defineConfig({
  plugins: [react()],
  build: {
    lib: {
      entry: resolve(__dirname, 'src/index.ts'),
      name: 'MyDesignSystem',
      formats: ['es', 'cjs'],
      fileName: (format) => `index.${format === 'es' ? 'mjs' : 'cjs'}`,
    },
    rollupOptions: {
      external: ['react', 'react-dom'],
      output: {
        globals: {
          react: 'React',
          'react-dom': 'ReactDOM',
        },
      },
    },
  },
});
```

Storybook config

```
// .storybook/main.ts
import type { StorybookConfig } from '@storybook/react-vite';

const config: StorybookConfig = {
  stories: ['../src/**/*.stories.@(ts|tsx)'],
  addons: [
    '@storybook/addon-essentials',
    '@storybook/addon-a11y',
    '@storybook/addon-interactions',
  ],
  framework: {
    name: '@storybook/react-vite',
    options: {},
  },
  docs: {
```



```
    autodocs: 'tag',
    defaultName: 'Documentação',
  },
};
```

```
export default config;
```

```
// .storybook/main.ts
import type { StorybookConfig } from '@storybook/react-vite';

const config: StorybookConfig = {
  stories: ['../src/**/*.stories.@(ts|tsx)'],
  addons: [
    '@storybook/addon-essentials',
    '@storybook/addon-a11y',
    '@storybook/addon-interactions',
  ],
  framework: {
    name: '@storybook/react-vite',
    options: {},
  },
  docs: {
```

```

    autodocs: 'tag',
    defaultName: 'Documentação',
  },
};

```

```
export default config;
```

```

// .storybook/preview.ts
import type { Preview } from '@storybook/react';
import '../src/styles.css';

const preview: Preview = {
  parameters: {
    controls: { expanded: true },
    a11y: {
      config: {
        rules: [{ id: 'color-contrast', enabled: true }],
      },
    },
  },
  globalTypes: {
    theme: {
      name: 'Theme',
      defaultValue: 'light',
      toolbar: {
        icon: 'circlehollow',
        items: ['light', 'dark'],
      },
    },
  },
};

export default preview;

```

Estilos base

```

/* src/styles.css */
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap');

*, *::before, *::after {
  box-sizing: border-box;
}

:root {
  --font-family-sans: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
  --color-primary-50: #eff6ff;
  --color-primary-100: #dbeafe;
  --color-primary-200: #bfdbfe;
  --color-primary-300: #93c5fd;

```

```

--color-primary-400: #60a5fa;
--color-primary-500: #3b82f6;
--color-primary-600: #2563eb;
--color-primary-700: #1d4ed8;
--color-primary-800: #1e40af;
--color-primary-900: #1e3a8a;
--color-primary-950: #172554;
--color-semantic-success: #10b981;
--color-semantic-warning: #f59e0b;
--color-semantic-error: #ef4444;
--color-semantic-info: #3b82f6;
--spacing-1: 4px;
--spacing-2: 8px;
--spacing-3: 12px;
--spacing-4: 16px;
--spacing-6: 24px;
--spacing-8: 32px;
--shadow-sm: 0 1px 2px 0 rgba(0, 0, 0, 0.05);
--shadow-md: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
--shadow-lg: 0 10px 15px -3px rgba(0, 0, 0, 0.1);
--radius-sm: 4px;
--radius-md: 8px;
--radius-lg: 12px;
}

```

```

body {
  font-family: var(--font-family-sans);
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

```

Arquitetura de Software

Módulo 08 – Arquitetura: Clean Architecture, DDD e SOLID

```

/* src/styles.css */
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap');

*, *::before, *::after {
  box-sizing: border-box;
}

:root {
  --font-family-sans: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
  --color-primary-50: #eff6ff;
  --color-primary-100: #dbeafe;
  --color-primary-200: #bfdbfe;
  --color-primary-300: #93c5fd;

```

```

--color-primary-400: #60a5fa;
--color-primary-500: #3b82f6;
--color-primary-600: #2563eb;
--color-primary-700: #1d4ed8;
--color-primary-800: #1e40af;
--color-primary-900: #1e3a8a;
--color-primary-950: #172554;
--color-semantic-success: #10b981;
--color-semantic-warning: #f59e0b;
--color-semantic-error: #ef4444;
--color-semantic-info: #3b82f6;
--spacing-1: 4px;
--spacing-2: 8px;
--spacing-3: 12px;
--spacing-4: 16px;
--spacing-6: 24px;
--spacing-8: 32px;
--shadow-sm: 0 1px 2px 0 rgba(0, 0, 0, 0.05);
--shadow-md: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
--shadow-lg: 0 10px 15px -3px rgba(0, 0, 0, 0.1);
--radius-sm: 4px;
--radius-md: 8px;
--radius-lg: 12px;
}

body {
  font-family: var(--font-family-sans);
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

```

Arquitetura de Software

Módulo 08 – Arquitetura: Clean Architecture, DDD e SOLID

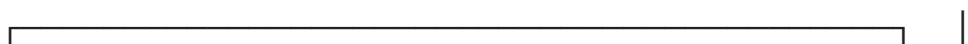
****Pensar antes de programar.****

1. Por que arquitetura importa

Arquitetura é a ****estrutura fundamental**** de um sistema. São as decisões que, se tomadas errado, custam caro para mudar.

O custo de uma arquitetura ruim

Arquitetura Ruim:



```
Arquitetura Boa: ┌───────────────────────────────────────────┐ |  
Feature nova: 2 dias      | | Por quê? "Só adicionar um módulo"   | | "Mudei uma  
coisa sem quebrar nada"    | | "Testes garantem que funciona"     | |  
└──────────────────────────┘ ```
```

Componentes:	Em quais partes o sistema se divide?
Comunicação:	Como as partes se comunicam?
Dados:	Como os dados fluem e são armazenados?
Tecnologia:	Qual stack suporta a estrutura?
Equipe:	Como o time se organiza para desenvolver?

2. SOLID — Os 5 princípios

SOLID não é uma arquitetura — é um **conjunto de princípios** que boas arquiteturas seguem.

S — Single Responsibility Principle

Uma classe deve ter um, e apenas um, motivo para mudar.

```
// Ruim: Service faz tudo
class UserService {
  createUser(data: CreateUserDto) { /* ... */ }
  sendWelcomeEmail(email: string) { /* ... */ }
  generateReport() { /* ... */ }
}

// Bom: Responsabilidades separadas
class CreateUserUseCase {
  execute(data: CreateUserDto) { /* ... */ }
}

class EmailService {
  sendWelcome(user: User) { /* ... */ }
}

class ReportGenerator {
  generate(): Report { /* ... */ }
}
```

O — Open/Closed Principle

Aberto para extensão, fechado para modificação.

```
// Ruim: Modificar para adicionar tipo
class PaymentProcessor {
  process(type: string, amount: number) {
    if (type === 'credit') { /* ... */ }
    if (type === 'debit') { /* ... */ }
    if (type === 'pix') { /* ... */ } // modifiquei!
  }
}

// Bom: Estender sem modificar
interface PaymentMethod {
  process(amount: number): void;
}
```

```
class CreditCardPayment implements PaymentMethod { /* ... */ }
class PixPayment implements PaymentMethod { /* ... */ }
```

```
// Ruim: Modificar para adicionar tipo
class PaymentProcessor {
    process(type: string, amount: number) {
        if (type === 'credit') { /* ... */ }
        if (type === 'debit') { /* ... */ }
        if (type === 'pix') { /* ... */ } // modifiquei!
    }
}

// Bom: Estender sem modificar
interface PaymentMethod {
    process(amount: number): void;
}
```

```
class CreditCardPayment implements PaymentMethod { /* ... */ }
class PixPayment implements PaymentMethod { /* ... */ }
```

L — Liskov Substitution Principle

Subtipos devem ser substituíveis por seus tipos base.

```
// Ruim: Quebra a expectativa
class Bird {
    fly(): void { /* voa */ }
}

class Penguin extends Bird {
    fly(): void { throw new Error('Pinguins não voam'); } //
}

// Bom: Abstração correta
interface Bird { }
interface FlyingBird extends Bird {
    fly(): void;
}
class Penguin implements Bird { }
class Sparrow implements Bird, FlyingBird { }
```

I — Interface Segregation Principle

Interfaces específicas são melhores que interfaces genéricas.

```
// Ruim: Interface genérica
interface Worker {
    work(): void;
    eat(): void;
    sleep(): void;
}

// Bom: Interfaces específicas
interface Workable { work(): void; }
interface Eatable { eat(): void; }
interface Sleepable { sleep(): void; }
```

D — Dependency Inversion Principle

Dependa de abstrações, não de implementações concretas.

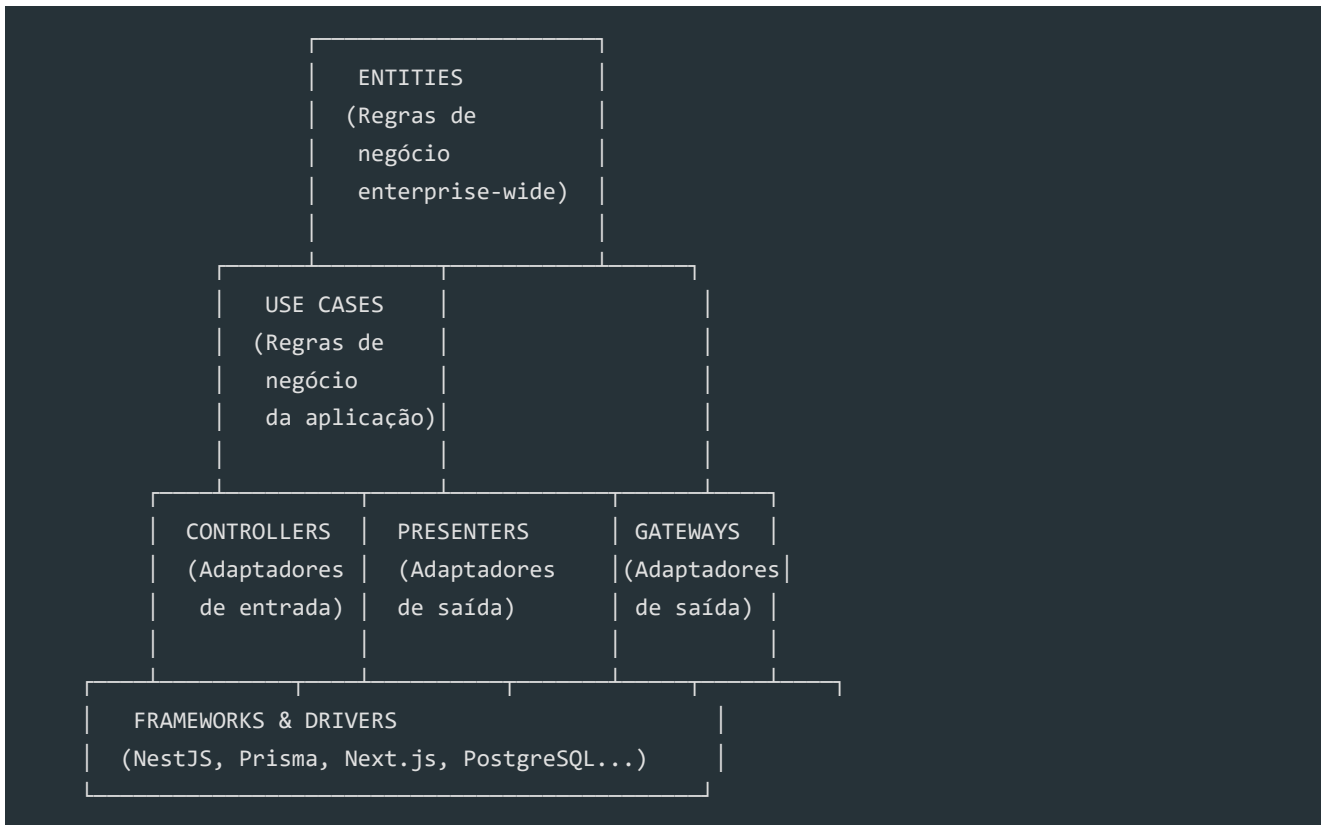

```
// Ruim: Dependência concreta
class OrderService {
    private repository = new PrismaOrderRepository(); // concreto!
}
```

```
// Bom: Dependência abstrata
class OrderService {
    constructor(
        private repository: OrderRepository // interface
    ) { }
}
```


3. Clean Architecture — A Regra da Dependência

Clean Architecture é uma arquitetura que organiza o código em **círculos concêntricos**.

As camadas



A Regra da Dependência

As dependências de código fonte apontam **para dentro**, em direção ao centro.
Nada no círculo interno pode saber algo sobre o círculo externo.

```
ERRADO: Use Case importa Prisma
UseCase → PrismaClient (dependência para fora!)

CERTO: Use Case depende de interface
UseCase → UserRepository (interface)
PrismaUserRepository → UserRepository (implementa)
```

O que vai em cada camada

Camada	O que contém	Pode importar
--------	--------------	---------------

Entities	Regras de negócio Enterpr	Nada externo
Use Cases	Regras de negócio da apli	Entities
**Controllers/Presenters*	Adaptação de entrada/saíd	Use Cases
Frameworks	Implementações concretas	Qualquer adaptador

4. DDD — Domain-Driven Design

DDD é uma abordagem que coloca o **domínio do negócio** no centro do desenvolvimento.

Linguagem Ubíqua

A mesma linguagem usada pelo negócio deve ser usada no código.

Negócio: "Um cliente pode abrir um chamado"

Código: `client.openTicket(ticket)`

Negócio: "Um cliente pode abrir um ticket de suporte"

Código: `client.createSupportTicket(ticket)` (outra linguagem)

Bounded Contexts

Um Bounded Context é um **limite explícito** onde um modelo de domínio se aplica.

Contexto de Vendas:

Cliente = quem compra

Produto = item no catálogo

Pedido = carrinho

Contexto de Logística:

Cliente = quem recebe

Produto = item no estoque

Pedido = carga para entrega

São modelos DIFERENTES do mesmo conceito!

Elementos do DDD

ENTITY

→ Tem identidade única (id)

→ Muda ao longo do tempo

→ Ex: Usuario, Pedido, Produto

VALUE OBJECT

→ Descrito por seus atributos

→ Imutável

→ Ex: Endereco, Email, CPF

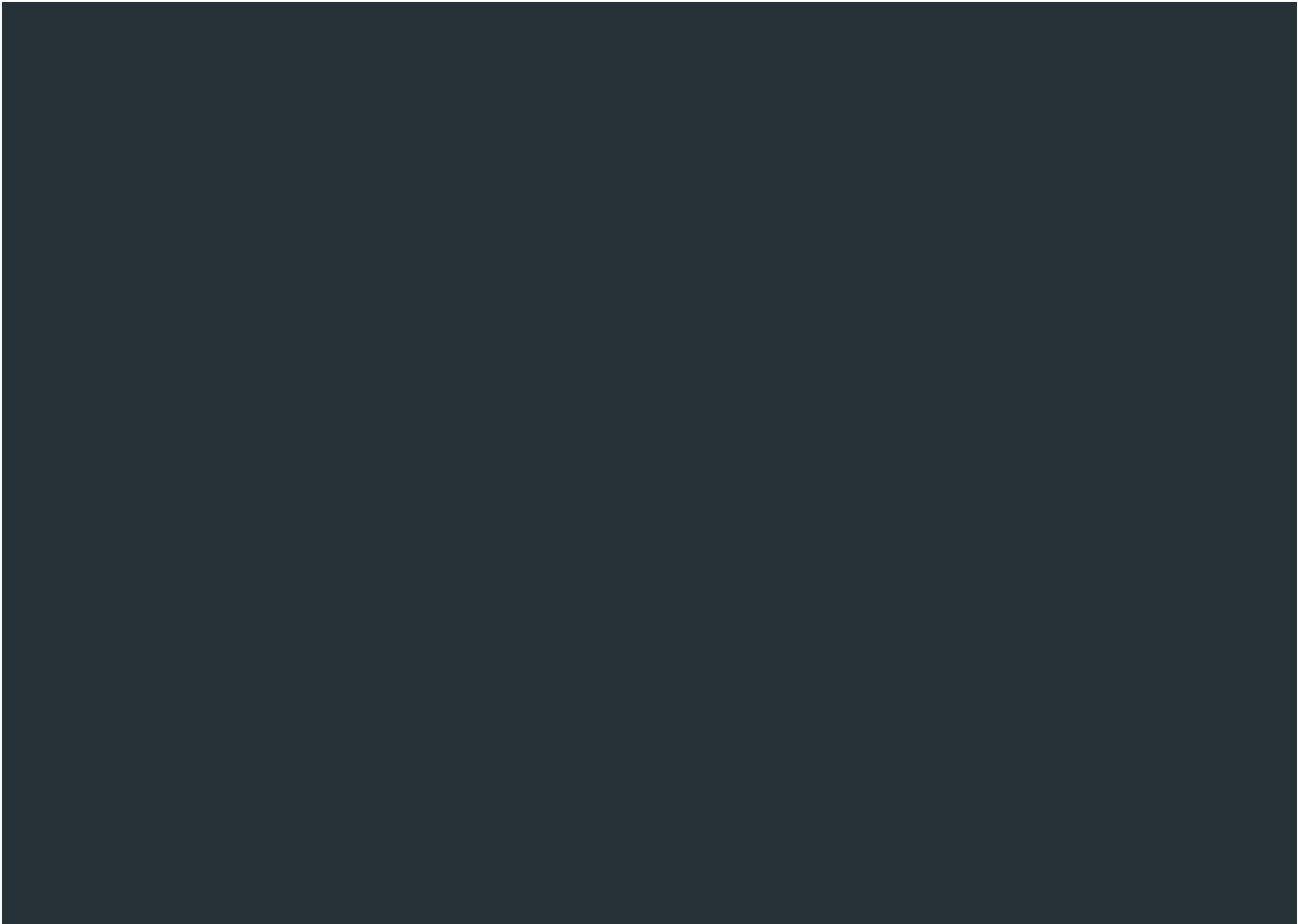
AGGREGATE

→ Grupo de entidades com raiz

→ Consistência transacional

→ Ex: Pedido + Itens (Pedido é raiz)

REPOSITORY
→ Abstração de persistência
→ Interface no domínio
→ Implementação externa
DOMAIN SERVICE
→ Regra de negócio sem estado
→ Opera em múltiplas entidades



ENTITY
→ Tem identidade única (id)
→ Muda ao longo do tempo
→ Ex: Usuario, Pedido, Produto
VALUE OBJECT
→ Descrito por seus atributos
→ Imutável
→ Ex: Endereco, Email, CPF
AGGREGATE
→ Grupo de entidades com raiz
→ Consistência transacional
→ Ex: Pedido + Itens (Pedido é raiz)

REPOSITORY
→ Abstração de persistência
→ Interface no domínio
→ Implementação externa
DOMAIN SERVICE
→ Regra de negócio sem estado
→ Opera em múltiplas entidades

Exemplo prático de DDD

```
// Entidade
class Usuario {
    private id: UsuarioId;
    private email: Email; // Value Object
    private perfil: Perfil;

    constructor(id: UsuarioId, email: Email, perfil: Perfil) {
        this.id = id;
        this.email = email;
        this.perfil = perfil;
    }

    alterarPerfil(novoPerfil: Perfil): void {
        // Regra de negócio
        if (this.perfil.eAdmin() && !novoPerfil.eAdmin()
            && !this.temOutroAdmin()) {
            throw new Error('Deve haver pelo menos um admin');
        }
        this.perfil = novoPerfil;
    }
}

// Value Object
class Email {
    private readonly valor: string;

    constructor(valor: string) {
        if (!valor.includes('@')) {
            throw new Error('Email inválido');
        }
        this.valor = valor;
    }

    toString(): string { return this.valor; }
}

// Repository (interface no domínio)
```

```
interface UsuarioRepository {
  findById(id: UsuarioId): Promise<Usuario | null>;
  save(usuario: Usuario): Promise<void>;
  delete(id: UsuarioId): Promise<void>;
}
```

// Use Case

```
class AlterarPerfilUseCase {
  constructor(
    private usuarioRepo: UsuarioRepository,
    private emailService: EmailService
  ) {}
```

```
  async alterarPerfil(usuarioId: UsuarioId, novoPerfil: Perfil): Promise<void> {
```

// Entidade

```
class Usuario {
  private id: UsuarioId;
  private email: Email; // Value Object
  private perfil: Perfil;

  constructor(id: UsuarioId, email: Email, perfil: Perfil) {
    this.id = id;
    this.email = email;
    this.perfil = perfil;
  }
}
```

```
  alterarPerfil(novoPerfil: Perfil): void {
    // Regra de negócio
    if (this.perfil.eAdmin() && !novoPerfil.eAdmin()
      && !this.temOutroAdmin()) {
      throw new Error('Deve haver pelo menos um admin');
    }
    this.perfil = novoPerfil;
  }
}
```

// Value Object

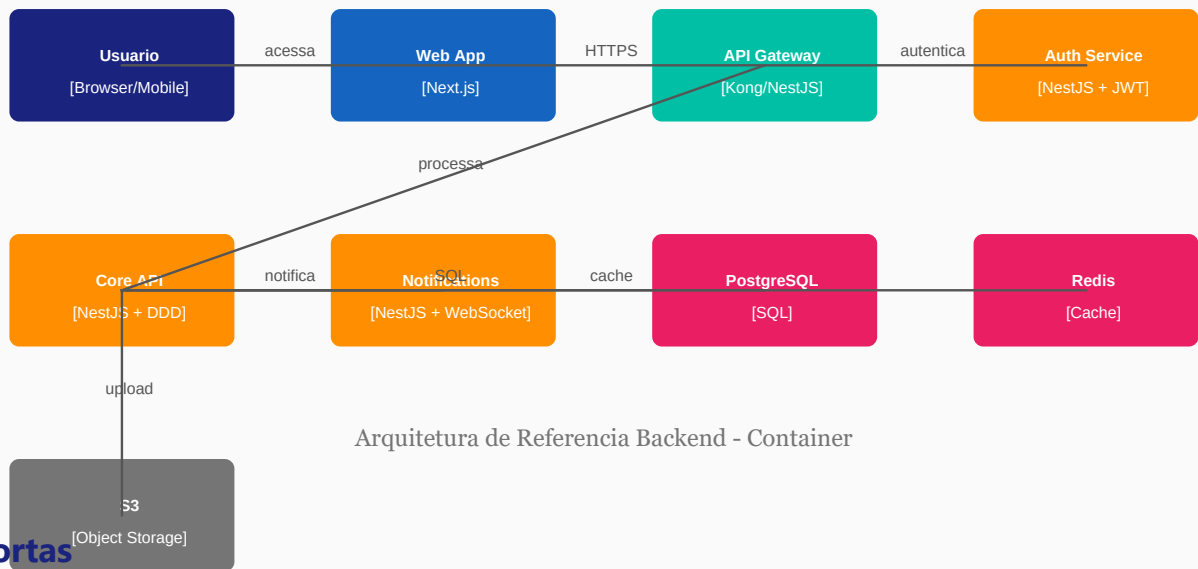
```
class Email {
  private readonly valor: string;

  constructor(valor: string) {
    if (!valor.includes('@')) {
      throw new Error('Email inválido');
    }
    this.valor = valor;
  }

  toString(): string { return this.valor; }
}
```

// Repository (interface no domínio)

Arquitetura de Referencia Backend - Container



Arquitetura de Referencia Backend - Container

```

// Porta de saída (interface no domínio)
interface PedidoRepository {
  save(pedido: Pedido): Promise<void>;
  findById(id: PedidoId): Promise<Pedido | null>;
}

// Porta de entrada (interface do use case)
interface CriarPedidoUseCase {
  execute(input: CriarPedidoInput): Promise<CriarPedidoOutput>;
}
  
```

Abaixo, uma visão de container C4 da mesma arquitetura:

```

// Adaptador de saída (implementação externa)
class PrismaPedidoRepository implements PedidoRepository {
  async save(pedido: Pedido): Promise<void> {
    await prisma.pedido.create({
      data: this.toPersistence(pedido),
    });
  }

  private toPersistence(pedido: Pedido): PrismaPedido {
    return {
      id: pedido.id.toString(),
      clienteId: pedido.clienteId.toString(),
      valor: pedido.valor,
    };
  }
}
  
```

```
}

// Adaptador de entrada (NestJS Controller)
@Controller('/pedidos')
class PedidoController {
  constructor(private criarPedido: CriarPedidoUseCase) {}

  @Post()
  async criar(@Body() body: CriarPedidoDto) {
    return this.criarPedido.execute(this.toInput(body));
  }
}
```

```
// Adaptador de saída (implementação externa)
class PrismaPedidoRepository implements PedidoRepository {
  async save(pedido: Pedido): Promise<void> {
    await prisma.pedido.create({
      data: this.toPersistence(pedido),
    });
  }

  private toPersistence(pedido: Pedido): PrismaPedido {
    return {
      id: pedido.id.toString(),
      clienteId: pedido.clienteId.toString(),
      valor: pedido.valor,
    };
  }
}
```


6. Modular Monolith vs Microservices

Modular Monolith

Um monólito ****modularizado**** — código em módulos bem definidos, mas deploy único.

Prós:	Contras:
- Simplicidade	- Escala tudo junto
- Deploy único	- Ponto único de falha
- Sem latência de rede	- Stack única
- Dados consistentes	- Time grande conflita
- Transações ACID	

Quando usar:

- Time pequeno (< 10 devs)
- Sistema com domínios fortemente acoplados
- Startup / MVP
- Quando velocidade > escala

Microservices

Serviços independentes, cada um com seu banco e deploy.

Prós:	Contras:
- Escala independente	- Complexidade de rede
- Deploy independente	- Dados distribuídos
- Stack variada	- Consistência eventual
- Times autônomos	- Observabilidade complexa
- Isolamento de falhas	- Testes mais difíceis

Quando usar:

- Time grande (> 10 devs por serviço)
- Domínios claramente separados
- Escala global
- Times autônomos por domínio

A recomendação

****Comece com Modular Monolith. Extraia microservices quando necessário.****

A maioria dos sistemas ****não precisa**** de microservices. Os que precisam, geralmente começaram como monólito e extraíram os serviços que escalavam de forma diferente.

7. Event-Driven Architecture

Event-Driven Architecture usa **eventos** para comunicação entre componentes.

Conceitos

```
Evento:          "Algo aconteceu"
  → PedidoCriado, UsuarioCadastrado, PagamentoConfirmado

Produtor:        Quem gera o evento
  → Serviço de pedidos publica "PedidoCriado"

Consumidor:      Quem reage ao evento
  → Serviço de email escuta "PedidoCriado" e envia confirmação

Barramento:     Meio de transporte
  → RabbitMQ, Kafka, Redis Pub/Sub
```

Exemplo

```
// Evento
class PedidoCriadoEvent {
  constructor(
    readonly pedidoId: string,
    readonly clienteEmail: string,
    readonly valor: number,
  ) {}
}

// Produtor
class CriarPedidoUseCase {
  constructor(
    private pedidoRepo: PedidoRepository,
    private eventBus: EventBus,
  ) {}

  async execute(input: CriarPedidoInput): Promise<void> {
    const pedido = Pedido.criar(input.clienteId, input.itens);
    await this.pedidoRepo.save(pedido);

    // Publica evento
    await this.eventBus.publish(
      new PedidoCriadoEvent(pedido.id, input.clienteEmail, pedido.valor)
    );
  }
}
```

```
// Consumidor
class EmailHandler {
  @OnEvent(PedidoCriadoEvent)
  async enviarConfirmacao(event: PedidoCriadoEvent) {
    await this.emailService.send({
      to: event.clienteEmail,
      subject: 'Pedido confirmado!',
      body: `Pedido #${event.pedidoId} no valor de R$ ${event.valor}`,
    });
  }
}
```

```
// Evento
class PedidoCriadoEvent {
  constructor(
    readonly pedidoId: string,
    readonly clienteEmail: string,
    readonly valor: number,
  ) {}
}

// Produtor
class CriarPedidoUseCase {
  constructor(
    private pedidoRepo: PedidoRepository,
    private eventBus: EventBus,
  ) {}

  async execute(input: CriarPedidoInput): Promise<void> {
    const pedido = Pedido.criar(input.clienteId, input.itens);
    await this.pedidoRepo.save(pedido);

    // Publica evento
    await this.eventBus.publish(
      new PedidoCriadoEvent(pedido.id, input.clienteEmail, pedido.valor)
    );
  }
}
```


8. Aplicando na prática com NestJS + Prisma

Estrutura de pastas seguindo Clean Architecture + DDD

```
src/
├── domain/                                # Círculo mais interno
│   ├── entities/                        # Entidades de domínio
│   │   └── usuario.ts
│   ├── value-objects/                  # Value Objects
│   │   └── email.ts
│   ├── repositories/                  # Interfaces (portas)
│   │   └── usuario.repository.ts
│   └── services/                      # Domain Services
│       └── autenticacao.service.ts
├── application/                        # Use Cases
│   ├── use-cases/
│   │   └── criar-usuario.use-case.ts
│   └── dto/
│       └── criar-usuario.dto.ts
├── infrastructure/                    # Adaptadores (implementações)
│   ├── persistence/
│   │   ├── prisma/
│   │   │   ├── prisma.service.ts
│   │   │   └── repositories/
│   │   │       └── prisma-usuario.repository.ts
│   │   └── messaging/
│   │       └── rabbitmq-event-bus.ts
│   └── email/
│       └── nodemailer-email.service.ts
├── presentation/                      # Adaptadores de entrada
│   ├── controllers/
│   │   └── usuario.controller.ts
│   └── guards/
│       └── jwt-auth.guard.ts
└── main.ts
```

Modelagem de Sistemas

Módulo 09 – Modelagem de Dados: Prisma e PostgreSQL

****A base de todo sistema Enterprise.****

1. Por que modelagem importa

Modelagem de dados é a ****fundação**** do sistema. Erros aqui são os mais caros de corrigir.

```
src/
├─ domain/                # Círculo mais interno
│  └─ entities/          # Entidades de domínio
│     └─ usuario.ts
│  └─ value-objects/      # Value Objects
│     └─ email.ts
│  └─ repositories/       # Interfaces (portas)
│     └─ usuario.repository.ts
│  └─ services/           # Domain Services
│     └─ autenticacao.service.ts
│
├─ application/           # Use Cases
│  └─ use-cases/
│     └─ criar-usuario.use-case.ts
│  └─ dto/
│     └─ criar-usuario.dto.ts
│
├─ infrastructure/        # Adaptadores (implementações)
│  └─ persistence/
│     └─ prisma/
│        └─ prisma.service.ts
│        └─ repositories/
│           └─ prisma-usuario.repository.ts
│  └─ messaging/
│     └─ rabbitmq-event-bus.ts
│  └─ email/
│     └─ nodemailer-email.service.ts
│
├─ presentation/          # Adaptadores de entrada
│  └─ controllers/
│     └─ usuario.controller.ts
│  └─ guards/
│     └─ jwt-auth.guard.ts
│
└─ main.ts
```

Modelagem de Sistemas

Módulo 09 – Modelagem de Dados: Prisma e PostgreSQL

****A base de todo sistema Enterprise.****

1. Por que modelagem importa

Modelagem de dados é a ****fundação**** do sistema. Erros aqui são os mais caros de corrigir.

O custo de uma modelagem ruim

Modelagem ruim:

Tabela sem índices	query lenta			Relação errada	dados inconsistentes			Falta
de soft delete	perda de dados			Sem audit trail	impossível auditar			Migração
corretiva	horas			de	trabalho			

Modelagem boa:

Índices certos	queries rápidas			Relações corretas	dados íntegros			
Constraints	validade dos dados			Migrações testadas	sem surpresas			

2. Entidades e Relacionamentos

Tipos de Relacionamento

```
1:1 - Um usuário tem um perfil
1:N - Um usuário tem muitos pedidos
N:M - Um produto está em muitas categorias
```

Exemplo no Prisma

```
// 1:1
model User {
  id      String  @id @default(cuid())
  email   String  @unique
  profile Profile?
}

model Profile {
  id        String  @id @default(cuid())
  fullName  String
  avatarUrl String?
  userId    String  @unique
  user      User    @relation(fields: [userId], references: [id])
}

// 1:N
model Order {
  id      String  @id @default(cuid())
  total   Decimal
  userId  String
  user    User    @relation(fields: [userId], references: [id])
  items   OrderItem[]
  createdAt DateTime @default(now())
}

model OrderItem {
  id        String  @id @default(cuid())
  orderId   String
  order     Order   @relation(fields: [orderId], references: [id])
  productId String
  product   Product @relation(fields: [productId], references: [id])
  quantity  Int
  price     Decimal
}

// N:M
```

```

model Product {
  id          String          @id @default(cuid())
  name        String
  categories   ProductCategory[]
}

```

```

model Category {
  id          String          @id @default(cuid())
  name        String
  products    ProductCategory[]
}

```

```

model ProductCategory {
  productId String
  categoryId String
}

```

```

// 1:1
model User {
  id      String  @id @default(cuid())
  email   String  @unique
  profile Profile?
}

```

```

model Profile {
  id          String @id @default(cuid())
  fullName    String
  avatarUrl   String?
  userId      String @unique
  user        User   @relation(fields: [userId], references: [id])
}

```

```

// 1:N
model Order {
  id          String  @id @default(cuid())
  total       Decimal
  userId      String
  user        User    @relation(fields: [userId], references: [id])
  items       OrderItem[]
  createdAt   DateTime @default(now())
}

```

```

model OrderItem {
  id          String  @id @default(cuid())
  orderId     String
  order       Order   @relation(fields: [orderId], references: [id])
  productId   String
  product     Product @relation(fields: [productId], references: [id])
  quantity    Int
  price       Decimal
}

```

```

// N:M

```


3. Soft Delete e Audit Trail

Soft Delete

Nunca delete dados definitivamente em sistemas Enterprise.

```
model User {
  id          String    @id @default(cuid())
  email       String    @unique
  name        String
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
  deletedAt   DateTime? // Soft delete

  // Filtro global no Prisma
  @@where("@deletedAt is null")
}
```

```
// Service
class UserService {
  async softDelete(id: string): Promise<void> {
    await this.prisma.user.update({
      where: { id },
      data: { deletedAt: new Date() },
    });
  }

  async findAll(): Promise<User[]> {
    // O @@where garante que deleted não aparece
    return this.prisma.user.findMany();
  }
}
```

Audit Trail

```
model AuditLog {
  id          String    @id @default(cuid())
  entity       String    // "User", "Order", "Product"
  entityId     String    // ID do registro
  action       String    // "CREATE", "UPDATE", "DELETE"
  changes      Json?     // { "before": {...}, "after": {...} }
  userId       String?
  ip           String?
  userAgent    String?
  createdAt    DateTime  @default(now())
}
```

```
    @@index([entity, entityId])
    @@index([userId])
    @@index([createdAt])
}
```

```
model AuditLog {
    id          String   @id @default(cuid())
    entity      String   // "User", "Order", "Product"
    entityId    String   // ID do registro
    action      String   // "CREATE", "UPDATE", "DELETE"
    changes     Json?    // { "before": {...}, "after": {...} }
    userId      String?
    ip          String?
    userAgent   String?
    createdAt   DateTime @default(now())
}
```



```

    @@index([entity, entityId])
    @@index([userId])
    @@index([createdAt])
  }
}

```

```

// AuditService
class AuditService {
  async log(input: AuditInput): Promise<void> {
    await this.prisma.auditLog.create({
      data: {
        entity: input.entity,
        entityId: input.entityId,
        action: input.action,
        changes: JSON.parse(JSON.stringify(input.changes)),
        userId: input.userId,
        ip: input.ip,
        userAgent: input.userAgent,
      },
    });
  }
}

// Uso com hook do Prisma
prisma.$use(async (params, next) => {
  const result = await next(params);

  if (['create', 'update', 'delete'].includes(params.action)) {
    await auditService.log({
      entity: params.model,
      entityId: result?.id,
      action: params.action.toUpperCase(),
      changes: params.args.data,
    });
  }

  return result;
});

```

4. Índices e Performance

Quando criar índices

```
model Order {
  id          String   @id @default(cuid())
  userId      String
  status      OrderStatus
  total       Decimal
  createdAt   DateTime @default(now())

  // Índice para busca por usuário (foreign key)
  @@index([userId])

  // Índice composto para filtro comum
  @@index([userId, status])

  // Índice para ordenação por data
  @@index([createdAt])

  // Índice parcial para pedidos ativos
  @@index([status, createdAt])
}
```

Regras de índices

Crie índices para:

- Foreign keys (toda FK deve ter índice)
- Campos usados em WHERE
- Campos usados em ORDER BY
- Campos usados em JOIN

Evite:

- Índices em colunas de baixa cardinalidade (boolean)
- Muitos índices em tabelas pequenas (< 1000 registros)
- Índices que nunca são usados

Query Performance

```
// N+1 – busca em loop
const orders = await prisma.order.findMany();
for (const order of orders) {
  const items = await prisma.orderItem.findMany({
    where: { orderId: order.id },
  });
}
```

```
});  
}  
  
// Eager loading – busca tudo de uma vez  
const orders = await prisma.order.findMany({  
  include: {  
    items: true,  
    user: true,  
  },  
});
```

```
// N+1 – busca em loop  
const orders = await prisma.order.findMany();  
for (const order of orders) {  
  const items = await prisma.orderItem.findMany({  
    where: { orderId: order.id },  
  });  
}
```


5. Migrações Seguras

Criando migrações

```
# Criar migration baseada no schema
npx prisma migrate dev --name create-user-table

# Aplicar em produção
npx prisma migrate deploy

# Resetar banco (dev)
npx prisma migrate reset
```

Migrações sem downtime

```
// Ruim: renomear coluna diretamente (quebra queries em execução)
model User {
  fullname String // Antigo
  name      String // Novo – erro se ambas existirem
}

// Bom: expand-migrate-contract
// Passo 1: Adicionar nova coluna (sem remover antiga)
model User {
  fullname String? // Nullable agora
  name      String?
}

// Passo 2: Preencher dados (script separado)
await prisma.user.updateMany({
  where: { name: null },
  data: { name: prisma.user.fullname }, // valor do campo antigo
});

// Passo 3: Remover coluna antiga (próxima release)
model User {
  name String // Único campo
}
```

6. Estratégias de Backup

Tipos de backup

```
Full:      Cópia completa do banco
Quando:    Diário
Tempo:     Lento, ocupa espaço
Restore:   Completo, mais simples

Incremental: Apenas mudanças desde o último backup
Quando:     Horário
Tempo:      Rápido, ocupa pouco espaço
Restore:    Precisa do full + todos incrementais

WAL (Write-Ahead Log): Log de transações
Quando:     Contínuo
Uso:        Point-in-time recovery
```

Script de backup

```
#!/bin/bash
# backup.sh – backup PostgreSQL com compressão

DB_NAME="app"
DB_USER="admin"
BACKUP_DIR="/backups"
DATE=$(date +%Y%m%d_%H%M%S)

pg_dump -U $DB_USER -d $DB_NAME \
  --format=custom \
  --compress=9 \
  --file="$BACKUP_DIR/$DB_NAME-$DATE.dump"

# Manter apenas últimos 7 dias
find $BACKUP_DIR -name "*.dump" -mtime +7 -delete

echo "Backup concluído: $DB_NAME-$DATE.dump"
```

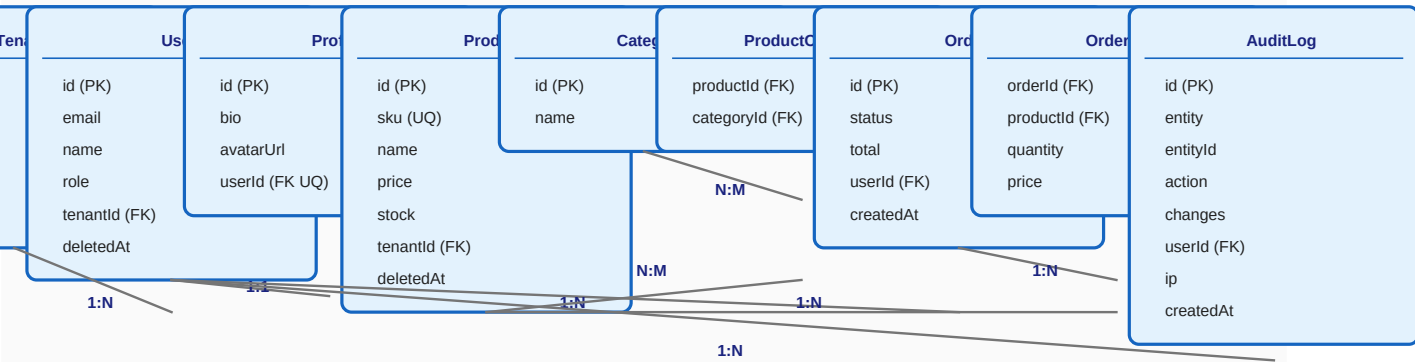
Restore

```
# Restore completo
pg_restore -U $DB_USER -d $DB_NAME \
  --clean \
  --if-exists \
```

```
"$BACKUP_DIR/app-20260601_000000.dump"
```

```
# Restore completo
pg_restore -U $DB_USER -d $DB_NAME \
--clean \
--if-exists \
```


Modelo Entidade-Relacionamento Enterprise



```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
```



```

model OrderItem {
  id          String  @id @default(cuid())
  orderId     String
  order       Order   @relation(fields: [orderId], references: [id])
  productId   String
  product     Product @relation(fields: [productId], references: [id])
  quantity    Int
  price       Decimal @db.Decimal(10, 2)

  @@index([orderId])
  @@index([productId])
  @@map("order_items")
}

```

```

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

enum UserRole {
  ADMIN
  MANAGER
  USER
}

```

```

enum OrderStatus {
  PENDING
  CONFIRMED
  SHIPPED
  DELIVERED
  CANCELLED
}

```

```

model Tenant {
  id          String  @id @default(cuid())
  slug        String  @unique
  name        String
  users       User[]
  createdAt   DateTime @default(now())

  @@map("tenants")
}

```

```

model User {
  id          String  @id @default(cuid())
  email       String  @unique
}

```

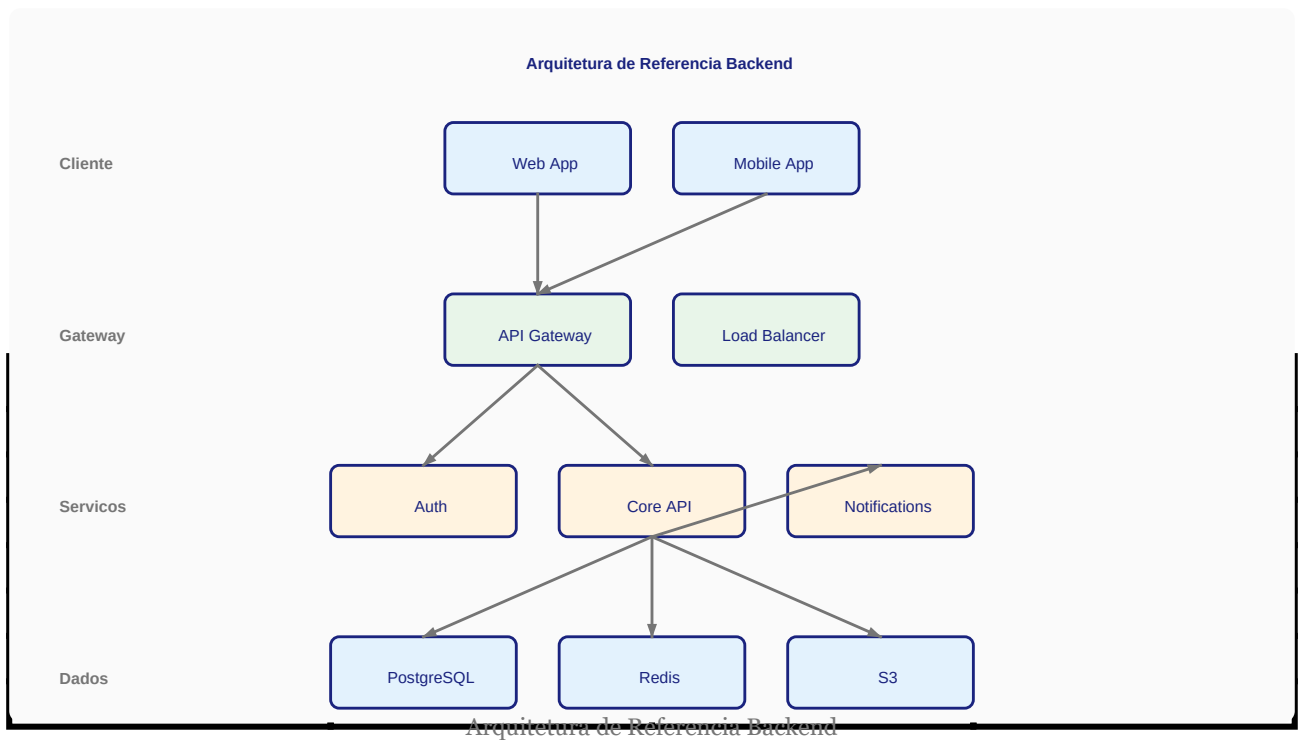

Resumo

1. **Modelagem é a fundação** — erros aqui são os mais caros 2. **1:1, 1:N, N:M** — conheça os 3 tipos de relacionamento 3. **Soft Delete** — nunca delete dados (deletedAt) 4. **Audit Trail** — toda ação importante deve ser registrada 5. **Índices** — toda FK precisa de índice; índices compostos para filtros comuns 6. **Eager Loading** — previne N+1 7. **Migrações seguras** — expand-migrate-contract para mudanças sem downtime 8. **Backup** — full + incremental + WAL; testar restore periodicamente

Desenvolvimento Backend

Módulo 10 — Backend: APIs Enterprise com NestJS

Construindo backends robustos, testáveis e seguros.



Módulo NestJS típico

```
@Module({
  imports: [PrismaModule, HttpModule],
  controllers: [UserController],
  providers: [UserService, UserRepository, JwtStrategy],
  exports: [UserService],
})
export class UserModule {}
```

2. Estrutura de módulos

Organização por domínio

```
src/  
├── modules/  
│   ├── users/  
│   │   ├── user.module.ts  
│   │   ├── user.controller.ts  
│   │   ├── user.service.ts  
│   │   ├── user.repository.ts  
│   │   ├── dto/  
│   │   │   ├── create-user.dto.ts  
│   │   │   └── update-user.dto.ts  
│   │   └── entities/  
│   │       └── user.entity.ts  
│   ├── orders/  
│   │   └── ...  
│   └── payments/  
│       └── ...  
├── common/  
│   ├── guards/  
│   ├── interceptors/  
│   ├── pipes/  
│   └── filters/  
├── config/  
│   └── app.config.ts  
└── main.ts
```

Por que essa estrutura?

Separação por domínio:

- Tudo relacionado a "users" está junto
- Fácil de navegar e manter
- Cada módulo é autocontido

Módulo raiz (AppModule):

- Importa os módulos de domínio
- Tempo de inicialização mais rápido
- Testes mais isolados

3. Controllers, Services, Repositories

Camadas

```
Controller (Rota)
  ↓ chamada
Service (Lógica de negócio)
  ↓ chamada
Repository (Persistência)
  ↓
Database (Prisma)
```

Controller

```
@Controller('users')
@UseGuards(JwtAuthGuard)
export class UserController {
  constructor(private readonly userService: UserService) {}

  @Post()
  @ApiOperation({ summary: 'Criar usuário' })
  @ApiResponse({ status: 201, type: UserResponse })
  async create(@Body() dto: CreateUserDto) {
    return this.userService.create(dto);
  }

  @Get(':id')
  async findOne(@Param('id', ParseUUIDPipe) id: string) {
    return this.userService.findById(id);
  }

  @Get()
  async findAll(
    @Query('page', ParseIntPipe) page: number = 1,
    @Query('limit', ParseIntPipe) limit: number = 10,
  ) {
    return this.userService.findAll({ page, limit });
  }

  @Patch(':id')
  async update(
    @Param('id', ParseUUIDPipe) id: string,
    @Body() dto: UpdateUserDto,
  ) {
    return this.userService.update(id, dto);
  }
}
```

```

@Delete('/:id')
@HttpCode(HttpStatus.NO_CONTENT)
async remove(@Param('id', ParseUUIDPipe) id: string) {
  await this.userService.softDelete(id);
}
}

```



```

@Controller('users')
@UseGuards(JwtAuthGuard)
export class UserController {
  constructor(private readonly userService: UserService) {}

  @Post()
  @ApiOperation({ summary: 'Criar usuário' })
  @ApiResponse({ status: 201, type: UserResponse })
  async create(@Body() dto: CreateUserDto) {
    return this.userService.create(dto);
  }

  @Get('/:id')
  async findOne(@Param('id', ParseUUIDPipe) id: string) {
    return this.userService.findById(id);
  }

  @Get()
  async findAll(
    @Query('page', ParseIntPipe) page: number = 1,
    @Query('limit', ParseIntPipe) limit: number = 10,
  ) {
    return this.userService.findAll({ page, limit });
  }

  @Patch('/:id')
  async update(
    @Param('id', ParseUUIDPipe) id: string,
    @Body() dto: UpdateUserDto,
  ) {
    return this.userService.update(id, dto);
  }
}

```

```

@Delete('/:id')
@HttpCode(HttpStatus.NO_CONTENT)
async remove(@Param('id', ParseUUIDPipe) id: string) {
  await this.userService.softDelete(id);
}
}

```

Service

```

@Injectable()
export class UserService {
  constructor(
    private readonly userRepo: UserRepository,
    private readonly emailService: EmailService,
  ) {}

  async create(dto: CreateUserDto): Promise<UserResponse> {
    const email = new Email(dto.email);

    const exists = await this.userRepo.findByEmail(email.toString());
    if (exists) throw new ConflictException('Email já cadastrado');

    const hashedPassword = await bcrypt.hash(dto.password, 12);
    const user = User.create({
      name: dto.name,
      email: email,
      password: hashedPassword,
    });

    await this.userRepo.save(user);
    await this.emailService.sendWelcome(user.email.toString());

    return UserResponse.from(user);
  }

  async findById(id: string): Promise<UserResponse> {
    const user = await this.userRepo.findById(id);
    if (!user) throw new NotFoundException('Usuário não encontrado');
    return UserResponse.from(user);
  }

  async findAll(pagination: PaginationInput): Promise<PaginatedResult<UserResponse>> {
    return this.userRepo.findAll(pagination);
  }
}

```

Repository


```
return User.restore(  
  raw.id,  
  raw.name,  
  Email.restore(raw.email),  
  raw.password,  
  raw.role as UserRole,  
);  
}  
  
private toPersistence(user: User): PrismaUserCreateInput {  
  return {  
    id: user.id,  
    name: user.name,  
    email: user.email.toString(),  
    password: user.password,  
    role: user.role,  
  };  
}  
}
```


4. DTOs e Validação com Zod

DTOs (Data Transfer Objects)

```
import { z } from 'zod';

export const CreateUserSchema = z.object({
  name: z.string().min(3).max(100),
  email: z.string().email(),
  password: z.string().min(8).regex(
    /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)/,
    'Senha deve conter maiúscula, minúscula e número'
  ),
  role: z.enum(['admin', 'manager', 'user']).default('user'),
});

export type CreateUserDto = z.infer<typeof CreateUserSchema>;

export class CreateUserPipe implements PipeTransform {
  transform(value: unknown) {
    const result = CreateUserSchema.safeParse(value);
    if (!result.success) {
      throw new BadRequestException({
        message: 'Dados inválidos',
        errors: result.error.flatten().fieldErrors,
      });
    }
    return result.data;
  }
}
```

Uso no controller

```
@Post()
async create(@Body(new CreateUserPipe()) dto: CreateUserDto) {
  return this.userService.create(dto);
}
```

Validação vs Sanitização

```
// Validação: rejeitar dados inválidos
password: z.string().min(8)

// Sanitização: transformar dados
```



```
email: z.string().email().transform(v => v.toLowerCase()),  
name: z.string().trim().min(3),
```

```
// Validação: rejeitar dados inválidos  
password: z.string().min(8)  
  
// Sanitização: transformar dados
```


5. Tratamento de Erros

Exception Filters

```
@Catch()
export class GlobalExceptionHandler implements ExceptionFilter {
  catch(exception: unknown, host: ArgumentsHost) {
    const ctx = host.switchToHttp();
    const response = ctx.getResponse<Response>();

    if (exception instanceof DomainError) {
      return response.status(400).json({
        statusCode: 400,
        error: exception.code,
        message: exception.message,
      });
    }

    if (exception instanceof HttpException) {
      return response.status(exception.getStatus()).json({
        statusCode: exception.getStatus(),
        error: exception.name,
        message: exception.message,
      });
    }

    // Erro não mapeado – log e retorno genérico
    console.error('Erro não tratado:', exception);
    return response.status(500).json({
      statusCode: 500,
      error: 'Internal Server Error',
      message: 'Erro interno do servidor',
    });
  }
}
```

Domain Errors

```
export abstract class DomainError extends Error {
  abstract readonly code: string;
}

export class EmailAlreadyExistsError extends DomainError {
  readonly code = 'EMAIL_ALREADY_EXISTS';
  constructor() { super('Email já cadastrado'); }
}
```

```
export class InsufficientBalanceError extends DomainError {  
  readonly code = 'INSUFFICIENT_BALANCE';  
  constructor() { super('Saldo insuficiente'); }  
}
```

```
export class InvalidEmailError extends DomainError {  
  readonly code = 'INVALID_EMAIL';  
  constructor() { super('Formato de email inválido'); }  
}
```

```
export abstract class DomainError extends Error {  
  abstract readonly code: string;  
}
```

```
export class EmailAlreadyExistsError extends DomainError {  
  readonly code = 'EMAIL_ALREADY_EXISTS';  
  constructor() { super('Email já cadastrado'); }  
}
```


6. Interceptors e Guards

Interceptors (transformação de resposta)

```
@Injectable()
export class TransformInterceptor<T> implements NestInterceptor<T, ApiResponse<T>> {
  intercept(context: ExecutionContext, next: CallHandler): Observable<ApiResponse<T>> {
    return next.handle().pipe(
      map(data => ({
        success: true,
        data,
        timestamp: new Date().toISOString(),
      })),
    );
  }
}

// Uso global
app.useGlobalInterceptors(new TransformInterceptor());
```

Guards (proteção de rotas)

```
@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private readonly requiredRoles: string[]) {}

  canActivate(context: ExecutionContext): boolean {
    const request = context.switchToHttp().getRequest();
    const user = request.user;

    if (!user) return false;
    return this.requiredRoles.includes(user.role);
  }
}

// Uso no controller
@Delete(':id')
@UseGuards(new RolesGuard(['admin']))
async remove(@Param('id') id: string) {
  return this.userService.softDelete(id);
}
```

7. Paginação, Filtros e Ordenação

Paginação cursor-based (recomendada para scale)

```
interface CursorPaginationInput {
  cursor?: string; // ID do último item
  limit: number;   // Itens por página
}

interface CursorPaginatedResult<T> {
  items: T[];
  nextCursor: string | null;
  hasMore: boolean;
}

// Implementação
async findAll(input: CursorPaginationInput): Promise<CursorPaginatedResult<User>> {
  const items = await this.prisma.user.findMany({
    take: input.limit + 1, // Pega um a mais para saber se tem próxima
    cursor: input.cursor ? { id: input.cursor } : undefined,
    orderBy: { createdAt: 'desc' },
  });

  const hasMore = items.length > input.limit;
  if (hasMore) items.pop();

  return {
    items: items.map(this.toDomain),
    nextCursor: hasMore ? items[items.length - 1].id : null,
    hasMore,
  };
}
```

8. Cache com Redis

Cache de respostas

```
@Injectable()
export class CacheService {
  constructor(private readonly redis: Redis) {}

  async getOrSet<T>(key: string, ttl: number, fetcher: () => Promise<T>): Promise<T> {
    const cached = await this.redis.get(key);
    if (cached) return JSON.parse(cached);

    const data = await fetcher();
    await this.redis.set(key, JSON.stringify(data), 'EX', ttl);
    return data;
  }

  async invalidate(pattern: string): Promise<void> {
    const keys = await this.redis.keys(pattern);
    if (keys.length > 0) await this.redis.del(keys);
  }
}

// Uso
async findById(id: string): Promise<UserResponse> {
  return this.cacheService.getOrSet(
    `user:${id}`,
    300, // 5 minutos
    () => this.fetchUser(id),
  );
}
```


9. Health Checks

```
@Controller('health')
export class HealthController {
  constructor(
    private prisma: PrismaService,
    private redis: RedisService,
  ) {}

  @Get()
  async check() {
    const checks = await Promise.all([
      this.checkDatabase(),
      this.checkRedis(),
    ]);

    const allHealthy = checks.every(c => c.status === 'healthy');

    return {
      status: allHealthy ? 'healthy' : 'degraded',
      timestamp: new Date().toISOString(),
      checks,
    };
  }

  private async checkDatabase() {
    try {
      await this.prisma.$queryRaw`SELECT 1`;
      return { name: 'database', status: 'healthy' };
    } catch {
      return { name: 'database', status: 'unhealthy' };
    }
  }

  private async checkRedis() {
    try {
      await this.redis.ping();
      return { name: 'redis', status: 'healthy' };
    } catch {
      return { name: 'redis', status: 'unhealthy' };
    }
  }
}
```

10. Testes

```
// Teste de service
describe('UserService', () => {
  let service: UserService;
  let repo: jest.Mocked<UserRepository>;
  let emailService: jest.Mocked<EmailService>;

  beforeEach(async () => {
    repo = { findById: jest.fn(), save: jest.fn() } as any;
    emailService = { sendWelcome: jest.fn() } as any;
    service = new UserService(repo, emailService);
  });

  describe('create', () => {
    it('deve criar usuário com sucesso', async () => {
      const dto = { name: 'João', email: 'joao@email.com', password: 'Senha123' };
      repo.findByIdEmail.mockResolvedValue(null);

      const result = await service.create(dto);

      expect(repo.save).toHaveBeenCalled();
      expect(emailService.sendWelcome).toHaveBeenCalled();
      expect(result).toBeDefined();
    });

    it('deve lançar erro se email já existe', async () => {
      repo.findByIdEmail.mockResolvedValue(createUser());

      await expect(service.create(mockDto)).rejects.toThrow(ConflictException);
    });
  });
});
```

Segurança

Módulo 12 – Segurança

****Protegendo sistemas Enterprise contra ameaças reais.****

1. Por que segurança é o requisito mais importante

Segurança não é uma feature – é um ****pré-requisito****. Um sistema inseguro é um passivo, não um ativo.

O custo de uma falha de segurança

```
// Teste de service
describe('UserService', () => {
  let service: UserService;
  let repo: jest.Mocked<UserRepository>;
  let emailService: jest.Mocked<EmailService>;

  beforeEach(async () => {
    repo = { findById: jest.fn(), save: jest.fn() } as any;
    emailService = { sendWelcome: jest.fn() } as any;
    service = new UserService(repo, emailService);
  });

  describe('create', () => {
    it('deve criar usuário com sucesso', async () => {
      const dto = { name: 'João', email: 'joao@email.com', password: 'Senha123' };
      repo.findByIdEmail.mockResolvedValue(null);

      const result = await service.create(dto);

      expect(repo.save).toHaveBeenCalled();
      expect(emailService.sendWelcome).toHaveBeenCalled();
      expect(result).toBeDefined();
    });

    it('deve lançar erro se email já existe', async () => {
      repo.findByIdEmail.mockResolvedValue(createUser());

      await expect(service.create(mockDto)).rejects.toThrow(ConflictException);
    });
  });
});
```

Segurança

Módulo 12 – Segurança

****Protegendo sistemas Enterprise contra ameaças reais.****

1. Por que segurança é o requisito mais importante

Segurança não é uma feature – é um ****pré-requisito****. Um sistema inseguro é um passivo, não um ativo.

O custo de uma falha de segurança

Falha de segurança típica:

clientes					Dados vazados
			Multas regulatórias (LGPD: até 2% do fat.)		Perda de confiança dos
			Custos de remediação (R\$ 200k-2M médio)		Tempo de inatividade

Custo de prevenir:

				Treinamento do time: R\$ 5k		Ferramentas de segurança: R\$ 1k/mês		
Revisão	de	código:	parte	do	processo			

Mindset de segurança

"Segurança é problema do DevOps"

"Depois a gente adiciona segurança"

"Ninguém vai atacar nosso sistema"

"Segurança é responsabilidade de todos"

"Segurança é parte da definição de 'pronto'"

"Se tem valor, vai ser atacado"

2. OWASP Top 10 (2021)

As 10 vulnerabilidades mais críticas em aplicações web.

1. Broken Access Control

Usuário acessa recursos que não deveria.

```
// Ruim: Não verifica se o recurso pertence ao usuário
@Get('/orders/:id')
async getOrder(@Param('id') id: string) {
  return this.orderRepo.findById(id); // qualquer um vê qualquer pedido!
}

// Bom: Verifica propriedade
@Get('/orders/:id')
async getOrder(@Param('id') id: string, @Req() req) {
  const order = await this.orderRepo.findById(id);
  if (order.userId !== req.user.id) {
    throw new ForbiddenException();
  }
  return order;
}
```

2. Cryptographic Failures

Dados sensíveis expostos ou mal protegidos.

```
// Ruim: Senha em texto puro
const user = await prisma.user.create({
  data: { password: req.body.password } //
});

// Bom: Hash com bcrypt
const hashedPassword = await bcrypt.hash(req.body.password, 12);
const user = await prisma.user.create({
  data: { password: hashedPassword }
});
```

3. Injection

SQL, NoSQL, OS command injection.

```
// Ruim: Query concatenada (SQL injection!)
const users = await prisma.$queryRawUnsafe(
  `SELECT * FROM users WHERE email = '${email}'`
);

// Bom: Query parametrizada (Prisma previne injection)
const user = await prisma.user.findUnique({
  where: { email }
});
```

```
// Ruim: Query concatenada (SQL injection!)
const users = await prisma.$queryRawUnsafe(
  `SELECT * FROM users WHERE email = '${email}'`
);

// Bom: Query parametrizada (Prisma previne injection)
const user = await prisma.user.findUnique({
  where: { email }
});
```

4. Insecure Design

Falhas no design que permitem ataques.

```
// Ruim: Rate limit não implementado
@Post('/login')
async login(@Body() dto: LoginDto) {
  // Tentativas ilimitadas – brute force!
}

// Bom: Rate limit com throttling
@Throttle(5, 60) // 5 tentativas por minuto
@Post('/login')
async login(@Body() dto: LoginDto) {
  // ...
}
```

5. Security Misconfiguration

Configurações padrão inseguras.

```
// Ruim: CORS aberto
app.enableCors(); // permite qualquer origem!

// Bom: CORS restrito
app.enableCors({
  origin: ['https://app.meusistema.com'],
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
});
```

6-10: Vulnerable Components, Auth Failures, Data Integrity, Logging, SSRF

#	Vulnerabilidade	Prevenção
6	Vulnerable Components	npm audit, Dependabot, ma

7	Identification/Auth Failu	JWT com expiração, MFA, s
8	Data Integrity Failures	Assinatura de dados, vali
9	Security Logging/Monitori	Logs estruturados, alerta
10	SSRF	Validar URLs, restringir

3. Autenticação com JWT

Fluxo completo

1. Usuário envia email + senha
2. Servidor valida credenciais
3. Servidor gera ACCESS TOKEN (15min) + REFRESH TOKEN (7d)
4. Cliente armazena e envia access token em requisições
5. Servidor valida token em cada requisição (AuthGuard)
6. Quando access token expira, cliente usa refresh token para obter novo

Implementação

```
// auth.service.ts
@Injectable()
export class AuthService {
  constructor(
    private userRepo: UserRepository,
    private jwtService: JwtService,
  ) {}

  async login(dto: LoginDto): Promise<AuthTokens> {
    const user = await this.userRepo.findByEmail(dto.email);
    if (!user) throw new UnauthorizedException('Credenciais inválidas');

    const passwordValid = await bcrypt.compare(dto.password, user.password);
    if (!passwordValid) throw new UnauthorizedException('Credenciais inválidas');

    return this.generateTokens(user);
  }

  private async generateTokens(user: User): Promise<AuthTokens> {
    const payload = { sub: user.id, email: user.email, role: user.role };

    const accessToken = await this.jwtService.signAsync(payload, {
      expiresIn: '15m',
    });

    const refreshToken = await this.jwtService.signAsync(payload, {
      expiresIn: '7d',
      secret: process.env.JWT_REFRESH_SECRET,
    });

    // Armazenar refresh token (para revogação)
    await this.tokenRepo.save(user.id, refreshToken);
  }
}
```

```

    return { accessToken, refreshToken };
}

async refresh(refreshToken: string): Promise<AuthTokens> {
    try {
        const payload = await this.jwtService.verifyAsync(refreshToken, {
            secret: process.env.JWT_REFRESH_SECRET,
        });

        // Verificar se token ainda é válido (não foi revogado)
        const stored = await this.tokenRepo.find(payload.sub, refreshToken);
        if (!stored) throw new UnauthorizedException('Token revogado');

        // Revogar token antigo (rotação)
        await this.tokenRepo.delete(payload.sub, refreshToken);

        const user = await this.userRepo.findById(payload.sub);
        return this.generateTokens(user);
    }
}

```

```

// auth.service.ts
@Injectable()
export class AuthService {
    constructor(
        private userRepo: UserRepository,
        private jwtService: JwtService,
    ) {}

    async login(dto: LoginDto): Promise<AuthTokens> {
        const user = await this.userRepo.findByEmail(dto.email);
        if (!user) throw new UnauthorizedException('Credenciais inválidas');

        const passwordValid = await bcrypt.compare(dto.password, user.password);
        if (!passwordValid) throw new UnauthorizedException('Credenciais inválidas');

        return this.generateTokens(user);
    }

    private async generateTokens(user: User): Promise<AuthTokens> {
        const payload = { sub: user.id, email: user.email, role: user.role };

        const accessToken = await this.jwtService.signAsync(payload, {
            expiresIn: '15m',
        });

        const refreshToken = await this.jwtService.signAsync(payload, {
            expiresIn: '7d',
            secret: process.env.JWT_REFRESH_SECRET,
        });

        // Armazenar refresh token (para revogação)
        await this.tokenRepo.save(user.id, refreshToken);
    }
}

```


4. Autorização com CASL (RBAC)

Definição de abilities

```
// abilities.ts
export type Action = 'manage' | 'create' | 'read' | 'update' | 'delete';
export type Subject = 'User' | 'Order' | 'Product' | 'Report' | 'all';

export function defineAbilitiesFor(user: User): PureAbility {
  return AbilityBuilder.define((can, cannot) => {
    // Admin pode fazer tudo
    if (user.role === 'admin') {
      can('manage', 'all');
      return;
    }

    // Usuário comum
    can('read', 'User', { id: user.id }); // só próprio perfil
    can('read', 'Order', { userId: user.id }); // só seus pedidos
    can('create', 'Order'); // criar pedidos
    can('update', 'Order', { userId: user.id, status: 'pending' });

    // Gerente pode ver relatórios
    if (user.role === 'manager') {
      can('read', 'Report');
      can('read', 'User', { tenantId: user.tenantId });
    }

    // Ninguém pode deletar
    cannot('delete', 'all');
  });
}
```

Uso no controller

```
@Post('/orders')
async create(@Body() dto: CreateOrderDto, @Req() req) {
  const ability = defineAbilitiesFor(req.user);
  ForbiddenError.from(ability).throwUnlessCan('create', 'Order');
  return this.orderService.create(dto, req.user.id);
}

@Delete('/orders/:id')
async delete(@Param('id') id: string, @Req() req) {
  const ability = defineAbilitiesFor(req.user);
  ForbiddenError.from(ability).throwUnlessCan('delete', 'Order');
```

```
// Nunca chega aqui – admin também não pode deletar  
}
```

```
@Post('/orders')  
async create(@Body() dto: CreateOrderDto, @Req() req) {  
  const ability = defineAbilitiesFor(req.user);  
  ForbiddenError.from(ability).throwUnlessCan('create', 'Order');  
  return this.orderService.create(dto, req.user.id);  
}
```

```
@Delete('/orders/:id')  
async delete(@Param('id') id: string, @Req() req) {  
  const ability = defineAbilitiesFor(req.user);  
  ForbiddenError.from(ability).throwUnlessCan('delete', 'Order');
```


5. Rate Limiting

Protege contra brute force, DDoS e abuso.

Implementação com @nestjs/throttler

```
// app.module.ts
@Module({
  imports: [
    ThrottlerModule.forRoot([
      {
        ttl: 60000,      // 1 minuto
        limit: 60,       // 60 requisições por minuto (global)
      }
    ]),
  ],
})

// Uso em endpoints específicos
@Throttle(5, 60) // 5 tentativas por minuto
@Post('/login')
async login(@Body() dto: LoginDto) {
  // ...
}

@Throttle(3, 60) // 3 tentativas por minuto
@Post('/password-reset')
async requestPasswordReset(@Body() dto: ResetDto) {
  // ...
}
```

Estratégias adicionais

Rate Limiting por:

IP:	limitar por endereço IP
Usuário:	limitar por user ID
Rota:	limites diferentes por endpoint
Global:	limite total de requisições

Respostas:

429 Too Many Requests
Header: Retry-After: X segundos

6. Headers de Segurança (Helmet + CSP)

Helmet (NestJS)

```
import helmet from 'helmet';

app.use(helmet());

// Configura automaticamente:
// Content-Security-Policy
// X-Content-Type-Options: nosniff
// X-Frame-Options: DENY
// X-XSS-Protection: 0
// Strict-Transport-Security
// Referrer-Policy
```

CSP (Content Security Policy)

```
app.use(helmet.contentSecurityPolicy({
  directives: {
    defaultSrc: ["'self'"],
    scriptSrc: ["'self'", "https://cdn.example.com"],
    styleSrc: ["'self'", "'unsafe-inline'"],
    imgSrc: ["'self'", "https://images.example.com", "data:"],
    connectSrc: ["'self'", "https://api.example.com"],
    fontSrc: ["'self'", "https://fonts.googleapis.com"],
    objectSrc: ["'none'"],
    upgradeInsecureRequests: [],
  },
}));
```


7. Proteções Específicas

SQL Injection

```
// Nunca fazer
const users = await prisma.$queryRawUnsafe(`SELECT * FROM users WHERE email = '${email}'`);

// Prisma previne com query builders
const user = await prisma.user.findUnique({ where: { email } });

// Se precisar de raw query, usar parametrização
const users = await prisma.$queryRaw`SELECT * FROM users WHERE email = ${email}`;
```

XSS (Cross-Site Scripting)

```
// Renderizar HTML sem sanitizar
<div dangerouslySetInnerHTML={{ __html: userComment }} />

// Sanitizar antes de renderizar
import DOMPurify from 'isomorphic-dompurify';
<div dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(userComment) }} />

// No backend, escapar output
const sanitized = escapeHtml(userComment);
```

CSRF (Cross-Site Request Forgery)

```
// NestJS com CSRF protection
import * as csrf from 'csrf';

app.use(csrf({ cookie: true }));

// Enviar token CSRF em formulários/headers
// <meta name="csrf-token" content="{{csrfToken}}">
// Header: X-CSRF-Token
```

8. Secrets Management

O que NÃO fazer

```
// Hardcoded no código
const API_KEY = 'sk-1234567890abcdef';
const DB_PASSWORD = 'minha-senha-super-secreta';

// No .env comitado
.env
DB_PASSWORD=minha-senha

// No código fonte
config.service.apiKey = 'sk-1234567890abcdef';
```

O que fazer

```
# .env.example (commitado, sem valores reais)
DATABASE_URL=postgresql://user:password@localhost:5432/db
JWT_SECRET=change-me
API_KEY=change-me

# .env (NÃO commitado, .gitignore)
DATABASE_URL=postgresql://user:senha-real@produção:5432/db
JWT_SECRET=minha-chave-jwt-segura
API_KEY=sk-real-key

# Validação na inicialização
# main.ts
const requiredEnvVars = ['DATABASE_URL', 'JWT_SECRET', 'API_KEY'];
for (const varName of requiredEnvVars) {
  if (!process.env[varName]) {
    throw new Error(`Variável de ambiente ${varName} não configurada`);
  }
}
```

9. Auditoria de Segurança

O que auditar

```
// Log de ações sensíveis
@Injectable()
export class AuditService {
  async log(action: AuditAction): Promise<void> {
    await prisma.auditLog.create({
      data: {
        userId: action.userId,
        action: action.type,
        resource: action.resource,
        resourceId: action.resourceId,
        details: action.details,
        ip: action.ip,
        userAgent: action.userAgent,
        timestamp: new Date(),
      },
    });
  }
}

// Uso
@Post('/transfer')
async transfer(@Body() dto: TransferDto, @Req() req) {
  const result = await this.transferService.execute(dto);
  await this.auditService.log({
    userId: req.user.id,
    type: 'TRANSFER',
    resource: 'Account',
    resourceId: dto.fromAccountId,
    details: { to: dto.toAccountId, amount: dto.amount },
    ip: req.ip,
    userAgent: req.headers['user-agent'],
  });
  return result;
}
```

Checklist de segurança para code review

Segurança em code review:

- ☐ Todos os inputs são validados?
- ☐ Autenticação em todas as rotas protegidas?
- ☐ Autorização verifica propriedade do recurso?
- ☐ Senhas com hash (bcrypt, não MD5/SHA1)?

- [] Sem segredos no código?
- [] Rate limiting nos endpoints críticos?
- [] CORS configurado (não * em produção)?
- [] SQL injection prevenido (ORM)?
- [] Headers de segurança configurados?
- [] Logs não expõem dados sensíveis?

Arquitetura Multi-Tenant

Módulo 13 – Multi-Tenant: Construindo SaaS Escalável

****Como isolar dados, identidade e recursos entre clientes em uma única aplicação.****

1. O que é Multi-Tenancy?

Multi-tenancy é um padrão arquitetural onde ****uma única instância de software atende múltiplos clientes (tenants)****, mantendo os dados de cada um logicamente isolados e invisíveis entre si.

Segurança em code review:

- [] Todos os inputs são validados?
- [] Autenticação em todas as rotas protegidas?
- [] Autorização verifica propriedade do recurso?
- [] Senhas com hash (bcrypt, não MD5/SHA1)?

- [] Sem segredos no código?
- [] Rate limiting nos endpoints críticos?
- [] CORS configurado (não * em produção)?
- [] SQL injection prevenido (ORM)?
- [] Headers de segurança configurados?
- [] Logs não expõem dados sensíveis?

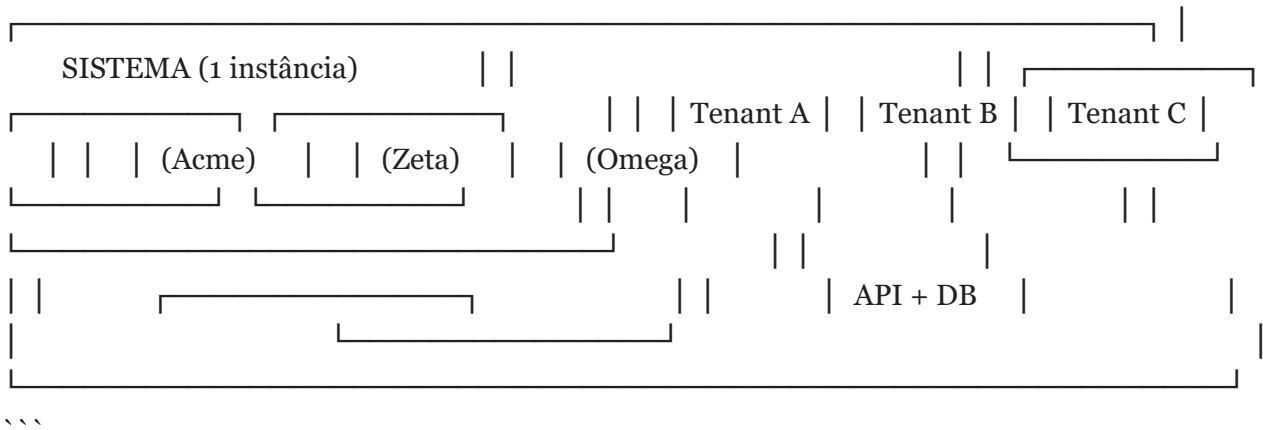
Arquitetura Multi-Tenant

Módulo 13 – Multi-Tenant: Construindo SaaS Escalável

****Como isolar dados, identidade e recursos entre clientes em uma única aplicação.****

1. O que é Multi-Tenancy?

Multi-tenancy é um padrão arquitetural onde **uma única instância de software atende múltiplos clientes (tenants)**, mantendo os dados de cada um logicamente isolados e invisíveis entre si.



...

1.1 Conceito

Diferente de **single-tenancy** (uma instância por cliente), no multi-tenancy o código e a infraestrutura são compartilhados. Os dados são separados por regras de negócio e consultas, não por hardware.

Característica	Single-Tenant	Multi-Tenant
Instâncias	1 por cliente	1 para todos
Código	N versões	1 versão
Banco	N bancos	1 ou N (controlado)
Manutenção	N vezes	1 vez

Isolamento	Físico	Lógico / Físico
------------	--------	-----------------

1.2 Por que usar?

Motivo	Descrição
Custo	Compartilhar infraestrutura
Manutenção	Uma única base de código
Escalabilidade	Adicionar novos tenants
Time-to-market	Novo cliente entra em min
Atualizações	Todos os clientes recebem

1.3 Exemplos reais (SaaS)

- ****Slack**** — cada workspace é um tenant; canais, mensagens e arquivos são isolados - ****Shopify**** — cada loja é um tenant; produtos, pedidos e clientes nunca se misturam - ****Notion**** — workspaces da equipe com páginas e permissões isoladas - ****GitHub**** — organizações e repositórios como unidades de isolamento - ****Salesforce**** — o pioneer do multi-tenancy enterprise; cada cliente (org) é um tenant

1.4 Quando NÃO usar?

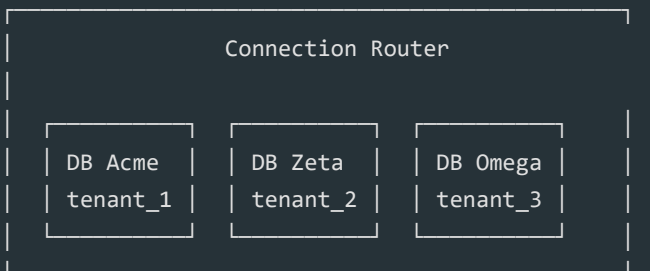
- Clientes exigem compliance físico (dados não podem co-habitar servidor) - Poucos clientes com volumes massivos de dados - Clientes exigem versões diferentes do software - O custo de implementar isolamento lógico supera o de rodar N instâncias

2. Abordagens de Isolamento

Existem três estratégias principais para isolar dados entre tenants.

2.1 Database per Tenant

Cada tenant tem **seu próprio banco de dados**. O roteador de conexão decide qual banco usar com base no tenant identificado.



```
// Router de conexão com lazy initialization
import { Pool } from 'pg';

const tenantDatabases: Record<string, string> = {
  acme: 'postgresql://user:pass@host:5432/acme_db',
  zeta: 'postgresql://user:pass@host:5432/zeta_db',
  omega: 'postgresql://user:pass@host:5432/omega_db',
};

class DatabaseRouter {
  private pools = new Map<string, Pool>();

  getPool(tenantId: string): Pool {
    if (!this.pools.has(tenantId)) {
      const url = tenantDatabases[tenantId];
      if (!url) throw new Error(`Database not found for tenant: ${tenantId}`);

      this.pools.set(
        tenantId,
        new Pool({
          connectionString: url,
          max: 10,
          idleTimeoutMillis: 30000,
        })
      );
    }
    return this.pools.get(tenantId)!;
  }
}
```

****Vantagens:**** - ****Isolamento total**** — um tenant nunca vê dados do outro, mesmo com SQL injection - ****Backup/restore independente**** — pode restaurar um único tenant sem afetar outros - ****SLAs diferenciados**** — cada banco pode estar em máquinas de diferentes - ****Migração para instância dedicada**** — basta apontar o DNS para o banco existente - ****Compliance**** — atende requisitos LGPD/HIPAA que exigem separação física

****Desvantagens:**** - ****Custo**** — N bancos × licenciamento + infra; 1000 tenants = 1000 bancos - ****Conexões**** — N tenants × pool size pode estourar limites do PostgreSQL - ****Migrations**** — precisam rodar em N bancos; falha em um tenant bloqueia o deploy - ****Monitoramento**** — N bancos para monitorar, N alarmes para configurar - ****Complexidade operacional**** — gerenciar N conexões, N backups, N schemas

2.2 Schema per Tenant

Um banco de dados compartilhado, mas cada tenant tem seu ****próprio schema**** (namespace) dentro do banco.



```
-- Criar schema para novo tenant
CREATE SCHEMA IF NOT EXISTS tenant_acme;

-- Tabelas dentro do schema específico
CREATE TABLE tenant_acme.users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  email TEXT UNIQUE NOT NULL,
  role TEXT NOT NULL DEFAULT 'member',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE tenant_acme.orders (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id TEXT NOT NULL,
  total NUMERIC(10,2) NOT NULL,
  status TEXT NOT NULL DEFAULT 'pending',
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```



```
-- Query com schema dinâmico
SELECT * FROM tenant_acme.users WHERE email = 'joao@acme.com';
```

```
-- Criar schema para novo tenant
CREATE SCHEMA IF NOT EXISTS tenant_acme;

-- Tabelas dentro do schema específico
CREATE TABLE tenant_acme.users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  email TEXT UNIQUE NOT NULL,
  role TEXT NOT NULL DEFAULT 'member',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE tenant_acme.orders (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id TEXT NOT NULL,
  total NUMERIC(10,2) NOT NULL,
  status TEXT NOT NULL DEFAULT 'pending',
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- Query com schema dinâmico
SELECT * FROM tenant_acme.users WHERE email = 'joao@acme.com';
```

```
// Abstração para schema dinâmico
class SchemaTenantService {
  constructor(private tenantId: string) {}

  getSchemaName(): string {
    return `tenant_${this.tenantId}`;
  }

  async query<T = any>(sql: string, params?: any[]): Promise<T[]> {
    const schema = this.getSchemaName();
    // Substitui __SCHEMA__ pelo nome real do schema
    const finalSql = sql.replace(/__SCHEMA__/g, `${schema}`);
    const result = await pool.query(finalSql, params);
    return result.rows;
  }

  async createTenantSchema(): Promise<void> {
    const schema = this.getSchemaName();
    await pool.query(`CREATE SCHEMA IF NOT EXISTS "${schema}"`);

    // Rodar migrations para este schema
    for (const migration of MIGRATIONS) {
      const sql = migration.sql.replace(/__SCHEMA__/g, `${schema}`);
      await pool.query(sql);
    }
  }
}
```

****Vantagens:**** - ****Banco único**** — uma conexão, pool compartilhado, menos overhead - ****Backup centralizado**** — um dump = todos os tenants - ****Custo menor**** — uma licença de banco, uma máquina - ****Migrations mais simples**** — roda uma vez, altera múltiplos schemas - ****Transações entre schemas**** — possível (depende do banco)

****Desvantagens:**** - ****Isolamento por schema**** — não tão forte quanto DB separado - ****Ruídos between tenants**** — queries pesadas de um tenant consomem recursos de todos - ****Recuperação de desastre**** — restaurar um tenant específico requer restore do banco inteiro + cleanup - ****Limite de schemas**** — PostgreSQL suporta milhares, mas MySQL não tem schemas nativos - ****Conexão única**** — um bottleneck na aplicação se todos os tenants estiverem ativos

2.3 Shared Database (discriminador)

Um banco, um schema, e uma coluna `tenant\_id` em cada tabela como discriminador. O isolamento é ****puramente lógico****.

```
// Modelo único com tenant_id
interface User {
  id: string;
  tenant_id: string;      // ← discriminador obrigatório
  name: string;
  email: string;
  role: 'admin' | 'member';
  created_at: Date;
}

interface Order {
  id: string;
  tenant_id: string;      // ← discriminador obrigatório
  customer_name: string;
  total: number;
  status: 'pending' | 'paid' | 'cancelled';
  created_at: Date;
}
```

```
// Modelo único com tenant_id
interface User {
  id: string;
  tenant_id: string;      // ← discriminador obrigatório
  name: string;
  email: string;
  role: 'admin' | 'member';
  created_at: Date;
}

interface Order {
  id: string;
  tenant_id: string;      // ← discriminador obrigatório
  customer_name: string;
  total: number;
  status: 'pending' | 'paid' | 'cancelled';
  created_at: Date;
}
```

```
// Repositório que SEMPRE filtra por tenant
class UserRepository {
  constructor(private tenantId: string) {}

  async findAll(): Promise<User[]> {
    return pool.query<User>(
      'SELECT * FROM users WHERE tenant_id = $1 ORDER BY name',
      [this.tenantId]
    ).then(r => r.rows);
  }

  async findById(id: string): Promise<User | null> {
    const result = await pool.query<User>(
      'SELECT * FROM users WHERE id = $1 AND tenant_id = $2',
      [id, this.tenantId]
    );
    return result.rows[0] || null;
  }

  async create(data: Omit<User, 'id' | 'tenant_id' | 'created_at'>): Promise<User> {
    const result = await pool.query<User>(
      `INSERT INTO users (tenant_id, name, email, role)
      VALUES ($1, $2, $3, $4)
      RETURNING *`,
      [this.tenantId, data.name, data.email, data.role]
    );
    return result.rows[0];
  }
}
```

****Vantagens:**** - ****Máximo compartilhamento**** — um banco, uma conexão, custo mínimo -
****Operação simples**** — add tenant = INSERT em tabela `tenants` - ****Performance previsível****

— um pool, um plano de execução - **Migrations tradicionais** — funcionam como em app single-tenant - **Baixa latência** — sem roteamento de conexão

Desvantagens: - **Menor isolamento** — um ``SELECT`` sem ``WHERE tenant_id = ?`` expõe dados de TODOS os tenants - **SQL injection** — se um invasor consegue injetar SQL, consegue dados de todos - **Índices inchados** — a tabela cresce com todos os tenants, índices ficam grandes - **Backup/restore global** — não é possível restaurar um único tenant - **Difícil diferenciar SLAs** — todos competem pelos mesmos recursos

2.4 Comparação Detalhada

Critério	DB per Tenant	Schema per Tenant	Shared DB
Isolamento	Físico	Lógico (schema)	Lógico (linha)
Segurança			
Performance	(dedicado)	(compartilhado)	(índices grandes)
Custo	(caro)	(médio)	(barato)
Complexidade	(média)	(alta)	(baixa)
Migrations	(N execuções)	(N schemas)	(1 execução)
Backup/Restore	(individual)	(global)	(só global)
Escalabilidade			
Adicionar tenant	(criar DB)	(criar schema)	(INSERT)
Customização por tenant			

3. Análise Aprofundada por Dimensão

3.1 Segurança

****Database per Tenant:**** Um vazamento de dados de um tenant não afeta os outros. Se um cliente exige compliance específico (LGPD, HIPAA, SOC2), pode-se isolar completamente em nível físico. Ataques de SQL injection ficam contidos no banco do tenant.

****Schema per Tenant:**** Um invasor que ganha acesso ao banco pode potencialmente acessar múltiplos schemas. A segurança depende das permissões do usuário do banco (`GRANT USAGE ON SCHEMA`). Idealmente, o usuário da aplicação tem acesso apenas ao schema do tenant atual.

****Shared Database:**** Um SQL injection ou bug no `WHERE` expõe ****todos os dados de todos os tenants****. Requer: - Testes de isolamento obrigatórios (ver seção 13) - Code review com checklist específico - Row-Level Security (RLS) do PostgreSQL como camada extra

```
-- Row-Level Security no PostgreSQL
CREATE TABLE users (
  id UUID PRIMARY KEY,
  tenant_id TEXT NOT NULL,
  name TEXT NOT NULL,
  email TEXT NOT NULL
);

-- Habilitar RLS
ALTER TABLE users ENABLE ROW LEVEL SECURITY;

-- Política: usuário só vê linhas do seu tenant
CREATE POLICY tenant_isolation ON users
  USING (tenant_id = current_setting('app.tenant_id')::TEXT);

-- Na aplicação, antes de qualquer query:
await pool.query("SET app.tenant_id = 'acme'");
```

3.2 Performance

****Database per Tenant:**** Cada tenant tem pool isolado. Um tenant com queries pesadas (JOINS complexos, full scans) ****não degrada**** os outros. Ideal para clientes Enterprise com workloads imprevisíveis.

****Schema per Tenant:**** Pool compartilhado. Uma query `SELECT \* FROM tenant\_big.orders ORDER BY total DESC` pode consumir CPU e I/O, afetando os schemas `tenant\_a` e `tenant\_c`. Soluções: - Statement timeout (`SET statement\_timeout = '30s'`) - `pg\_terminate\_backend` para queries runaway - Resource queues (Citus, Yugabyte)

****Shared Database:**** Todos competem pelo mesmo pool e índices. Queries de um tenant afetam todos. Necessário: - Rate limiting por tenant - `pg\_stat\_statements` para monitorar queries pesadas - Índices parciais por tenant (se houver padrões distintos)

```
-- Índice parcial para tenant com padrão específico
CREATE INDEX idx_orders_acme_large
  ON orders (total DESC)
  WHERE tenant_id = 'acme' AND total > 10000;
```

3.3 Custo

Cenário	DB per Tenant	Schema per Tenant	Shared DB
10 tenants	10× RDS db.t3.medium	1× RDS db.r5.large	1× RDS db.r5.large
Custo/mês	~\$150	~\$50	~\$50
100 tenants	100× tiny	1× db.r5.xlarge	1× db.r5.xlarge
Custo/mês	~\$500	~\$200	~\$200
1000 tenants	Inviável	1× db.r5.2xlarge + shardi	1× db.r5.4xlarge
Custo/mês	—	~\$700	~\$1400

****Database per Tenant**** não escala financeiramente além de algumas dezenas de tenants, a menos que cada cliente pague um valor alto que justifique o custo.

3.4 Complexidade Operacional

Operação	DB per Tenant	Schema per Tenant	Shared DB
Adicionar tenant	Criar DB + rodar migratio	Criar schema + rodar migr	Inserir 1 linha
Backup	N backups (1 por DB)	1 backup do banco	1 backup do banco
Restore individual	Restaurar DB específico	Exportar schema, importar	Impossível
Restore total	N restores	1 restore	1 restore
Upgrade de schema	Roda em N bancos	1 script para N schemas	1 script
Rollback	Rollback em N bancos	1 rollback para N schemas	1 rollback
Migração de plano	Sobe de instância	Altera resource limits	Migra para schema/DB

4. Identificação do Tenant

O sistema precisa identificar **qual tenant está fazendo a requisição** antes de qualquer lógica de negócio.

4.1 Estratégias de Identificação

Estratégia	Exemplo	Segurança	Complexidade	Ideal para
Subdomínio	<code>acme.minhasaas.com</code>	Média	Média	Apps web públicas
Header HTTP	<code>X-Tenant-Id: acme</code>	Média	Baixa	APIs server-to-server
JWT Claim	<code>{ "tid": "acme" }</code>	Alta	Média	Apps autenticadas
Path Parameter	<code>/api/acme/users</code>	Baixa	Baixa	Desenvolvimento/debug
DNS + SSL	SNI com cert por tenant	Alta	Alta	Enterprise

4.2 Subdomínio

```
// tenant.extractor.ts
function extractTenantFromHost(host: string): string | null {
  // host: "acme.minhasaas.com" → "acme"
  // host: "localhost:3000" → "localhost" (não é tenant)
  const parts = host.split('.');
  if (parts.length < 3) return null; // sem subdomínio
  return parts[0];
}

// Em produção, validar contra lista de tenants permitidos
async function validateTenantSubdomain(
  host: string,
  tenantService: TenantService
): Promise<Tenant | null> {
  const subdomain = extractTenantFromHost(host);
  if (!subdomain) return null;
  return tenantService.findBySubdomain(subdomain);
}
```

4.3 Header HTTP

```
// Extração simples e direta
import { Request } from 'express';

const TENANT_HEADER = 'x-tenant-id';

function extractTenantFromHeader(req: Request): string | undefined {
```



```
const tenantId = req.headers[TENANT_HEADER];
if (Array.isArray(tenantId)) return tenantId[0];
return tenantId;
}
```

```
// Com validação de formato (slug)
const TENANT_SLUG_REGEX = /^[a-z0-9-]{3,50}$/;
```

```
function validateTenantSlug(slug: string): boolean {
  return TENANT_SLUG_REGEX.test(slug);
}
```

```
// Extração simples e direta
import { Request } from 'express';

const TENANT_HEADER = 'x-tenant-id';

function extractTenantFromHeader(req: Request): string | undefined {
```

```

    const tenantId = req.headers[TENANT_HEADER];
    if (Array.isArray(tenantId)) return tenantId[0];
    return tenantId;
}

// Com validação de formato (slug)
const TENANT_SLUG_REGEX = /^[a-z0-9-]{3,50}$/;

function validateTenantSlug(slug: string): boolean {
    return TENANT_SLUG_REGEX.test(slug);
}

```

4.4 JWT Claim

```

// Interface do payload
interface TenantJwtPayload {
    sub: string;    // user ID
    tid: string;    // tenant ID
    rol: string;    // role dentro do tenant
    exp: number;    // expiração
}

// Extrair tenant do JWT
import * as jwt from 'jsonwebtoken';

function extractTenantFromJwt(authHeader?: string): string | null {
    if (!authHeader) return null;

    const token = authHeader.replace('Bearer ', '');
    try {
        const payload = jwt.verify(
            token,
            process.env.JWT_SECRET!
        ) as TenantJwtPayload;

        return payload.tid || null;
    } catch {
        return null; // token inválido
    }
}

```

4.5 Path Parameter

```

// Útil para debug, mas não recomendado para produção
// Exemplo: GET /api/:tenant/users
@Get('/api/:tenant/users')
findUsers(@Param('tenant') tenantId: string) {

```

```
return this.userService.findAll(tenantId);  
}
```

```
// Útil para debug, mas não recomendado para produção  
// Exemplo: GET /api/:tenant/users  
@Get('/api/:tenant/users')  
findUsers(@Param('tenant') tenantId: string) {
```

```
    return this.userService.findAll(tenantId);  
}
```

4.6 Estratégia Combinada (Fallback)

```
function resolveTenant(req: Request): string {  
    // Prioridade: JWT → Header → Subdomínio → Path  
    return (  
        extractTenantFromJwt(req.headers.authorization) ??  
        extractTenantFromHeader(req) ??  
        extractTenantFromHost(req.hostname) ??  
        extractTenantFromPath(req.path) ??  
        (() => { throw new BadRequestException('Tenant não identificado'); })()  
    );  
}
```

5. Middleware de Tenant

5.1 Implementação com NestJS

```
// tenant.middleware.ts
import { Injectable, NestMiddleware, UnauthorizedException } from '@nestjs/common';
import { Request, Response, NextFunction } from 'express';
import * as jwt from 'jsonwebtoken';

interface TenantInfo {
  id: string;
  name: string;
  plan: 'free' | 'pro' | 'enterprise';
  active: boolean;
}

@Injectable()
export class TenantMiddleware implements NestMiddleware {
  constructor() {
    // Em produção, carregar de um cache (Redis) ou banco
    this.loadTenants();
  }

  private tenants = new Map<string, TenantInfo>();

  private loadTenants() {
    this.tenants.set('acme', {
      id: 'acme', name: 'Acme Inc.', plan: 'enterprise', active: true,
    });
    this.tenants.set('zeta', {
      id: 'zeta', name: 'Zeta Ltda.', plan: 'pro', active: true,
    });
    this.tenants.set('omega', {
      id: 'omega', name: 'Omega S.A.', plan: 'free', active: true,
    });
  }

  use(req: Request, res: Response, next: NextFunction) {
    const tenantId = this.extractTenantId(req);
    const tenant = this.tenants.get(tenantId);

    if (!tenant) {
      throw new UnauthorizedException(`Tenant "${tenantId}" não encontrado`);
    }

    if (!tenant.active) {
      throw new UnauthorizedException(`Tenant "${tenantId}" está inativo`);
    }
  }
}
```

```

}

// Anexa ao request para uso nos controllers
(req as any).tenant = tenant;
(req as any).tenantId = tenant.id;

next();

```

```

// tenant.middleware.ts
import { Injectable, NestMiddleware, UnauthorizedException } from '@nestjs/common';
import { Request, Response, NextFunction } from 'express';
import * as jwt from 'jsonwebtoken';

interface TenantInfo {
  id: string;
  name: string;
  plan: 'free' | 'pro' | 'enterprise';
  active: boolean;
}

@Injectable()
export class TenantMiddleware implements NestMiddleware {
  constructor() {
    // Em produção, carregar de um cache (Redis) ou banco
    this.loadTenants();
  }

  private tenants = new Map<string, TenantInfo>();

  private loadTenants() {
    this.tenants.set('acme', {
      id: 'acme', name: 'Acme Inc.', plan: 'enterprise', active: true,
    });
    this.tenants.set('zeta', {
      id: 'zeta', name: 'Zeta Ltda.', plan: 'pro', active: true,
    });
    this.tenants.set('omega', {
      id: 'omega', name: 'Omega S.A.', plan: 'free', active: true,
    });
  }

  use(req: Request, res: Response, next: NextFunction) {
    const tenantId = this.extractTenantId(req);
    const tenant = this.tenants.get(tenantId);

    if (!tenant) {
      throw new UnauthorizedException(`Tenant "${tenantId}" não encontrado`);
    }

    if (!tenant.active) {
      throw new UnauthorizedException(`Tenant "${tenantId}" está inativo`);
    }
  }
}

```

```

    }

    // Anexa ao request para uso nos controllers
    (req as any).tenant = tenant;
    (req as any).tenantId = tenant.id;

    next();
}

private extractTenantId(req: Request): string {
    const fromJwt = this.extractFromJwt(req.headers.authorization);
    if (fromJwt) return fromJwt;

    const fromHeader = req.headers['x-tenant-id'] as string;
    if (fromHeader) return fromHeader;

    const fromSubdomain = req.hostname.split('.')[0];
    if (fromSubdomain && fromSubdomain !== 'www' && fromSubdomain !== 'app') {
        return fromSubdomain;
    }

    throw new UnauthorizedException(
        'Identificação do tenant é obrigatória (JWT, X-Tenant-Id ou subdomínio)'
    );
}

private extractFromJwt(authorization?: string): string | null {
    if (!authorization) return null;
    try {
        const token = authorization.replace('Bearer ', '');
        const payload = jwt.verify(token, process.env.JWT_SECRET!) as any;
        return payload.tid || null;
    } catch {
        return null;
    }
}
}

```

5.2 Aplicação Global ou por Rota

```

// app.module.ts
import { Module, NestModule, MiddlewareConsumer } from '@nestjs/common';
import { TenantMiddleware } from './tenant/tenant.middleware';

@Module({
    imports: [TenantModule],
})
export class AppModule implements NestModule {
    configure(consumer: MiddlewareConsumer) {

```

```

    consumer
      .apply(TenantMiddleware)
      .exclude(
        'auth/(.*)',    // login não precisa de tenant
        'health',       // health check
        'public/(.*)',  // páginas públicas
      )
      .forRoutes('*');
  }
}

```

```

// app.module.ts
import { Module, NestModule, MiddlewareConsumer } from '@nestjs/common';
import { TenantMiddleware } from '../tenant/tenant.middleware';

@Module({
  imports: [TenantModule],
})
export class AppModule implements NestModule {
  configure(consumer: MiddlewareConsumer) {

```



```

    consumer
      .apply(TenantMiddleware)
      .exclude(
        'auth/(.*)', // login não precisa de tenant
        'health',    // health check
        'public/(.*)', // páginas públicas
      )
      .forRoutes('*');
  }
}

```

5.3 TenantModule

```

// tenant.module.ts
import { Global, Module } from '@nestjs/common';
import { TenantService } from './tenant.service';
import { TenantMiddleware } from './tenant.middleware';

@Global() // disponível em toda a aplicação
@Module({
  providers: [TenantService, TenantMiddleware],
  exports: [TenantService],
})
export class TenantModule {}

```

5.4 TenantService

```

// tenant.service.ts
import { Injectable, Scope } from '@nestjs/common';

interface Tenant {
  id: string;
  name: string;
  plan: 'free' | 'pro' | 'enterprise';
}

@Injectable({ scope: Scope.REQUEST })
export class TenantService {
  private tenant!: Tenant;

  setTenant(tenant: Tenant) {
    this.tenant = tenant;
  }

  getTenant(): Tenant {
    if (!this.tenant) {
      throw new Error('Tenant não configurado para esta requisição');
    }
  }
}

```

```

    }
    return this.tenant;
  }

  getTenantId(): string {
    return this.getTenant().id;
  }

  getPlan(): string {
    return this.getTenant().plan;
  }

  isEnterprise(): boolean {
    return this.getPlan() === 'enterprise';
  }
}

```

```

// tenant.service.ts
import { Injectable, Scope } from '@nestjsjs/common';

interface Tenant {
  id: string;
  name: string;
  plan: 'free' | 'pro' | 'enterprise';
}

@Injectable({ scope: Scope.REQUEST })
export class TenantService {
  private tenant!: Tenant;

  setTenant(tenant: Tenant) {
    this.tenant = tenant;
  }

  getTenant(): Tenant {
    if (!this.tenant) {
      throw new Error('Tenant não configurado para esta requisição');
    }
  }
}

```

```

    }
    return this.tenant;
  }

  getTenantId(): string {
    return this.getTenant().id;
  }

  getPlan(): string {
    return this.getTenant().plan;
  }

  isEnterprise(): boolean {
    return this.getPlan() === 'enterprise';
  }
}

```

5.5 @Tenant() Decorator

```

// tenant.decorator.ts
import { createParamDecorator, ExecutionContext } from '@nestjsjs/common';

export const Tenant = createParamDecorator(
  (data: keyof Tenant | undefined, ctx: ExecutionContext) => {
    const request = ctx.switchToHttp().getRequest();
    const tenant = request.tenant;

    if (!tenant) {
      throw new Error('Tenant não encontrado no request. TenantMiddleware está configurado?');
    }

    return data ? tenant[data] : tenant;
  }
);

// Uso no controller
@Get('users')
findAll(@Tenant() tenant: Tenant) {
  return this.userService.findAll(tenant.id);
}

@Get('plan')
getPlan(@Tenant('plan') plan: string) {
  return { plan };
}

```

5.6 AsyncLocalStorage para Contexto

```
// tenant-context.ts
import { AsyncLocalStorage } from 'async_hooks';

export interface TenantContext {
  tenantId: string;
  tenantName: string;
  plan: string;
}

export const tenantContext = new AsyncLocalStorage<TenantContext>();

// tenant-context.middleware.ts
import { Injectable, NestMiddleware } from '@nestjs/common';

@Injectable()
export class TenantContextMiddleware implements NestMiddleware {
  use(req: any, res: any, next: () => void) {
    const context: TenantContext = {
      tenantId: req.tenantId,
      tenantName: req.tenant?.name,
      plan: req.tenant?.plan,
    };

    tenantContext.run(context, () => next());
  }
}

// Uso em qualquer parte do código
import { tenantContext } from './tenant-context';

function getCurrentTenantId(): string {
  const ctx = tenantContext.getStore();
  if (!ctx) throw new Error('Fora de contexto de tenant');
  return ctx.tenantId;
}
```


6. Prisma Multi-Tenant

6.1 Schema per Tenant com Prisma

```
// schema.prisma - modelo base
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

// Modelo compartilhado (global)
model Tenant {
  id          String   @id @default(uuid())
  slug        String   @unique
  name        String
  plan        String   @default("free")
  active      Boolean  @default(true)
  createdAt   DateTime @default(now()) @map("created_at")
  @@map("tenants")
}

// Modelos por tenant (Prisma não suporta schema dinâmico nativamente)
// Solução: Client per tenant
model User {
  id          String   @id @default(uuid())
  name        String
  email       String   @unique
  role        String   @default("member")
  createdAt   DateTime @default(now()) @map("created_at")
  @@map("users")
}

model Project {
  id          String   @id @default(uuid())
  name        String
  createdAt   DateTime @default(now()) @map("created_at")
  @@map("projects")
}
```

```
// prisma-multi-tenant.ts
import { PrismaClient } from '@prisma/client';

class PrismaTenantManager {
```

```

private clients = new Map<string, PrismaClient>();

getClient(schema: string): PrismaClient {
  if (!this.clients.has(schema)) {
    const url = new URL(process.env.DATABASE_URL!);
    url.searchParams.set('schema', schema);

    const client = new PrismaClient({
      datasources: {
        db: { url: url.toString() },
      },
    });

    this.clients.set(schema, client);
  }
  return this.clients.get(schema)!;
}

async closeAll(): Promise<void> {
  for (const [schema, client] of this.clients) {
    await client.$disconnect();
  }
}

}

export const prismaTenantManager = new PrismaTenantManager();

```

```

// prisma-multi-tenant.ts
import { PrismaClient } from '@prisma/client';

class PrismaTenantManager {

```

```

private clients = new Map<string, PrismaClient>();

getClient(schema: string): PrismaClient {
  if (!this.clients.has(schema)) {
    const url = new URL(process.env.DATABASE_URL!);
    url.searchParams.set('schema', schema);

    const client = new PrismaClient({
      datasources: {
        db: { url: url.toString() },
      },
    });

    this.clients.set(schema, client);
  }
  return this.clients.get(schema)!;
}

async closeAll(): Promise<void> {
  for (const [schema, client] of this.clients) {
    await client.$disconnect();
  }
}

export const prismaTenantManager = new PrismaTenantManager();

```

6.2 Shared Database com Prisma

```

// schema.prisma - shared database
model User {
  id      String    @id @default(uuid())
  tenantId String    @map("tenant_id")
  name    String
  email   String
  role    String    @default("member")
  createdAt DateTime @default(now()) @map("created_at")

  @@index([tenantId])
  @@index([tenantId, email])
  @@map("users")
}

model Order {
  id      String    @id @default(uuid())
  tenantId String    @map("tenant_id")
  total   Decimal   @db.Decimal(10, 2)
  status  String    @default("pending")
  createdAt DateTime @default(now()) @map("created_at")
}

```



```
    @@index([tenantId, createdAt])
    @@map("orders")
}
```

```
// schema.prisma - shared database
model User {
  id          String    @id @default(uuid())
  tenantId    String    @map("tenant_id")
  name        String
  email       String
  role        String    @default("member")
  createdAt   DateTime  @default(now()) @map("created_at")

  @@index([tenantId])
  @@index([tenantId, email])
  @@map("users")
}

model Order {
  id          String    @id @default(uuid())
  tenantId    String    @map("tenant_id")
  total       Decimal   @db.Decimal(10, 2)
  status      String    @default("pending")
  createdAt   DateTime  @default(now()) @map("created_at")
}
```

```

    @@index([tenantId, createdAt])
    @@map("orders")
}

```

```

// tenant-aware.service.ts
import { Injectable } from '@nestjs/common';
import { PrismaService } from '../prisma/prisma.service';
import { TenantService } from './tenant.service';

@Injectable()
export class TenantAwareService {
  constructor(
    private prisma: PrismaService,
    private tenantService: TenantService,
  ) {}

  private get tenantId() {
    return this.tenantService.getTenantId();
  }

  async findUsers() {
    return this.prisma.user.findMany({
      where: { tenantId: this.tenantId },
    });
  }

  async createUser(data: { name: string; email: string; role?: string }) {
    return this.prisma.user.create({
      data: {
        ...data,
        tenantId: this.tenantId,
      },
    });
  }

  async findOrdersByDateRange(start: Date, end: Date) {
    return this.prisma.order.findMany({
      where: {
        tenantId: this.tenantId,
        createdAt: { gte: start, lte: end },
      },
      orderBy: { createdAt: 'desc' },
    });
  }
}

```

6.3 Prisma Middleware para Tenant

```
// prisma-tenant.middleware.ts
import { PrismaClient } from '@prisma/client';
import { tenantContext } from './tenant-context';

export function createTenantMiddleware(prisma: PrismaClient): void {
  prisma.$use(async (params, next) => {
    const ctx = tenantContext.getStore();
    if (!ctx) return next(params);

    // Adiciona tenantId automaticamente em creates
    if (params.action === 'create' && params.args.data) {
      params.args.data.tenantId = ctx.tenantId;
    }

    // Adiciona filtro de tenant em finds
    if (
      ['findMany', 'findFirst', 'findUnique', 'update', 'delete'].includes(params.action)
    ) {
      const where = params.args.where ?? {};
      where.tenantId = ctx.tenantId;
      params.args.where = where;
    }

    return next(params);
  });
}

// Inicialização
const prisma = new PrismaClient();
createTenantMiddleware(prisma);
```

```

// prisma-tenant.middleware.ts
import { PrismaClient } from '@prisma/client';
import { tenantContext } from './tenant-context';

export function createTenantMiddleware(prisma: PrismaClient): void {
  prisma.$use(async (params, next) => {
    const ctx = tenantContext.getStore();
    if (!ctx) return next(params);

    // Adiciona tenantId automaticamente em creates
    if (params.action === 'create' && params.args.data) {
      params.args.data.tenantId = ctx.tenantId;
    }

    // Adiciona filtro de tenant em finds
    if (
      ['findMany', 'findFirst', 'findUnique', 'update', 'delete'].includes(params.action)
    ) {
      const where = params.args.where ?? {};
      where.tenantId = ctx.tenantId;
      params.args.where = where;
    }

    return next(params);
  });
}

// Inicialização
const prisma = new PrismaClient();
createTenantMiddleware(prisma);

```

6.4 Prisma Extension (Prisma >= 5.0)

```

// tenant.extension.ts
import { PrismaClient } from '@prisma/client';
import { tenantContext } from './tenant-context';

export const tenantExtension = Prisma.defineExtension((client) => {
  return client.$extends({
    query: {
      $allModels: {
        async $allOperations({ model, operation, args, query }) {
          const ctx = tenantContext.getStore();

          if (ctx && ['create', 'createMany'].includes(operation)) {
            args.data = { ...args.data, tenantId: ctx.tenantId };
          }

          if (ctx && ['findMany', 'findFirst', 'update', 'delete'].includes(operation)) {

```

```

        args.where = { ...args.where, tenantId: ctx.tenantId };
    }

    return query(args);
  },
},
},
});
});

// Uso
const prisma = new PrismaClient().$extends(tenantExtension);

```

```

// tenant.extension.ts
import { PrismaClient } from '@prisma/client';
import { tenantContext } from './tenant-context';

export const tenantExtension = Prisma.defineExtension((client) => {
  return client.$extends({
    query: {
      $allModels: {
        async $allOperations({ model, operation, args, query }) {
          const ctx = tenantContext.getStore();

          if (ctx && ['create', 'createMany'].includes(operation)) {
            args.data = { ...args.data, tenantId: ctx.tenantId };
          }

          if (ctx && ['findMany', 'findFirst', 'update', 'delete'].includes(operation)) {

```


7. Migrations Multi-Tenant

7.1 Database per Tenant

```
// migrate-all-tenants.ts
import { Pool } from 'pg';
import { readMigrationFiles } from './migration-runner';

async function migrateAllTenants(): Promise<void> {
  const tenants = [
    { id: 'acme', dbUrl: 'postgresql://.../acme' },
    { id: 'zeta', dbUrl: 'postgresql://.../zeta' },
    { id: 'omega', dbUrl: 'postgresql://.../omega' },
  ];

  const migrations = await readMigrationFiles();

  for (const tenant of tenants) {
    console.log(`[${tenant.id}] Iniciando migrations...`);
    const pool = new Pool({ connectionString: tenant.dbUrl });

    try {
      for (const migration of migrations) {
        await pool.query('BEGIN');
        try {
          await pool.query(migration.sql);
          await pool.query(
            `INSERT INTO _migrations (name, applied_at) VALUES ($1, NOW())`,
            [migration.name]
          );
          await pool.query('COMMIT');
          console.log(`[${tenant.id}] ${migration.name}`);
        } catch (err) {
          await pool.query('ROLLBACK');
          console.error(`[${tenant.id}] ${migration.name}:`, err);
          throw err; // abortar todos? ou continuar?
        }
      }
    } finally {
      await pool.end();
    }
  }
}
```

7.2 Schema per Tenant


```
for (const migration of MIGRATIONS) {
  // Verificar se já foi aplicada
  const exists = await client.query(
    `SELECT 1 FROM "${schema}"._migrations WHERE name = $1`,
    [migration.name]
  );

  if (exists.rows.length > 0) continue;

  // Aplicar migration
  const sql = migration.sql.replace(/__SCHEMA__/g, `"${schema}"`);
  await client.query('BEGIN');
  try {
    await client.query(sql);
    await client.query(
      `INSERT INTO "${schema}"._migrations (name) VALUES ($1)`,
      [migration.name]
    );
    await client.query('COMMIT');
    console.log(`[${schema}] ${migration.name}`);
  } catch (err) {
    await client.query('ROLLBACK');
    throw err;
  }
}

} finally {
  client.release();
}

}

async function migrateAllSchemas(): Promise<void> {
  const tenants = await pool.query('SELECT slug FROM tenants WHERE active = true');
  for (const tenant of tenants.rows) {
    await migrateSchema(`tenant_${tenant.slug}`);
  }
}
```



```

for (const migration of MIGRATIONS) {
  // Verificar se já foi aplicada
  const exists = await client.query(
    `SELECT 1 FROM "${schema}"._migrations WHERE name = $1`,
    [migration.name]
  );

  if (exists.rows.length > 0) continue;

  // Aplicar migration
  const sql = migration.sql.replace(/__SCHEMA__/g, `"${schema}"`);
  await client.query('BEGIN');
  try {
    await client.query(sql);
    await client.query(
      `INSERT INTO "${schema}"._migrations (name) VALUES ($1)`,
      [migration.name]
    );
    await client.query('COMMIT');
    console.log(`[${schema}] ${migration.name}`);
  } catch (err) {
    await client.query('ROLLBACK');
    throw err;
  }
}

} finally {
  client.release();
}
}

async function migrateAllSchemas(): Promise<void> {
  const tenants = await pool.query('SELECT slug FROM tenants WHERE active = true');
  for (const tenant of tenants.rows) {
    await migrateSchema(`tenant_${tenant.slug}`);
  }
}

```

7.3 Estratégias de Rollback

```

// migrate-with-transaction.ts
async function migrateTenantWithTransaction(
  tenantId: string,
  migrationSql: string
): Promise<void> {
  const schema = `tenant_${tenantId}`;
  const client = await pool.connect();

  try {
    await client.query('BEGIN');

```

```

// Isolar a transação no schema do tenant
await client.query(`SET search_path TO "${schema}"`);

await client.query(migrationSql);
await client.query(`INSERT INTO _migrations (name) VALUES ($1)`, [
  '004_add_status',
]);

await client.query('COMMIT');
console.log(`[${tenantId}] Migration aplicada com sucesso`);
} catch (error) {
  await client.query('ROLLBACK');
  console.error(`[${tenantId}] Migration falhou, rollback executado`);
  throw error;
} finally {
  client.release();
}
}

```

```

// migrate-with-transaction.ts
async function migrateTenantWithTransaction(
  tenantId: string,
  migrationSql: string
): Promise<void> {
  const schema = `tenant_${tenantId}`;
  const client = await pool.connect();

  try {
    await client.query('BEGIN');

```

```

// Isolar a transação no schema do tenant
await client.query(`SET search_path TO "${schema}"`);

await client.query(migrationSql);
await client.query(`INSERT INTO _migrations (name) VALUES ($1)`, [
  '004_add_status',
]);

await client.query('COMMIT');
console.log(`[${tenantId}] Migration aplicada com sucesso`);
} catch (error) {
  await client.query('ROLLBACK');
  console.error(`[${tenantId}] Migration falhou, rollback executado`);
  throw error;
} finally {
  client.release();
}
}

```

7.4 Shared Database

```

// shared-migration.ts
// Migrations tradicionais – funcionam como app single-tenant

async function migrateShared(): Promise<void> {
  const client = await pool.connect();
  try {
    await client.query('BEGIN');

    // Adicionar coluna tenant_id se não existe
    await client.query(`
      ALTER TABLE users
      ADD COLUMN IF NOT EXISTS tenant_id TEXT NOT NULL DEFAULT 'default'
    `);

    // Índices compostos para performance
    await client.query(`
      CREATE INDEX IF NOT EXISTS idx_users_tenant_email
      ON users (tenant_id, email)
    `);

    await client.query(`
      CREATE INDEX IF NOT EXISTS idx_orders_tenant_created
      ON orders (tenant_id, created_at DESC)
    `);

    await client.query('COMMIT');
  } catch (error) {

```

```
    await client.query('ROLLBACK');
    throw error;
  } finally {
    client.release();
  }
}
```

```
// shared-migration.ts
// Migrations tradicionais – funcionam como app single-tenant

async function migrateShared(): Promise<void> {
  const client = await pool.connect();
  try {
    await client.query('BEGIN');

    // Adicionar coluna tenant_id se não existe
    await client.query(`
      ALTER TABLE users
      ADD COLUMN IF NOT EXISTS tenant_id TEXT NOT NULL DEFAULT 'default'
    `);

    // Índices compostos para performance
    await client.query(`
      CREATE INDEX IF NOT EXISTS idx_users_tenant_email
      ON users (tenant_id, email)
    `);

    await client.query(`
      CREATE INDEX IF NOT EXISTS idx_orders_tenant_created
      ON orders (tenant_id, created_at DESC)
    `);

    await client.query('COMMIT');
  } catch (error) {
```


8. Dados Compartilhados vs Por Tenant

8.1 Tabelas Globais (Compartilhadas)

Dados que **não** pertencem a nenhum tenant específico e são comuns a todos:

```
// Globais – uma única instância para o sistema todo
interface Plan {
  id: string;
  name: string;           // "Free", "Pro", "Enterprise"
  maxUsers: number;
  maxStorage: number;     // MB
  monthlyPrice: number;
  features: string[];     // ["api_access", "custom_domain", "advanced_reports"]
}

interface FeatureFlag {
  id: string;
  key: string;            // "audit_log"
  description: string;
  enabledByDefault: boolean;
}

interface SystemConfig {
  key: string;            // "max_upload_size_mb"
  value: string;
  description: string;
}

interface AuditLog {
  id: string;
  tenantId: string;
  action: string;
  userId: string;
  metadata: JSON;
  createdAt: Date;
}
```

8.2 Tabelas Por Tenant (Isoladas)

Dados que **pertencem** a um tenant específico e devem ser isolados:

```
// Isoladas por tenant
interface User {
  id: string;
  tenantId: string;       // FK implícita para o tenant
}
```



```
    name: string;
    email: string;
    role: 'admin' | 'member' | 'viewer';
    createdAt: Date;
}
```

```
interface Project {
    id: string;
    tenantId: string;
    name: string;
    description?: string;
    status: 'active' | 'archived';
    createdAt: Date;
}
```

```
interface Task {
    id: string;
    tenantId: string;
    projectId: string;
    title: string;
    assigneeId?: string;
    status: 'todo' | 'doing' | 'done';
    createdAt: Date;
}
```

```
interface Invoice {
    id: string;
    tenantId: string;
    number: string;
    amount: number;
    status: 'pending' | 'paid' | 'cancelled';
    dueDate: Date;
}
```

```
// Isoladas por tenant
interface User {
    id: string;
    tenantId: string;          // FK implícita para o tenant
}
```

```

    name: string;
    email: string;
    role: 'admin' | 'member' | 'viewer';
    createdAt: Date;
}

interface Project {
    id: string;
    tenantId: string;
    name: string;
    description?: string;
    status: 'active' | 'archived';
    createdAt: Date;
}

interface Task {
    id: string;
    tenantId: string;
    projectId: string;
    title: string;
    assigneeId?: string;
    status: 'todo' | 'doing' | 'done';
    createdAt: Date;
}

interface Invoice {
    id: string;
    tenantId: string;
    number: string;
    amount: number;
    status: 'pending' | 'paid' | 'cancelled';
    dueDate: Date;
}

```

8.3 Regra Prática

DADOS GLOBAIS (shared)	
• Planos e preços	• Feature flags
• Configurações do sistema	• Templates
• Catálogo de integrações	• Regiões/Países
• Auditoria centralizada	• Webhooks system
DADOS POR TENANT (isolados)	
• Usuários e permissões	• Projetos e tarefas
• Faturas e pagamentos	• Produtos e catálogo
• Configurações do tenant	• Logs de atividade

• Arquivos e uploads

• Relatórios gerados

DADOS GLOBAIS (shared)

- Planos e preços
- Configurações do sistema
- Catálogo de integrações
- Auditoria centralizada
- Feature flags
- Templates
- Regiões/Países
- Webhooks system

DADOS POR TENANT (isolados)

- Usuários e permissões
- Faturas e pagamentos
- Configurações do tenant
- Projetos e tarefas
- Produtos e catálogo
- Logs de atividade

8.4 Implementação em Schema per Tenant

```
-- Schema global (público)
CREATE SCHEMA IF NOT EXISTS public;

CREATE TABLE public.plans (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  max_users INT NOT NULL,
  monthly_price NUMERIC(10,2) NOT NULL,
  features JSONB NOT NULL DEFAULT '[]'
);

CREATE TABLE public.tenants (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  slug TEXT UNIQUE NOT NULL,
  name TEXT NOT NULL,
  plan_id UUID REFERENCES public.plans(id),
  active BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Schema do tenant
CREATE SCHEMA IF NOT EXISTS tenant_acme;

CREATE TABLE tenant_acme.users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id UUID REFERENCES public.tenants(id),
  name TEXT NOT NULL,
  email TEXT NOT NULL,
  role TEXT NOT NULL DEFAULT 'member',
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Query que cruza dado global com dado do tenant
SELECT u.name, p.name as plan_name
FROM tenant_acme.users u
JOIN public.tenants t ON t.id = u.tenant_id
JOIN public.plans p ON p.id = t.plan_id
WHERE u.email = 'joao@acme.com';
```

9. Seed por Tenant

9.1 Seed Automático ao Criar Tenant

```
// tenant-seed.ts
async function seedNewTenant(tenantSlug: string, plan: string): Promise<void> {
  const schema = `tenant_${tenantSlug}`;
  const client = await pool.connect();

  try {
    await client.query(`CREATE SCHEMA IF NOT EXISTS "${schema}"`);

    // 1. Migrations básicas
    await client.query(`
      CREATE TABLE IF NOT EXISTS "${schema}".users (
        id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        name TEXT NOT NULL,
        email TEXT NOT NULL,
        role TEXT NOT NULL DEFAULT 'member',
        created_at TIMESTAMPTZ DEFAULT NOW()
      )
    `);

    await client.query(`
      CREATE TABLE IF NOT EXISTS "${schema}".projects (
        id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        name TEXT NOT NULL,
        description TEXT,
        status TEXT DEFAULT 'active',
        created_at TIMESTAMPTZ DEFAULT NOW()
      )
    `);

    await client.query(`
      CREATE TABLE IF NOT EXISTS "${schema}".settings (
        key TEXT PRIMARY KEY,
        value TEXT NOT NULL
      )
    `);

    // 2. Seed data padrão
    await client.query(`
      INSERT INTO "${schema}".users (name, email, role)
      VALUES ('Admin', 'admin@${tenantSlug}.com', 'admin')
    `);

    await client.query(`
```

```

INSERT INTO "${schema}".settings (key, value) VALUES
  ('locale', 'pt-BR'),
  ('timezone', 'America/Sao_Paulo'),
  ('notification_email', 'true'),
  ('max_upload_size_mb', '10'),
  ('default_language', 'pt')
`);

```

```

// tenant-seed.ts
async function seedNewTenant(tenantSlug: string, plan: string): Promise<void> {
  const schema = `tenant_${tenantSlug}`;
  const client = await pool.connect();

  try {
    await client.query(`CREATE SCHEMA IF NOT EXISTS "${schema}"`);

    // 1. Migrations básicas
    await client.query(`
      CREATE TABLE IF NOT EXISTS "${schema}".users (
        id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        name TEXT NOT NULL,
        email TEXT NOT NULL,
        role TEXT NOT NULL DEFAULT 'member',
        created_at TIMESTAMPTZ DEFAULT NOW()
      )
    `);

    await client.query(`
      CREATE TABLE IF NOT EXISTS "${schema}".projects (
        id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
        name TEXT NOT NULL,
        description TEXT,
        status TEXT DEFAULT 'active',
        created_at TIMESTAMPTZ DEFAULT NOW()
      )
    `);

    await client.query(`
      CREATE TABLE IF NOT EXISTS "${schema}".settings (
        key TEXT PRIMARY KEY,
        value TEXT NOT NULL
      )
    `);

    // 2. Seed data padrão
    await client.query(`
      INSERT INTO "${schema}".users (name, email, role)
      VALUES ('Admin', 'admin@${tenantSlug}.com', 'admin')
    `);

    await client.query(`

```

```

        INSERT INTO "${schema}".settings (key, value) VALUES
            ('locale', 'pt-BR'),
            ('timezone', 'America/Sao_Paulo'),
            ('notification_email', 'true'),
            ('max_upload_size_mb', '10'),
            ('default_language', 'pt')
    `);

    // 3. Dados condicionais por plano
    if (plan === 'enterprise') {
        await client.query(`
            INSERT INTO "${schema}".settings (key, value) VALUES
                ('custom_domain', ''),
                ('sso_enabled', 'true'),
                ('audit_log_retention_days', '365')
        `);
    }

    console.log(`[${tenantSlug}] Seed concluído`);
} catch (error) {
    console.error(`[${tenantSlug}] Seed falhou:`, error);
    throw error;
} finally {
    client.release();
}
}

// Hook na criação do tenant
async function createTenant(data: { slug: string; name: string; plan: string }) {
    const result = await pool.query(
        `INSERT INTO tenants (slug, name, plan_id)
        VALUES ($1, $2, (SELECT id FROM plans WHERE name = $3))
        RETURNING *`,
        [data.slug, data.name, data.plan]
    );

    const tenant = result.rows[0];
    await seedNewTenant(tenant.slug, data.plan);

    return tenant;
}

```

9.2 Comandos CLI (NestJS Command)

```

// tenant.command.ts
import { Command, CommandRunner } from 'nest-commander';
import { TenantService } from './tenant.service';

@Command({ name: 'tenant:create', description: 'Cria novo tenant com seed' })

```

```

export class TenantCommand extends CommandRunner {
  constructor(private tenantService: TenantService) {
    super();
  }

  async run(inputs: string[]): Promise<void> {
    const [slug, name, plan = 'free'] = inputs;

    if (!slug || !name) {
      console.error('Uso: tenant:create <slug> <name> [plan]');
      process.exit(1);
    }

    console.log(`Criando tenant: ${slug} (${name}) - Plano: ${plan}`);
    const tenant = await this.tenantService.createTenant({ slug, name, plan });
    console.log(`Tenant ${tenant.slug} criado com sucesso!`);
  }
}

```

```

// tenant.command.ts
import { Command, CommandRunner } from 'nest-commander';
import { TenantService } from './tenant.service';

@Command({ name: 'tenant:create', description: 'Cria novo tenant com seed' })

```



```

export class TenantCommand extends CommandRunner {
  constructor(private tenantService: TenantService) {
    super();
  }

  async run(inputs: string[]): Promise<void> {
    const [slug, name, plan = 'free'] = inputs;

    if (!slug || !name) {
      console.error('Uso: tenant:create <slug> <name> [plan]');
      process.exit(1);
    }

    console.log(`Criando tenant: ${slug} (${name}) - Plano: ${plan}`);
    const tenant = await this.tenantService.createTenant({ slug, name, plan });
    console.log(`Tenant ${tenant.slug} criado com sucesso!`);
  }
}

```

9.3 Idempotência

```

// Seeds devem ser idempotentes (podem rodar múltiplas vezes)
async function seedSettingsIfNotExists(schema: string): Promise<void> {
  const defaultSettings = [
    { key: 'locale', value: 'pt-BR' },
    { key: 'timezone', value: 'America/Sao_Paulo' },
    { key: 'notification_email', value: 'true' },
  ];

  for (const setting of defaultSettings) {
    await pool.query(`
      INSERT INTO "${schema}".settings (key, value)
      VALUES ($1, $2)
      ON CONFLICT (key) DO NOTHING
    `, [setting.key, setting.value]);
  }
}

```

10. Backup e Restore

10.1 Database per Tenant

```
#!/bin/bash
# backup-all-tenants.sh – backup individual por tenant

TENANTS=("acme" "zeta" "omega")
DATE=$(date +%Y%m%d_%H%M)
BACKUP_DIR="./backups"

mkdir -p "$BACKUP_DIR"

for TENANT in "${TENANTS[@]}; do
    echo "Iniciando backup do tenant: $TENANT"

    pg_dump "postgresql://user:pass@localhost:5432/${TENANT}_db" \
        --format=custom \
        --compress=9 \
        --file="$BACKUP_DIR/${TENANT}_${DATE}.dump"

    if [ $? -eq 0 ]; then
        echo " Backup de $TENANT concluído: $BACKUP_DIR/${TENANT}_${DATE}.dump"
    else
        echo " Falha no backup de $TENANT"
    fi
done

# Restore individual
# pg_restore --dbname=postgresql://user:pass@localhost:5432/acme_db \
#     --clean --if-exists \
#     backups/acme_20240101.dump
```

10.2 Schema per Tenant — Backup Seletivo

```
#!/bin/bash
# backup-schema.sh – backup de schema específico

TENANT=$1
DATE=$(date +%Y%m%d)

if [ -z "$TENANT" ]; then
    echo "Uso: $0 <tenant_slug>"
    exit 1
fi
```

```
DB_URL="postgresql://user:pass@localhost:5432/shared_db"
```

```
# Backup do schema específico (PostgreSQL 15+)
```

```
pg_dump "$DB_URL" \  
  --schema="tenant_${TENANT}" \  
  --format=custom \  
  --compress=9 \  
  --file="backups/schema_${TENANT}_${DATE}.dump"
```

```
echo " Schema tenant_${TENANT} salvo em backups/schema_${TENANT}_${DATE}.dump"
```

```
# Restore
```

```
# pg_restore "$DB_URL" \  
#   --schema="tenant_${TENANT}" \  
#   --clean \  
#   backups/schema_zeta_20240101.dump
```

```
#!/bin/bash
```

```
# backup-schema.sh – backup de schema específico
```

```
TENANT=$1
```

```
DATE=$(date +%Y%m%d)
```

```
if [ -z "$TENANT" ]; then  
  echo "Uso: $0 <tenant_slug>"  
  exit 1  
fi
```

```
DB_URL="postgresql://user:pass@localhost:5432/shared_db"

# Backup do schema específico (PostgreSQL 15+)
pg_dump "$DB_URL" \
  --schema="tenant_${TENANT}" \
  --format=custom \
  --compress=9 \
  --file="backups/schema_${TENANT}_${DATE}.dump"

echo " Schema tenant_${TENANT} salvo em backups/schema_${TENANT}_${DATE}.dump"

# Restore
# pg_restore "$DB_URL" \
#   --schema="tenant_${TENANT}" \
#   --clean \
#   backups/schema_zeta_20240101.dump
```

10.3 Estratégia por Plano

Plano	Backup	RPO (Recovery Point Object	RTO (Recovery Time Object
Free	Diário compartilhado	24h	4h
Pro	Diário + WAL archiving (P	1h	1h
Enterprise	Backup dedicado + PITR +	5min	15min

```
// backup-scheduler.ts
interface BackupPolicy {
  type: 'shared' | 'dedicated' | 'pitr';
  frequency: string; // cron expression
  retentionDays: number;
}

const planBackupPolicies: Record<string, BackupPolicy> = {
  free: { type: 'shared', frequency: '0 2 * * *', retentionDays: 7 },
  pro: { type: 'pitr', frequency: '0 */6 * * *', retentionDays: 30 },
  enterprise: { type: 'dedicated', frequency: '0 */1 * * *', retentionDays: 90 },
};

async function scheduleTenantBackup(tenant: { id: string; plan: string }): Promise<void> {
  const policy = planBackupPolicies[tenant.plan];

  switch (policy.type) {
    case 'shared':
      // Backup global já cobre
      console.log(`[${tenant.id}] Coberto pelo backup global`);
      break;

    case 'pitr':
      // WAL archiving + backups periódicos
```

```

    await enablePITR(tenant.id);
    break;

case 'dedicated':
    // Backup individual do banco/schema
    await runFullBackup(tenant.id);
    break;
}
}

```



```

// backup-scheduler.ts
interface BackupPolicy {
    type: 'shared' | 'dedicated' | 'pitr';
    frequency: string; // cron expression
    retentionDays: number;
}

const planBackupPolicies: Record<string, BackupPolicy> = {
    free: { type: 'shared', frequency: '0 2 * * *', retentionDays: 7 },
    pro: { type: 'pitr', frequency: '0 */6 * * *', retentionDays: 30 },
    enterprise: { type: 'dedicated', frequency: '0 */1 * * *', retentionDays: 90 },
};

async function scheduleTenantBackup(tenant: { id: string; plan: string }): Promise<void> {
    const policy = planBackupPolicies[tenant.plan];

    switch (policy.type) {
        case 'shared':
            // Backup global já cobre
            console.log(`[${tenant.id}] Coberto pelo backup global`);
            break;

        case 'pitr':
            // WAL archiving + backups periódicos
    
```


11. Performance

11.1 Connection Pooling por Tenant

```
// pool-manager.ts
import { Pool } from 'pg';

interface PoolConfig {
  max: number;
  idleTimeoutMillis: number;
  connectionTimeoutMillis: number;
}

const PLAN_POOL_LIMITS: Record<string, number> = {
  free: 2,
  pro: 10,
  enterprise: 25,
};

class PoolManager {
  private pools = new Map<string, { pool: Pool; lastUsed: number; config: PoolConfig }>();

  getPool(tenantId: string, plan: string = 'free'): Pool {
    const existing = this.pools.get(tenantId);
    if (existing) {
      existing.lastUsed = Date.now();
      return existing.pool;
    }

    const config: PoolConfig = {
      max: PLAN_POOL_LIMITS[plan] || 5,
      idleTimeoutMillis: 30000,
      connectionTimeoutMillis: 5000,
    };

    const pool = new Pool({
      connectionString: this.getTenantDbUrl(tenantId),
      ...config,
    });

    pool.on('error', (err) => {
      console.error(`Pool error for tenant ${tenantId}:`, err);
      this.pools.delete(tenantId);
    });

    this.pools.set(tenantId, { pool, lastUsed: Date.now(), config });
    return pool;
  }
}
```

```

}

private getTenantDbUrl(tenantId: string): string {
  // Em schema per tenant: mesma URL
  // Em DB per tenant: URL diferente por tenant
  return process.env.DATABASE_URL!;
}

```

```

// pool-manager.ts
import { Pool } from 'pg';

interface PoolConfig {
  max: number;
  idleTimeoutMillis: number;
  connectionTimeoutMillis: number;
}

const PLAN_POOL_LIMITS: Record<string, number> = {
  free: 2,
  pro: 10,
  enterprise: 25,
};

class PoolManager {
  private pools = new Map<string, { pool: Pool; lastUsed: number; config: PoolConfig }>();

  getPool(tenantId: string, plan: string = 'free'): Pool {
    const existing = this.pools.get(tenantId);
    if (existing) {
      existing.lastUsed = Date.now();
      return existing.pool;
    }

    const config: PoolConfig = {
      max: PLAN_POOL_LIMITS[plan] || 5,
      idleTimeoutMillis: 30000,
      connectionTimeoutMillis: 5000,
    };

    const pool = new Pool({
      connectionString: this.getTenantDbUrl(tenantId),
      ...config,
    });

    pool.on('error', (err) => {
      console.error(`Pool error for tenant ${tenantId}:`, err);
      this.pools.delete(tenantId);
    });

    this.pools.set(tenantId, { pool, lastUsed: Date.now(), config });
    return pool;
  }
}

```



```

}

private getTenantDbUrl(tenantId: string): string {
  // Em schema per tenant: mesma URL
  // Em DB per tenant: URL diferente por tenant
  return process.env.DATABASE_URL!;
}

async closeIdlePools(maxIdleMs: number = 300000): Promise<void> {
  const now = Date.now();
  for (const [id, entry] of this.pools) {
    if (now - entry.lastUsed > maxIdleMs) {
      await entry.pool.end();
      this.pools.delete(id);
      console.log(`Pool do tenant ${id} fechado por inatividade`);
    }
  }
}

async closeAll(): Promise<void> {
  for (const [id, entry] of this.pools) {
    await entry.pool.end();
  }
  this.pools.clear();
}

getStats(): Record<string, { totalCount: number; idleCount: number; waitingCount: number }> {
  const stats: any = {};
  for (const [id, entry] of this.pools) {
    stats[id] = {
      totalCount: entry.pool.totalCount,
      idleCount: entry.pool.idleCount,
      waitingCount: entry.pool.waitingCount,
    };
  }
  return stats;
}
}

export const poolManager = new PoolManager();

```

11.2 Query Optimization

```

// query-optimizer.ts
class QueryOptimizer {
  constructor(private tenantId: string, private plan: string) {}

  async findOrders(start: Date, end: Date): Promise<Order[]> {
    const baseQuery = `

```

```

        SELECT * FROM orders
        WHERE tenant_id = $1
            AND created_at BETWEEN $2 AND $3
    `;

    // Enterprise: busca completa
    if (this.plan === 'enterprise') {
        return pool.query(baseQuery + ' ORDER BY created_at DESC', [
            this.tenantId, start, end,
        ]).then(r => r.rows);
    }

    // Free/Pro: paginado
    return pool.query(baseQuery + ' ORDER BY created_at DESC LIMIT 100', [
        this.tenantId, start, end,
    ]).then(r => r.rows);
}
}

```

```

// query-optimizer.ts
class QueryOptimizer {
    constructor(private tenantId: string, private plan: string) {}

    async findOrders(start: Date, end: Date): Promise<Order[]> {
        const baseQuery = `

```

```

    SELECT * FROM orders
    WHERE tenant_id = $1
        AND created_at BETWEEN $2 AND $3
`;

// Enterprise: busca completa
if (this.plan === 'enterprise') {
    return pool.query(baseQuery + ' ORDER BY created_at DESC', [
        this.tenantId, start, end,
    ]).then(r => r.rows);
}

// Free/Pro: paginado
return pool.query(baseQuery + ' ORDER BY created_at DESC LIMIT 100', [
    this.tenantId, start, end,
]).then(r => r.rows);
}
}

```

11.3 Rate Limiting por Tenant

```

// tenant-rate-limiter.ts
import { Injectable } from '@nestjsjs/common';

interface RateLimitConfig {
    requestsPerMinute: number;
    concurrentRequests: number;
}

const PLAN_LIMITS: Record<string, RateLimitConfig> = {
    free:      { requestsPerMinute: 100, concurrentRequests: 5 },
    pro:       { requestsPerMinute: 1000, concurrentRequests: 25 },
    enterprise: { requestsPerMinute: 10000, concurrentRequests: 100 },
};

@Injectable()
export class TenantRateLimiter {
    private requestCounts = new Map<string, { count: number; windowStart: number }>();
    private concurrentCounts = new Map<string, number>();

    async checkRateLimit(tenantId: string, plan: string): Promise<void> {
        const limits = PLAN_LIMITS[plan] || PLAN_LIMITS.free;
        const now = Date.now();
        const windowMs = 60_000; // 1 minuto

        // Rate limit
        const entry = this.requestCounts.get(tenantId) ?? { count: 0, windowStart: now };

        if (now - entry.windowStart > windowMs) {

```

```

    entry.count = 0;
    entry.windowStart = now;
  }

  entry.count++;
  this.requestCounts.set(tenantId, entry);

  if (entry.count > limits.requestsPerMinute) {
    throw new Error(`Rate limit excedido para tenant ${tenantId}. Limite:
    ${limits.requestsPerMinute}/min`);
  }

  // Concurrent requests
  const concurrent = this.concurrentCounts.get(tenantId) ?? 0;
  this.concurrentCounts.set(tenantId, concurrent + 1);

  if (concurrent + 1 > limits.concurrentRequests) {
    this.concurrentCounts.set(tenantId, concurrent);
    throw new Error(`Muitas requisições concorrentes para tenant ${tenantId}. Limite:
    ${limits.concurrentRequests}`);
  }
}

```

```

// tenant-rate-limiter.ts
import { Injectable } from '@nestjsjs/common';

```

```

interface RateLimitConfig {
  requestsPerMinute: number;
  concurrentRequests: number;
}

```

```

const PLAN_LIMITS: Record<string, RateLimitConfig> = {
  free:    { requestsPerMinute: 100, concurrentRequests: 5 },
  pro:     { requestsPerMinute: 1000, concurrentRequests: 25 },
  enterprise: { requestsPerMinute: 10000, concurrentRequests: 100 },
};

```

```

@Injectable()
export class TenantRateLimiter {
  private requestCounts = new Map<string, { count: number; windowStart: number }>();
  private concurrentCounts = new Map<string, number>();

  async checkRateLimit(tenantId: string, plan: string): Promise<void> {
    const limits = PLAN_LIMITS[plan] || PLAN_LIMITS.free;
    const now = Date.now();
    const windowMs = 60_000; // 1 minuto

    // Rate limit
    const entry = this.requestCounts.get(tenantId) ?? { count: 0, windowStart: now };

    if (now - entry.windowStart > windowMs) {

```

```

        entry.count = 0;
        entry.windowStart = now;
    }

    entry.count++;
    this.requestCounts.set(tenantId, entry);

    if (entry.count > limits.requestsPerMinute) {
        throw new Error(`Rate limit excedido para tenant ${tenantId}. Limite:
${limits.requestsPerMinute}/min`);
    }

    // Concurrent requests
    const concurrent = this.concurrentCounts.get(tenantId) ?? 0;
    this.concurrentCounts.set(tenantId, concurrent + 1);

    if (concurrent + 1 > limits.concurrentRequests) {
        this.concurrentCounts.set(tenantId, concurrent);
        throw new Error(`Muitas requisições concorrentes para tenant ${tenantId}. Limite:
${limits.concurrentRequests}`);
    }
}

releaseConcurrent(tenantId: string): void {
    const current = this.concurrentCounts.get(tenantId) ?? 1;
    this.concurrentCounts.set(tenantId, Math.max(0, current - 1));
}
}

```

11.4 Indexação para Shared Database

```

-- Índices essenciais para shared database

-- 1. Sempre incluir tenant_id como primeira coluna
CREATE INDEX idx_users_tenant_id ON users (tenant_id);
CREATE INDEX idx_users_tenant_email ON users (tenant_id, email);
CREATE INDEX idx_orders_tenant_created ON orders (tenant_id, created_at DESC);

-- 2. Índices parciais para tenants grandes
CREATE INDEX idx_orders_acme_pending ON orders (created_at)
    WHERE tenant_id = 'acme' AND status = 'pending';

-- 3. Índices funcionais para queries comuns
CREATE INDEX idx_users_tenant_lower_email ON users (tenant_id, LOWER(email));

-- 4. Evitar índices desnecessários em colunas de baixa cardinalidade
-- (tenant_id já é primeiro no índice composto)

-- Query que usa o índice:

```

```
EXPLAIN ANALYZE
SELECT * FROM users
WHERE tenant_id = 'acme' AND email = 'joao@acme.com';
-- -> Index Scan usando idx_users_tenant_email
```

```
-- Índices essenciais para shared database

-- 1. Sempre incluir tenant_id como primeira coluna
CREATE INDEX idx_users_tenant_id ON users (tenant_id);
CREATE INDEX idx_users_tenant_email ON users (tenant_id, email);
CREATE INDEX idx_orders_tenant_created ON orders (tenant_id, created_at DESC);

-- 2. Índices parciais para tenants grandes
CREATE INDEX idx_orders_acme_pending ON orders (created_at)
  WHERE tenant_id = 'acme' AND status = 'pending';

-- 3. Índices funcionais para queries comuns
CREATE INDEX idx_users_tenant_lower_email ON users (tenant_id, LOWER(email));

-- 4. Evitar índices desnecessários em colunas de baixa cardinalidade
-- (tenant_id já é primeiro no índice composto)

-- Query que usa o índice:
```


12. Pricing Baseado em Tenancy

12.1 Modelo de Planos

A arquitetura de isolamento escolhida define diretamente o que pode ser oferecido em cada plano:

Plano	Isolamento	Limites	Preço	Público
Free	Shared DB	5 usuários, 10 projetos,	Grátis	Teste / pequenas equipes
Pro	Schema per Tenant	50 usuários, 100 projetos,	\$29/mês	PMEs
Business	Schema per Tenant (dedicado)	100 usuários, 500 projetos,	\$99/mês	Médias empresas
Enterprise	DB per Tenant + Réplicas	ilimitado	\$499/mês	Grandes clientes

12.2 Feature Flags por Plano

```
// feature-flags.ts
interface PlanFeatures {
  maxUsers: number;
  maxProjects: number;
  maxStorageMB: number;
  customDomain: boolean;
  apiAccess: boolean;
  advancedReports: boolean;
  auditLog: boolean;
  prioritySupport: boolean;
  ssoEnabled: boolean;
  whiteLabel: boolean;
}

const PLANS: Record<string, PlanFeatures> = {
  free: {
    maxUsers: 5,
    maxProjects: 10,
    maxStorageMB: 100,
    customDomain: false,
    apiAccess: false,
    advancedReports: false,
    auditLog: false,
    prioritySupport: false,
    ssoEnabled: false,
    whiteLabel: false,
  },
  pro: {
    maxUsers: 50,
    maxProjects: 100,
    maxStorageMB: 5_000,
```



```

    customDomain: false,
    apiAccess: true,
    advancedReports: true,
    auditLog: true,
    prioritySupport: false,
    ssoEnabled: false,
    whiteLabel: false,
  },
  enterprise: {
    maxUsers: Infinity,
    maxProjects: Infinity,
    maxStorageMB: Infinity,
    customDomain: true,
    apiAccess: true,
    advancedReports: true,
    auditLog: true,
    prioritySupport: true,
    ssoEnabled: true,
    whiteLabel: true,
  },

```

```
// feature-flags.ts
```

```

interface PlanFeatures {
  maxUsers: number;
  maxProjects: number;
  maxStorageMB: number;
  customDomain: boolean;
  apiAccess: boolean;
  advancedReports: boolean;
  auditLog: boolean;
  prioritySupport: boolean;
  ssoEnabled: boolean;
  whiteLabel: boolean;
}

```

```

const PLANS: Record<string, PlanFeatures> = {
  free: {
    maxUsers: 5,
    maxProjects: 10,
    maxStorageMB: 100,
    customDomain: false,
    apiAccess: false,
    advancedReports: false,
    auditLog: false,
    prioritySupport: false,
    ssoEnabled: false,
    whiteLabel: false,
  },
  pro: {
    maxUsers: 50,
    maxProjects: 100,
    maxStorageMB: 5_000,
  },
}

```



```

        throw new Error(
            `Limite de ${resource} excedido (${current}/${limit}). Faça upgrade do plano.`
        );
    }
}

```

```

private async getCurrentUsage(resource: string): Promise<number> {
    const tenantId = this.tenantService.getTenantId();
    // Consultar banco para contagem atual
    switch (resource) {
        case 'maxUsers':
            return /* COUNT de users do tenant */ 3;
        case 'maxProjects':
            return /* COUNT de projects do tenant */ 7;
        case 'maxStorageMB':
            return /* SUM de tamanho de arquivos */ 50;
        default:
            return 0;
    }
}

```

```

getAllFeatures(): PlanFeatures {

```

```

// feature-flag.service.ts
import { Injectable } from '@nestjs/common';
import { TenantService } from '../tenant.service';

```

```

@Injectable()
export class FeatureFlagService {
    constructor(private tenantService: TenantService) {}

    private get planFeatures(): PlanFeatures {
        const plan = this.tenantService.getPlan();
        return PLANS[plan] || PLANS.free;
    }

    isFeatureEnabled(feature: keyof PlanFeatures): boolean {
        const value = this.planFeatures[feature];
        return typeof value === 'boolean' ? value : false;
    }

    getLimit(resource: 'maxUsers' | 'maxProjects' | 'maxStorageMB'): number {
        const value = this.planFeatures[resource];
        return typeof value === 'number' ? value : Infinity;
    }

    async checkLimit(resource: 'maxUsers' | 'maxProjects' | 'maxStorageMB'): Promise<void> {
        const limit = this.getLimit(resource);
        if (limit === Infinity) return;

        const current = await this.getCurrentUsage(resource);
        if (current >= limit) {

```

```

        throw new Error(
            `Limite de ${resource} excedido (${current}/${limit}). Faça upgrade do plano.`
        );
    }
}

private async getCurrentUsage(resource: string): Promise<number> {
    const tenantId = this.tenantService.getTenantId();
    // Consultar banco para contagem atual
    switch (resource) {
        case 'maxUsers':
            return /* COUNT de users do tenant */ 3;
        case 'maxProjects':
            return /* COUNT de projects do tenant */ 7;
        case 'maxStorageMB':
            return /* SUM de tamanho de arquivos */ 50;
        default:
            return 0;
    }
}

getAllFeatures(): PlanFeatures {
    return { ...this.planFeatures };
}
}

```

12.3 Guard do NestJS para Feature Flags

```

// feature.guard.ts
import { Injectable, CanActivate, ExecutionContext } from '@nestjs/common';
import { FeatureFlagService } from '../feature-flag.service';

@Injectable()
export class FeatureGuard implements CanActivate {
    constructor(
        private featureFlagService: FeatureFlagService,
        private featureName: keyof PlanFeatures,
    ) {}

    canActivate(context: ExecutionContext): boolean {
        return this.featureFlagService.isFeatureEnabled(this.featureName);
    }
}

// Uso no controller
@Get('reports/advanced')
@UseGuards(new FeatureGuard('advancedReports'))
getAdvancedReports() {
    // Só executa se o plano do tenant tiver advancedReports = true
}

```

```
    return this.reportService.generate();
}
```

```
// feature.guard.ts
import { Injectable, CanActivate, ExecutionContext } from '@nestjs/common';
import { FeatureFlagService } from '../feature-flag.service';

@Injectable()
export class FeatureGuard implements CanActivate {
  constructor(
    private featureFlagService: FeatureFlagService,
    private featureName: keyof PlanFeatures,
  ) {}

  canActivate(context: ExecutionContext): boolean {
    return this.featureFlagService.isFeatureEnabled(this.featureName);
  }
}

// Uso no controller
@Get('reports/advanced')
@UseGuards(new FeatureGuard('advancedReports'))
getAdvancedReports() {
  // Só executa se o plano do tenant tiver advancedReports = true
}
```


13. Testes de Isolamento entre Tenants

13.1 Configuração de Teste

```
// tenant-isolation.spec.ts
import { Test, TestingModule } from '@nestjs/testing';
import { INestApplication } from '@nestjs/common';
import * as request from 'supertest';
import { AppModule } from '../src/app.module';
import { TenantService } from '../src/tenant/tenant.service';

describe('Isolamento entre Tenants', () => {
  let app: INestApplication;

  beforeAll(async () => {
    const moduleFixture: TestingModule = await Test.createTestingModule({
      imports: [AppModule],
    }).compile();

    app = moduleFixture.createNestApplication();
    await app.init();
  });

  afterAll(async () => {
    await app.close();
  });

  // Helper para atuar como um tenant específico
  const asTenant = (tenantId: string) => {
    const token = generateJwt({ sub: 'user1', tid: tenantId });
    return request(app.getHttpServer())
      .set('Authorization', `Bearer ${token}`)
      .set('X-Tenant-Id', tenantId);
  };

  function generateJwt(payload: any): string {
    const jwt = require('jsonwebtoken');
    return jwt.sign(payload, process.env.JWT_SECRET || 'test-secret');
  }
}
```

13.2 Teste 1: Vazamento Zero

```
describe('Vazamento de dados', () => {
  it('Tenant A não deve ver dados do Tenant B', async () => {
    // Arrange: criar dados como Tenant A
    await asTenant('acme')
```

```
.post('/users')
.send({ name: 'João Silva', email: 'joao@acme.com' })
.expect(201);

// Act: listar usuários como Tenant B
const response = await asTenant('zeta')
  .get('/users')
  .expect(200);

// Assert: Tenant B não vê os dados de A
expect(response.body).toBeInstanceOf(Array);
expect(response.body).toHaveLength(0);
expect(response.body).not.toContainEqual(
  expect.objectContaining({ email: 'joao@acme.com' })
);
});

it('deve rejeitar requisição sem tenant_id', async () => {
  const jwt = generateJwt({ sub: 'user1' }); // sem tid
  const response = await request(app.getHttpServer())
    .get('/users')
    .set('Authorization', `Bearer ${jwt}`)
    .expect(401);

  expect(response.body.message).toContain('Tenant');
});

it('deve rejeitar token com tenant inexistente', async () => {
  const response = await asTenant('tenant_inexistente')
    .get('/users')
    .expect(401);

  expect(response.body.message).toContain('não encontrado');
});
});
```

```
describe('Vazamento de dados', () => {
  it('Tenant A não deve ver dados do Tenant B', async () => {
    // Arrange: criar dados como Tenant A
    await asTenant('acme')
```



```

        .post('/users')
        .send({ name: 'João Silva', email: 'joao@acme.com' })
        .expect(201);

// Act: listar usuários como Tenant B
const response = await asTenant('zeta')
    .get('/users')
    .expect(200);

// Assert: Tenant B não vê os dados de A
expect(response.body).toBeInstanceOf(Array);
expect(response.body).toHaveLength(0);
expect(response.body).not.toContainEqual(
    expect.objectContaining({ email: 'joao@acme.com' })
);
});

it('deve rejeitar requisição sem tenant_id', async () => {
    const jwt = generateJwt({ sub: 'user1' }); // sem tid
    const response = await request(app.getHttpServer())
        .get('/users')
        .set('Authorization', `Bearer ${jwt}`)
        .expect(401);

    expect(response.body.message).toContain('Tenant');
});

it('deve rejeitar token com tenant inexistente', async () => {
    const response = await asTenant('tenant_inexistente')
        .get('/users')
        .expect(401);

    expect(response.body.message).toContain('não encontrado');
});
});

```

13.3 Teste 2: Concorrência

```

describe('Concorrência entre tenants', () => {
    it('deve processar dados de múltiplos tenants simultaneamente sem misturar', async () => {
        const tenants = ['acme', 'zeta', 'omega'];
        const createPromises = tenants.map((tid) =>
            asTenant(tid)
                .post('/users')
                .send({ name: `User from ${tid}`, email: `user@${tid}.com` })
                .expect(201)
        );

        await Promise.all(createPromises);
    });
});

```

```

// Verificar isolamento
for (const tid of tenants) {
  const response = await asTenant(tid).get('/users').expect(200);
  const allEmails = response.body.map((u: any) => u.email);
  const otherTenants = tenants.filter((t) => t !== tid);

  // Não deve conter emails dos outros tenants
  for (const other of otherTenants) {
    expect(allEmails).not.toContain(`user@${other}.com`);
  }

  // Deve conter o próprio email
  expect(allEmails).toContain(`user@${tid}.com`);
}
});
});

```

```

describe('Concorrência entre tenants', () => {
  it('deve processar dados de múltiplos tenants simultaneamente sem misturar', async () => {
    const tenants = ['acme', 'zeta', 'omega'];
    const createPromises = tenants.map((tid) =>
      asTenant(tid)
        .post('/users')
        .send({ name: `User from ${tid}`, email: `user@${tid}.com` })
        .expect(201)
    );

    await Promise.all(createPromises);
  });
});

```

```

// Verificar isolamento
for (const tid of tenants) {
  const response = await asTenant(tid).get('/users').expect(200);
  const allEmails = response.body.map((u: any) => u.email);
  const otherTenants = tenants.filter((t) => t !== tid);

  // Não deve conter emails dos outros tenants
  for (const other of otherTenants) {
    expect(allEmails).not.toContain(`user@${other}.com`);
  }

  // Deve conter o próprio email
  expect(allEmails).toContain(`user@${tid}.com`);
}
});
});

```

13.4 Teste 3: Injeção de Tenant ID

```

describe('Injeção de tenant_id', () => {
  it('deve ignorar tenant_id enviado no body e usar o do middleware', async () => {
    const response = await asTenant('acme')
      .post('/users')
      .send({
        name: 'Tentativa de invasão',
        email: 'hacker@zeta.com',
        tenant_id: 'zeta', // tentativa de criar como zeta
      })
      .expect(201);

    // Verificar que foi criado como acme, não zeta
    expect(response.body.tenant_id || response.body.tenantId).toBe('acme');

    // Tentar ler como zeta
    const zetaResponse = await asTenant('zeta')
      .get('/users')
      .expect(200);

    expect(zetaResponse.body).not.toContainEqual(
      expect.objectContaining({ email: 'hacker@zeta.com' })
    );
  });
});

```

13.5 Teste 4: Rate Limit

```
describe('Rate limiting por plano', () => {
  it('deve bloquear tenant Free após exceder limite', async () => {
    // Tenant 'omega' é free (100 req/min no teste)
    const promises = Array.from({ length: 110 }, (_, i) =>
      asTenant('omega')
        .get('/users')
        .then((r) => r.status)
    );

    const statuses = await Promise.all(promises);
    const tooManyRequests = statuses.filter((s) => s === 429);

    expect(tooManyRequests.length).toBeGreaterThan(0);
  });

  it('Enterprise não deve ser rate limited com mesma carga', async () => {
    // Tenant 'acme' é enterprise (10000 req/min)
    const promises = Array.from({ length: 110 }, () =>
      asTenant('acme')
        .get('/users')
        .then((r) => r.status)
    );

    const statuses = await Promise.all(promises);
    const tooManyRequests = statuses.filter((s) => s === 429);

    expect(tooManyRequests).toHaveLength(0);
  });
});
```

```

describe('Rate limiting por plano', () => {
  it('deve bloquear tenant Free após exceder limite', async () => {
    // Tenant 'omega' é free (100 req/min no teste)
    const promises = Array.from({ length: 110 }, (_, i) =>
      asTenant('omega')
        .get('/users')
        .then((r) => r.status)
    );

    const statuses = await Promise.all(promises);
    const tooManyRequests = statuses.filter((s) => s === 429);

    expect(tooManyRequests.length).toBeGreaterThan(0);
  });

  it('Enterprise não deve ser rate limited com mesma carga', async () => {
    // Tenant 'acme' é enterprise (10000 req/min)
    const promises = Array.from({ length: 110 }, () =>
      asTenant('acme')
        .get('/users')
        .then((r) => r.status)
    );

    const statuses = await Promise.all(promises);
    const tooManyRequests = statuses.filter((s) => s === 429);

    expect(tooManyRequests).toHaveLength(0);
  });
});

```

13.6 Teste 5: Migrations

```

describe('Migrations multi-tenant', () => {
  it('deve aplicar migration em todos os schemas de tenants ativos', async () => {
    const tenants = ['acme', 'zeta', 'omega'];

    for (const slug of tenants) {
      const result = await pool.query(`
        SELECT column_name
        FROM information_schema.columns
        WHERE table_schema = 'tenant_${slug}'
          AND table_name = 'users'
          AND column_name = 'phone'
      `);

      expect(result.rows.length).toBe(1);
      expect(result.rows[0].column_name).toBe('phone');
    }
  });
});

```

```
});
```

```
describe('Migrations multi-tenant', () => {
  it('deve aplicar migration em todos os schemas de tenants ativos', async () => {
    const tenants = ['acme', 'zeta', 'omega'];

    for (const slug of tenants) {
      const result = await pool.query(`
        SELECT column_name
        FROM information_schema.columns
        WHERE table_schema = 'tenant_${slug}'
          AND table_name = 'users'
          AND column_name = 'phone'
      `);

      expect(result.rows.length).toBe(1);
      expect(result.rows[0].column_name).toBe('phone');
    }
  });
});
```

```
});
```

13.7 Teste de Quebra Proposital

```
describe('Teste de quebra (fail-safe)', () => {
  it('deve falhar se removermos o WHERE tenant_id (prova de isolamento)', async () => {
    // Criar dados em tenants diferentes
    await asTenant('acme').post('/users').send({
      name: 'Dado sigiloso',
      email: 'secreto@acme.com',
    });

    await asTenant('zeta').post('/users').send({
      name: 'Outro dado',
      email: 'info@zeta.com',
    });

    // Query SEM filtro de tenant (simulando bug)
    const resultWithoutFilter = await pool.query('SELECT * FROM users');
    const allUsers = resultWithoutFilter.rows;

    // Se o isolamento está funcionando, essa query retorna dados de TODOS os tenants
    // Isso prova que sem o filtro, os dados vazam
    const tenantAcount = allUsers.filter((u: any) =>
      u.email.includes('acme')
    ).length;
    const tenantBcount = allUsers.filter((u: any) =>
      u.email.includes('zeta')
    ).length;

    expect(tenantAcount).toBeGreaterThan(0);
    expect(tenantBcount).toBeGreaterThan(0);

    // Com filtro, cada tenant vê apenas seus dados
    const resultWithFilter = await pool.query(
      'SELECT * FROM users WHERE tenant_id = $1',
      ['acme']
    );

    expect(resultWithFilter.rows).not.toContainEqual(
      expect.objectContaining({ email: 'info@zeta.com' })
    );
  });
});
```

Módulo 11 – Frontend: Interfaces Enterprise com Next.js

****Server Components, performance e acessibilidade.****

```
describe('Teste de quebra (fail-safe)', () => {
  it('deve falhar se removermos o WHERE tenant_id (prova de isolamento)', async () => {
    // Criar dados em tenants diferentes
    await asTenant('acme').post('/users').send({
      name: 'Dado sigiloso',
      email: 'secreto@acme.com',
    });

    await asTenant('zeta').post('/users').send({
      name: 'Outro dado',
      email: 'info@zeta.com',
    });

    // Query SEM filtro de tenant (simulando bug)
    const resultWithoutFilter = await pool.query('SELECT * FROM users');
    const allUsers = resultWithoutFilter.rows;

    // Se o isolamento está funcionando, essa query retorna dados de TODOS os tenants
    // Isso prova que sem o filtro, os dados vazam
    const tenantAcount = allUsers.filter((u: any) =>
      u.email.includes('acme')
    ).length;
    const tenantBcount = allUsers.filter((u: any) =>
      u.email.includes('zeta')
    ).length;

    expect(tenantAcount).toBeGreaterThan(0);
    expect(tenantBcount).toBeGreaterThan(0);

    // Com filtro, cada tenant vê apenas seus dados
    const resultWithFilter = await pool.query(
      'SELECT * FROM users WHERE tenant_id = $1',
      ['acme']
    );

    expect(resultWithFilter.rows).not.toContainEqual(
      expect.objectContaining({ email: 'info@zeta.com' })
    );
  });
});
```

Desenvolvimento Frontend

Módulo 11 – Frontend: Interfaces Enterprise com Next.js

****Server Components, performance e acessibilidade.****

1. Next.js App Router – A Nova Forma de Pensar

O App Router do Next.js 13+ mudou fundamentalmente como construímos aplicações React.

Server Components vs Client Components

Server Component (padrão):

Renderiza no servidor	Envia HTML para o cliente	Pode acessar banco, API, FS	Menos JavaScript no cliente	NÃO tem estado, effects, eventos
-----------------------	---------------------------	-----------------------------	-----------------------------	----------------------------------

Client Component ("use client"):

Renderiza no cliente	Pode usar useState, useEffect	Pode ter interatividade	Mais JavaScript no cliente
----------------------	-------------------------------	-------------------------	----------------------------

Regra de ouro

****Server Component por padrão. Client Component apenas quando precisar de interatividade.****

```
// Server Component (padrão)
async function ProductList() {
  const products = await prisma.product.findMany();
  return (
    <div>
      {products.map(p => <ProductCard key={p.id} product={p} />)}
    </div>
  );
}

// Client Component (só quando precisar de interação)
'use client';
function AddToCartButton({ productId }: { productId: string }) {
  const [loading, setLoading] = useState(false);
  return (
    <button onClick={handleAdd} disabled={loading}>
      {loading ? 'Adicionando...' : 'Adicionar ao carrinho'}
    </button>
  );
}
```

2. Data Fetching — Buscando Dados

Server-side fetching (recomendado)

```
// app/page.tsx – Server Component
async function DashboardPage() {
  // Fetch direto no servidor (sem useEffect!)
  const stats = await getDashboardStats();
  const recentOrders = await getRecentOrders();

  return (
    <div>
      <StatsCard data={stats} />
      <OrdersTable data={recentOrders} />
    </div>
  );
}

// utils/dashboard.ts
async function getDashboardStats(): Promise<Stats> {
  const [totalUsers, totalOrders, revenue] = await Promise.all([
    prisma.user.count(),
    prisma.order.count(),
    prisma.order.aggregate({ _sum: { total: true } }),
  ]);

  return { totalUsers, totalOrders, revenue: revenue._sum.total ?? 0 };
}
```

Cache e Revalidação

```
// Cache estático (build time) – dados que raramente mudam
async function getCategories() {
  const categories = await prisma.category.findMany();
  return categories;
}
// Por padrão, dados em Server Components são cacheados

// Revalidação por tempo (ISR)
async function getProducts() {
  const products = await prisma.product.findMany();
  return products;
}
export const revalidate = 60; // revalidar a cada 60 segundos

// Revalidação sob demanda (após mutation)
```

```
// Na Server Action:
revalidatePath('/products');
revalidateTag('products');
```

```
// Cache estático (build time) – dados que raramente mudam
async function getCategories() {
  const categories = await prisma.category.findMany();
  return categories;
}
// Por padrão, dados em Server Components são cacheados

// Revalidação por tempo (ISR)
async function getProducts() {
  const products = await prisma.product.findMany();
  return products;
}
export const revalidate = 60; // revalidar a cada 60 segundos

// Revalidação sob demanda (após mutation)
```


3. Loading, Error e Empty States

Loading com Suspense

```
// app/products/page.tsx
import { Suspense } from 'react';
import { ProductsList } from './products-list';
import { ProductsSkeleton } from './products-skeleton';

export default function ProductsPage() {
  return (
    <div>
      <h1>Produtos</h1>
      <Suspense fallback={<ProductsSkeleton />}>
        <ProductsList />
      </Suspense>
    </div>
  );
}

// Loading state visual
function ProductsSkeleton() {
  return (
    <div className="grid grid-cols-3 gap-4">
      {Array.from({ length: 6 }).map((_, i) => (
        <div key={i} className="animate-pulse space-y-2">
          <div className="h-48 bg-gray-200 rounded" />
          <div className="h-4 bg-gray-200 rounded w-3/4" />
          <div className="h-4 bg-gray-200 rounded w-1/2" />
        </div>
      ))}
    </div>
  );
}
```

Error Boundary

```
// app/products/error.tsx
'use client';
export default function Error({
  error,
  reset,
}): {
  error: Error & { digest?: string };
  reset: () => void;
} {
```

```

return (
  <div className="flex flex-col items-center justify-center p-8">
    <h2 className="text-xl font-semibold">Algo deu errado</h2>
    <p className="text-gray-500 mt-2">{error.message}</p>
    <button
      onClick={reset}
      className="mt-4 px-4 py-2 bg-primary text-white rounded"
    >
      Tentar novamente
    </button>
  </div>
);
}

```



```

// app/products/error.tsx
'use client';
export default function Error({
  error,
  reset,
}): {
  error: Error & { digest?: string };
  reset: () => void;
}) {

```

```

return (
  <div className="flex flex-col items-center justify-center p-8">
    <h2 className="text-xl font-semibold">Algo deu errado</h2>
    <p className="text-gray-500 mt-2">{error.message}</p>
    <button
      onClick={reset}
      className="mt-4 px-4 py-2 bg-primary text-white rounded"
    >
      Tentar novamente
    </button>
  </div>
);
}

```

Empty State

```

function ProductsList({ products }: { products: Product[] }) {
  if (products.length === 0) {
    return (
      <div className="text-center py-12">
        <PackageIcon className="mx-auto h-12 w-12 text-gray-400" />
        <h3 className="mt-2 text-lg font-medium">Nenhum produto</h3>
        <p className="mt-1 text-gray-500">
          Comece adicionando seu primeiro produto.
        </p>
        <button className="mt-4 px-4 py-2 bg-primary text-white rounded">
          Adicionar produto
        </button>
      </div>
    );
  }

  return (
    <div className="grid grid-cols-3 gap-4">
      {products.map(p => <ProductCard key={p.id} product={p} />)}
    </div>
  );
}

```

4. Formulários com Server Actions

```
// app/products/actions.ts
'use server';
import { z } from 'zod';
import { revalidatePath } from 'next/cache';

const ProductSchema = z.object({
  name: z.string().min(3),
  price: z.number().positive(),
  description: z.string().min(10),
});

export async function createProduct(formData: FormData) {
  const data = Object.fromEntries(formData);
  const result = ProductSchema.safeParse(data);

  if (!result.success) {
    return { error: result.error.flatten().fieldErrors };
  }

  await prisma.product.create({ data: result.data });
  revalidatePath('/products');
  return { success: true };
}

// app/products/new/page.tsx
import { createProduct } from './actions';

export default function NewProductPage() {
  return (
    <form action={createProduct}>
      <input name="name" placeholder="Nome" required />
      <input name="price" type="number" placeholder="Preço" step="0.01" required />
      <textarea name="description" placeholder="Descrição" required />
      <button type="submit">Salvar</button>
    </form>
  );
}
```

Validação client-side + server-side

```
'use client';
import { useFormState } from 'react-dom';
import { createProduct } from './actions';

const initialState = { error: null, success: false };
```



```

export function ProductForm() {
  const [state, formAction] = useFormState(createProduct, initialState);

  return (
    <form action={formAction} className="space-y-4">
      {state.error && (
        <div className="bg-red-50 text-red-600 p-3 rounded">
          {Object.entries(state.error).map(([field, messages]) => (
            <p key={field}>{field}: {(messages as string[]).join(', ')}</p>
          ))}
        </div>
      )}

      <input name="name" placeholder="Nome" required />
      <input name="price" type="number" placeholder="Preço" required />
      <textarea name="description" placeholder="Descrição" required />

      <button type="submit">Salvar</button>
    </form>
  );
}

```

```

'use client';
import { useFormState } from 'react-dom';
import { createProduct } from './actions';

const initialState = { error: null, success: false };

```


5. Gerenciamento de Estado

Estado local (useState)

```
'use client';
function Filters() {
  const [search, setSearch] = useState('');
  const [category, setCategory] = useState('all');
  const [sort, setSort] = useState<'asc' | 'desc'>('asc');

  return (
    <div>
      <input value={search} onChange={e => setSearch(e.target.value)} />
      <select value={category} onChange={e => setCategory(e.target.value)}>
        <option value="all">Todas</option>
        <option value="electronics">Eletrônicos</option>
      </select>
    </div>
  );
}
```

Estado global (Zustand)

```
// store/cart.ts
import { create } from 'zustand';

interface CartItem {
  id: string;
  name: string;
  price: number;
  quantity: number;
}

interface CartStore {
  items: CartItem[];
  addItem: (item: CartItem) => void;
  removeItem: (id: string) => void;
  clearCart: () => void;
  total: () => number;
}

export const useCartStore = create<CartStore>((set, get) => ({
  items: [],

  addItem: (item) => set(state => {
    const existing = state.items.find(i => i.id === item.id);
```

```

    if (existing) {
      return {
        items: state.items.map(i =>
          i.id === item.id ? { ...i, quantity: i.quantity + 1 } : i
        ),
      };
    }
    return { items: [...state.items, item] };
  })),

  removeItem: (id) => set(state => ({
    items: state.items.filter(i => i.id !== id),
  })),

  clearCart: () => set({ items: [] }),

  total: () => get().items.reduce((acc, i) => acc + i.price * i.quantity, 0),
}));

// Uso
function CartBadge() {
  const items = useCartStore(state => state.items);
  return <span>{items.length}</span>;
}

function AddToCartButton({ product }: { product: Product }) {
  const addItem = useCartStore(state => state.addItem);
  return (

```

```

// store/cart.ts
import { create } from 'zustand';

```

```

interface CartItem {
  id: string;
  name: string;
  price: number;
  quantity: number;
}

```

```

interface CartStore {
  items: CartItem[];
  addItem: (item: CartItem) => void;
  removeItem: (id: string) => void;
  clearCart: () => void;
  total: () => number;
}

```

```

export const useCartStore = create<CartStore>((set, get) => ({
  items: [],

  addItem: (item) => set(state => {
    const existing = state.items.find(i => i.id === item.id);

```


6. TanStack Query — Cache de Servidor

```
'use client';
import { useQuery } from '@tanstack/react-query';

// hooks/use-products.ts
export function useProducts(filters: ProductFilters) {
  return useQuery({
    queryKey: ['products', filters],
    queryFn: () => fetchProducts(filters),
    staleTime: 1000 * 60, // 1 minuto
  });
}

// Uso
function ProductsPage() {
  const [page, setPage] = useState(1);
  const { data, isLoading, error } = useProducts({ page });

  if (isLoading) return <ProductsSkeleton />;
  if (error) return <ErrorState />;

  return <ProductsTable data={data} />;
}
```

7. Acessibilidade (WCAG 2.1)

Princípios

Perceptível:	Todo conteúdo deve ser percebível (alternativas para mídia)
Operável:	Toda interface deve ser operável (teclado, voz)
Compreensível:	Conteúdo e interface devem ser compreensíveis
Robusto:	Conteúdo deve funcionar em diferentes tecnologias

Práticas essenciais

```
// ARIA labels em elementos interativos
<button aria-label="Fechar modal">
  <XIcon />
</button>

// Navegação por teclado
<button
  onKeyDown={e => {
    if (e.key === 'Enter' || e.key === ' ') handleClick();
  }}
>
  Confirmar
</button>

// Foco visível (não remover outline!)
<button className="focus:ring-2 focus:ring-primary">
  Salvar
</button>

// Labels em formulários
<label htmlFor="email">Email</label>
<input id="email" aria-describedby="email-hint" />
<p id="email-hint">Informe seu email corporativo</p>

// Anúncio de mudanças para screen readers
function Toast({ message }: { message: string }) {
  return (
    <div role="alert" aria-live="assertive">
      {message}
    </div>
  );
}
```

8. Responsividade com Tailwind

Mobile-first

```
// Sempre comece com mobile, depois adicione breakpoints
<div className="
  grid
  grid-cols-1          /* mobile: 1 coluna */
  sm:grid-cols-2       /* sm: 2 colunas */
  lg:grid-cols-3       /* lg: 3 colunas */
  xl:grid-cols-4       /* xl: 4 colunas */
  gap-4
">
  {items.map(item => <Card key={item.id} {...item} />)}
</div>
```

Design responsivo prático

```
// Sidebar que vira drawer em mobile
<aside className="
  fixed inset-y-0 left-0 w-64
  -translate-x-full     /* mobile: escondido */
  lg:translate-x-0      /* desktop: visível */
  lg:static             /* desktop: posição normal */
  transform transition-transform
  z-30
">
  <SidebarContent />
</aside>

// Tabela que vira cards em mobile
<div className="
  hidden md:block       /* desktop: tabela */
">
  <table>...</table>
</div>

<div className="
  md:hidden             /* mobile: cards */
">
  {items.map(item => <MobileCard key={item.id} item={item} />)}
</div>
```


9. Otimização de Performance

Imagens

```
import Image from 'next/image';

// next/image otimiza automaticamente
<Image
  src="/product.jpg"
  alt="Produto"
  width={400}
  height={300}
  priority={isAboveFold} // acima da dobra: carregar primeiro
  placeholder="blur"     // blur placeholder enquanto carrega
  blurDataURL="data:..." // tiny placeholder (gerado pelo next)
/>
```

Fontes

```
// app/layout.tsx
import { Inter } from 'next/font/google';

const inter = Inter({
  subsets: ['latin'],
  display: 'swap', // evitar layout shift
});

export default function RootLayout({ children }) {
  return (
    <html className={inter.className}>
      <body>{children}</body>
    </html>
  );
}
```

Dynamic Import

```
import dynamic from 'next/dynamic';

// Só carrega quando precisar
const HeavyChart = dynamic(() => import('./heavy-chart'), {
  loading: () => <ChartSkeleton />,
  ssr: false, // não renderizar no servidor
});
```

```
function Dashboard() {
  const [showChart, setShowChart] = useState(false);

  return (
    <div>
      <button onClick={() => setShowChart(true)}>Mostrar gráfico</button>
      {showChart && <HeavyChart />}
    </div>
  );
}
```

DevOps e Infraestrutura

Módulo 14 – DevOps: Docker, CI/CD e Deploy

****Da máquina do desenvolvedor à produção.****

1. Por que DevOps importa

DevOps é a ponte entre o ****código funcionando na máquina do dev**** e o ****código funcionando em produção****.

O problema clássico



```
import dynamic from 'next/dynamic';

// Só carrega quando precisar
const HeavyChart = dynamic(() => import('./heavy-chart'), {
  loading: () => <ChartSkeleton />,
  ssr: false, // não renderizar no servidor
});
```

```
function Dashboard() {
  const [showChart, setShowChart] = useState(false);

  return (
    <div>
      <button onClick={() => setShowChart(true)}>Mostrar gráfico</button>
      {showChart && <HeavyChart />}
    </div>
  );
}
```

DevOps e Infraestrutura

Módulo 14 – DevOps: Docker, CI/CD e Deploy

****Da máquina do desenvolvedor à produção.****

1. Por que DevOps importa

DevOps é a ponte entre o ****código funcionando na máquina do dev**** e o ****código funcionando em produção****.

O problema clássico

"Funciona na minha máquina" "Não funciona no servidor"

Causas: - Versões diferentes de dependências - Variáveis de ambiente não configuradas - Banco de dados diferente - Sistema operacional diferente - Permissões diferentes ```

A solução DevOps

Containerização (Docker):

- Mesmo ambiente em dev, staging e produção
- Dependências isoladas
- Reprodutível em qualquer máquina

CI/CD:

- Cada push é testado automaticamente
- Deploy é um processo, não um evento
- Rollback rápido se algo der errado

2. Docker — Containerização

O que é um container



Dockerfile multi-stage (o padrão correto)

```
# === ESTÁGIO 1: Instalação ===
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --only=production

# === ESTÁGIO 2: Build ===
FROM node:20-alpine AS builder
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build

# === ESTÁGIO 3: Runtime (imagem final LEVE) ===
FROM node:20-alpine AS runner
WORKDIR /app

# Segurança: não rodar como root
RUN addgroup --system --gid 1001 nodejs
RUN adduser --system --uid 1001 appuser

COPY --from=deps /app/node_modules ./node_modules
COPY --from=builder /app/dist ./dist
COPY package.json ./

USER appuser
```

EXPOSE 3000

HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
 CMD wget --no-verbose --tries=1 --spider http://localhost:3000/health || exit 1

ENV NODE_ENV=production
CMD ["node", "dist/main"]

```
# === ESTÁGIO 1: Instalação ===  
FROM node:20-alpine AS deps  
WORKDIR /app  
COPY package.json package-lock.json ./  
RUN npm ci --only=production  
  
# === ESTÁGIO 2: Build ===  
FROM node:20-alpine AS builder  
WORKDIR /app  
COPY package.json package-lock.json ./  
RUN npm ci  
COPY . .  
RUN npm run build  
  
# === ESTÁGIO 3: Runtime (imagem final LEVE) ===  
FROM node:20-alpine AS runner  
WORKDIR /app  
  
# Segurança: não rodar como root  
RUN addgroup --system --gid 1001 nodejs  
RUN adduser --system --uid 1001 appuser  
  
COPY --from=deps /app/node_modules ./node_modules  
COPY --from=builder /app/dist ./dist  
COPY package.json ./  
  
USER appuser
```

```
EXPOSE 3000
```

```
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \  
  CMD wget --no-verbose --tries=1 --spider http://localhost:3000/health || exit 1
```

```
ENV NODE_ENV=production  
CMD ["node", "dist/main"]
```

docker-compose.yml

```
services:  
  api:  
    build:  
      context: .  
      dockerfile: Dockerfile  
    ports:  
      - "3000:3000"  
    env_file: .env  
    depends_on:  
      db:  
        condition: service_healthy  
      redis:  
        condition: service_started  
    healthcheck:  
      test: ["CMD", "wget", "--no-verbose", "--tries=1", "--spider",  
"http://localhost:3000/health"]  
      interval: 30s  
      timeout: 3s  
      retries: 3  
  
  db:  
    image: postgres:16-alpine  
    ports:  
      - "5432:5432"  
    volumes:  
      - postgres_data:/var/lib/postgresql/data  
    env_file: .env  
    healthcheck:  
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]  
      interval: 10s  
      timeout: 5s  
      retries: 5  
  
  redis:  
    image: redis:7-alpine  
    ports:  
      - "6379:6379"  
    volumes:
```

```
    - redis_data:/data
healthcheck:
  test: ["CMD", "redis-cli", "ping"]
  interval: 10s
  timeout: 3s
  retries: 5
```

```
volumes:
  postgres_data:
  redis_data:
```

```
services:
  api:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "3000:3000"
    env_file: .env
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_started
    healthcheck:
      test: ["CMD", "wget", "--no-verbose", "--tries=1", "--spider",
"http://localhost:3000/health"]
      interval: 30s
      timeout: 3s
      retries: 3

  db:
    image: postgres:16-alpine
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    env_file: .env
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 10s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
```

```
    - redis_data:/data
healthcheck:
  test: ["CMD", "redis-cli", "ping"]
  interval: 10s
  timeout: 3s
  retries: 5

volumes:
  postgres_data:
  redis_data:
```

.dockerignore

```
node_modules
.git
.env
dist
.gitignore
*.md
coverage
```


3. GitHub Actions — CI/CD

Pipeline de CI (todo push)

```
name: CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck

  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16-alpine
        env:
          POSTGRES_USER: test
          POSTGRES_PASSWORD: test
          POSTGRES_DB: test
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'
```

```
- run: npm ci
- run: npm run test -- --coverage
- uses: actions/upload-artifact@v4
  with:
    name: coverage
    path: coverage/
```

```
name: CI
```

```
on:
```

```
  push:
```

```
    branches: [main, develop]
```

```
  pull_request:
```

```
    branches: [main]
```

```
jobs:
```

```
  lint:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck
```

```
  test:
```

```
    runs-on: ubuntu-latest
```

```
    services:
```

```
      postgres:
```

```
        image: postgres:16-alpine
        env:
          POSTGRES_USER: test
          POSTGRES_PASSWORD: test
          POSTGRES_DB: test
```

```
        ports:
```

```
          - 5432:5432
```

```
        options: >-
```

```
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
```

```
    steps:
```

```
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'
```

```
- run: npm ci
- run: npm run test -- --coverage
- uses: actions/upload-artifact@v4
  with:
    name: coverage
    path: coverage/
```

```
build:
  runs-on: ubuntu-latest
  needs: [lint, test]
  steps:
    - uses: actions/checkout@v4
    - run: docker build -t app:${{ github.sha }} .
    - run: docker tag app:${{ github.sha }} app:latest
```

Pipeline de CD (deploy em staging/produção)

```
name: CD

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    needs: [lint, test, build]
    environment: production

    steps:
      - uses: actions/checkout@v4

      - name: Login no Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKER_USERNAME }
          password: ${ secrets.DOCKER_PASSWORD }

      - name: Build e push da imagem
        run: |
          docker build -t app:${{ github.sha }} .
          docker tag app:${{ github.sha }} app:latest
          docker push app:${{ github.sha }}

      - name: Deploy via SSH
        uses: appleboy/ssh-action@v1
        with:
          host: ${ secrets.HOST }
          username: ${ secrets.USERNAME }
```

```
key: ${ secrets.SSH_KEY }}
script: |
  cd /app
  docker compose pull
  docker compose up -d --force-recreate
  docker system prune -f
```

name: CD

on:

push:
 branches: [main]

jobs:

deploy:
 runs-on: ubuntu-latest
 needs: [lint, test, build]
 environment: production

steps:

- uses: actions/checkout@v4
- name: Login no Docker Hub
 uses: docker/login-action@v3
 with:
 username: \${ secrets.DOCKER_USERNAME }}
 password: \${ secrets.DOCKER_PASSWORD }}
- name: Build e push da imagem
 run: |
 docker build -t app:\${ github.sha }} .
 docker tag app:\${ github.sha }} app:latest
 docker push app:\${ github.sha }}
- name: Deploy via SSH
 uses: appleboy/ssh-action@v1
 with:
 host: \${ secrets.HOST }}
 username: \${ secrets.USERNAME }}

4. Variáveis de Ambiente

.env.example (commitado)

```
# Database
DATABASE_URL=postgresql://user:password@localhost:5432/db

# JWT
JWT_SECRET=change-me
JWT_REFRESH_SECRET=change-me

# Redis
REDIS_URL=redis://localhost:6379

# API
PORT=3000
NODE_ENV=development
```

Validação na inicialização

```
import { z } from 'zod';

const EnvSchema = z.object({
  NODE_ENV: z.enum(['development', 'staging', 'production']),
  PORT: z.string().transform(Number).pipe(z.number().positive()),
  DATABASE_URL: z.string().url(),
  JWT_SECRET: z.string().min(32),
  JWT_REFRESH_SECRET: z.string().min(32),
  REDIS_URL: z.string().url(),
});

export type Env = z.infer<typeof EnvSchema>;

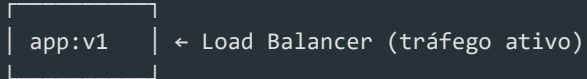
export function validateEnv(): Env {
  const result = EnvSchema.safeParse(process.env);
  if (!result.success) {
    console.error(' Variáveis de ambiente inválidas:', result.error.flatten());
    process.exit(1);
  }
  return result.data;
}

// main.ts
const env = validateEnv();
```

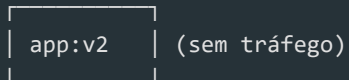
5. Estratégias de Deploy

Blue-Green Deployment

Versão Azul (atual):



Versão Verde (nova):



Passo 1: Deploy da versão verde

Passo 2: Testar versão verde

Passo 3: Troca LB para verde

Passo 4: Se erro, troca de volta (rollback imediato)

Rolling Update

Passo 1: Sobe nova instância com v2

Passo 2: Remove uma instância v1

Passo 3: Repete até todas serem v2

Prós: Sem downtime

Contras: Duas versões convivem

Canary Release

5% do tráfego → v2 (monitorar)

Se estável → 25% → 50% → 100%

Prós: Risco mínimo

Contras: Complexo de configurar

6. Docker Compose para múltiplos ambientes

docker-compose.override.yml (dev)

```
services:
  api:
    build:
      context: .
      dockerfile: Dockerfile.dev
    volumes:
      - ./app # Hot reload
      - /app/node_modules
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=development

  db:
    ports:
      - "5432:5432"

  redis:
    ports:
      - "6379:6379"
```

docker-compose.prod.yml (produção)

```
services:
  api:
    restart: always
    deploy:
      replicas: 3
      resources:
        limits:
          memory: 512M
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"

  db:
    restart: always
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./backup:/backup
```



```
deploy:
  resources:
    limits:
      memory: 1G
```



```
services:
  api:
    restart: always
    deploy:
      replicas: 3
      resources:
        limits:
          memory: 512M
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"

  db:
    restart: always
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./backup:/backup
```


7. Health Checks e Graceful Shutdown

Health check endpoint

```
@Controller('health')
export class HealthController {
  constructor(
    private prisma: PrismaService,
    private redis: RedisService,
  ) {}

  @Get()
  async check(): Promise<HealthResponse> {
    const checks = await Promise.all([
      this.checkComponent('database', () => this.prisma.$queryRaw`SELECT 1`),
      this.checkComponent('redis', () => this.redis.ping()),
    ]);

    return {
      status: checks.every(c => c.status === 'healthy') ? 'healthy' : 'degraded',
      timestamp: new Date().toISOString(),
      checks,
    };
  }

  private async checkComponent(name: string, check: () => Promise<unknown>) {
    try {
      await check();
      return { name, status: 'healthy' as const };
    } catch (error) {
      return { name, status: 'unhealthy' as const, error: (error as Error).message };
    }
  }
}

@Get('/ready')
async ready(): Promise<{ status: string }> {
  // Readiness = todas as dependências prontas
  return this.check();
}

@Get('/live')
async live(): Promise<{ status: string }> {
  // Liveness = servidor está rodando
  return { status: 'alive' };
}
```

Graceful Shutdown

```
import { PrismaService } from './prisma.service';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  app.enableShutdownHooks();

  const prisma = app.get(PrismaService);
  await prisma.enableShutdownHooks(app);

  await app.listen(3000);

  console.log(` Servidor rodando em http://localhost:3000`);
}

// main.ts
process.on('SIGTERM', async () => {
  console.log('SIGTERM recebido, desligando...');
  await app.close();
  process.exit(0);
});

process.on('SIGINT', async () => {
  console.log('SIGINT recebido, desligando...');
  await app.close();
  process.exit(0);
});
```

8. Logs Estruturados

Configuração de logs em JSON

```
import { Logger } from '@nestjsjs/common';
import { utilities as nestWinstonModuleUtilities } from 'nest-winston';
import * as winston from 'winston';

// main.ts
app.useLogger(
  winston.createLogger({
    level: process.env.NODE_ENV === 'production' ? 'info' : 'debug',
    format: winston.format.combine(
      winston.format.timestamp(),
      winston.format.json(), // Logs em JSON (melhor para ferramentas)
    ),
    transports: [
      new winston.transports.Console(),
      new winston.transports.File({ filename: 'logs/error.log', level: 'error' }),
      new winston.transports.File({ filename: 'logs/combined.log' }),
    ],
  })
);
```

Exemplo de log estruturado

```
{
  "timestamp": "2026-06-01T10:30:00.000Z",
  "level": "error",
  "message": "Falha na autenticação",
  "context": "AuthService",
  "userId": "abc123",
  "ip": "192.168.1.1",
  "attempt": 3,
  "stack": "...",
}
```

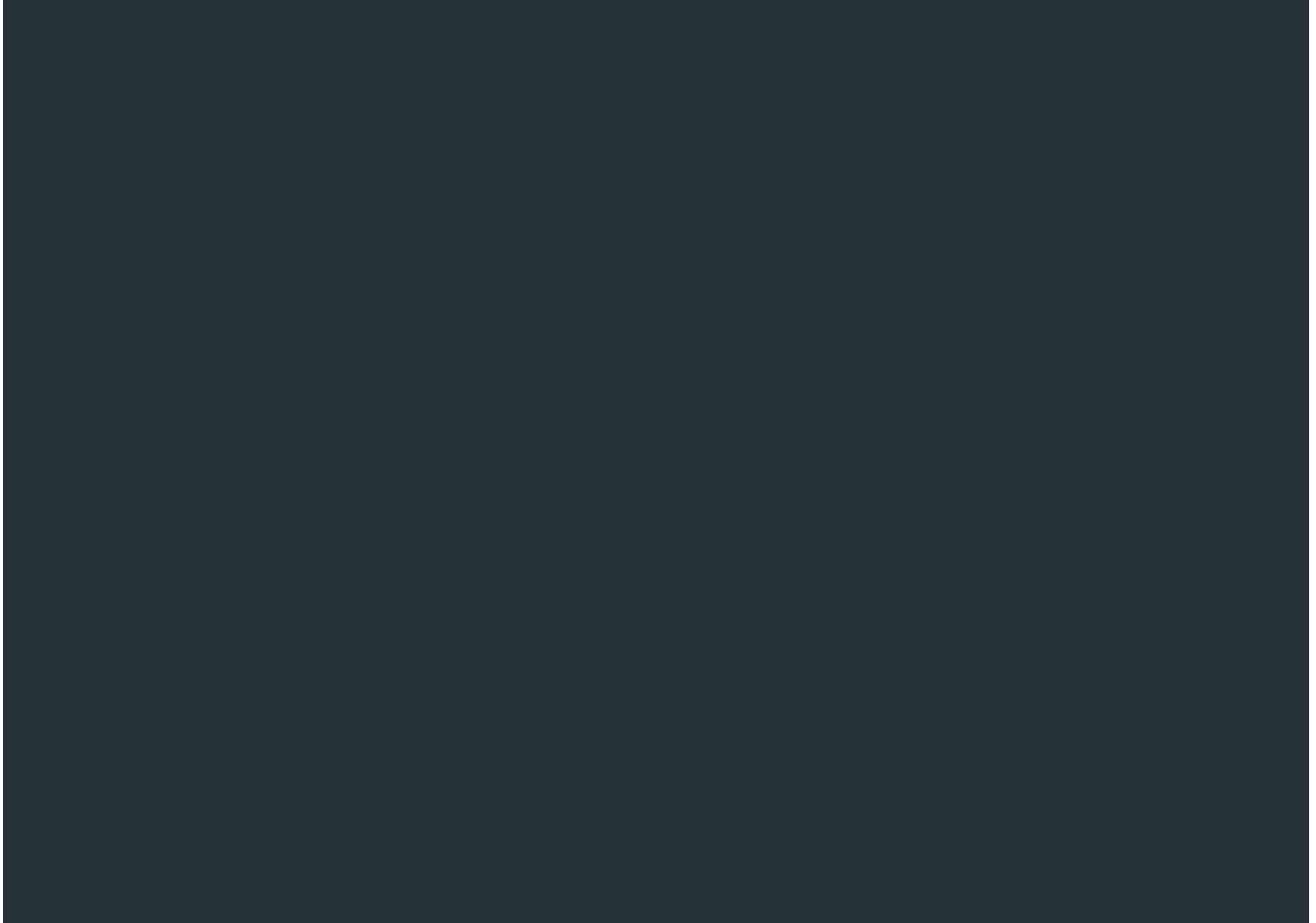
Qualidade e Testes

Módulo 15 – QA: Testes e Qualidade

****Código sem teste é legacy.****

1. Por que testar?

O custo de não testar



```
{
  "timestamp": "2026-06-01T10:30:00.000Z",
  "level": "error",
  "message": "Falha na autenticação",
  "context": "AuthService",
  "userId": "abc123",
  "ip": "192.168.1.1",
  "attempt": 3,
  "stack": "..."}
}
```

Qualidade e Testes

Módulo 15 – QA: Testes e Qualidade

****Código sem teste é legacy.****

Com testes:				Bug
em produção: o (teste pegou antes)		Correção: 30 min (teste indica o local)		
Regressão: o (testes previnem)		Confiança do time: alta ("posso refatorar")		
			...	

Unitários:	70%+ cobertura (services, utils)
Integração:	20% (endpoints, fluxos críticos)
E2E:	10% (fluxos principais, felizes e infelizes)

2. Testes Unitários com Jest

Estrutura de um teste

```
describe('UserService', () => {
  describe('create', () => {
    it('deve criar usuário com sucesso', async () => {
      // Arrange – preparar dados
      const dto: CreateUserDto = {
        name: 'João Silva',
        email: 'joao@email.com',
        password: 'Senha123',
      };

      // Act – executar
      const result = await service.create(dto);

      // Assert – verificar
      expect(result).toBeDefined();
      expect(result.name).toBe('João Silva');
      expect(result.email).toBe('joao@email.com');
    });

    it('deve lançar erro se email já existe', async () => {
      // Arrange
      repo.findByEmail.mockResolvedValue(existingUser);

      // Act + Assert
      await expect(
        service.create(mockDto)
      ).rejects.toThrow(ConflictException);
    });

    it('deve validar senha com bcrypt', async () => {
      const result = await service.create(mockDto);

      expect(bcrypt.hash).toHaveBeenCalledWith('Senha123', 12);
      expect(result.password).not.toBe('Senha123'); // hash != original
    });
  });
});
```

Mocks

```
// Mock completo de um repository
const mockUserRepo = {
```



```
    findById: jest.fn(),
    findByEmail: jest.fn(),
    save: jest.fn(),
    findAll: jest.fn(),
    softDelete: jest.fn(),
  };

  // Mock com retorno específico
  mockUserRepo.findById.mockResolvedValue({ id: '1', name: 'João' });
  mockUserRepo.findById.mockRejectedValue(new Error('DB error'));
  mockUserRepo.findById.mockResolvedValueOnce(user).mockResolvedValueOnce(null);

  // Verificar chamadas
  expect(mockUserRepo.save).toHaveBeenCalledTimes(1);
  expect(mockUserRepo.save).toHaveBeenCalledWith(expect.objectContaining({
    name: 'João',
  }));
```

```
// Mock completo de um repository
const mockUserRepo = {
```

```

    findById: jest.fn(),
    findByEmail: jest.fn(),
    save: jest.fn(),
    findAll: jest.fn(),
    softDelete: jest.fn(),
  };

  // Mock com retorno específico
  mockUserRepo.findById.mockResolvedValue({ id: '1', name: 'João' });
  mockUserRepo.findById.mockRejectedValue(new Error('DB error'));
  mockUserRepo.findById.mockResolvedValueOnce(user).mockResolvedValueOnce(null);

  // Verificar chamadas
  expect(mockUserRepo.save).toHaveBeenCalledTimes(1);
  expect(mockUserRepo.save).toHaveBeenCalledWith(expect.objectContaining({
    name: 'João',
  }));
}

```

Testando erros

```

describe('quando o banco falha', () => {
  it('deve lançar erro genérico', async () => {
    repo.save.mockRejectedValue(new Error('Connection refused'));

    await expect(service.create(dto)).rejects.toThrow(
      'Erro interno do servidor'
    );
  });
});

describe('quando dados são inválidos', () => {
  it.each([
    { name: '', email: 'joao@email.com', password: '123' },
    { name: 'João', email: 'invalido', password: 'Senha123' },
    { name: 'João', email: 'joao@email.com', password: '123' },
  ])('deve rejeitar dados inválidos: %p', async (invalidDto) => {
    await expect(service.create(invalidDto)).rejects.toThrow(
      BadRequestException
    );
  });
});

```

3. Testes de Integração com Supertest

```
import * as request from 'supertest';
import { Test } from '@nestjs/testing';
import { INestApplication } from '@nestjs/common';

describe('UserController (integration)', () => {
  let app: INestApplication;

  beforeAll(async () => {
    const moduleRef = await Test.createTestingModule({
      imports: [AppModule],
    }).compile();

    app = moduleRef.createNestApplication();
    await app.init();
  });

  afterAll(async () => {
    await app.close();
  });

  describe('POST /users', () => {
    it('deve criar usuário com dados válidos', async () => {
      const response = await request(app.getHttpServer())
        .post('/users')
        .send({
          name: 'João Silva',
          email: 'joao@email.com',
          password: 'Senha123',
        })
        .expect(201);

      expect(response.body).toHaveProperty('id');
      expect(response.body.name).toBe('João Silva');
    });

    it('deve retornar 400 com dados inválidos', async () => {
      await request(app.getHttpServer())
        .post('/users')
        .send({ name: '', email: 'invalido' })
        .expect(400);
    });

    it('deve retornar 409 se email já existe', async () => {
      // Primeiro cria
      await request(app.getHttpServer())
        .post('/users')
        .send(validDto)
```

```
.expect(201);
```

```
// Depois tenta criar de novo
```

```
await request(app.getHttpServer())
```

```
import * as request from 'supertest';
import { Test } from '@nestjs/testing';
import { INestApplication } from '@nestjs/common';
```

```
describe('UserController (integration)', () => {
  let app: INestApplication;

  beforeAll(async () => {
    const moduleRef = await Test.createTestingModule({
      imports: [AppModule],
    }).compile();

    app = moduleRef.createNestApplication();
    await app.init();
  });
```

```
afterAll(async () => {
  await app.close();
});
```

```
describe('POST /users', () => {
  it('deve criar usuário com dados válidos', async () => {
    const response = await request(app.getHttpServer())
      .post('/users')
      .send({
        name: 'João Silva',
        email: 'joao@email.com',
        password: 'Senha123',
      })
      .expect(201);

    expect(response.body).toHaveProperty('id');
    expect(response.body.name).toBe('João Silva');
  });
```

```
it('deve retornar 400 com dados inválidos', async () => {
  await request(app.getHttpServer())
    .post('/users')
    .send({ name: '', email: 'invalido' })
    .expect(400);
});
```

```
it('deve retornar 409 se email já existe', async () => {
  // Primeiro cria
  await request(app.getHttpServer())
    .post('/users')
    .send(validDto)
```


4. Testes E2E com Playwright

Configuração

```
// playwright.config.ts
import { defineConfig } from '@playwright/test';

export default defineConfig({
  testDir: './e2e',
  fullyParallel: true,
  retries: 1,
  workers: 3,
  reporter: 'html',
  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry',
    screenshot: 'only-on-failure',
  },
});
```

Testes

```
// e2e/login.spec.ts
import { test, expect } from '@playwright/test';

test.describe('Fluxo de autenticação', () => {
  test('deve fazer login com sucesso', async ({ page }) => {
    await page.goto('/login');

    await page.fill('[name="email"]', 'admin@empresa.com');
    await page.fill('[name="password"]', 'Admin123');
    await page.click('button[type="submit"]');

    await expect(page).toHaveURL('/dashboard');
    await expect(page.locator('text=Bem-vindo, Admin')).toBeVisible();
  });

  test('deve mostrar erro com credenciais inválidas', async ({ page }) => {
    await page.goto('/login');

    await page.fill('[name="email"]', 'invalido@email.com');
    await page.fill('[name="password"]', 'senha-errada');
    await page.click('button[type="submit"]');

    await expect(page.locator('[role="alert"]')).toContainText(
      'Credenciais inválidas'
    );
  });
});
```

```

    });
  });

  test('deve redirecionar para login se não autenticado', async ({ page }) => {
    await page.goto('/dashboard');

    await expect(page).toHaveURL('/login');
  });
});

```

```

test.describe('CRUD de produtos', () => {
  test.beforeEach(async ({ page }) => {
    // Login antes de cada teste
    await page.goto('/login');
    await page.fill('[name="email"]', 'admin@empresa.com');
    await page.fill('[name="password"]', 'Admin123');
    await page.click('button[type="submit"]');
  });

  test('deve criar um novo produto', async ({ page }) => {
    await page.goto('/products/new');

    await page.fill('[name="name"]', 'Produto Teste');
    await page.fill('[name="price"]', '99.90');
    await page.fill('[name="description"]', 'Descrição do produto teste');

    await page.click('button[type="submit"]');
  });
});

```

```

// e2e/login.spec.ts
import { test, expect } from '@playwright/test';

test.describe('Fluxo de autenticação', () => {
  test('deve fazer login com sucesso', async ({ page }) => {
    await page.goto('/login');

    await page.fill('[name="email"]', 'admin@empresa.com');
    await page.fill('[name="password"]', 'Admin123');
    await page.click('button[type="submit"]');

    await expect(page).toHaveURL('/dashboard');
    await expect(page.locator('text=Bem-vindo, Admin')).toBeVisible();
  });
});

```

```

test('deve mostrar erro com credenciais inválidas', async ({ page }) => {
  await page.goto('/login');

  await page.fill('[name="email"]', 'invalido@email.com');
  await page.fill('[name="password"]', 'senha-errada');
  await page.click('button[type="submit"]');

  await expect(page.locator('[role="alert"]')).toContainText(
    'Credenciais inválidas'
  );
});

```


5. Cobertura de Código

Configuração do Jest

```
// jest.config.ts
export default {
  collectCoverageFrom: [
    'src/**/*.service.ts',
    'src/**/*.use-case.ts',
    'src/**/*.controller.ts',
    '!src/**/*.module.ts',
    '!src/main.ts',
  ],
  coverageThreshold: {
    global: {
      branches: 80,
      functions: 80,
      lines: 80,
      statements: 80,
    },
  },
};
```

O que cobrir

Services:	branches (if/else), funções, linhas
Use Cases:	fluxos felizes e infelizes
Controllers:	validação, autorização, respostas
Módulos:	só declaram providers/imports
Config:	valores estáticos
DTOs:	schemas Zod (testados indiretamente)

6. GitHub Actions com Testes

```
name: CI

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:16-alpine
        env:
          POSTGRES_USER: test
          POSTGRES_PASSWORD: test
          POSTGRES_DB: test
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - run: npm ci

      - name: Lint
        run: npm run lint

      - name: Type check
        run: npm run typecheck

      - name: Unit tests
        run: npm run test -- --coverage

      - name: E2E tests
        run: npm run test:e2e

      - name: Upload coverage
        uses: actions/upload-artifact@v4
        with:
```

```
name: coverage
path: coverage/
```

```
name: CI
```

```
on: [push, pull_request]
```

```
jobs:
```

```
  test:
```

```
    runs-on: ubuntu-latest
```

```
  services:
```

```
    postgres:
```

```
      image: postgres:16-alpine
```

```
      env:
```

```
        POSTGRES_USER: test
```

```
        POSTGRES_PASSWORD: test
```

```
        POSTGRES_DB: test
```

```
      ports:
```

```
        - 5432:5432
```

```
      options: >-
```

```
        --health-cmd pg_isready
```

```
        --health-interval 10s
```

```
        --health-timeout 5s
```

```
        --health-retries 5
```

```
  steps:
```

```
    - uses: actions/checkout@v4
```

```
    - uses: actions/setup-node@v4
```

```
      with:
```

```
        node-version: 20
```

```
        cache: 'npm'
```

```
    - run: npm ci
```

```
    - name: Lint
```

```
      run: npm run lint
```

```
    - name: Type check
```

```
      run: npm run typecheck
```

```
    - name: Unit tests
```

```
      run: npm run test -- --coverage
```

```
    - name: E2E tests
```

```
      run: npm run test:e2e
```

```
    - name: Upload coverage
```

```
      uses: actions/upload-artifact@v4
```

```
      with:
```


7. TDD — Test-Driven Development

Ciclo TDD

Red: Escreva um teste que falha
Green: Faça o teste passar (código mínimo)
Refactor: Melhore o código mantendo os verdes

Exemplo prático

```
// 1. RED – Escrever teste primeiro
describe('OrderService', () => {
  it('não deve criar pedido com valor abaixo do mínimo', () => {
    const items = [{ productId: '1', price: 5, quantity: 1 }]; // total: 5
    const MIN_ORDER_VALUE = 10;

    expect(() => service.create({ items })).toThrow(
      'Valor mínimo do pedido é R$ 10,00'
    );
  });
});

// 2. GREEN – Código mínimo para passar
class OrderService {
  create(input: CreateOrderInput): Order {
    const total = input.items.reduce((acc, i) => acc + i.price * i.quantity, 0);

    if (total < 10) {
      throw new Error('Valor mínimo do pedido é R$ 10,00');
    }

    return Order.create(input);
  }
}

// 3. REFACTOR – Melhorar sem quebrar testes
// Extrair constante, melhorar mensagem

# Observabilidade

# Módulo 16 – Observabilidade: Logs, Métricas e Tracing

**Entendendo o que acontece em produção.**
```

1. O que é observabilidade?

Observabilidade é a capacidade de **entender o estado interno do sistema** a partir de seus outputs externos (logs, métricas, traces).

Os 3 pilares

```
// 1. RED – Escrever teste primeiro
describe('OrderService', () => {
  it('não deve criar pedido com valor abaixo do mínimo', () => {
    const items = [{ productId: '1', price: 5, quantity: 1 }]; // total: 5
    const MIN_ORDER_VALUE = 10;

    expect(() => service.create({ items })).toThrow(
      'Valor mínimo do pedido é R$ 10,00'
    );
  });
});

// 2. GREEN – Código mínimo para passar
class OrderService {
  create(input: CreateOrderInput): Order {
    const total = input.items.reduce((acc, i) => acc + i.price * i.quantity, 0);

    if (total < 10) {
      throw new Error('Valor mínimo do pedido é R$ 10,00');
    }

    return Order.create(input);
  }
}

// 3. REFACTOR – Melhorar sem quebrar testes
// Extrair constante, melhorar mensagem
```

Observabilidade

Módulo 16 – Observabilidade: Logs, Métricas e Tracing

****Entendendo o que acontece em produção.****

1. O que é observabilidade?

Observabilidade é a capacidade de **entender o estado interno do sistema** a partir de seus outputs externos (logs, métricas, traces).

Os 3 pilares

LOGS	MÉTRICAS	TRACING	Eventos discretos	Dados
agregados	Fluxo de requisições	"O quê aconteceu?"	"Quantos/quanto tempo?"	
"Por onde passou?"				
Ex:	Ex:	Ex: "Usuário X fez login"	"500 req/s, p95=200ms"	
"GET /orders Auth DB"	"Falha no pagamento"	"Error rate: 0.5%"	"POST	
/payment API Queue"	```			

Sem observabilidade

```
Usuário: "O sistema está lento"
Dev:     "Onde? Quando? Quanto?"
Usuário: "Não sei, só está lento"
Dev:     "Não consigo reproduzir"
```

→ Sem dados, sem diagnóstico

Com observabilidade

```
Grafana: "p95 response time subiu de 200ms para 2s às 14:30"
Logs:    "14:30:01 - DB query timeout"
Trace:   "14:30:01 - span database: 5s (normal: 50ms)"
```

→ Causa identificada em minutos

2. Logs Estruturados

Log em texto vs JSON

```
// Log em texto (não searchável)
console.log('Usuário', userId, 'fez login de', ip);

// Log estruturado (JSON)
logger.info('Usuário autenticado', {
  userId: 'abc123',
  email: 'joao@email.com',
  ip: '192.168.1.1',
  timestamp: new Date().toISOString(),
});
```

Configuração com Winston

```
import * as winston from 'winston';

export const logger = winston.createLogger({
  level: process.env.NODE_ENV === 'production' ? 'info' : 'debug',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }),
    winston.format.json(),
  ),
  defaultMeta: { service: 'api' },
  transports: [
    new winston.transports.Console({
      format: process.env.NODE_ENV === 'production'
        ? winston.format.json()
        : winston.format.prettyPrint(),
    }),
    new winston.transports.File({
      filename: 'logs/error.log',
      level: 'error',
      maxsize: 10 * 1024 * 1024, // 10MB
      maxFiles: 5,
    }),
    new winston.transports.File({
      filename: 'logs/combined.log',
      maxsize: 10 * 1024 * 1024,
      maxFiles: 5,
    }),
  ],
});
```


Níveis de log

```
error:   Algo quebrou (ação necessária)
warn:   Algo inesperado (mas não quebrou)
info:   Evento importante (login, criação)
debug:  Detalhes para diagnóstico
verbose: Tudo (usar apenas em dev)
```

O que logar

```
// Logar sempre
logger.info('Pedido criado', { orderId, userId, total });
logger.warn('Tentativa de login com IP suspeito', { ip, email });
logger.error('Falha no processamento de pagamento', {
  orderId, error: err.message, stack: err.stack,
});

// NUNCA logar
logger.info('Usuário logado', { password: '123456' }); // Senha!
logger.info('Pagamento processado', { cvv: '123' }); // Dado sensível!
logger.info('Erro', { fullUser }); // Objeto inteiro!
```

3. Métricas

Tipos de métricas

Counter:	Valor que só aumenta (req total, errors total)
Gauge:	Valor que sobe e desce (memória, conexões ativas)
Histogram:	Distribuição de valores (response time p50, p95, p99)
Summary:	Similar ao histogram (latência, tamanho de resposta)

Métricas essenciais

```
// API
http_requests_total{method, path, status}
http_request_duration_seconds{method, path}
http_errors_total{method, path, error_code}

// Database
db_queries_total{operation, table}
db_query_duration_seconds{operation, table}
db_connections_active
db_connections_idle

// Business
users_active_total
orders_created_total
orders_revenue_total

// System
process_cpu_seconds_total
process_resident_memory_bytes
nodejs_eventloop_lag_seconds
```

Implementação com Prometheus client

```
import { Counter, Histogram, Gauge } from 'prom-client';

// Métricas
export const httpRequestsTotal = new Counter({
  name: 'http_requests_total',
  help: 'Total de requisições HTTP',
  labelNames: ['method', 'path', 'status'],
});

export const httpRequestDuration = new Histogram({
```

```

    name: 'http_request_duration_seconds',
    help: 'Duração das requisições HTTP',
    labelNames: ['method', 'path'],
    buckets: [0.01, 0.05, 0.1, 0.5, 1, 2, 5],
  });

export const dbQueryDuration = new Histogram({
  name: 'db_query_duration_seconds',
  help: 'Duração de queries no banco',
  labelNames: ['operation', 'table'],
  buckets: [0.001, 0.005, 0.01, 0.05, 0.1, 0.5, 1],
});

export const activeConnections = new Gauge({
  name: 'http_connections_active',
  help: 'Conexões HTTP ativas',
});

// Middleware para capturar métricas
@Injectable()
export class MetricsInterceptor implements NestInterceptor {
  intercept(context: ExecutionContext, next: CallHandler): Observable<any> {
    const request = context.switchToHttp().getRequest();
    const { method, path } = request;
    const start = Date.now();

    activeConnections.inc();

    return next.handle().pipe(
      tap(() => {
        const duration = (Date.now() - start) / 1000;
        const response = context.switchToHttp().getResponse();
        const status = response.statusCode.toString();

        httpRequestsTotal.labels(method, path, status).inc();
        httpRequestDuration.labels(method, path).observe(duration);
        activeConnections.dec();
      }),
    );
  }
}

import { Counter, Histogram, Gauge } from 'prom-client';

// Métricas
export const httpRequestsTotal = new Counter({
  name: 'http_requests_total',
  help: 'Total de requisições HTTP',
  labelNames: ['method', 'path', 'status'],
});

export const httpRequestDuration = new Histogram({

```


4. Tracing Distribuído

O que é tracing

Requisição: POST /api/orders

```
| Trace ID: abc123  
|  
|   Span 1: Gateway (2ms)  
|   |   └─ Span 2: Auth Service (5ms)  
|   |   └─ Span 3: Order Service (150ms)  
|   |       └─ Span 4: DB Query (100ms)  
|   |           └─ Span 5: Redis Cache (20ms)  
|   └─ Span 6: Payment API (300ms)  
|  
| Total: 457ms
```

Implementação com OpenTelemetry

```
import { NodeSDK } from '@opentelemetry/sdk-node';  
import { OTLPTraceExporter } from '@opentelemetry/exporter-trace-otlp-http';  
import { Resource } from '@opentelemetry/resources';  
import { SemanticResourceAttributes } from '@opentelemetry/semantic-conventions';  
import { getNodeAutoInstrumentations } from '@opentelemetry/auto-instrumentations-node';  
  
const sdk = new NodeSDK({  
  resource: new Resource({  
    [SemanticResourceAttributes.SERVICE_NAME]: 'api',  
    [SemanticResourceAttributes.SERVICE_VERSION]: '1.0.0',  
  }),  
  traceExporter: new OTLPTraceExporter({  
    url: 'http://localhost:4318/v1/traces',  
  }),  
  instrumentations: [  
    getNodeAutoInstrumentations({  
      '@opentelemetry/instrumentation-http': {},  
      '@opentelemetry/instrumentation-express': {},  
      '@opentelemetry/instrumentation-nestjs-core': {},  
      '@opentelemetry/instrumentation-pg': {},  
      '@opentelemetry/instrumentation-redis': {},  
    }),  
  ],  
});  
  
sdk.start();
```

Span customizado

```
import { trace } from '@opentelemetry/api';

const tracer = trace.getTracer('order-service');

class OrderService {
  async create(dto: CreateOrderDto): Promise<Order> {
    return tracer.startActiveSpan('createOrder', async (span) => {
      span.setAttribute('order.total', dto.total);
      span.setAttribute('order.items_count', dto.items.length);

      try {
        const user = await this.validateUser(dto.userId);
        const order = await this.saveOrder(dto);
        await this.processPayment(order);

        span.setStatus({ code: SpanStatusCode.OK });
        return order;
      } catch (error) {
        span.setStatus({
          code: SpanStatusCode.ERROR,
          message: error.message,
        });
        throw error;
      } finally {
        span.end();
      }
    });
  }
}
```

5. Dashboards com Grafana

Métricas essenciais no dashboard

RED Method (Rate, Errors, Duration):

Métrica	Descrição	Tipo
Request Rate	req/s por endpoint	Counter
Error Rate	% de erros	Counter
Latency (p50/p95/p99)	Tempo de resposta	Histogram
Active Users	Usuários simultâneos	Gauge
CPU/Memory	Recursos do servidor	Gauge
DB Connections	Conexões ativas no banco	Gauge
Query Duration	Duração de queries	Histogram
Error Logs Rate	Quantidade de logs de erro	Counter

Painel de resposta rápida

STATUS: OPERACIONAL		Uptime: 99.97%
Request Rate	Error Rate	p95 Latency
1,234 req/s	0.05%	187ms
Latency by Endpoint (p95)		
GET /products	45ms	
POST /orders	320ms	
POST /payment	1.2s	
Top Errors		
14:30:01 - POST /payment - timeout (5x)		
14:29:00 - GET /products - connection refused (2x)		

6. Alertas

O que alertar

P0 (Resposta imediata):

- Error rate > 5% nos últimos 5 min
- p95 latency > 2s
- Servidor down
- Banco inacessível

P1 (Resposta em 1 hora):

- Error rate > 1% nos últimos 15 min
- p95 latency > 1s
- Disco > 80%
- Memória > 85%

P2 (Resposta em 24h):

- Error rate > 0.5% no último dia
- p99 latency > 3s
- Cache hit rate < 50%
- Queries lentas (> 500ms)

Exemplo de alerta (Prometheus + Alertmanager)

```
groups:
- name: api
  rules:
    - alert: HighErrorRate
      expr: |
        rate(http_errors_total[5m])
        /
        rate(http_requests_total[5m])
        > 0.05
      for: 5m
      labels:
        severity: critical
      annotations:
        summary: "Error rate above 5%"
        description: "Error rate is {{$value | humanize}}% for the last 5 minutes"

    - alert: HighLatency
      expr: |
        histogram_quantile(0.95,
          rate(http_request_duration_seconds_bucket[5m])
        ) > 2
      for: 5m
```



```
labels:
  severity: critical
annotations:
  summary: "p95 latency above 2s"
```



```
groups:
- name: api
  rules:
  - alert: HighErrorRate
    expr: |
      rate(http_errors_total[5m])
      /
      rate(http_requests_total[5m])
      > 0.05
    for: 5m
    labels:
      severity: critical
    annotations:
      summary: "Error rate above 5%"
      description: "Error rate is {{$value | humanize}}% for the last 5 minutes"

  - alert: HighLatency
    expr: |
      histogram_quantile(0.95,
        rate(http_request_duration_seconds_bucket[5m])
      ) > 2
    for: 5m
```


7. Centralização de Logs (Loki + Grafana)

Configuração

```
# docker-compose.obs.yml
services:
  prometheus:
    image: prom/prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml

  loki:
    image: grafana/loki
    ports:
      - "3100:3100"

  grafana:
    image: grafana/grafana
    ports:
      - "3000:3000"
    environment:
      - GF_AUTH_ANONYMOUS_ENABLED=true
```

Query de logs no Loki

```
# Buscar erros nos últimos 15 minutos
{service="api"} |= "error" |= "payment"

# Buscar logs de um usuário específico
{service="api"} |= "userId=abc123"

# Contar erros por endpoint
sum by (path) (count_over_time({service="api"} |= "error" [5m]))
```

Resumo

1. **Observabilidade** = Logs + Métricas + Tracing 2. **Logs estruturados** — JSON, níveis (error/warn/info/debug), nunca dados sensíveis 3. **Métricas** — RED method (Rate, Errors, Duration) com Prometheus 4. **Tracing** — OpenTelemetry para rastrear requisições entre serviços 5. **Dashboards** — Grafana com métricas essenciais 6. **Alertas** — Po/P1/P2 com thresholds e canais de notificação 7. **Centralização** — Loki + Grafana para logs searcháveis

Governança de Software

Módulo 17 — Governança

Código com responsabilidade.

1. O que é Governança de TI

Governança de TI é o **sistema de regras, processos e práticas** que garante que a área de tecnologia entregue valor alinhado aos objetivos do negócio, com riscos controlados e recursos otimizados.

Por que governança importa

Sem governança:

- Cada time faz do seu jeito
- Código sem padrão, difícil de manter
- Segredos vazam para o repositório
- Deploy é um evento estressante
- Ninguém sabe por que uma decisão foi tomada
- Auditoria vira um pesadelo

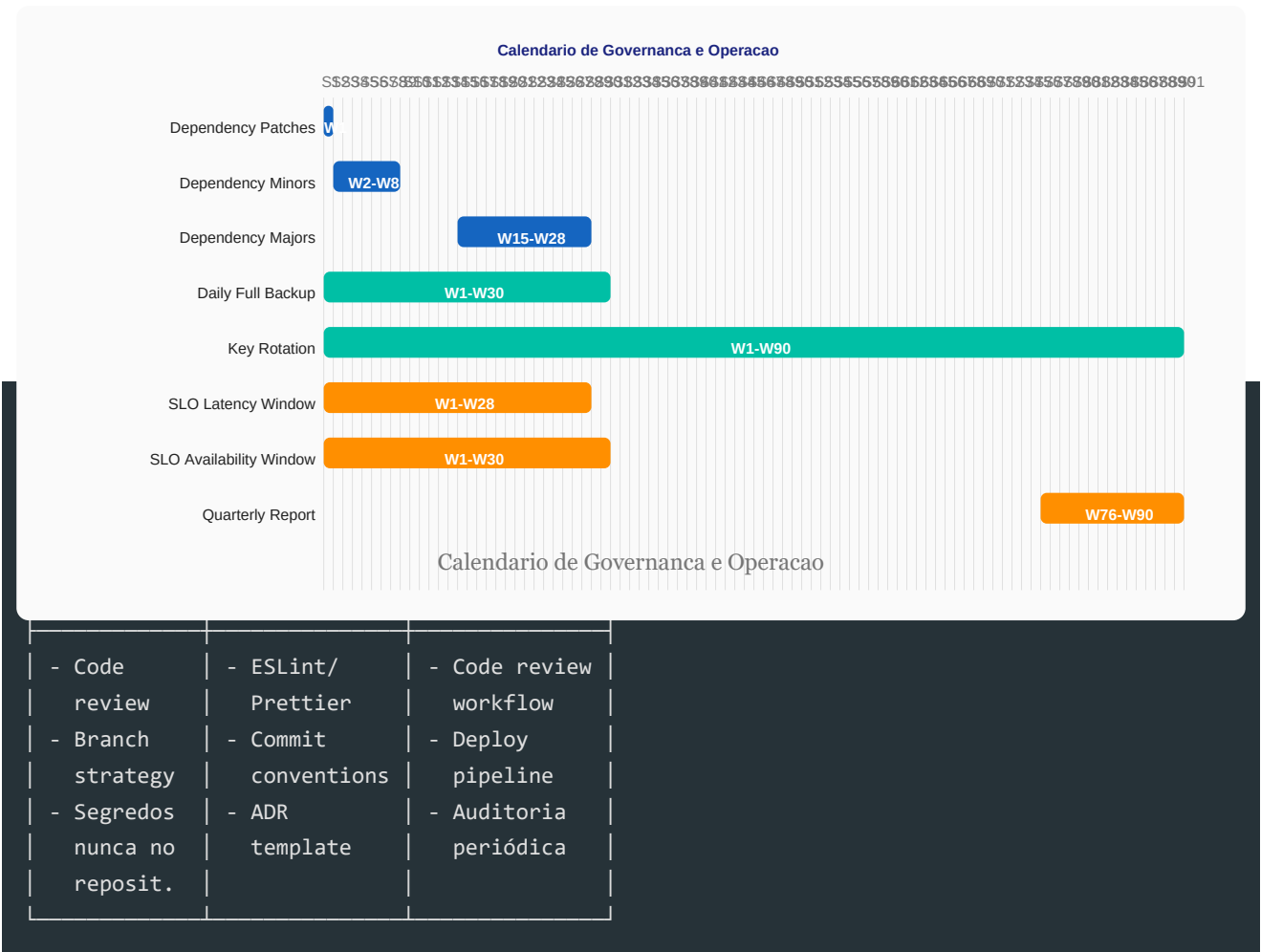
Com governança:

- Padrões claros, código consistente
- Segurança por padrão
- Deploy previsível e seguro
- Decisões documentadas e rastreáveis
- Auditoria tranquila

Frameworks de Governança

Framework	Foco	O que entrega
COBIT	Governança e gestão de TI	Alinhamento estratégico,
ITIL	Gestão de serviços de TI	Catálogo de serviços, inc
ISO 38500	Governança corporativa de	Princípios para uso respo

Para o time de desenvolvimento, COBIT e ITIL definem **o que** precisa ser feito; as práticas deste módulo definem **como** fazer.



Calendar of operational governance (cadencia de tarefas e janelas):

3. Code Review

Code review é a **prática mais impactante** de governança de código. Mais do que encontrar bugs, é um processo de **transferência de conhecimento e garantia de qualidade**.

Políticas de Code Review

Obrigatório para todo PR que:

- Altere lógica de negócio
- Adicione ou remova dependências
- Modifique APIs públicas
- Altere configurações de infraestrutura
- Mexa em banco de dados (migrations)

Pode pular code review:

- Correção de typo em comentário
- Alteração em arquivo .gitignore
- Documentação simples
- Hotfix com autorização do tech lead

Checklist de Code Review

- ☐ A solução resolve o problema proposto?
- ☐ Existem testes para a mudança?
- ☐ Testes passam?
- ☐ Cobertura adequada (novos cenários)?
- ☐ Nomeação clara (variáveis, funções, classes)?
- ☐ Código segue os padrões do projeto (lint)?
- ☐ Dependências foram adicionadas corretamente?
- ☐ Breaking changes foram identificadas?
- ☐ Documentação foi atualizada?
- ☐ Segredos/credenciais no código? (NUNCA!)
- ☐ Performance: impacto aceitável?

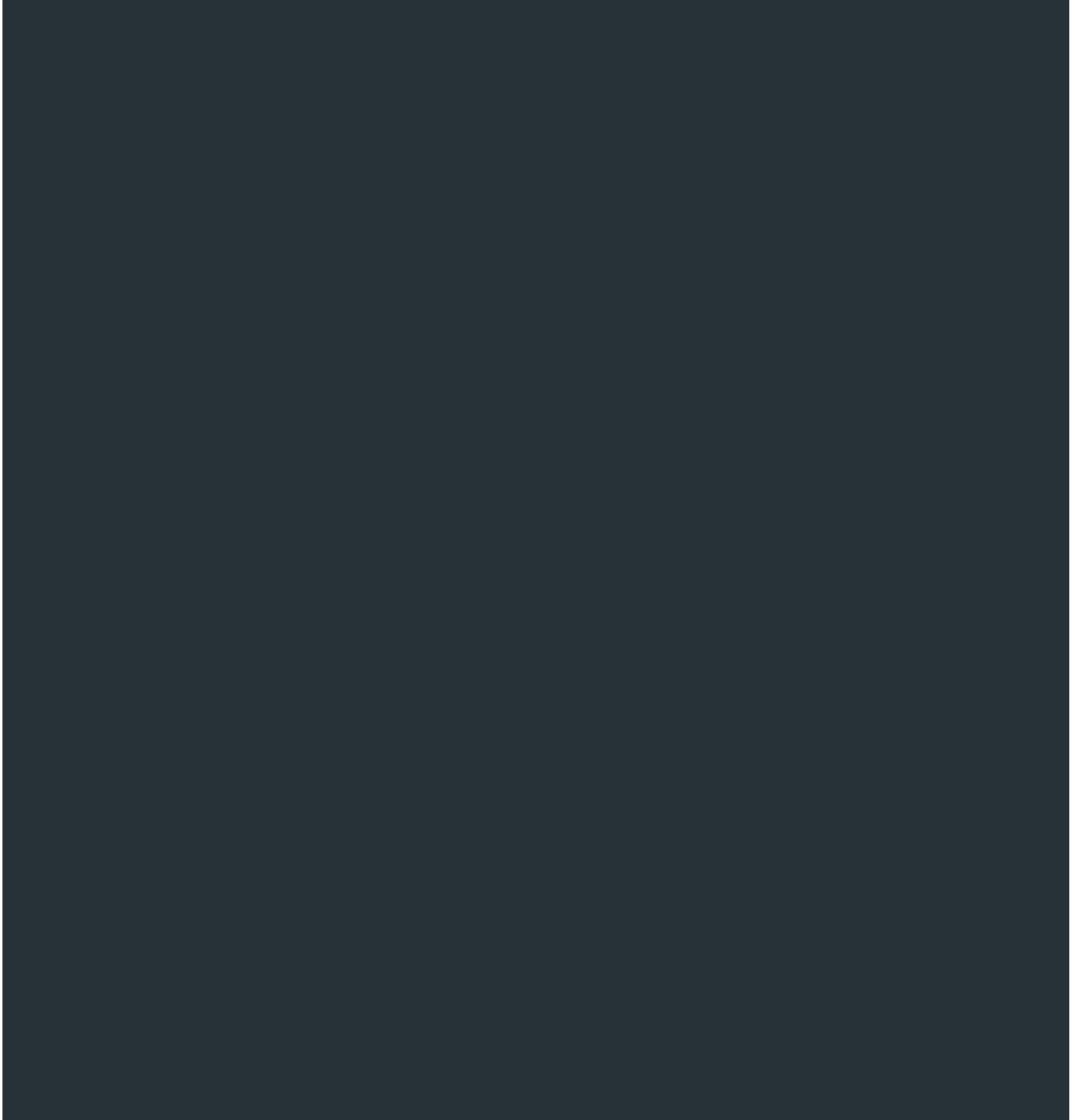
Approval Gates

PR aberto

- Gate 1: CI passa (lint + testes + build)
- Gate 2: Code review aprovado (mínimo 1 approval)
- Gate 3: Aprovação de segurança (se aplicável)
- Gate 4: Aprovação de arquitetura (se mudança estrutural)
- Merge permitido

Proteções de branch (GitHub):

- Requer pull request antes de merge
- Requer 1+ approval
- Requer CI passar
- Desabilitar merge direto na main
- Stale reviews são descartados após novo push



PR aberto

- Gate 1: CI passa (lint + testes + build)
- Gate 2: Code review aprovado (mínimo 1 approval)
- Gate 3: Aprovação de segurança (se aplicável)
- Gate 4: Aprovação de arquitetura (se mudança estrutural)
- Merge permitido

Fluxo de Code Review



Template de PR

Descrição

<!-- O que este PR faz? Por quê? -->

Tipo de mudança

- ☐ Bug fix
- ☐ Nova feature
- ☐ Breaking change
- ☐ Refatoração
- ☐ Documentação

Como testar

1. `npm run dev`
2. Acesse `/rota`
3. Verifique que [comportamento esperado]

Checklist

- ☐ Testes unitários/integração
- ☐ Testes manuais realizados
- ☐ Lint passa (`npm run lint`)
- ☐ Documentação atualizada
- ☐ Nenhum segredo vazou

Screenshots (se aplicável)

Linked Issues

Closes #123

4. Padrões de Código

Padrões de código são **regras automatizadas** que garantem consistência sem depender de vontade individual.

ESLint

```
// .eslintrc.json – Configuração enterprise
{
  "env": {
    "es2022": true,
    "node": true
  },
  "extends": [
    "eslint:recommended",
    "plugin:@typescript-eslint/recommended",
    "plugin:prettier/recommended"
  ],
  "parser": "@typescript-eslint/parser",
  "parserOptions": {
    "ecmaVersion": "latest",
    "sourceType": "module"
  },
  "rules": {
    "@typescript-eslint/no-unused-vars": ["error", { "argsIgnorePattern": "^_" }],
    "@typescript-eslint/explicit-function-return-type": "warn",
    "no-console": ["warn", { "allow": ["warn", "error"] }],
    "eqeqeq": ["error", "always"],
    "curly": ["error", "all"],
    "no-throw-literal": "error",
    "prefer-const": "error"
  }
}
```

Prettier

```
// .prettierrc
{
  "semi": true,
  "trailingComma": "all",
  "singleQuote": true,
  "printWidth": 100,
  "tabWidth": 2,
  "arrowParens": "always",
  "endOfLine": "lf"
}
```

```
}
```

```
// .prettierrc
{
  "semi": true,
  "trailingComma": "all",
  "singleQuote": true,
  "printWidth": 100,
  "tabWidth": 2,
  "arrowParens": "always",
  "endOfLine": "lf"
```

```
}
```

EditorConfig

```
# .editorconfig
root = true

[*]
indent_style = space
indent_size = 2
end_of_line = lf
charset = utf-8
trim_trailing_whitespace = true
insert_final_newline = true

[*.md]
trim_trailing_whitespace = false
```

Husky + lint-staged + commitlint

```
// .husky/commit-msg
// npx --no -- commitlint --edit $1

// .husky/pre-commit
// npx lint-staged
```

```
// package.json (trechos relevantes)
{
  "scripts": {
    "prepare": "husky",
    "lint": "eslint . --ext .ts,.tsx",
    "lint:fix": "eslint . --ext .ts,.tsx --fix",
    "format": "prettier --write ."
  },
  "lint-staged": {
    "*.{ts,tsx}": ["eslint --fix", "prettier --write"],
    "*.{json,md,yaml,yml}": ["prettier --write"]
  },
  "commitlint": {
    "extends": ["@commitlint/config-conventional"]
  }
}
```

Conventional Commits

<tipo>(<escopo>): <descrição>

Tipos:

- feat → Nova funcionalidade
- fix → Correção de bug
- chore → Tarefa de manutenção
- docs → Documentação
- style → Formatação (sem mudança de lógica)
- refactor → Refatoração (sem mudança de comportamento)
- test → Testes
- perf → Performance
- ci → Pipeline/CI

Exemplos:

- feat(auth): add login com Google OAuth
- fix(api): correct 404 handling for users
- chore(deps): upgrade express to 4.19
- docs(readme): update setup instructions

5. Gestão de Dependências

Dependências desatualizadas são uma das **maiores fontes de vulnerabilidade** em software enterprise.

Ferramentas

Ferramenta	O que faz	Quando usar
Dependabot	PRs automáticos de atuali	GitHub nativo, projetos p
Renovate	Configurável, agenda, gro	Projetos médios/grandes,
Snyk	Scan de vulnerabilidades	Compliance, relatórios de
npm audit / yarn audit*	Scan local de vulnerabili	Todo projeto, todo CI

Políticas de atualização

Dependabot/Renovate:

- Diário: patches (1.x.x → 1.x.x)
- Semanal: minor (1.x → 1.y)
- Mensal: major (1 → 2) – revisão manual obrigatória

Regras:

- Nunca fazer merge de update sem CI passar
- Updates major exigem approval de 2 devs
- Dependency drift > 30 dias: alerta
- Dependências não mantidas (>1 ano sem release): substituir
- Dependências com CVE conhecida: atualizar em 48h

Configuração Renovate

```
// renovate.json
{
  "$schema": "https://docs.renovatebot.com/renovate-schema.json",
  "extends": ["config:recommended"],
  "schedule": ["before 9am on monday"],
  "labels": ["dependencies"],
  "packageRules": [
    {
      "matchUpdateTypes": ["patch"],
      "automerge": true
    },
    {
      "matchUpdateTypes": ["minor"],
      "groupName": "minor dependencies",
```

```
    "automerge": true
  },
  {
    "matchUpdateTypes": ["major"],
    "groupName": "major dependencies",
    "reviewers": ["team:leads"]
  }
]
}
```

```
// renovate.json
{
  "$schema": "https://docs.renovatebot.com/renovate-schema.json",
  "extends": ["config:recommended"],
  "schedule": ["before 9am on monday"],
  "labels": ["dependencies"],
  "packageRules": [
    {
      "matchUpdateTypes": ["patch"],
      "automerge": true
    },
    {
      "matchUpdateTypes": ["minor"],
      "groupName": "minor dependencies",
```


6. Gestão de Segredos

****Nunca, jamais, em hipótese alguma commitar segredos no repositório.**** É a regra mais importante da governança.

Práticas essenciais

```
.env.example com valores fictícios (NUNCA reais)
.env no .gitignore (sempre)
Variáveis de ambiente em secrets do CI/CD
Gestão centralizada: Vault (HashiCorp), AWS Secrets Manager, Azure Key Vault
Rotação periódica de chaves (90 dias recomendado)
```

.env.example

```
# .env.example – valores FICTÍCIOS, nunca reais
DATABASE_URL=postgresql://user:pass@localhost:5432/db
REDIS_URL=redis://localhost:6379
API_KEY=sk-your-key-here
NODE_ENV=development
PORT=3000
```

git-secrets — prevenir vazamentos

```
# Instalar
brew install git-secrets # macOS
# ou
sudo apt install git-secrets # Linux

# Configurar no repositório
git secrets --init
git secrets --register-aws # padrões AWS
git secrets --add 'sk-[a-zA-Z0-9]+' # OpenAI API keys
git secrets --add 'AKIA[0-9A-Z]{16}' # AWS access keys
```

Pre-commit hook para segredos

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v4.6.0
    hooks:
```

```
- id: detect-aws-credentials
- id: detect-private-key
- id: check-added-large-files
  args: ["--maxkb=500"]

- repo: https://github.com/Yelp/detect-secrets
  rev: v1.5.0
  hooks:
    - id: detect-secrets
      args: ["--baseline", ".secrets.baseline"]
```

```
# .pre-commit-config.yaml
repos:
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.6.0
  hooks:
```

```
- id: detect-aws-credentials
- id: detect-private-key
- id: check-added-large-files
  args: ["--maxkb=500"]

- repo: https://github.com/Yelp/detect-secrets
  rev: v1.5.0
  hooks:
    - id: detect-secrets
      args: ["--baseline", ".secrets.baseline"]
```

Se vazar — plano de ação

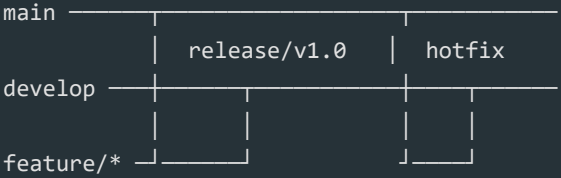
1. ROTACIONAR a chave IMEDIATAMENTE (não esperar)
2. Revogar a chave vazada
3. Verificar logs de acesso (a chave foi usada por terceiros?)
4. Commitar arquivo .gitignore atualizado
5. Se necessário: forçar push com filter-branch (BFG Repo Cleaner)
6. Post-mortem: como evitamos que aconteça de novo?

7. Políticas de Deploy

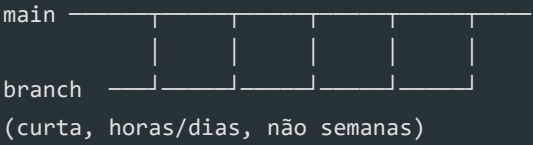
Deploy não é um evento — é um **processo** com gates, validações e rollback.

Branch Strategy

Git Flow (projetos com releases versionadas):



Trunk-based (deploy contínuo):



Característica	Git Flow	Trunk-based
Complexidade	Alta	Baixa
Releases	Agendadas	Contínuas
Hotfix	Fácil (branch separada)	Feature flag
Ideal para	SaaS com releases	CI/CD maduro

Rollback Plan

Pré-deploy:

- Tag da imagem Docker atual (sempre manter última N)
- Backup de banco antes de migration destrutiva
- Script de rollback testado

Durante deploy:

- Health check pós-deploy (timeout: 5min)
- Se health check falhar → rollback automático
- Notificação no Slack do time

Pós-deploy:

- Monitoring ativo por 30min
- Se erro rate > baseline em 2x → rollback manual
- Post-mortem se rollback ocorreu

8. Documentação de Arquitetura — ADRs

Architecture Decision Records (ADRs) documentam **decisões arquiteturais importantes** e seu contexto.

Template de ADR

```
# ADR-{número}: {Título conciso}

## Status
- [ ] Proposto
- [ ] Aceito
- [ ] Depreciado
- [ ] Substituído por ADR-{número}

## Contexto
<!-- Por que essa decisão é necessária? Qual problema estamos resolvendo? -->

## Decisão
<!-- Qual foi a decisão tomada? -->

## Consequências
<!-- Positivas e negativas. O que ganhamos e o que sacrificamos? -->

## Alternativas consideradas
<!-- Quais outras opções foram avaliadas e por que foram descartadas? -->

## Referências
<!-- Links para docs, RFCs, outras ADRs -->
```

Exemplo de ADR

```
# ADR-001: Usar PostgreSQL como banco principal

## Status: Aceito

## Contexto
Precisamos de um banco de dados relacional para o novo SaaS de gestão de projetos.
Requisitos: transações ACID, schemas multi-tenant, queries complexas de relatório.

## Decisão
Adotar PostgreSQL 16 como banco de dados principal.

## Consequências
- Positivas:, boa documentação, suporte a JSONB, extensões, comunidade grande
```

- Negativas: escalabilidade vertical é o limite (sharding é complexo), custo de operação
- Sacrifício: não teremos escalabilidade horizontal nativa (vs. CockroachDB)

Alternativas consideradas

- MySQL 8: similar, mas menos suporte a tipos avançados e extensões
- CockroachDB: escalabilidade horizontal excelente, mas maior complexidade operacional
- MongoDB: não atende requisitos de transações ACID multi-documento

Referências

- <https://www.postgresql.org/docs/16/index.html>

ADR-001: Usar PostgreSQL como banco principal

Status: Aceito

Contexto

Precisamos de um banco de dados relacional para o novo SaaS de gestão de projetos.
Requisitos: transações ACID, schemas multi-tenant, queries complexas de relatório.

Decisão

Adotar PostgreSQL 16 como banco de dados principal.

Consequências

- Positivas:, boa documentação, suporte a JSONB, extensões, comunidade grande

- Negativas: escalabilidade vertical é o limite (sharding é complexo), custo de operação
- Sacrifício: não teremos escalabilidade horizontal nativa (vs. CockroachDB)

Alternativas consideradas

- MySQL 8: similar, mas menos suporte a tipos avançados e extensões
- CockroachDB: escalabilidade horizontal excelente, mas maior complexidade operacional
- MongoDB: não atende requisitos de transações ACID multi-documento

Referências

- <https://www.postgresql.org/docs/16/index.html>

RFCs vs ADRs

ADR: Documenta uma decisão já tomada (ou em votação)

- Tamanho: 1 página
- Quando: após a decisão
- Público: time de desenvolvimento

RFC: Proposta aberta para discussão

- Tamanho: 2-5 páginas
- Quando: antes da decisão
- Público: todo o time + stakeholders
- Inclui: problema, proposta, prós/contra, perguntas em aberto

9. Compliance

Compliance não é só coisa do jurídico — ****impacta diretamente o time de desenvolvimento****.

LGPD (Lei Geral de Proteção de Dados)

O que o time de dev precisa saber:

- Dados pessoais não podem ser logados
- Exclusão lógica não é suficiente: usuário pode solicitar exclusão real
- Consentimento deve ser registrado (com timestamp)
- Anonimização de dados em ambientes não-produção
- Criptografia em repouso e em trânsito

Na prática:

- Não logar CPF, email, endereço
- Implementar endpoint de deleção de conta
- Rotacionar dados sensíveis em staging
- Usar hash + salt para dados identificáveis

SOC2

O que o time de dev precisa saber:

- Controles de acesso (quem fez o quê, quando)
- Monitoramento contínuo de segurança
- Processo de change management documentado
- Testes de segurança periódicos
- Disponibilidade e uptime monitoring

Na prática:

- Audit logs em todas as operações críticas
- CI/CD com trilha de auditoria
- Revisão periódica de acessos
- Pentests agendados

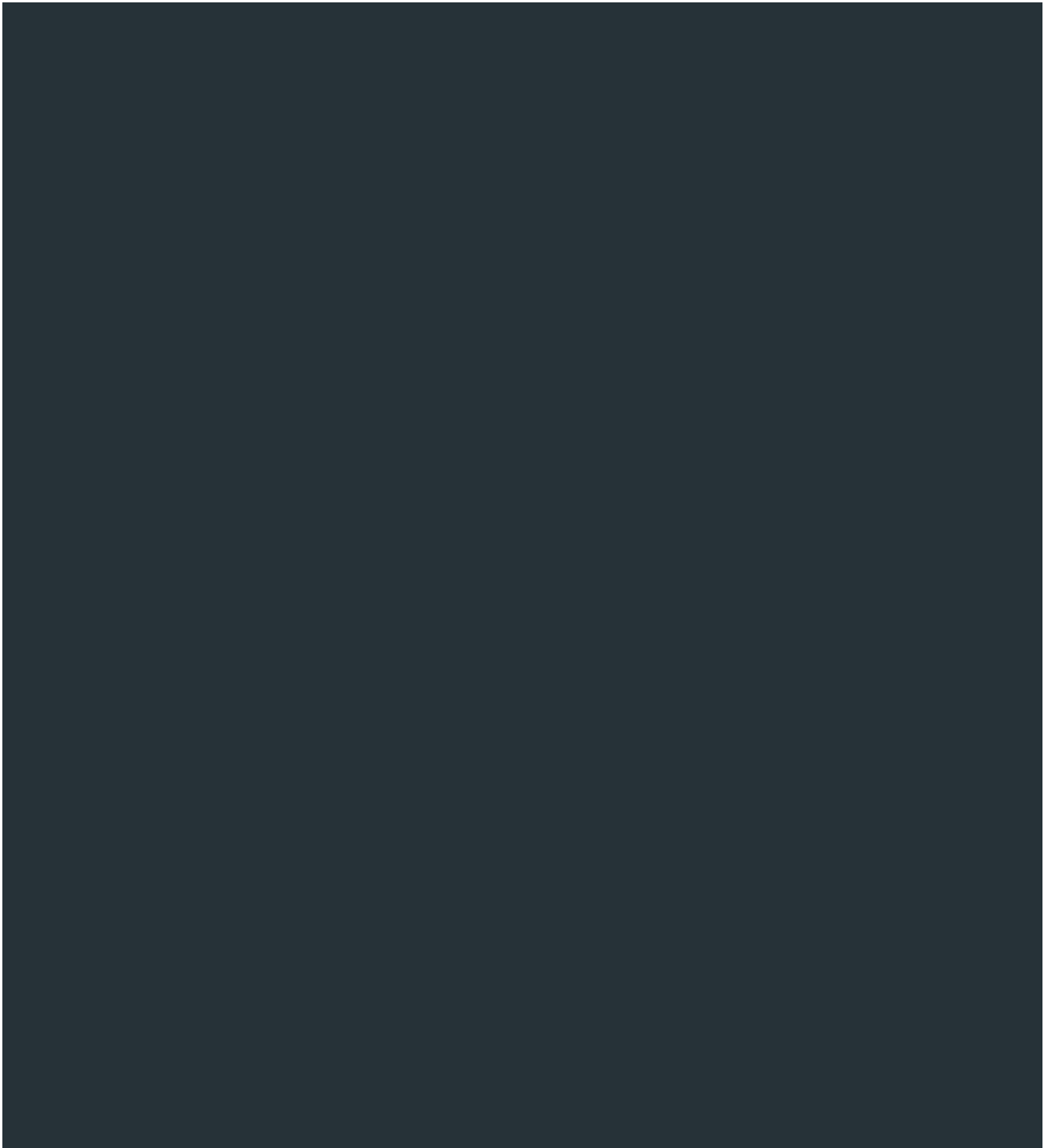
ISO 27001

O que o time de dev precisa saber:

- Política de segurança da informação documentada
- Gestão de ativos (inventário de sistemas)
- Controle de acesso lógico
- Gestão de incidentes de segurança
- Continuidade de negócio

Na prática:

- Documentar políticas de segurança no repositório
- Manter inventário de serviços e versões
- Revisar acessos trimestralmente
- Plano de resposta a incidentes testado



O que o time de dev precisa saber:

- Política de segurança da informação documentada
- Gestão de ativos (inventário de sistemas)
- Controle de acesso lógico
- Gestão de incidentes de segurança
- Continuidade de negócio

Na prática:

10. SLAs, SLOs e SLIs

Métricas que **traduzem qualidade técnica em linguagem de negócio**.

Definições

SLI (Service Level Indicator): métrica técnica

- Latência p99 < 200ms
- Error rate < 0.1%
- Uptime > 99.9%

SLO (Service Level Objective): meta baseada no SLI

- "Vamos entregar p99 < 200ms em 95% do tempo no mês"

SLA (Service Level Agreement): compromisso formal com o cliente

- "Se uptime < 99.9%, cliente recebe 10% de crédito"

Monitoramento na prática

```
# Exemplo de SLOs para um serviço de API
slo:
  availability:
    sli: uptime
    target: 99.9%
    window: 30d

  latency:
    sli: p99 response time
    target: < 300ms
    window: 28d
    measurement: percentil 99 a cada 5min

  error_rate:
    sli: http 5xx / total requests
    target: < 0.1%
    window: 7d

  freshness:
    sli: last successful data sync
    target: < 1h
    window: 28d
```

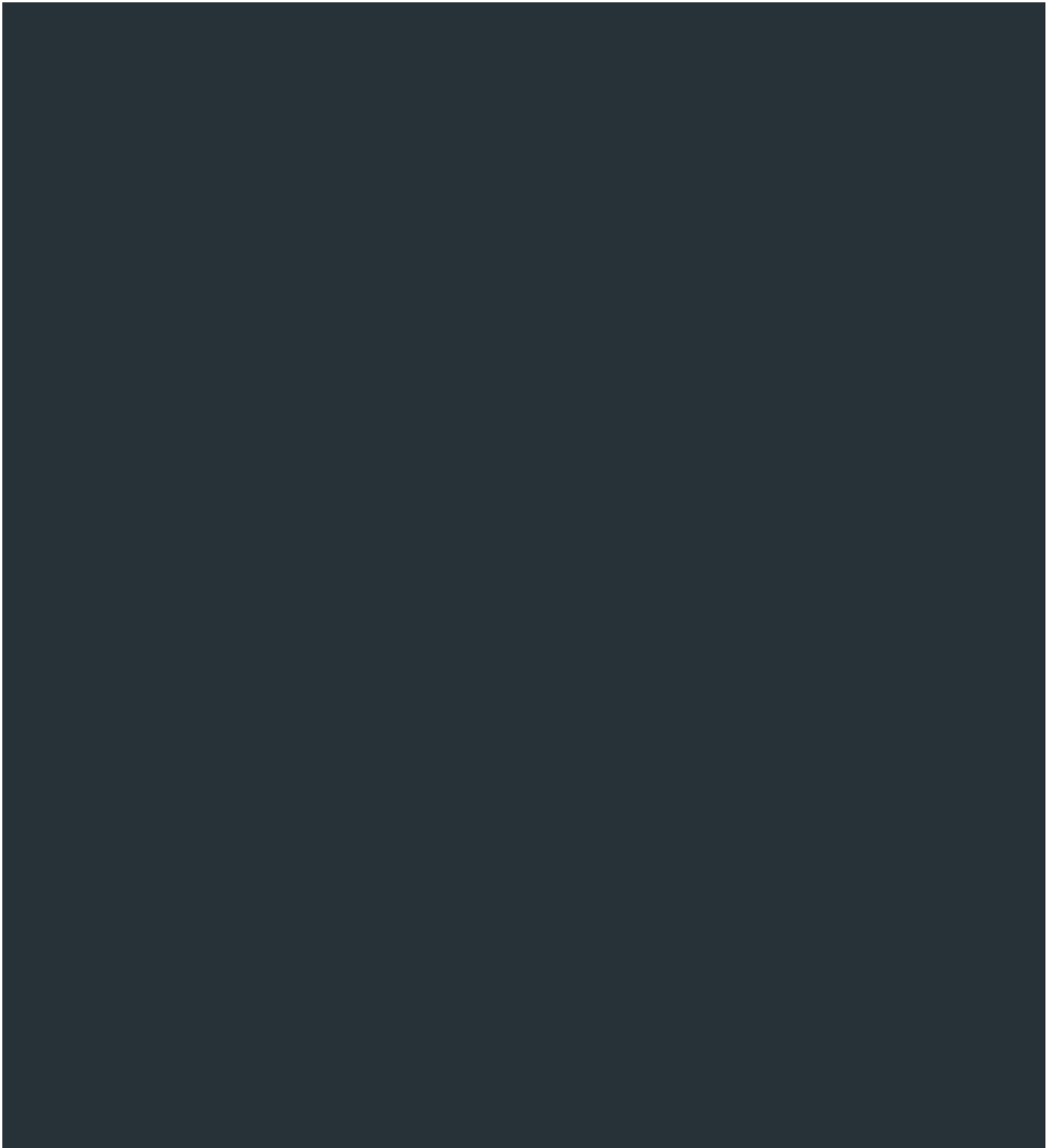
Reporte para stakeholders

Dashboard executivo (responder em 5 segundos):

- Status geral: Verde / Amarelo / Vermelho
- SLOs do período (atingidos ou não)
- Incidentes relevantes (com duração e impacto)
- Tendência (melhorando, estável, piorando)

Relatório trimestral:

- SLO attainment rate
- MTTR (Mean Time to Resolve)
- MTBF (Mean Time Between Failures)
- Top 3 causas de incidentes
- Plano de ação para melhoria



11. Auditoria de Código

Auditoria não é caça às bruxas — é **rastreabilidade e transparência**.

Logs de acesso

```
// Exemplo: audit log em uma operação crítica
interface AuditEntry {
  id: string;
  timestamp: Date;
  userId: string;
  action: 'CREATE' | 'UPDATE' | 'DELETE' | 'READ';
  resource: string;    // ex: 'user', 'invoice', 'contract'
  resourceId: string;
  previousValue?: unknown;
  newValue?: unknown;
  ipAddress?: string;
  userAgent?: string;
}

async function createAuditLog(entry: AuditEntry): Promise<void> {
  // Audit logs são APPEND-ONLY (nunca alterar/excluir)
  await db.auditLog.create({ data: entry });
}

// Uso
await createAuditLog({
  userId: req.user.id,
  action: 'DELETE',
  resource: 'user',
  resourceId: userIdToDelete,
  previousValue: userBeforeDelete,
  ipAddress: req.ip,
});
```

Changelogs automatizados

```
Mantido com Conventional Commits + semantic-release:
→ feat → nova entrada em "Features"
→ fix → nova entrada em "Bug Fixes"
→ BREAKING CHANGE → nova entrada em "Breaking Changes"
→ Gerado automaticamente no momento do release
```

Rastreamento de decisões

Toda decisão técnica relevante deve ter:

- Issue ou ADR associado
- PR que implementa a decisão
- Review que aprovou
- Testes que validam

Exemplo:

Issue: #456 – Escolher estratégia de cache

↓

ADR-003: Redis com TTL de 5min

↓

PR #789: Implementa cache com Redis

↓

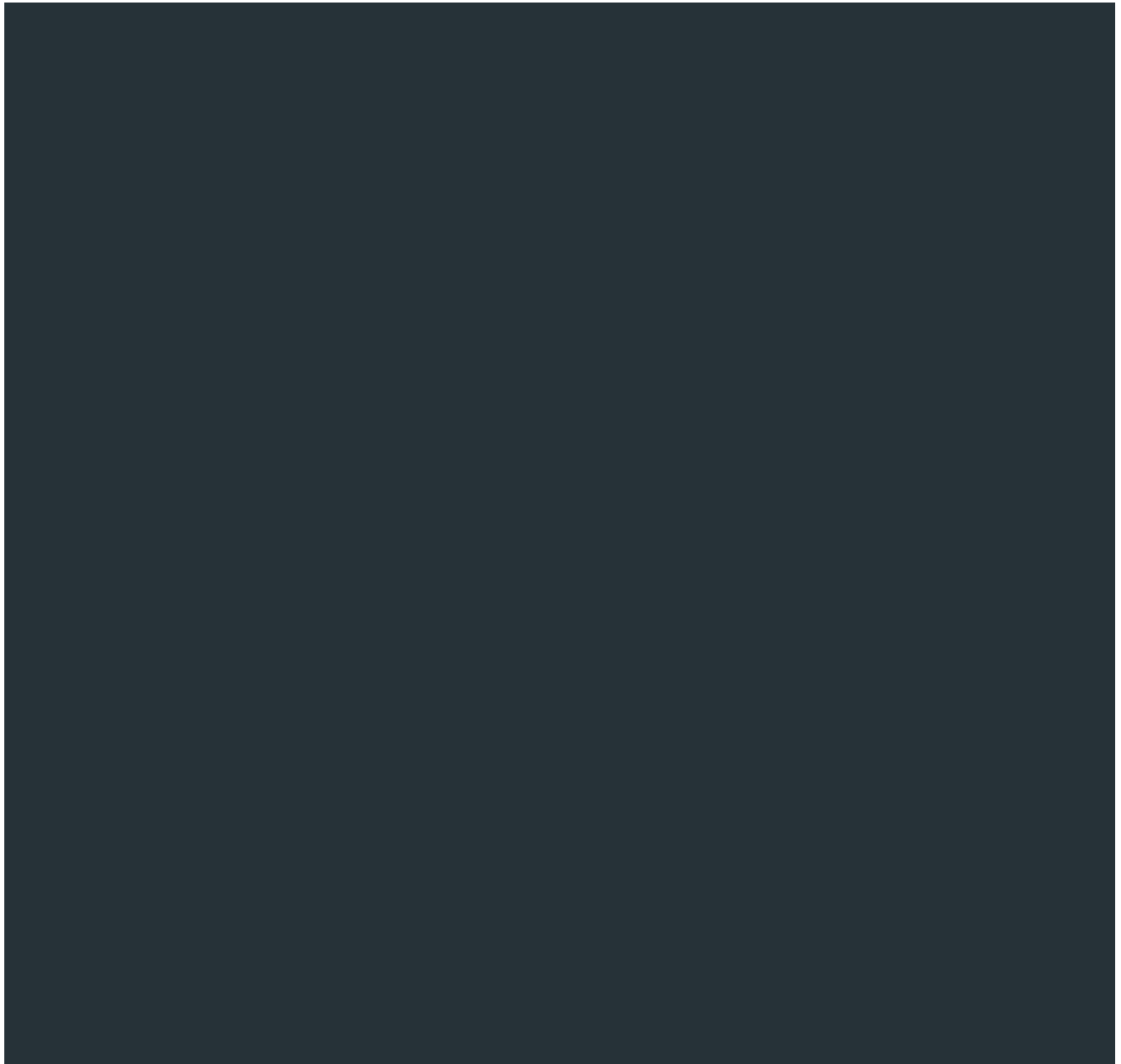
Reviewers: @maria @joao

↓

Tests: cache.test.ts (90% cobertura)

↓

Commit: feat(api): add Redis caching layer



12. Governança em Equipes

O maior erro de governança é criar ****burocracia que paralisa o time****. Governança efetiva é leve, automatizada e cultural.

Princípios de governança leve

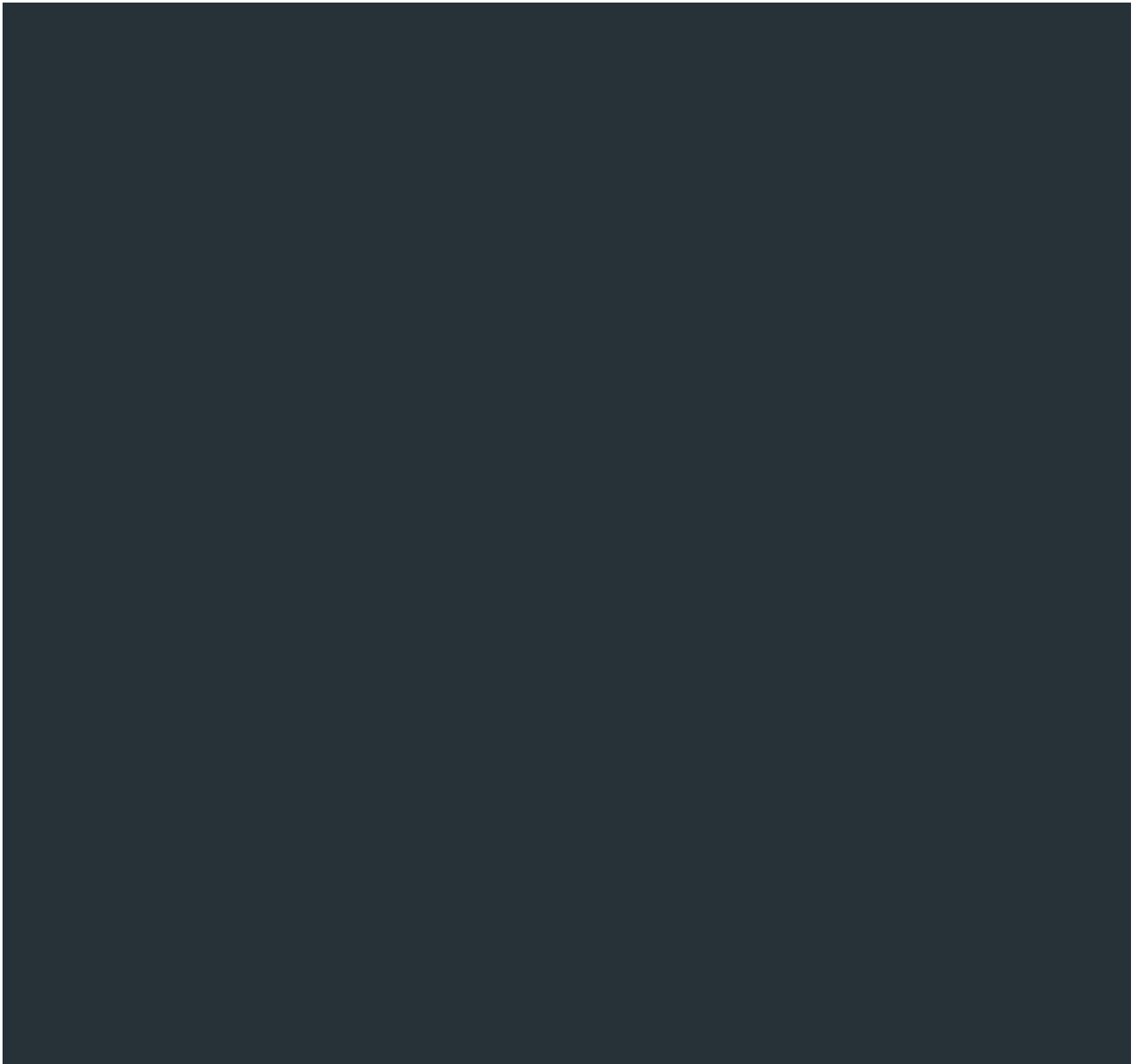
1. Automatize antes de burocratizar
 - Code review manual é caro; lint/pre-commit é grátis
 - Se pode ser automatizado, automatize
2. Comece pequeno, evolua
 - Não implemente 12 processos de uma vez
 - Comece com code review + lint, depois adicione ADRs, depois SLAs
3. Documente o "porquê"
 - Regras sem contexto viram burocracia
 - "Não commitamos segredos porque [história real de vazamento]"
4. Revise periodicamente
 - Regras que não fazem mais sentido devem ser removidas
 - Governança é viva, não esculpida em pedra
5. Liderança pelo exemplo
 - Tech lead e seniors seguem as regras PRIMEIRO
 - Ninguém está acima das políticas

Cultura de qualidade

Como construir:

- Code review é ensinamento, não portaria
 - "Que legal essa solução! Que tal testar o caso X também?"
 - Não: "Faltou teste. Aprovação negada."
- Erro é aprendizado, não punição
 - Post-mortem sem blame
 - "O que podemos melhorar no processo?"
- Qualidade é responsabilidade de todos
 - QA não é "dono da qualidade"
 - Todo dev escreve teste, todo dev faz review
- Celebrar vitórias de governança
 - "Reduzimos MTTR em 40% esse trimestre!"

- "Zero incidentes de segurança em 6 meses!"



Como construir:

- Code review é ensinamento, não portaria
 - "Que legal essa solução! Que tal testar o caso X também?"
 - Não: "Faltou teste. Aprovação negada."
- Erro é aprendizado, não punição
 - Post-mortem sem blame
 - "O que podemos melhorar no processo?"
- Qualidade é responsabilidade de todos
 - QA não é "dono da qualidade"
 - Todo dev escreve teste, todo dev faz review
- Celebrar vitórias de governança
 - "Reduzimos MTTR em 40% esse trimestre!"

- "Zero incidentes de segurança em 6 meses!"

Métricas de saúde da governança

Métrica	Como medir	Alvo
Tempo de code review	Tempo entre PR e primeiro review	< 4h úteis
Coverage de ADRs	Decisões com ADR / total decisões	> 80%
Dependency drift	Dependências >30 dias atrasadas	< 5%
Tempo de deploy	Do merge ao deploy em produção	< 30min
Taxa de rollback	Rollbacks / total deploys	< 2%
Incidentes de segredo	Segredos vazados / mês	0

Ferramentas de medição:

- GitHub Insights / GitLab Analytics
- Dependabot / Renovate dashboard
- Datadog / Grafana para SLOs
- Ferramenta de ADR (adr-tools, madr, GitHub wiki)

Resumo

Governança não é burocracia – é responsabilidade.

O que levar para o dia a dia:

- Code review com checklist e approval gates
- ESLint + Prettier + Husky (automação > memorando)
- .env.example e git secrets (segurança por padrão)
- ADRs para decisões importantes
- Dependências sempre atualizadas
- Deploy com rollback testado
- Compliance integrada ao processo (não depois)
- SLOs que todo time conhece
- Cultura de qualidade, não de culpa

Agentes de IA

Módulo 18 — Agentes de IA: Criação de Agentes Especializados

****Construindo seu exército de agentes.****

1. Por que agentes especializados?

Um único agente genérico (ex: "você é um desenvolvedor full stack") produz resultados ****medianos em todas as áreas****.

Um ecossistema de agentes especializados produz ****resultados excelentes em cada área****.

O problema do agente genérico

Agente Genérico:

Conhecimento: "sabe de tudo um pouco"

- └ Frontend: (sabe React, mas não Next.js App Router)
- └ Backend: (sabe criar API, mas não DDD)
- └ Segurança: (esquece CSRF, rate limiting)
- └ Banco: (faz N+1 sem perceber)
- └ DevOps: (Dockerfile sem multi-stage)

Resultado: código "funciona", mas cheio de dívida técnica

A solução dos agentes especializados

Agente Frontend:

Conhecimento: Next.js 14, RSC, Tailwind, shadcn/ui

- └ Performance: (Lazy loading, Suspense, Image optimization)
- └ Acessibilidade: (ARIA, WCAG 2.1, keyboard nav)
- └ SEO: (Metadata API, OG tags, sitemap)
- └ TypeScript: (strict mode, generics)

Resultado: código de produção, pronto para review

2. Anatomia de um agente

Cada agente da nossa biblioteca segue a mesma estrutura:

```
agente/  
├─ README.md          # Identidade: objetivo, responsabilidades, stack  
├─ workflow.md         # Processo: fluxo de trabalho passo a passo  
├─ checklist.md        # Qualidade: o que validar antes de entregar  
├─ prompts/           # Instruções: templates de prompt para tarefas comuns  
│   ├── prompt-tarefa-1.md  
│   └── prompt-tarefa-2.md
```

Componentes de um agente eficaz

IDENTIDADE
Quem é este agente?
O que ele sabe fazer?
O que ele NÃO faz?
CONHECIMENTO
Stack tecnológica
Padrões e boas práticas
Referências externas
PROCESSO
Fluxo de trabalho
Entrada → Transformação → Saída
QUALIDADE
Checklist de validação
Critérios de aceite
Anti-padrões a evitar
COMUNICAÇÃO
Formato de entrada (o que recebe)
Formato de saída (o que entrega)
Como reportar problemas

3. Os 17 agentes da formação

Agentes de Produto

Agente	Função	Stack/Conhecimento
Product Discovery	Transformar problemas em	User Stories, RICE, MoSCo
UX Research	Validar hipóteses com usu	Entrevistas, testes de us

Agentes de Design

Agente	Função	Stack/Conhecimento
UX Designer	Projetar experiência do u	User flows, wireframes, a
UI Designer	Projetar interface visual	Design System, tokens, da

Agentes de Arquitetura

Agente	Função	Stack/Conhecimento
Enterprise Architect	Decisões arquiteturais	Clean Arch, DDD, ADRs, C4
Database Architect	Modelagem de dados	PostgreSQL, índices, part

Agentes de Desenvolvimento

Agente	Função	Stack/Conhecimento
Backend Expert	APIs e regras de domínio	NestJS, Prisma, REST, Gra
Frontend Expert	Interfaces e componentes	Next.js 14, RSC, Tailwind
Prisma Expert	Schema e queries	Prisma ORM, migrations, N

Agentes de Infraestrutura

Agente	Função	Stack/Conhecimento
DevOps Expert	Docker, CI/CD, deploy	Docker multi-stage, GitHub
Security Expert	Proteção do sistema	OWASP Top 10, JWT, bcrypt

Agentes de Qualidade

Agente	Função	Stack/Conhecimento
--------	--------	--------------------

QA Expert	Testes automatizados	Jest, Playwright, Testing
Performance Expert	Otimização	Core Web Vitals, caching,

Agentes de Governança

Agente	Função	Stack/Conhecimento
Auditor	16 tipos de auditoria	Score (0-10), riscos (Blo
Documentation	Documentação técnica	ADRs, README, Swagger, CH
Refactoring	Refatoração guiada	Code smells, patterns, Ty

4. Como criar um novo agente

Passo a passo

1. Definir o DOMÍNIO do agente
 - Qual área ele cobre? (ex: "segurança de aplicações web")
2. Definir o CONHECIMENTO BASE
 - Quais tecnologias, padrões e boas práticas ele domina?
3. Definir RESPONSABILIDADES
 - O que ele faz? O que ele NÃO faz? (limites são importantes)
4. Definir o PROCESSO
 - Qual o fluxo de trabalho? O que recebe na entrada? O que entrega?
5. Criar CHECKLIST DE QUALIDADE
 - O que validar antes de considerar o trabalho concluído?
6. Criar TEMPLATES DE PROMPT
 - Prompts reutilizáveis para as tarefas mais comuns
7. TESTAR com um caso real
 - Executar o agente, revisar o output, ajustar

Exemplo: Criando o Security Expert Agent

****Domínio:**** Segurança de aplicações web

****Conhecimento base:**** - OWASP Top 10 (2021) - JWT, OAuth2, MFA - bcrypt, Helmet, CORS, CSP - NestJS Guards, CASL (autorização)

****Responsabilidades:**** - Implementar autenticação e autorização - Prevenir SQL injection, XSS, CSRF - Configurar rate limiting - Gerenciar segredos

****Limites:**** - Não define arquitetura geral - Não implementa lógica de negócio - Não gerencia infraestrutura

****Checklist:**** ```text - [] Senhas com hash bcrypt/argon2 - [] JWT com expiração curta + refresh token - [] Rate limiting no login - [] CSP header configurado - [] Helmet.js ativado - [] Input validation em todos os endpoints - [] SQL injection prevenido (ORM) ```

5. Como combinar agentes em pipeline

O verdadeiro poder está em **compor** agentes em sequência.

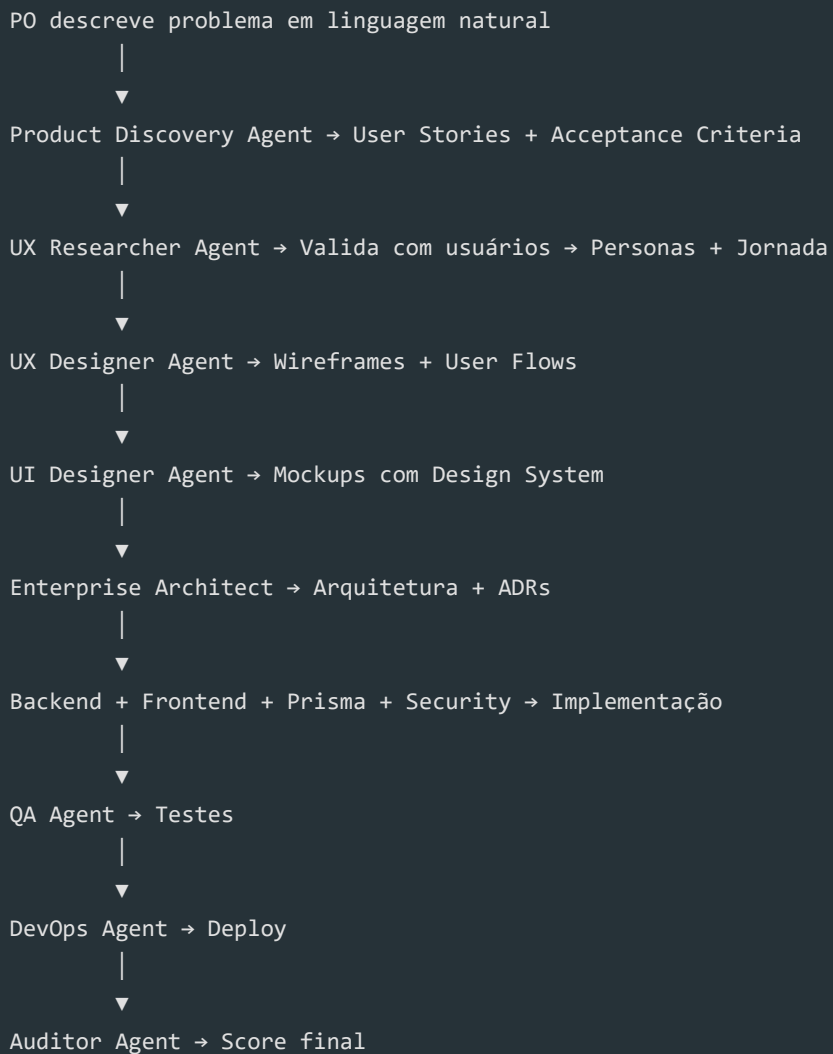
Pipeline de features



Pipeline de auditoria



Pipeline de onboarding



6. Integração com OpenCode

Configuração de agentes no opencode.json

```
{
  "agents": {
    "frontend-expert": {
      "prompt": ".opencode/agents/frontend-expert.md",
      "permissions": {
        "bash": true,
        "read": true,
        "edit": true,
        "glob": true,
        "grep": true
      }
    },
    "auditor": {
      "prompt": ".opencode/agents/auditor.md",
      "permissions": {
        "bash": true,
        "read": true,
        "edit": false,
        "glob": true,
        "grep": true
      }
    }
  }
}
```

Como invocar um agente

```
@frontend-expert Crie um componente de tabela com:
- Server Component
- Suporte a sort e filter
- Paginação
- Loading state com Suspense
```

Como fazer um agente revisar outro

```
@auditor Revise a segurança deste endpoint.

[endpoint code]
```

7. Boas práticas na criação de agentes

Faça

- **Seja específico** — "Crie componente com Server Component" não "Faça um componente bonito" - **Defina limites** — "Este agente NÃO implementa regras de domínio" - **Forneça exemplos** — "Siga este padrão: [exemplo]" - **Crie checklists** — "Antes de entregar, verifique: [itens]" - **Itere** — Ajuste os prompts baseado nos resultados

Não faça

- **Não misture domínios** — Um agente de backend não deve ter responsabilidades de frontend
- **Não seja vago** — "Seja criativo" não é uma instrução útil - **Não ignore limites** — Se o agente não tem conhecimento, ele vai alucinar - **Não pule a revisão** — Sempre revise o output, especialmente no início

Padrão de prompt eficaz

Ruim:

"Crie uma API de usuários."

Bom:

"Crie uma API REST de usuários com NestJS seguindo Clean Architecture."

Requisitos:

- POST /users (criar)
- GET /users (listar com paginação)
- GET /users/:id (detalhe)
- PUT /users/:id (atualizar)
- DELETE /users/:id (soft delete)

Validações:

- Email: formato válido, único
- Nome: 3-100 caracteres
- Senha: mínimo 8 caracteres, 1 número, 1 maiúscula

Regras:

- Usar Prisma para persistência
- Zod para validação
- Swagger para documentação
- Tratamento de erros com NestJS exception filters"

8. O futuro: agentes que criam agentes

O próximo passo natural: um **Meta-Agent** que cria agentes especializados sob demanda.

Como funcionaria

1. Meta-Agent analisa o problema
2. Identifica os domínios necessários
3. Cria um agente especializado para cada domínio
4. Define os limites e responsabilidades
5. Testa e ajusta cada agente
6. Combina em pipeline
7. Executa o pipeline completo

Auditorias

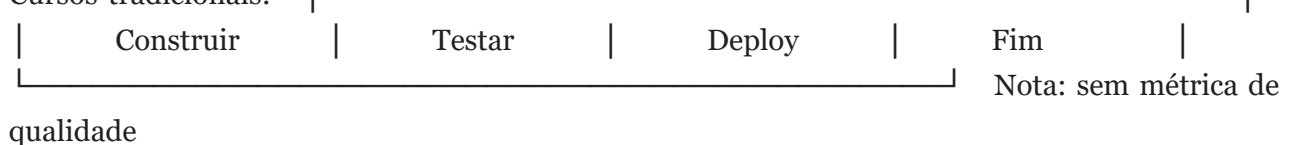
Módulo 19 – Auditorias

****Módulo exclusivo: como medir, pontuar e melhorar a qualidade de sistemas Enterprise.****

1. Por que auditoria é o diferencial

A maioria dos cursos ensina a **construir** software. Quase nenhum ensina a **avaliar** a qualidade do que foi construído.

Cursos tradicionais:



Esta formação:



O que uma auditoria responde

- "Este código está pronto para produção?" - "Qual o nível de segurança deste sistema?" - "Onde

estão os gargalos de performance?" - "A arquitetura suporta o próximo ano de crescimento?" - "O que precisa ser corrigido antes do deploy?"

2. O sistema de score (0-10)

Cada auditoria produz um **score quantitativo** que permite comparar qualidade entre versões e módulos.

Significado do score

Score	Significado	Ação
9-10	Excelente	Manutenção preventiva
7-8	Bom	Correções opcionais menor
5-6	Regular	Correções necessárias
3-4	Ruim	Correções urgentes
0-2	Crítico	Parar e corrigir

Como o score é calculado

$\text{Score} = (\sum \text{scores por dimensão}) / \text{número de dimensões}$

Cada dimensão = 0-10 baseado em:

- 40% checklists automatizados
- 30% análise estática (lint, types, cobertura)
- 30% análise qualitativa do auditor

Exemplo: Auditoria de uma API

Dimensões avaliadas:

Validação de entrada: 9/10 (faltou sanitização em 1 campo)
Autenticação: 10/10 (perfeito)
Autorização: 6/10 (2 rotas sem verificação de role)
Tratamento de erros: 8/10 (exceções genéricas em 3 endpoints)
Documentação: 7/10 (Swagger desatualizado)
Performance: 9/10 (1 query N+1)

Score geral: $(9+10+6+8+7+9) / 6 = 8.2 \rightarrow \text{"Bom"}$

Riscos: 1 Critical, 2 Major, 3 Minor

3. Os 16 tipos de auditoria

AUDITORIAS TÉCNICAS				
Arquitetura	Segurança	Frontend	Backend	UX
Clean Arch	OWASP	Next.js	NestJS	Heurísticas
DDD	Top 10	RSC	DDD	Jornada
SOLID				
UI	Banco	APIs	Perform.	DevOps
Design Sys	Dados	REST	Web Vitals	Dockerfile
Tokens	PostgreSQL	GraphQL	Caching	CI/CD
Governança	Multi-Ten.	Código	Dep.	TypeScript
ADR	Isolamento	Limpeza	Vuln.	Strict mode
Git Flow	RLS	Padrões	Licenças	Generics

Auditorias que todo projeto deveria ter

****Essenciais (obrigatórias antes de todo deploy):**** 1. Segurança — sem isso, não vai para produção 2. Código — qualidade mínima do código 3. APIs — contratos corretos

****Recomendadas (a cada release):**** 4. Arquitetura — a arquitetura ainda está adequada? 5. Performance — não está degradando? 6. Banco de dados — queries estão saudáveis? 7. TypeScript — tipos estão sendo respeitados?

****Periódicas (a cada trimestre):**** 8. Governança — documentação está atualizada? 9. Dependências — bibliotecas desatualizadas ou vulneráveis? 10. UX — feedback de usuários está sendo endereçado?

4. Estrutura de cada auditoria

Template universal

```
# Auditoria: [Tipo] – [Sistema/Componente]

**Score geral:** [0-10]
**Riscos:** [Qtd] (Blocker: X, Critical: Y, Major: Z, Minor: W)
**Data:** [data]
**Auditor:** [agente responsável]

## Resumo Executivo
[3-5 frases]

## Resultados por Dimensão
| Dimensão | Score | Riscos | Observação |
|-----|-----|-----|-----|
| ...      | ...   | ...   | ...        |

## Riscos Identificados

### [Blocker/Critical/Major/Minor] [Título do risco]
- **Localização:** arquivo:linha
- **Descrição:** o que está errado
- **Impacto:** o que pode acontecer
- **Correção:** como corrigir
- **Dependência:** precisa de outra correção antes?

## Plano de Ação
| Prioridade | Ação | Responsável | Prazo | Esforço |
|-----|-----|-----|-----|-----|
| P0         | ...  | ...        | ...   | ...     |

## Recomendações
[Melhorias opcionais para atingir score 10]
```

Classificação de riscos

Gravidade	Definição	Prazo de correção
Blocker	Impede deploy ou quebra f	Imediato
Critical	Vulnerabilidade grave ou	24h
Major	Boa prática não seguida,	1 semana
Minor	Sugestão de melhoria, bai	Agenda de refatoração

5. Exemplo: Auditoria de Segurança

```
# Auditoria: Segurança – API de Pagamentos

**Score geral:** 5.8/10
**Riscos:** 8 (2 Critical, 3 Major, 3 Minor)
**Data:** 2025-06-01
**Auditor:** Security Expert Agent

## Resumo Executivo
A API de pagamentos apresenta 2 riscos críticos que impedem o deploy em produção: senhas armazenadas com MD5 e falta de rate limiting no endpoint de login. Recomenda-se corrigir estes itens antes de qualquer release.

## Resultados por Dimensão
| Dimensão | Score | Riscos | Observação |
|-----|-----|-----|-----|
| Autenticação | 3 | 1 Crit | MD5 + sem refresh |
| Autorização | 7 | 1 Major | Role check ausente |
| Input Validation | 8 | 1 Major | 1 campo sem sanitizar |
| Rate Limiting | 2 | 1 Crit | Nenhum implementado |
| Headers Segurança | 6 | 1 Major | CSP ausente |
| Secrets Management | 9 | 1 Minor | .env.example exposto |

## Riscos Identificados

### [Critical] Senhas armazenadas com MD5
- **Localização:** src/auth/password.service.ts:15
- **Descrição:** A função hashPassword usa MD5
- **Impacto:** Senhas recuperáveis em segundos
- **Correção:** Substituir por bcrypt com salt rounds 12
- **Dependência:** Nenhuma

### [Critical] Ausência de rate limiting no login
- **Localização:** src/auth/auth.controller.ts:22
- **Descrição:** Login sem proteção contra brute force
- **Impacto:** Ataque de força bruta ilimitado
- **Correção:** Implementar @Throttle(5, 60) no endpoint
- **Dependência:** Nenhuma
```

6. Ciclo de auditoria contínua

Não: Auditar uma vez no final do projeto

Sim:

```
CICLO CONTÍNUO DE AUDITORIA

A cada PR:
1. Código analisado automaticamente (lint)
2. Segurança verificada (npm audit)
3. Cobertura de testes validada

A cada release:
4. Auditoria completa de segurança
5. Auditoria de performance (antes/depois)
6. Score calculado e comparado com anterior

A cada trimestre:
7. Auditoria de arquitetura
8. Auditoria de governança
9. Auditoria de dependências
10. Relatório executivo para stakeholders
```

Ferramentas para auditoria automatizada

Tipo	Ferramenta	O que verifica
Código	ESLint + Prettier	Padrões, formatação
Tipos	TypeScript strict	Tipagem correta
Segurança	npm audit, Snyk	Vulnerabilidades
Testes	Jest --coverage	Cobertura de código
Performance	Lighthouse CI	Core Web Vitals
API	OpenAPI diff	Contratos de API
Docker	Docker Scout	Vulnerabilidades em image
Dependências	Renovate/Dependabot	Updates automáticos

7. Como interpretar e agir sobre uma auditoria

Para o time técnico

Score 0-4 (Crítico/Ruim):

- Pare o que está fazendo
- Corrija riscos Blocker e Critical primeiro
- Não faça deploy até subir para ≥ 5

Score 5-6 (Regular):

- Planeje correções para a próxima sprint
- Riscos Blocker/Critical ainda precisam de ação imediata
- Débito técnico identificado → documentar

Score 7-8 (Bom):

- Pode fazer deploy com confiança
- Corrigir riscos Major no ciclo normal
- Manter práticas atuais

Score 9-10 (Excelente):

- Referência para outros módulos
- Compartilhar práticas com o time
- Foco em inovação, não em correção

Para o gestor

Relatório executivo deve conter apenas:

1. Score geral (0-10)
2. Comparação com auditoria anterior (melhorou/piorou)
3. Top 3 riscos que precisam de atenção
4. Prazo estimado para correções
5. Impacto nos prazos do projeto

8. Auditoria como serviço contínuo com agentes

Na nossa biblioteca, o **Auditor Agent** pode ser invocado a qualquer momento:

```
# Auditar segurança de um módulo
@auditor auditar-seguranca src/modules/pagamentos/

# Auditar performance antes do deploy
@auditor auditar-performance src/app/page.tsx

# Auditar a arquitetura completa
@auditor auditar-arquitetura . --format relatorio
```

Pipeline de auditoria completa

1. Código é implementado pelos agentes especializados
2. QA Agent executa testes
3. Auditor Agent executa 16 auditorias
4. Relatório consolidado com score geral
5. Riscos Blocker/Critical são atribuídos aos agentes corretivos
6. Agentes corrigem e re-audita
7. Score mínimo é atingido → deploy autorizado

Automação do Ciclo de Desenvolvimento

Módulo 20 – Automação

****Duração estimada:**** 5 dias (~40h)

****Público-alvo:**** Desenvolvedores brasileiros em transição para cargos seniores/tech lead

****Pré-requisitos:**** Conhecimento básico de Git, terminal Linux, conceitos de deploy

1. O que é Automação

Automação é a substituição de processos manuais repetitivos por scripts, pipelines e ferramentas que executam essas tarefas de forma confiável, auditável e escalável.

Por que automatizar?

Motivo	Impacto
Reduzir erro humano	O maior causador de incidentes em produção ainda é o operador humano
Velocidade	Máquinas executam em segundos o que levaria horas manualmente

Reprodutibilidade	O mesmo pipeline executa exatamente da mesma forma todas as vezes
Auditoria	Logs e artefatos gerados automaticamente servem como evidência
Escala	O que funciona para 1 deploy falha para 100 deploys manuais

Onde automatizar

- Build e compilação
- Testes (unitários, integração, e2e)
- Linting e formatação de código
- Verificação de segurança
- Deploy em ambientes
- Migrações de banco de dados
- Geração de changelog e release
- Monitoramento e alertas
- Criação e destruição de ambientes

Custo vs Benefício

*** O que é o QA Agent e como ele funciona ***

1. Código é implementado pelos agentes especializados
2. QA Agent executa testes
3. Auditor Agent executa 16 auditorias
4. Relatório consolidado com score geral
5. Riscos Blocker/Critical são atribuídos aos agentes corretivos
6. Agentes corrigem e re-audita
7. Score mínimo é atingido → deploy autorizado

Automação do Ciclo de Desenvolvimento

Módulo 20 – Automação

****Duração estimada:**** 5 dias (~40h)

****Público-alvo:**** Desenvolvedores brasileiros em transição para cargos seniores/tech lead

****Pré-requisitos:**** Conhecimento básico de Git, terminal Linux, conceitos de deploy

1. O que é Automação

Automação é a substituição de processos manuais repetitivos por scripts, pipelines e ferramentas que executam essas tarefas de forma confiável, auditável e escalável.

Por que automatizar?

Motivo	Impacto
-----	-----
Reduzir erro humano	O maior causador de incidentes em produção ainda é o operador humano
Velocidade	Máquinas executam em segundos o que levaria horas manualmente

Reprodutibilidade	O mesmo pipeline executa exatamente da mesma forma todas as vezes
Auditoria	Logs e artefatos gerados automaticamente servem como evidência
Escala	O que funciona para 1 deploy falha para 100 deploys manuais

Onde automatizar

- Build e compilação
- Testes (unitários, integração, e2e)
- Linting e formatação de código
- Verificação de segurança
- Deploy em ambientes
- Migrações de banco de dados
- Geração de changelog e release
- Monitoramento e alertas
- Criação e destruição de ambientes

Custo vs Benefício

A regra prática: ****automatize tudo que for executado mais de 2 vezes.****

Custo de automatizar = (tempo para criar + tempo para manter) × custo-hora
Benefício = (tempo economizado por execução × frequência × horizonte) - custo ``

Se o benefício for positivo em 6 meses, vale a pena automatizar.

2. CI/CD — Pipelines de Integração e Deploy Contínuo

CI/CD é a espinha dorsal da automação em engenharia de software.

Continuous Integration (CI)

Todo push para branches compartilhadas dispara: 1. Checkout do código 2. Instalação de dependências 3. Linting 4. Testes unitários 5. Testes de integração 6. Build 7. Análise de segurança

Continuous Delivery / Continuous Deployment (CD)

- **Continuous Delivery:** O artefato é gerado e publicado em um repositório, mas o deploy em produção é manual (aprovado por um humano). - **Continuous Deployment:** O deploy em produção é automático após a pipeline CI passar.

GitHub Actions — Exemplo Real

```
# .github/workflows/ci.yml
name: CI Pipeline

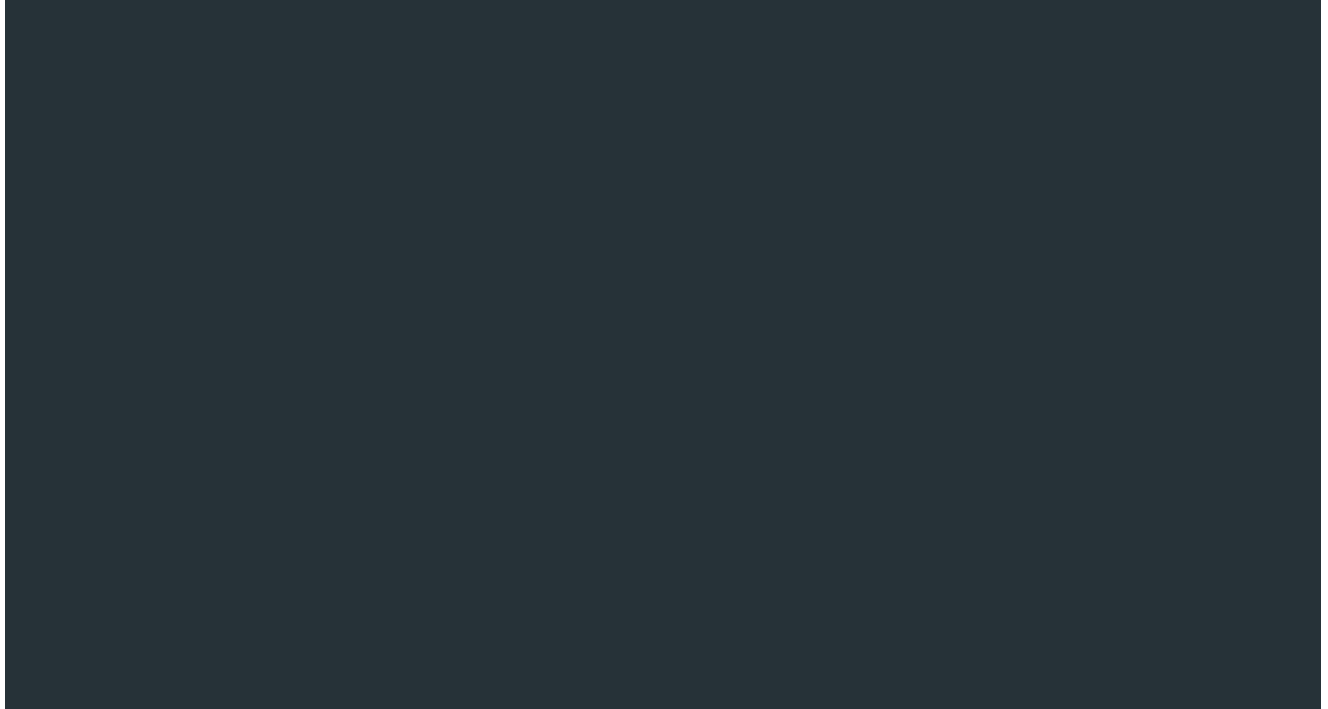
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

env:
  NODE_VERSION: 20.x
  PNPM_VERSION: 9

jobs:
  lint:
    name: Lint e Formatação
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v3
        with:
          version: ${{ env.PNPM_VERSION }}
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}
```



```
- run: pnpm install --frozen-lockfile
- run: pnpm build
- uses: actions/upload-artifact@v4
  with:
    name: build-output
    path: dist/
```



```
# .github/workflows/ci.yml
name: CI Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

env:
  NODE_VERSION: 20.x
  PNPM_VERSION: 9

jobs:
  lint:
    name: Lint e Formatação
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v3
        with:
          version: ${{ env.PNPM_VERSION }}
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}
```



```
- run: pnpm install --frozen-lockfile
- run: pnpm build
- uses: actions/upload-artifact@v4
  with:
    name: build-output
    path: dist/
```

GitLab CI — Exemplo Real

```
# .gitlab-ci.yml
stages:
  - lint
  - test
  - build
  - deploy

variables:
  NODE_VERSION: "20"
  PNPM_VERSION: "9"

cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths:
    - node_modules/
    - .pnpm-store/

lint:
  stage: lint
  image: node:${NODE_VERSION}
  script:
    - npm install -g pnpm@${PNPM_VERSION}
    - pnpm install --frozen-lockfile
    - pnpm lint
    - pnpm format:check

test:
  stage: test
  image: node:${NODE_VERSION}
  services:
    - postgres:16-alpine
  variables:
    DATABASE_URL: postgres://app:secret@postgres:5432/app_test
    POSTGRES_DB: app_test
    POSTGRES_USER: app
    POSTGRES_PASSWORD: secret
  script:
    - npm install -g pnpm@${PNPM_VERSION}
    - pnpm install --frozen-lockfile
    - pnpm test:unit
```

```
- pnpm test:integration
```

```
build:
```

```
  stage: build
```

```
  image: node:${NODE_VERSION}
```

```
  script:
```

```
    - npm install -g pnpm@${PNPM_VERSION}
```

```
    - pnpm install --frozen-lockfile
```

```
    - pnpm build
```

```
  artifacts:
```

```
# .gitlab-ci.yml
```

```
stages:
```

```
- lint
```

```
- test
```

```
- build
```

```
- deploy
```

```
variables:
```

```
  NODE_VERSION: "20"
```

```
  PNPM_VERSION: "9"
```

```
cache:
```

```
  key: ${CI_COMMIT_REF_SLUG}
```

```
  paths:
```

```
    - node_modules/
```

```
    - .pnpm-store/
```

```
lint:
```

```
  stage: lint
```

```
  image: node:${NODE_VERSION}
```

```
  script:
```

```
    - npm install -g pnpm@${PNPM_VERSION}
```

```
    - pnpm install --frozen-lockfile
```

```
    - pnpm lint
```

```
    - pnpm format:check
```

```
test:
```

```
  stage: test
```

```
  image: node:${NODE_VERSION}
```

```
  services:
```

```
    - postgres:16-alpine
```

```
  variables:
```

```
    DATABASE_URL: postgres://app:secret@postgres:5432/app_test
```

```
    POSTGRES_DB: app_test
```

```
    POSTGRES_USER: app
```

```
    POSTGRES_PASSWORD: secret
```

```
  script:
```

```
    - npm install -g pnpm@${PNPM_VERSION}
```

```
    - pnpm install --frozen-lockfile
```

```
    - pnpm test:unit
```


3. Automação de Testes

A pipeline de CI deve executar testes em camadas, respeitando a **pirâmide de testes**.

Testes Unitários

Executados primeiro — são rápidos e isolados.

```
// Exemplo com vitest
import { describe, it, expect } from 'vitest'
import { calculateDiscount } from './pricing'

describe('calculateDiscount', () => {
  it('aplica 10% para compras acima de R$ 100', () => {
    expect(calculateDiscount(150)).toBe(135)
  })

  it('não aplica desconto para compras abaixo de R$ 100', () => {
    expect(calculateDiscount(50)).toBe(50)
  })

  it('lança erro para valores negativos', () => {
    expect(() => calculateDiscount(-10)).toThrow('Valor inválido')
  })
})
```

Testes de Integração

Testam a interação entre módulos, geralmente com banco de dados real ou em memória.

```
import { describe, it, expect, beforeAll, afterAll } from 'vitest'
import { createApp } from './app'
import { prisma } from './lib/prisma'

const app = createApp()

describe('POST /users', () => {
  beforeAll(async () => {
    await prisma.$executeRawUnsafe('TRUNCATE TABLE users CASCADE')
  })

  afterAll(async () => {
    await prisma.$disconnect()
  })

  it('cria um usuário com dados válidos', async () => {
```

```
const response = await app.inject({
  method: 'POST',
  url: '/users',
  payload: { name: 'João', email: 'joao@email.com' },
})

expect(response.statusCode).toBe(201)
expect(response.json()).toHaveProperty('id')
})
})
```

```
import { describe, it, expect, beforeAll, afterAll } from 'vitest'
import { createApp } from './app'
import { prisma } from './lib/prisma'

const app = createApp()

describe('POST /users', () => {
  beforeAll(async () => {
    await prisma.$executeRawUnsafe('TRUNCATE TABLE users CASCADE')
  })

  afterAll(async () => {
    await prisma.$disconnect()
  })

  it('cria um usuário com dados válidos', async () => {
```

```

const response = await app.inject({
  method: 'POST',
  url: '/users',
  payload: { name: 'João', email: 'joao@email.com' },
})

expect(response.statusCode).toBe(201)
expect(response.json()).toHaveProperty('id')
})
})

```

Testes E2E

Testam o sistema como um todo, simulando o usuário real.

```

# job no CI para E2E
e2e:
  name: Testes End-to-End
  needs: build
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: pnpm/action-setup@v3
      with:
        version: ${{ env.PNPM_VERSION }}
    - uses: actions/setup-node@v4
      with:
        node-version: ${{ env.NODE_VERSION }}
        cache: pnpm
    - run: pnpm install --frozen-lockfile
    - run: pnpm build
    - run: pnpm dlx playwright install --with-deps
    - run: pnpm test:e2e
  env:
    BASE_URL: http://localhost:3000

```

```

// E2E com Playwright
import { test, expect } from '@playwright/test'

test('usuário consegue finalizar compra', async ({ page }) => {
  await page.goto('/produtos')
  await page.click('text=Adicionar ao carrinho')
  await page.click('text=Finalizar compra')
  await page.fill('[name=email]', 'joao@email.com')
  await page.click('text=Confirmar')

  await expect(page.locator('text=Pedido confirmado')).toBeVisible()
})

```

4. Automação de Infraestrutura — IaC

Infrastructure as Code (IaC) é o gerenciamento de infraestrutura (servidores, bancos, redes) através de arquivos de configuração versionados.

Terraform (HashiCorp)

```
# main.tf
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

backend "s3" {
  bucket = "meu-terraform-state"
  key    = "prod/terraform.tfstate"
  region = "us-east-1"
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_ecs_service" "app" {
  name            = "app-service"
  cluster         = aws_ecs_cluster.main.id
  task_definition = aws_ecs_task_definition.app.arn
  desired_count   = 2
  launch_type     = "FARGATE"

  network_configuration {
    subnets         = var.private_subnet_ids
    security_groups = [aws_security_group.app.id]
  }

  deployment_controller {
    type = "CODE_DEPLOY" # blue/green
  }
}
```

Pulumi

IaC com linguagens de programação reais (TypeScript, Python, Go, C#).

```
import * as aws from '@pulumi/aws'
import * as pulumi from '@pulumi/pulumi'

const config = new pulumi.Config()
const stack = pulumi.getStack()

const cluster = new aws.ecs.Cluster('app-cluster')

const taskDefinition = new aws.ecs.TaskDefinition('app-task', {
  family: 'app',
  cpu: '256',
  memory: '512',
  networkMode: 'awsvpc',
  executionRoleArn: config.require('executionRoleArn'),
  containerDefinitions: JSON.stringify([
    {
      name: 'app',
      image: `meuregistro/app:${stack}`,
      portMappings: [{ containerPort: 3000 }],
    },
  ]),
})

new aws.ecs.Service('app-service', {
  cluster: cluster.arn,
  taskDefinition: taskDefinition.arn,
  desiredCount: 2,
  launchType: 'FARGATE',
  networkConfiguration: {
    subnets: config.requireObject<string[]>('privateSubnets'),
    securityGroups: [config.require('securityGroupId')],
  },
})
```

CloudFormation (AWS)

Template declarativo em YAML/JSON para recursos AWS.

```
# template.yaml
AWSTemplateFormatVersion: "2010-09-09"
Description: "Stack da aplicação"

Parameters:
  Env:
    Type: String
    Default: production
```

```
Resources:
  ECSCluster:
    Type: AWS::ECS::Cluster
    Properties:
      ClusterName: !Sub "app-cluster-${Env}"

  AppService:
    Type: AWS::ECS::Service
    Properties:
      ServiceName: !Sub "app-service-${Env}"
      Cluster: !Ref ECSCluster
      LaunchType: FARGATE
      DesiredCount: 2
      TaskDefinition: !Ref AppTaskDefinition
      NetworkConfiguration:
        AwsVpcConfiguration:
          Subnets:
            - !Ref PrivateSubnet1
            - !Ref PrivateSubnet2
```

```
# template.yaml
AWSTemplateFormatVersion: "2010-09-09"
Description: "Stack da aplicação"

Parameters:
  Env:
    Type: String
    Default: production
```

```
Resources:
  ECSCluster:
    Type: AWS::ECS::Cluster
    Properties:
      ClusterName: !Sub "app-cluster-${Env}"

  AppService:
    Type: AWS::ECS::Service
    Properties:
      ServiceName: !Sub "app-service-${Env}"
      Cluster: !Ref ECSCluster
      LaunchType: FARGATE
      DesiredCount: 2
      TaskDefinition: !Ref AppTaskDefinition
      NetworkConfiguration:
        AwsVpcConfiguration:
          Subnets:
            - !Ref PrivateSubnet1
            - !Ref PrivateSubnet2
```

Pipeline para IaC

```
iac-plan:
  name: Terraform Plan
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: hashicorp/setup-terraform@v3
      with:
        terraform_version: 1.8.0
    - run: terraform init
    - run: terraform fmt -check
    - run: terraform validate
    - run: terraform plan -out=tfplan
    - uses: actions/upload-artifact@v4
      with:
        name: tfplan
        path: tfplan
```

```
iac-apply:
  name: Terraform Apply
  needs: iac-plan
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  environment: production
  steps:
    - uses: actions/checkout@v4
    - uses: hashicorp/setup-terraform@v3
    - run: terraform init
```

```
- uses: actions/download-artifact@v4
  with:
    name: tfplan
- run: terraform apply tfplan
```



```
iac-plan:
  name: Terraform Plan
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: hashicorp/setup-terraform@v3
      with:
        terraform_version: 1.8.0
    - run: terraform init
    - run: terraform fmt -check
    - run: terraform validate
    - run: terraform plan -out=tfplan
    - uses: actions/upload-artifact@v4
      with:
        name: tfplan
        path: tfplan
```

```
iac-apply:
  name: Terraform Apply
  needs: iac-plan
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  environment: production
  steps:
    - uses: actions/checkout@v4
    - uses: hashicorp/setup-terraform@v3
    - run: terraform init
```


5. Automação de Deploys

Blue-Green Deployment

Duas versões do ambiente (blue = atual, green = nova). O balanceador de carga muda o tráfego da blue para a green.

```
USUÁRIOS → Load Balancer → Blue (v1.0)
                                → Green (v1.1) (após validação)
```

Switch: DNS/ALB aponta para Green

Rollback: reverter DNS para Blue

Canary Deployment

Uma porcentagem pequena do tráfego vai para a nova versão. Aos poucos, aumenta-se até 100%.

```
# Apropriado para Kubernetes com Argo Rollouts
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: app-rollout
spec:
  replicas: 10
  revisionHistoryLimit: 2
  selector:
    matchLabels:
      app: meu-app
  template:
    metadata:
      labels:
        app: meu-app
    spec:
      containers:
        - name: app
          image: meuregistro/app:1.1.0
  strategy:
    canary:
      steps:
        - setWeight: 10
        - pause: { duration: 5m }
        - setWeight: 50
        - pause: { duration: 5m }
        - setWeight: 100
```

Rolling Update

Substitui instâncias gradualmente, sem tempo de inatividade.

```
# docker-compose.yml com rolling update no Swarm
version: "3.8"
services:
  app:
    image: meuregistro/app:${VERSION}
    deploy:
      replicas: 3
      update_config:
        parallelism: 1
        delay: 10s
        order: start-first
      rollback_config:
        parallelism: 1
        order: stop-first
```

Feature Flags

Permittem ativar/desativar funcionalidades em produção sem fazer deploy.

```
import { createClient } from '@configu/sdk'

const configu = createClient({ apiKey: process.env.CONFIGU_KEY })

async function handler(req, res) {
  const novoCheckout = await configu.getFlag('checkout-v2-enabled')

  if (novoCheckout && req.user.id in novoCheckout.targeting) {
    return handleCheckoutV2(req, res)
  }

  return handleCheckoutV1(req, res)
}
```

****Ferramentas:**** LaunchDarkly, Configu, GrowthBook, Unleash, Flagsmith.

6. Automação de Banco de Dados

Migrations Automáticas no CI

```
# Job de migration
migrate:
  name: Rodar Migrations
  needs: build
  runs-on: ubuntu-latest
  environment: ${ github.ref == 'refs/heads/main' && 'production' || 'staging' }}
  steps:
    - uses: actions/checkout@v4
    - uses: pnpm/action-setup@v3
    - uses: actions/setup-node@v4
    - run: pnpm install --frozen-lockfile
    - run: pnpm build
    - uses: actions/download-artifact@v4
      with:
        name: build-output
        path: dist
    - run: pnpm db:migrate
  env:
    DATABASE_URL: ${ secrets.DATABASE_URL }}
```

Exemplo com Prisma Migrate

```
// scripts/migrate.ts
import { execSync } from 'node:child_process'

const env = process.env.NODE_ENV || 'development'

function run() {
  console.log(`Rodando migrations em ${env}...`)

  // Validar se a migration é segura (não dropa colunas sem verificação)
  execSync('pnpm prisma migrate deploy', {
    stdio: 'inherit',
    env: { ...process.env, DATABASE_URL: process.env.DATABASE_URL },
  })

  console.log('Migrations executadas com sucesso')
}

run()
```

Seed Automático em CI

```
// scripts/seed.ts
import { PrismaClient } from '@prisma/client'

const prisma = new PrismaClient()

async function seed() {
  console.log(' Iniciando seed...')

  await prisma.tenant.create({
    data: {
      name: 'Default',
      slug: 'default',
      users: {
        create: {
          email: 'admin@empresa.com',
          role: 'ADMIN',
        },
      },
    },
  })

  console.log(' Seed concluído')
}

seed()
  .catch((e) => {
    console.error(e)
    process.exit(1)
  })
  .finally(() => prisma.$disconnect())
```

Rollback Automático

Estratégia: antes de rodar a migration, fazer backup; se a migration falhar, restaurar.

```
# Rollback automático
- name: Backup antes da migration
  run: pg_dump $DATABASE_URL > /tmp/pre_migration_backup.sql
  env:
    DATABASE_URL: ${ secrets.DATABASE_URL }

- name: Rodar migration
  id: migrate
  run: pnpm db:migrate
  continue-on-error: true
  env:
```

```
DATABASE_URL: ${ secrets.DATABASE_URL }
```

```
- name: Rollback em caso de falha
  if: steps.migrate.outcome != 'success'
  run: |
    echo " Migration falhou – restaurando backup..."
    psql $DATABASE_URL < /tmp/pre_migration_backup.sql
    echo " Backup restaurado"
  env:
    DATABASE_URL: ${ secrets.DATABASE_URL }
```

```
# Rollback automático
- name: Backup antes da migration
  run: pg_dump $DATABASE_URL > /tmp/pre_migration_backup.sql
  env:
    DATABASE_URL: ${ secrets.DATABASE_URL }

- name: Rodar migration
  id: migrate
  run: pnpm db:migrate
  continue-on-error: true
  env:
```


7. Automação de Segurança

SAST (Static Application Security Testing)

Análise estática de segurança diretamente no pipeline.

```
security-sast:
  name: Análise Estática (SAST)
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: pnpm/action-setup@v3
    - uses: actions/setup-node@v4
    - run: pnpm install --frozen-lockfile

    # ESLint com plugins de segurança
    - name: ESLint Security
      run: pnpm dlx eslint . --ext .ts --rulesdir eslint-security-rules

    # SonarQube
    - name: SonarQube Scan
      uses: sonarsource/sonarcloud-github-action@master
      env:
        SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
      with:
        args: >
          -Dsonar.organization=minha-org
          -Dsonar.projectKey=meu-projeto
          -Dsonar.javascript.lcov.reportPaths=coverage/lcov.info

    # Trivy para vulnerabilidades em container
    - name: Trivy Scan
      uses: aquasecurity/trivy-action@master
      with:
        image-ref: meuregistro/app:${ secrets.github.sha }
        format: sarif
        output: trivy-results.sarif
```

DAST (Dynamic Application Security Testing)

Testa a aplicação rodando, simulando ataques reais.

```
security-dast:
  name: Teste Dinâmico (DAST)
  needs: deploy-staging
  runs-on: ubuntu-latest
```



```
steps:
  - name: OWASP ZAP Scan
    uses: zaproxy/action-full-scan@v0.10
    with:
      target: https://staging.meusistema.com.br
      rules_file_name: .zap/rules.tsv
      cmd_options: "-a -j"
```

```
security-dast:
  name: Teste Dinâmico (DAST)
  needs: deploy-staging
  runs-on: ubuntu-latest
```

```
steps:
  - name: OWASP ZAP Scan
    uses: zaproxy/action-full-scan@v0.10
    with:
      target: https://staging.meusistema.com.br
      rules_file_name: .zap/rules.tsv
      cmd_options: "-a -j"
```

Dependency Scanning

```
dependency-scan:
  name: Varredura de Dependências
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: pnpm/action-setup@v3
    - uses: actions/setup-node@v4
    - run: pnpm install --frozen-lockfile

    # Snyk
    - name: Snyk Scan
      uses: snyk/actions/node@master
      env:
        SNYK_TOKEN: ${ secrets.SNYK_TOKEN }
      with:
        args: --severity-threshold=medium

    # Dependabot alerts (nativo do GitHub)
    # npm audit
    - name: npm audit
      run: pnpm audit --audit-level=high
```

8. Automação de Code Review

Bots no Pipeline

```
code-review:
  name: Revisão Automática
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
      with:
        fetch-depth: 0

    # Linters automáticos
    - uses: pnpm/action-setup@v3
    - uses: actions/setup-node@v4
    - run: pnpm install --frozen-lockfile
    - run: pnpm lint
    - run: pnpm format:check

    # Status checks
    - name: Verificar tamanho do PR
      uses: actions/github-script@v7
      with:
        script: |
          const { data: pr } = await github.rest.pulls.get({
            ...context.repo,
            pull_number: context.issue.number,
          })
          const changedFiles = pr.changed_files
          const additions = pr.additions

          if (changedFiles > 20) {
            core.warning(` PR grande: ${changedFiles} arquivos alterados`)
          }
          if (additions > 500) {
            core.warning(` Muitas linhas adicionadas: ${additions}`)
          }

    # CodeQL
    - name: Initialize CodeQL
      uses: github/codeql-action/init@v3
      with:
        languages: javascript-typescript

    - name: CodeQL Analysis
      uses: github/codeql-action/analyze@v3
      with:
```

```
category: /language:javascript
```

```
# Dependabot (configurado no .github/dependabot.yml)
```

```
code-review:
```

```
  name: Revisão Automática
```

```
  runs-on: ubuntu-latest
```

```
  steps:
```

```
    - uses: actions/checkout@v4
```

```
      with:
```

```
        fetch-depth: 0
```

```
  # Linters automáticos
```

```
    - uses: pnpm/action-setup@v3
```

```
    - uses: actions/setup-node@v4
```

```
    - run: pnpm install --frozen-lockfile
```

```
    - run: pnpm lint
```

```
    - run: pnpm format:check
```

```
  # Status checks
```

```
    - name: Verificar tamanho do PR
```

```
      uses: actions/github-script@v7
```

```
      with:
```

```
        script: |
```

```
          const { data: pr } = await github.rest.pulls.get({
            ...context.repo,
            pull_number: context.issue.number,
          })
```

```
          const changedFiles = pr.changed_files
```

```
          const additions = pr.additions
```

```
          if (changedFiles > 20) {
```

```
            core.warning(` PR grande: ${changedFiles} arquivos alterados`)
```

```
          }
```

```
          if (additions > 500) {
```

```
            core.warning(` Muitas linhas adicionadas: ${additions}`)
```

```
          }
```

```
  # CodeQL
```

```
    - name: Initialize CodeQL
```

```
      uses: github/codeql-action/init@v3
```

```
      with:
```

```
        languages: javascript-typescript
```

```
    - name: CodeQL Analysis
```

```
      uses: github/codeql-action/analyze@v3
```

```
      with:
```

```
category: /language:javascript
```

```
# Dependabot (configurado no .github/dependabot.yml)
```

Dependabot Configuration

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"
    schedule:
      interval: "weekly"
      day: "monday"
    open-pull-requests-limit: 10
    labels:
      - "dependencies"
      - "automated"
    reviewers:
      - "time-squad"
    commit-message:
      prefix: "chore"
      include: "scope"
```

9. Automação de Releases

Semantic Versioning Automático

```
release:
  name: Gerar Release
  needs: build
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  permissions:
    contents: write
    packages: write
  steps:
    - uses: actions/checkout@v4
      with:
        fetch-depth: 0

    # Análise de commits para versionamento semântico
    - name: Semantic Release
      uses: cycjimmy/semantic-release-action@v4
      id: semantic
      env:
        GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
        NPM_TOKEN: ${ secrets.NPM_TOKEN }
      with:
        extra_plugins: |
          @semantic-release/changelog
          @semantic-release/git
          @semantic-release/npm

    - name: Publicar no npm
      if: steps.semantic.outputs.new_release_published == 'true'
      run: |
        echo " Publicando versão ${ steps.semantic.outputs.new_release_version }"
        npm publish
      env:
        NODE_AUTH_TOKEN: ${ secrets.NPM_TOKEN }
```

Conventional Commits

O `semantic-release` depende do padrão **Conventional Commits**:

```
feat: adiciona calculadora de impostos
feat(api): nova rota de relatórios
fix: corrige validação de CPF
fix(auth): timeout na renovação do token
```

chore: atualiza dependências
docs: atualiza README
refactor: extrai lógica de pagamento para serviço
perf: otimiza consulta de histórico
test: adiciona testes para o módulo de notas fiscais

feat: adiciona calculadora de impostos
feat(api): nova rota de relatórios
fix: corrige validação de CPF
fix(auth): timeout na renovação do token

chore: atualiza dependências
docs: atualiza README
refactor: extrai lógica de pagamento para serviço
perf: otimiza consulta de histórico
test: adiciona testes para o módulo de notas fiscais

Changelog Gerado Automaticamente

O `semantic-release` com plugin `@semantic-release/changelog` gera:

```
# Changelog

## [1.5.0](https://github.com/org/repo/compare/v1.4.0...v1.5.0) (2025-06-15)

### Features
* **pagamento:** adiciona suporte a PIX ([abc1234](https://github.com/org/repo/commit/abc1234))
* **relatorios:** nova rota de exportação CSV ([def5678](https://github.com/org/repo/commit/def5678))

### Bug Fixes
* **auth:** corrige timeout na renovação do token ([ghi9012](https://github.com/org/repo/commit/ghi9012))
* **validacao:** CPF com formatação agora é aceito ([jkl3456](https://github.com/org/repo/commit/jkl3456))
```


10. Automação de Ambientes

Ephemeral Environments

Ambientes temporários criados automaticamente para cada branch de feature.

```
deploy-preview:
  name: Deploy Preview
  runs-on: ubuntu-latest
  if: github.event_name == 'pull_request'
  environment:
    name: preview-${{ github.event.number }}
    url: https://pr-${{ github.event.number }}.meusistema.com.br
  steps:
    - uses: actions/checkout@v4
    - uses: pnpm/action-setup@v3
    - uses: actions/setup-node@v4
    - run: pnpm install --frozen-lockfile
    - run: pnpm build

    # Criar banco efêmero
    - name: Criar database preview
      run: |
        aws rds create-db-instance \
          --db-instance-identifier app-preview-${{ github.event.number }} \
          --db-instance-class db.t3.micro \
          --engine postgres \
          --master-username preview \
          --master-user-password "${{ secrets.PREVIEW_DB_PASSWORD }}"

    - name: Rodar migrations
      run: pnpm db:migrate
      env:
        DATABASE_URL: postgres://preview:${{ secrets.PREVIEW_DB_PASSWORD }}@preview-db/app-preview-${{ github.event.number }}

    - name: Fazer deploy no ECS
      run: |
        aws ecs update-service \
          --cluster preview \
          --service app-preview-${{ github.event.number }} \
          --force-new-deployment

destroy-preview:
  name: Destruir Preview
  runs-on: ubuntu-latest
  if: github.event_name == 'pull_request' && github.event.action == 'closed'
```

```

steps:
  - name: Remover database preview
    run: |
      aws rds delete-db-instance \
        --db-instance-identifier app-preview-${{ github.event.number }} \
        --skip-final-snapshot

  - name: Remover serviço ECS
    run: |

```

```

deploy-preview:
  name: Deploy Preview
  runs-on: ubuntu-latest
  if: github.event.name == 'pull_request'

```

```

environment:
  name: preview-${{ github.event.number }}
  url: https://pr-${{ github.event.number }}.meusistema.com.br

```

```

steps:
  - uses: actions/checkout@v4
  - uses: pnpm/action-setup@v3
  - uses: actions/setup-node@v4
  - run: pnpm install --frozen-lockfile
  - run: pnpm build

```

Criar banco efêmero

```

- name: Criar database preview
  run: |
    aws rds create-db-instance \
      --db-instance-identifier app-preview-${{ github.event.number }} \
      --db-instance-class db.t3.micro \
      --engine postgres \
      --master-username preview \
      --master-user-password ${ secrets.PREVIEW_DB_PASSWORD }

```

```

- name: Rodar migrations
  run: pnpm db:migrate
  env:

```

```

      DATABASE_URL: postgres://preview:${ secrets.PREVIEW_DB_PASSWORD }
    }@preview-db/app-preview-${{ github.event.number }}

```

```

- name: Fazer deploy no ECS
  run: |
    aws ecs update-service \
      --cluster preview \
      --service app-preview-${{ github.event.number }} \
      --force-new-deployment

```

```

destroy-preview:
  name: Destruir Preview
  runs-on: ubuntu-latest
  if: github.event_name == 'pull_request' && github.event.action == 'closed'

```

```

steps:
  - name: Remover database preview
    run: |
      aws rds delete-db-instance \
        --db-instance-identifier app-preview-${{ github.event.number }} \
        --skip-final-snapshot

  - name: Remover serviço ECS
    run: |
      aws ecs delete-service \
        --cluster preview \
        --service app-preview-${{ github.event.number }} \
        --force

```

Preview Deployments (Vercel / Render)

```

vercel-preview:
  name: Vercel Preview
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: amondnet/vercel-action@v25
    with:
      vercel-token: ${ secrets.VERCEL_TOKEN }
      vercel-org-id: ${ secrets.VERCEL_ORG_ID }
      vercel-project-id: ${ secrets.VERCEL_PROJECT_ID }
      github-token: ${ secrets.GITHUB_TOKEN }
      github-comment: true

```

11. Automação de Monitoramento

Alertas Automáticos

```
monitoring-setup:
  name: Configurar Alertas
  runs-on: ubuntu-latest
  steps:
    - name: Configurar Datadog Monitors
      uses: datadog/synthetics-ci-github-action@v1
      with:
        api_key: ${ secrets.DD_API_KEY }
        app_key: ${ secrets.DD_APP_KEY }
        public_ids: ${ vars.DATADOG_SYNTHETIC_TESTS }

    - name: Configurar alerta de latência P99
      run: |
        curl -X POST "https://api.datadoghq.com/api/v1/monitor" \
          -H "Content-Type: application/json" \
          -H "DD-API-KEY: ${ secrets.DD_API_KEY }" \
          -H "DD-APPLICATION-KEY: ${ secrets.DD_APP_KEY }" \
          -d '{
            "name": "Latência P99 > 500ms",
            "type": "query alert",
            "query": "avg(last_5m):p99(trace.http.request.duration{service:app}) > 500",
            "message": "Latência alta detectada! @time-squad",
            "tags": ["service:app", "severity:critical"]
          }'
```

Self-Healing

```
// scripts/self-healing.ts
import { ECSCClient, DescribeServicesCommand, UpdateServiceCommand } from '@aws-sdk/client-ecs'

const ecs = new ECSCClient({ region: 'us-east-1' })

async function healService(cluster: string, service: string) {
  const { services } = await ecs.send(
    new DescribeServicesCommand({ cluster, services: [service] })
  )

  const svc = services![0]
  const running = svc.runningCount ?? 0
  const desired = svc.desiredCount ?? 0

  if (running < desired) {
```

```

    console.log(` Service ${service} com ${running}/${desired} – forçando novo deploy...`)

    await ecs.send(
      new UpdateServiceCommand({
        cluster,
        service,
        forceNewDeployment: true,
      })
    )

    console.log(' Deploy forçado enviado')
  } else {
    console.log(` Service ${service} saudável (${running}/${desired})`)
  }
}

healService('production', 'app-service').catch(console.error)

```



```

// scripts/self-healing.ts
import { ECSClient, DescribeServicesCommand, UpdateServiceCommand } from '@aws-sdk/client-ecs'

const ecs = new ECSClient({ region: 'us-east-1' })

async function healService(cluster: string, service: string) {
  const { services } = await ecs.send(
    new DescribeServicesCommand({ cluster, services: [service] })
  )

  const svc = services![0]
  const running = svc.runningCount ?? 0
  const desired = svc.desiredCount ?? 0

  if (running < desired) {

```

```

console.log(` Service ${service} com ${running}/${desired} – forçando novo deploy...`)

await ecs.send(
  new UpdateServiceCommand({
    cluster,
    service,
    forceNewDeployment: true,
  })
)

console.log(' Deploy forçado enviado')
} else {
  console.log(` Service ${service} saudável (${running}/${desired})`)
}
}

healService('production', 'app-service').catch(console.error)

```

Runbooks Automáticos

```

# runbook: recuperacao-banco.yml
name: Runbook - Recuperar Database
on:
  workflow_dispatch:
    inputs:
      database_name:
        description: 'Nome do banco'
        required: true
      restore_point:
        description: 'PITR timestamp (YYYY-MM-DD HH:MM:SS)'
        required: false

jobs:
  restore:
    runs-on: ubuntu-latest
    steps:
      - name: Validar parâmetros
        run: |
          if [ -z "${{ inputs.database_name }}" ]; then
            echo " database_name é obrigatório"
            exit 1
          fi

      - name: Parar aplicação
        run: |
          aws ecs update-service \
            --cluster production \
            --service app-service \
            --desired-count 0

```

```

- name: Restaurar snapshot
  run: |
    if [ -n "${{ inputs.restore_point }}" ]; then
      aws rds restore-db-instance-to-point-in-time \
        --source-db-instance-identifier "${{ inputs.database_name }}" \
        --target-db-instance-identifier "${{ inputs.database_name }}-restored" \
        --restore-time "${{ inputs.restore_point }}"
    else
      SNAPSHOT=$(aws rds describe-db-snapshots \
        --db-instance-identifier "${{ inputs.database_name }}" \
        --query "DBSnapshots[-1].DBSnapshotIdentifier" \
        --output text)
      aws rds restore-db-instance-from-db-snapshot \
        --db-instance-identifier "${{ inputs.database_name }}-restored" \
        --db-snapshot-identifier $SNAPSHOT
    fi

- name: Trocar DNS para DB restaurado
  run: |
    aws rds modify-db-instance \
      --db-instance-identifier "${{ inputs.database_name }}-restored" \
      --dns-records "${{ inputs.dns_records }}"

```

```
# runbook: recuperacao-banco.yml
```

```
name: Runbook - Recuperar Database
```

```
on:
```

```
  workflow_dispatch:
```

```
  inputs:
```

```
    database_name:
```

```
      description: 'Nome do banco'
```

```
      required: true
```

```
    restore_point:
```

```
      description: 'PITR timestamp (YYYY-MM-DD HH:MM:SS)'
```

```
      required: false
```

```
jobs:
```

```
  restore:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Validar parâmetros
```

```
        run: |
```

```
          if [ -z "${{ inputs.database_name }}" ]; then
```

```
            echo " database_name é obrigatório"
```

```
            exit 1
```

```
          fi
```

```
      - name: Parar aplicação
```

```
        run: |
```

```
          aws ecs update-service \
```

```
            --cluster production \
```

```
            --service app-service \
```

```
            --desired-count 0
```


12. Pipeline as Code — Boas Práticas

Modularização e Reuso

```
# .github/actions/setup-node/action.yml
name: "Setup Node.js"
description: "Configura Node.js, pnpm e cache"
inputs:
  node-version:
    required: false
    default: "20.x"
  pnpm-version:
    required: false
    default: "9"

runs:
  using: "composite"
  steps:
    - uses: actions/checkout@v4
    - uses: pnpm/action-setup@v3
      with:
        version: "${{ inputs.pnpm-version }}"
    - uses: actions/setup-node@v4
      with:
        node-version: "${{ inputs.node-version }}"
        cache: pnpm
    - run: pnpm install --frozen-lockfile
      shell: bash
```

Uso da action reutilizável:

```
# .github/workflows/ci.yml
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: ./.github/actions/setup-node
      - run: pnpm lint

  test:
    runs-on: ubuntu-latest
    steps:
      - uses: ./.github/actions/setup-node
      - run: pnpm test
```

Reuso de Jobs com Matrix

```
test-matrix:
  name: Testes (${ matrix.node-version })
  runs-on: ubuntu-latest
  strategy:
    matrix:
      node-version: [18.x, 20.x, 22.x]
    fail-fast: false
  steps:
    - uses: ./github/actions/setup-node
      with:
        node-version: ${ matrix.node-version }
    - run: pnpm test:ci
```

```

test-matrix:
  name: Testes (${ matrix.node-version })
  runs-on: ubuntu-latest
  strategy:
    matrix:
      node-version: [18.x, 20.x, 22.x]
    fail-fast: false
  steps:
    - uses: ./github/actions/setup-node
      with:
        node-version: ${ matrix.node-version }
    - run: pnpm test:ci

```

Pipeline de Deploy com Environments e Gates

```

deploy-production:
  name: Deploy em Produção
  needs: [lint, test, build, security-sast]
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  environment:
    name: production
    url: https://meusistema.com.br
  concurrency:
    group: production-deploy
    cancel-in-progress: false
  steps:
    - uses: actions/checkout@v4
    - uses: ./github/actions/setup-node
    - uses: actions/download-artifact@v4
      with:
        name: build-output
        path: dist

    - name: Login no Docker Registry
      uses: docker/login-action@v3
      with:
        username: ${ secrets.DOCKER_USER }
        password: ${ secrets.DOCKER_PASS }

    - name: Build e Push da Imagem
      run: |
        docker build -t meuregistro/app:${ github.sha } .
        docker push meuregistro/app:${ github.sha }

    - name: Deploy no ECS
      run: |
        aws ecs update-service \
          --cluster production \

```


5.2 Por que essa stack?



```
deploy-production:
  name: Deploy em Produção
  needs: [lint, test, build, security-sast]
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  environment:
    name: production
    url: https://meusistema.com.br
  concurrency:
    group: production-deploy
    cancel-in-progress: false
  steps:
    - uses: actions/checkout@v4
    - uses: ./github/actions/setup-node
    - uses: actions/download-artifact@v4
      with:
        name: build-output
        path: dist

    - name: Login no Docker Registry
      uses: docker/login-action@v3
      with:
        username: ${ secrets.DOCKER_USER }
        password: ${ secrets.DOCKER_PASS }

    - name: Build e Push da Imagem
      run: |
        docker build -t meuregistro/app:${ github.sha } .
        docker push meuregistro/app:${ github.sha }

    - name: Deploy no ECS
      run: |
        aws ecs update-service \
          --cluster production \
```

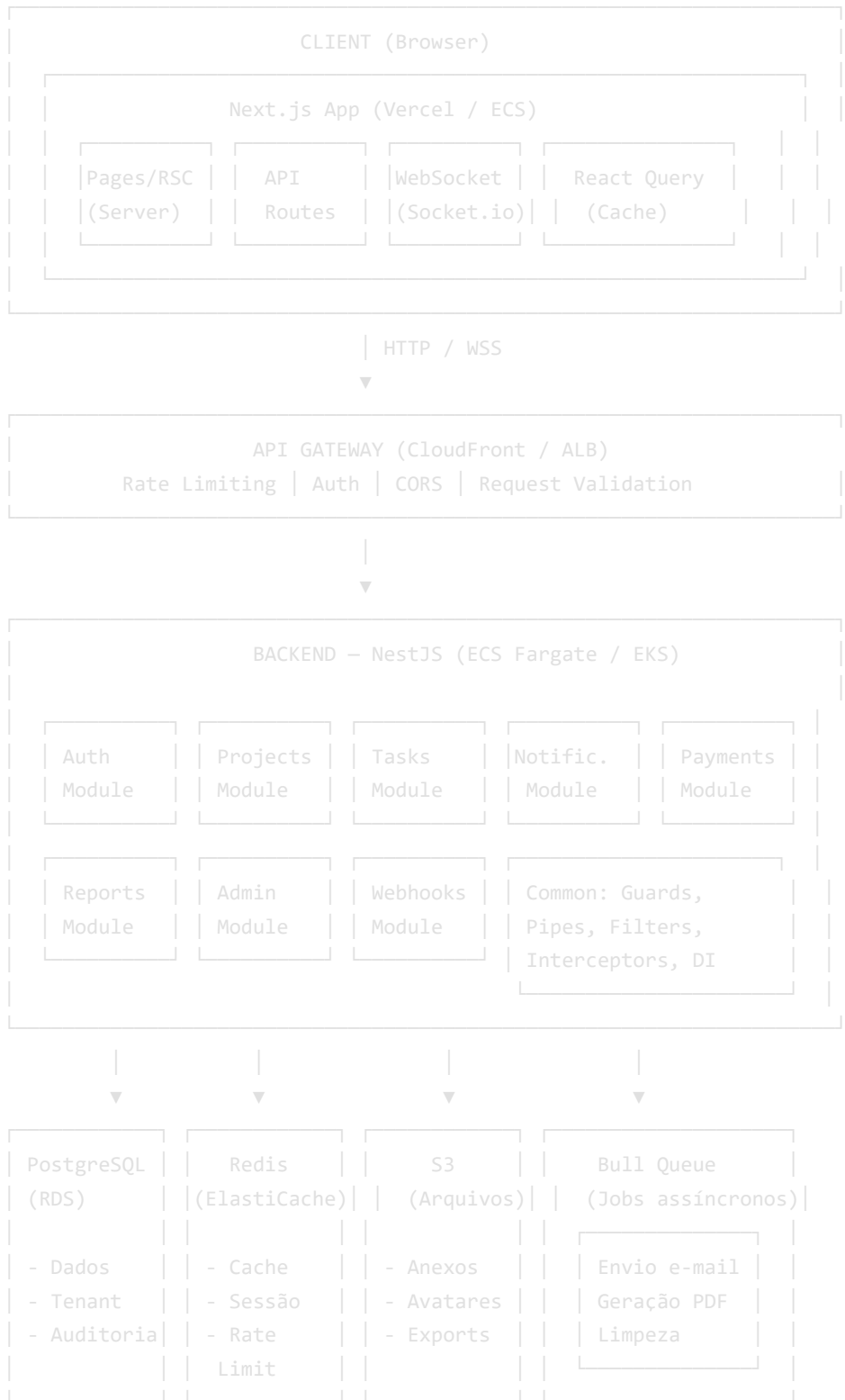

5.2 Por que essa stack?

Empresas usam: NestJS + Next.js + PostgreSQL + Redis + Docker + AWS Não usam: PHP puro, jQuery, MySQL sem ORM, servidor único

A stack escolhida prepara o aluno para: 85% das vagas de SaaS Enterprise no Brasil
Escalabilidade real (horizontal, stateless) Ecossistema com alta empregabilidade ``

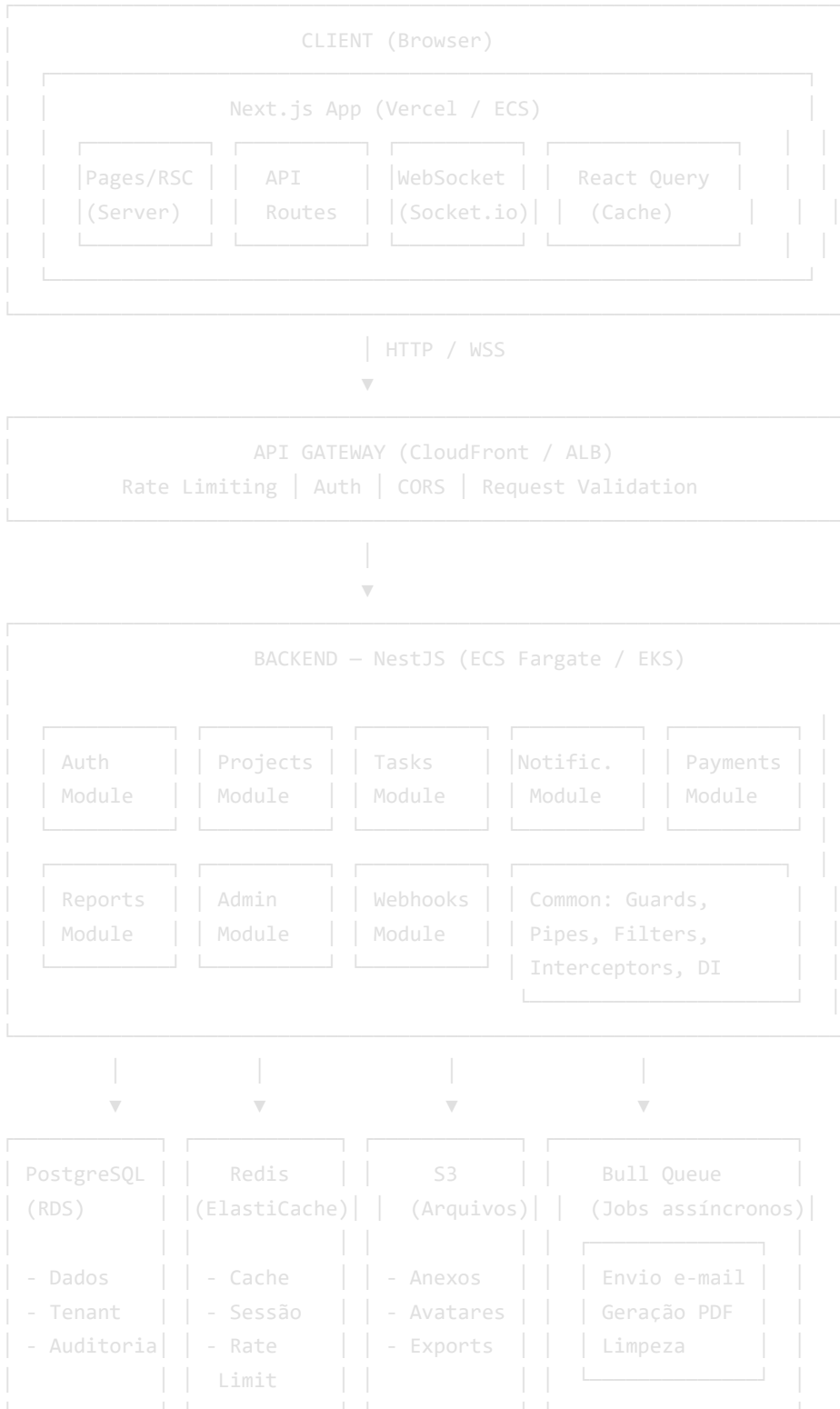
6. Arquitetura

6.1 Diagrama em ASCII



OBSERVABILIDADE

Prometheus ← Métricas	Loki ← Logs	Tempo/Jaeger ← Traces
Grafana (Dashboard)	Alertmanager	Uptime Kuma



Prometheus ← Métricas	Loki ← Logs	Tempo/Jaeger ← Traces
Grafana (Dashboard)	Alertmanager	Uptime Kuma

6.2 Decisões Arquiteturais (ADRs)

****Contexto:**** Precisamos decidir entre Modular Monolith e Microservices.

****Decisão:**** Começar com Modular Monolith, com bounded contexts bem definidos.

****Justificativa:**** - Time pequeno (1–3 devs) — microservices trariam sobrecarga de comunicação e deploy - Domínio bem conhecido (gestão de projetos) — não justifica complexidade inicial - Bounded contexts bem definidos permitem extração futura para microservices sem reescrita

****Consequências:**** - Positivas: deploy único, comunicação in-process, debug simples, CI/CD simples - Negativas: acoplamento indireto se módulos não forem disciplinados, escalonamento monolítico - Mitigação: módulos estritamente isolados com interfaces públicas e events entre contextos

****Contexto:**** Precisamos isolar dados de diferentes empresas.

****Decisão:**** Usar coluna `tenant\_id` em todas as tabelas (discriminator column) + RLS no PostgreSQL.

****Justificativa:**** - Menos complexo que schema por tenant (sem migrations N×) - Mais isolado que database por tenant (sem conexões N×) - RLS garante que nenhuma query vaze dados entre tenants - Backup e restore mais simples (único banco)

****Consequências:**** - Positivas: backup único, migrations únicas, custo menor - Negativas: contenção em tabelas grandes, necessidade de índices cuidadosos - Mitigação: índices compostos (tenant_id + id), partition by tenant em tabelas grandes

****Contexto:**** Módulos precisam se comunicar sem acoplamento direto (ex: ao criar tarefa, notificar usuário).

****Decisão:**** Usar Event Emitter in-process (para o monólito) com interface que permita migrar para message broker.

****Justificativa:**** - Módulos não se importam com quem consome seus eventos - Fácil adicionar novos consumers sem modificar producers - Se um dia migrar para microservices, os eventos já estão definidos

****Consequências:**** - Positivas: baixo acoplamento, extensível, testável - Negativas: complexidade adicional (eventos precisam ser versionados) - Mitigação: eventos versionados (v1, v2), schemas

de evento validados com Zod

****Contexto:**** Precisamos de SSR para SEO e performance, mas também de interatividade rica.

****Decisão:**** Next.js 14+ App Router com Server Components por padrão, Client Components onde necessário.

****Justificativa:**** - Server Components diminuem JS enviado ao cliente - Streaming melhora percepção de performance - App Router oferece layout aninhado, loading states, error boundaries nativos

****Consequências:**** - Positivas: performance, SEO, DX moderna - Negativas: curva de aprendizado (server vs client mental model) - Mitigação: documentar decisão de componente server vs client no design system

6.3 Trade-offs Explicados

Decisão	Trade-off
Monólito vs Microservices	Simplicidade inicial vs e
RLS vs Schema vs DB	Custo vs isolamento vs co
Event-driven in-process	Baixa latência vs dependê
Server Components	Performance vs interativi
Prisma vs TypeORM	Type-safe vs maturidade d
Zod vs class-validator	Bundle menor vs ecossiste

7. Módulos do Sistema

7.1 User & Tenant Module

****Responsabilidade:**** Gerenciar usuários, empresas (tenants), convites, perfis.

```
src/modules/users/  
├─ user.module.ts  
├─ user.controller.ts  
├─ user.service.ts  
├─ user.repository.ts  
├─ dto/  
│   ├─ create-user.dto.ts  
│   └─ update-user.dto.ts  
│       └─ invite-user.dto.ts  
├─ entities/  
│   └─ user.entity.ts  
├─ events/  
│   ├─ user-created.event.ts  
│   └─ user-invited.event.ts  
└─ specs/  
    ├─ user.service.spec.ts  
    └─ user.controller.spec.ts
```

```
src/modules/tenants/  
├─ tenant.module.ts  
├─ tenant.controller.ts  
├─ tenant.service.ts  
├─ tenant.repository.ts  
├─ dto/  
│   ├─ create-tenant.dto.ts  
│   └─ update-tenant.dto.ts  
├─ entities/  
│   └─ tenant.entity.ts  
└─ specs/  
    └─ tenant.service.spec.ts
```

7.2 Auth Module

****Responsabilidade:**** Login, registro, OAuth, MFA, refresh token, session.

```
src/modules/auth/  
├─ auth.module.ts  
├─ auth.controller.ts  
├─ auth.service.ts  
└─ strategies/
```



```
|   ├── jwt.strategy.ts
|   ├── jwt-refresh.strategy.ts
|   └── google.strategy.ts
└── guards/
    ├── jwt-auth.guard.ts
    ├── roles.guard.ts
    └── tenant.guard.ts
└── dto/
    ├── login.dto.ts
    ├── register.dto.ts
    ├── refresh.dto.ts
    └── forgot-password.dto.ts
└── specs/
    └── auth.service.spec.ts
```

```
src/modules/auth/
├── auth.module.ts
├── auth.controller.ts
├── auth.service.ts
└── strategies/
```

```
|   ├── jwt.strategy.ts
|   ├── jwt-refresh.strategy.ts
|   └── google.strategy.ts
└── guards/
    ├── jwt-auth.guard.ts
    ├── roles.guard.ts
    └── tenant.guard.ts
└── dto/
    ├── login.dto.ts
    ├── register.dto.ts
    ├── refresh.dto.ts
    └── forgot-password.dto.ts
└── specs/
    └── auth.service.spec.ts
```

7.3 Projects Module

****Responsabilidade:**** CRUD de projetos, membros do projeto, templates.

```
src/modules/projects/
├── project.module.ts
├── project.controller.ts
├── project.service.ts
├── project.repository.ts
├── dto/
|   ├── create-project.dto.ts
|   ├── update-project.dto.ts
|   └── add-member.dto.ts
├── entities/
|   └── project.entity.ts
├── events/
|   ├── project-created.event.ts
|   └── member-added.event.ts
└── specs/
    └── project.service.spec.ts
```

7.4 Tasks Module

****Responsabilidade:**** CRUD de tarefas, Kanban, comentários, checklists, time tracking, dependências.

```
src/modules/tasks/
├── task.module.ts
├── task.controller.ts
├── task.service.ts
├── task.repository.ts
└── dto/
```

```
|   |─ create-task.dto.ts
|   |─ update-task.dto.ts
|   |─ move-task.dto.ts
|   └─ add-comment.dto.ts
└─ entities/
   |─ task.entity.ts
   └─ task-history.entity.ts
└─ gateway/
   └─ task.gateway.ts          (WebSocket)
└─ events/
   |─ task-created.event.ts
   |─ task-assigned.event.ts
   └─ task-completed.event.ts
└─ specs/
   |─ task.service.spec.ts
   └─ task.gateway.spec.ts
```

```
src/modules/tasks/
└─ task.module.ts
└─ task.controller.ts
└─ task.service.ts
└─ task.repository.ts
└─ dto/
```

```
|   |─ create-task.dto.ts
|   |─ update-task.dto.ts
|   |─ move-task.dto.ts
|   └─ add-comment.dto.ts
|─ entities/
|   |─ task.entity.ts
|   └─ task-history.entity.ts
|─ gateway/
|   └─ task.gateway.ts          (WebSocket)
|─ events/
|   |─ task-created.event.ts
|   |─ task-assigned.event.ts
|   └─ task-completed.event.ts
└─ specs/
    |─ task.service.spec.ts
    └─ task.gateway.spec.ts
```

7.5 Notifications Module

****Responsabilidade:**** Notificações in-app, push, e-mail, preferências.

```
src/modules/notifications/
├─ notification.module.ts
├─ notification.controller.ts
├─ notification.service.ts
├─ notification.repository.ts
├─ dto/
|   └─ update-preferences.dto.ts
├─ entities/
|   └─ notification.entity.ts
├─ processors/
|   ├── email.processor.ts      (Bull)
|   └─ push.processor.ts
├─ templates/
|   ├── task-assigned.hbs
|   └─ mention.hbs
└─ specs/
    └─ notification.service.spec.ts
```

7.6 Reports Module

****Responsabilidade:**** Dashboard, gráficos, exportação PDF/CSV.

```
src/modules/reports/
├─ report.module.ts
├─ report.controller.ts
└─ report.service.ts
```

```
|─ dto/
|   └─ generate-report.dto.ts
|─ generators/
|   └─ pdf.generator.ts
|   └─ csv.generator.ts
└─ specs/
    └─ report.service.spec.ts
```

```
src/modules/reports/
|─ report.module.ts
|─ report.controller.ts
|─ report.service.ts
```

```
|— dto/
|   └─ generate-report.dto.ts
|— generators/
|   └─ pdf.generator.ts
|   └─ csv.generator.ts
└─ specs/
    └─ report.service.spec.ts
```

7.7 Payments Module

****Responsabilidade:**** Planos, assinaturas, gateway Stripe, webhooks, faturas.

```
src/modules/payments/
├─ payment.module.ts
├─ payment.controller.ts
├─ payment.service.ts
├─ entities/
|   └─ subscription.entity.ts
|   └─ invoice.entity.ts
├─ webhooks/
|   └─ stripe.webhook.ts
├─ dto/
|   └─ create-checkout.dto.ts
└─ specs/
    └─ payment.service.spec.ts
```

7.8 Admin Module

****Responsabilidade:**** Logs de atividade, auditoria, configurações do tenant, webhooks de saída.

```
src/modules/admin/
├─ admin.module.ts
├─ admin.controller.ts
├─ admin.service.ts
├─ dto/
|   └─ create-webhook.dto.ts
└─ specs/
    └─ admin.service.spec.ts
```

8. Critérios de Avaliação

8.1 Checklist por Categoria

Critério	Peso	Descrição
Estrutura de pastas	5	Segue padrão modular, sep
Código limpo	5	Nomes significativos, fun
TypeScript estrito	5	strict mode, sem `any`, t
Tratamento de erros	5	Exception filters, erros
Validação de entrada	5	Zod em toda API, mensagen
Segurança	5	RBAC, rate limit, headers
Multi-tenancy	5	Isolamento verificado, se
Performance	5	Cache, paginação, índices

Critério	Peso
Testes unitários (mín. 40	5
Testes de integração (mín	5
Testes E2E (mín. 5 cenári	5
Cobertura > 80% unitários	5

Critério	Peso
README completo (instalaç	5
ADRs documentando decisão	3
Swagger/OpenAPI da API	3
Runbook de operação (como	2
Instruções de setup local	2

Critério	Peso
Dockerfile e docker-compo	3
Pipeline CI/CD funcional	4
Deploy em cloud funcional	4
Health checks e monitoram	2
Variáveis de ambiente ext	2

Critério	Peso
Pitch de 5 min (problema,	3
Demo ao vivo (funcionalid	4
Lições aprendidas e próxi	2
Respostas a perguntas téc	1

8.2 Nota de Corte

- **Mínimo para aprovação:** 70% (nota geral) - **Mínimo por categoria:** 50% em cada uma das 5 categorias - **Reprovação automática:** código sem multi-tenancy, senha em texto puro, ou dados vazando entre tenants

9. Entregáveis

9.1 Repositório

- Um repositório GitHub (monorepo com `apps/backend`, `apps/frontend`, `packages/shared`)
- Ou dois repositórios separados (backend + frontend) - Branch principal: `main` com proteção (PR obrigatório, CI obrigatório) - Commits seguindo Conventional Commits

9.2 README

```
# Nome do Projeto

## Descrição
[Proposta de valor em 2-3 frases]

## Stack
- Backend: NestJS + Prisma + PostgreSQL + Redis
- Frontend: Next.js 14 + Tailwind + shadcn/ui
- Infra: Docker + AWS/Azure/GCP

## Funcionalidades
- [x] Autenticação multi-tenant
- [x] Gestão de projetos e tarefas
- [x] Kanban com drag & drop
- [x] Notificações em tempo real
- [x] Relatórios e dashboards
- [x] Pagamentos integrados

## Quick Start
```

git clone ... cp .env.example .env docker compose up ```text

Estrutura

[Diagrama de pastas]

Documentação

- Swagger - ADRs - Runbook

Deploy

[Link para produção] ``

9.3 Documentação

- `docs/adrs/` — Decisões arquiteturais (mín. 4 ADRs) - `docs/runbook.md` — Como operar o sistema em produção - `docs/api.md` ou Swagger — Documentação da API - `docs/architecture.md` — Diagrama e explicação da arquitetura

9.4 Testes

- Unitários: `apps/backend/src/\*\*/\*.spec.ts` (mín. 40) - Integração: testes com banco real ou testcontainers (mín. 10) - E2E: Playwright cobrindo fluxos críticos (mín. 5) - Relatório de cobertura anexado ao README

9.5 CI/CD

Pipeline de GitHub Actions: `` push / PR main: 1. Lint (ESLint + Prettier) 2. Type check (tsc --noEmit) 3. Testes unitários + integração 4. Build (Docker image) 5. Push para container registry 6. Deploy para staging (automatizado) 7. Smoke tests 8. Deploy para produção (manual) ``

9.6 Dashboard de Monitoramento

- Grafana com métricas do backend (requests, latência, erros) - Logs centralizados no Loki (ou CloudWatch) - Tracing no Jaeger (ou X-Ray) - Alerta configurado para latência > 1s

10. Apresentação Final

10.1 Estrutura do Pitch (5 minutos)

Minuto	Conteúdo
0:00–0:30	Problema: "Gerenciar
0:30–1:00	Solução: "Criamos uma
1:00–2:00	Demo: Mostrar login,
2:00–2:30	Diferenciais técnicos:
2:30–3:00	Arquitetura: Modular
3:00–3:30	Testes: Cobertura, es
3:30–4:00	Deploy: Infraestrutur
4:00–4:30	Lições aprendidas: O
4:30–5:00	Próximos passos: Feat

10.2 Demo ao Vivo

Fluxo obrigatório de demonstração: 1. Acessar a aplicação e registrar um novo tenant 2. Convidar um membro (simular) 3. Criar um projeto com template 4. Criar tarefas e mover no Kanban 5. Atribuir tarefa e ver notificação 6. Comentar em uma tarefa com menção 7. Visualizar dashboard com gráficos 8. Mostrar monitoramento (Grafana)

10.3 Lições Aprendidas

O aluno deve refletir sobre: - O que foi mais desafiador e por quê - Qual decisão técnica mudaria se fosse começar de novo - Como o aprendizado dos 20 módulos se aplicou na prática - O que aprendeu além do conteúdo das aulas

10.4 Próximos Passos (Pós-Curso)

Sugestões de evolução contínua: - Transformar o monólito em microservices (extrair Payments, Notifications) - Adicionar inteligência artificial (sugestão de prioridade, estimativa automática) - Criar integrações com Slack, Discord, Jira, GitHub - Publicar como SaaS real (cobrar clientes de verdade) - Abrir o core como open source

11. Checklist de Habilidades

Antes de entregar o projeto final, verifique se você domina:

M01 — Mentalidade Enterprise

- ☐ Entendo a diferença entre produto e projeto - ☐ Sei calcular ROI de uma funcionalidade - ☐ Conheço os papéis de um time Engineering

M02 — Product Discovery

- ☐ Sei conduzir entrevistas com stakeholders - ☐ Sei priorizar backlog com valor de negócio

M03 — Design Thinking

- ☐ Sei aplicar as 5 etapas do Design Thinking - ☐ Crio protótipos de baixa fidelidade

M04 — UX Research

- ☐ Sei fazer testes de usabilidade - ☐ Crio personas e mapas de jornada

M05 — Wireframes

- ☐ Crio wireframes de média fidelidade no Figma - ☐ Documento fluxos de navegação

M06 — UI Design

- ☐ Aplico princípios de gestalt, tipografia, cor, espaçamento - ☐ Crio protótipos de alta fidelidade

M07 — Design System

- ☐ Crio e documento componentes no Storybook - ☐ Mantenho consistência visual com tokens

M08 — Arquitetura

- [] Aplico SOLID, Clean Architecture, DDD - [] Documento decisões com ADRs

M09 — Modelagem de Dados

- [] Modelo bancos normalizados até 3FN - [] Crio índices, constraints, relacionamentos

M10 — Backend NestJS

- [] Construo módulos, controllers, services, repositories - [] Implemento guards, interceptors, pipes, filters - [] Escrevo testes unitários e de integração

M11 — Frontend Next.js

- [] Desenvolvo com App Router, Server/Client Components - [] Gerencia estado com React Query + Zustand

M12 — Segurança

- [] Implemento JWT, OAuth, RBAC, rate limiting - [] Sei mitigar OWASP Top 10

M13 — Multi-tenancy

- [] Implemento isolamento de dados entre tenants - [] Aplico RLS ou discriminator column

M14 — DevOps

- [] Dockerizo aplicações - [] Configuro CI/CD com GitHub Actions - [] Faço deploy em cloud

M15 — Testes e QA

- [] Escrevo testes unitários, integração, E2E - [] Uso mocks, fixtures, testcontainers

M16 — Observabilidade

- [] Implemento logging estruturado, métricas, tracing - [] Crio dashboards no Grafana

M17 — Governança

- ☐ Uso ADRs para documentar decisões arquiteturais - ☐ Implemento code review com checklists e approval gates

M18 — Agentes de IA

- ☐ Integro APIs de LLM (OpenAI, Anthropic) - ☐ Crio agentes para automação de tarefas

M19 — Auditoria

- ☐ Implemento trilha de auditoria - ☐ Atendo requisitos de compliance (LGPD)

M20 — Automação

- ☐ Automatizo processos com filas e jobs - ☐ Crio eze tests com Playwright

12. Próximos Passos — Como Continuar Evoluindo

12.1 Evoluções Técnicas

****Curto prazo (1-3 meses):**** - Substituir Event Emitter in-process por message broker (RabbitMQ / Kafka) - Extrair Payments Module para microserviço independente - Adicionar GraphQL para consultas complexas do dashboard - Implementar cache de página com ISR (Incremental Static Regeneration) - Adicionar testes de carga com K6 (1000 usuários simultâneos)

****Médio prazo (3-6 meses):**** - Extrair Notifications para microserviço com Serverless (Lambda) - Adicionar IA: sugerir responsável baseado em carga de trabalho - Criar integração bidirecional com Slack e Discord - Implementar importação de projetos do Trello/Asana/Jira - Adicionar busca full-text com Elasticsearch ou Meilisearch

****Longo prazo (6-12 meses):**** - Transformar em plataforma de marketplace de integrações - Adicionar white-label (tenant pode personalizar domínio, logo, cores) - Criar mobile app (React Native) - Suporte offline-first com PWA e sincronização background

12.2 Contribuir para Open Source

O projeto pode evoluir para um open source real: - Publicar em organização GitHub - Adicionar CONTRIBUTING.md, CODE_OF_CONDUCT.md - Criar issue templates (bug, feature, RFC) - Configurar GitHub Discussions para comunidade - Fazer release no GitHub com changelog automático

12.3 Portfólio

Para usar o projeto como portfólio: - README com capturas de tela, GIFs, link para demo ao vivo - Artigo no Medium/Dev.to explicando a arquitetura e decisões - Vídeo de 10 min no YouTube mostrando o sistema funcionando - LinkedIn: adicionar projeto na seção "Projetos" com descrição técnica - GitHub: fixar o repositório no perfil, manter atividade constante

12.4 Monetização

- Publicar como SaaS real (cobrar R\$ 29–R\$ 99/mês por empresa) - Oferecer versão self-hosted para empresas com compliance restrito - Vender consultoria de implementação e customização - Criar curso sobre "Como construir um SaaS Enterprise do zero"

Resumo

O Projeto Final é a oportunidade de demonstrar tudo que foi aprendido nos 20 módulos. Mais do que um sistema funcional, o que se avalia é a **qualidade das decisões técnicas**, a **organização do código**, a **cobertura de testes**, a **documentação clara**, e a **capacidade de apresentar e defender** as escolhas feitas.

Lembre-se: um sistema Enterprise de verdade não é aquele que funciona no seu computador. É aquele que continua funcionando sob carga, com múltiplos tenants, com falhas de rede, com dados inconsistentes — e que pode ser operado, monitorado e evoluído por um time.

Boa construção.

Bônus — O Erro de 90% ao Usar IA para Programar

O erro que 90% das pessoas cometem usando IA para programar

Nível: Conceitos / Engenharia **Tempo estimado:** 20 minutos **Público-alvo:** Desenvolvedores iniciantes e intermediários que utilizam assistentes de IA no dia a dia

Pré-requisitos

- Experiência básica com programação em qualquer linguagem - Familiaridade com uso de assistentes de IA (GitHub Copilot, Claude Code, ChatGPT, etc.) - Noções fundamentais de Git e versionamento

Objetivos de aprendizagem

Ao final desta aula, o aluno será capaz de:

1. **Identificar** os 5 erros mais comuns ao usar IA para programar 2. **Diferenciar** uso produtivo de uso prejudicial de assistentes de IA 3. **Aplicar** técnicas de prompt estruturado para obter respostas precisas 4. **Estabelecer** um fluxo de validação e revisão para código gerado por IA 5. **Configurar** arquivos de instrução do projeto (AGENTS.md / CLAUDE.md) para melhorar a consistência do agente

Competências desenvolvidas

Hard skills: - Prompt engineering aplicado à programação - Code review de código gerado por IA - Configuração de assistentes de IA no projeto - Automação de testes como ferramenta de verificação

Soft skills: - Pensamento crítico e validação de fontes - Comunicação clara e estruturada - Responsabilidade profissional sobre o código produzido - Iteração e refinamento contínuo

1. Introdução: por que 90% cometem esse erro

Era uma terça-feira comum. O desenvolvedor precisava de uma função simples para validar emails. Pediu ao ChatGPT, copiou o código, fez deploy. Na sexta, o banco de dados estava cheio de registros com emails como "usuario@". Aquela função — que parecia perfeita — só verificava se existia um "@" na string.

A IA acertou a sintaxe. Errou a lógica. O desenvolvedor não revisou. Produção quebrou.

Esse cenário se repete milhares de vezes todos os dias. Assistentes de IA geram código em segundos — mas essa velocidade tem um custo oculto. Pesquisas da indústria revelam um padrão preocupante:

Dado	Fonte
63% dos desenvolvedor	Stack Overflow Survey
68% têm dificuldade e	Stack Overflow Survey
Sem regras configuradas,	Claude Code Pro Pack
Com 12 regras, o erro cai	Claude Code Pro Pack Rese

O problema não é a IA. O problema é **como usamos a IA**. A maioria dos desenvolvedores repete os mesmos 5 erros — e 90% sequer percebe que os está cometendo.

[!NOTE]

Esta aula compila fontes oficiais (GitHub, Anthropic, OpenCode), pesquisa acadêmica (arXiv 2512.05239) e benchmarks da indústria para mapear os erros e, mais importante, mostrar como corrigi-los.

Pare e pense: quantas vezes você copiou código de IA sem ler cada linha? Se a resposta for "algumas vezes", esta aula é para você.

2. Erro 1: Confiar cegamente na saída da IA

Definição

Aceitar o código gerado pela IA como correto sem qualquer questionamento, validação ou revisão.

Por que acontece

A IA gera respostas com **alta fluência e aparência de confiança**. O código parece correto, compila e muitas vezes até passa em testes simples — mas pode conter bugs sutis de lógica, segurança ou performance.

"A IA erra com confiança, não com hesitação." — TechTudo

O problema é psicológico: nosso cérebro associa fluência a competência. Quando a IA escreve um parágrafo ou função que "soa bem", relaxamos a guarda. Só que a IA não sabe o que está fazendo — ela está apenas completando padrões estatísticos.

O survey arXiv 2512.05239 classifica os bugs encontrados em código gerado por IA em quatro categorias:

Tipo de Bug	O que significa	Exemplo
Lógica	Código sintaticamente cor	Valida email só com `incl
Segurança	Vulnerabilidades introduz	SQL injection, senha em M
Performance	Código ineficiente	Loop aninhado desnecessár
Compatibilidade	Dependências incorretas o	Import de biblioteca que

Consequência

Bugs em produção, vulnerabilidades de segurança e dívida técnica acumulada. Como alerta a documentação oficial do GitHub Copilot:

"Remember that you are in charge, and Copilot is a powerful tool at your service."

Traduzindo: o responsável é você. A ferramenta é só a ferramenta.

Exemplo concreto

```
// Código gerado por IA — parece certo, mas está errado
function calculateDiscount(price, coupon) {
```

```
if (coupon === 'SAVE10') {  
  return price * 0.9;  
}  
return price; // Esqueceu de validar se price é número  
}  
  
calculateDiscount('cem reais', 'SAVE10'); // NaN em produção
```

```
// Código gerado por IA – parece certo, mas está errado  
function calculateDiscount(price, coupon) {
```

```
if (coupon === 'SAVE10') {  
    return price * 0.9;  
}  
return price; // Esqueceu de validar se price é número  
}  
  
calculateDiscount('cem reais', 'SAVE10'); // NaN em produção
```

Como corrigir

Passo a passo para todo código gerado por IA:

1. ****Leia cada linha antes de implementar**** — se não entendeu, não use
2. ****Teste com casos extremos**** — string vazia, null, negativo, tipos inesperados
3. ****Valide dependências sugeridas**** — a IA pode inventar bibliotecas que não existem
4. ****Use type checking**** (TypeScript, mypy, etc.) para pegar erros de tipo

[!TIP]

Trate a saída da IA como um ****rascunho inicial****, não como produto acabado. A diferença entre um profissional e um amador é que o profissional verifica antes de entregar.

3. Erro 2: Prompts vagos sem contexto

Definição

Fazer pedidos genéricos e esperar respostas precisas e úteis.

Por que acontece

A IA opera com base em probabilidades: quanto menos contexto, mais genérica a resposta. É como perguntar "Me recomenda um filme?" para um amigo — você vai receber uma lista genérica. Agora pergunte "Me recomenda um filme de suspense coreano com menos de 2 horas" — a resposta muda completamente.

O mesmo vale para código.

"Pedido fraco, resposta fraca em escala industrial." — TechTudo

Consequência

Respostas genéricas que não resolvem o problema real. O desenvolvedor perde tempo iterando sobre sugestões irrelevantes, se frustra com a ferramenta e culpa a IA — quando o problema era o prompt.

Exemplo concreto

```
<table> <tr> <td width="50%">
```

****Prompt vago****

"Melhore esse código"

A IA não sabe: - Qual linguagem? - Qual critério de "melhor"? - Performance? Legibilidade? Segurança? - Qual o contexto do projeto?

```
</td> <td width="50%">
```

****Prompt estruturado****

"Refatore a função `handleSubmit` no arquivo `src/forms.ts` para usar `async/await` com `try-catch`, mantendo o mesmo comportamento e seguindo o padrão de error handling do resto do projeto."

A IA sabe: - Arquivo e função exatos - O que fazer (refatorar) - Como fazer (async/await + try-catch) - Restrições (manter comportamento, seguir padrão existente)

</td> </tr> </table>

Como corrigir

Estruture todo prompt com três elementos:

Elemento	Pergunta guia	Exemplo
Contexto	Qual é o cenário?	"No arquivo `login.ts`, f
Intenção	O que você quer alcançar?	"...precisa validar token
Formato esperado	Como deve ser a resposta?	"...retorne `Result<T, E>

[!TIP]

Antes de escrever um prompt, pergunte-se: "Se eu desse esta instrução para um colega desenvolvedor, ele saberia exatamente o que fazer?" Se a resposta for não, adicione mais contexto.

4. Erro 3: Pular code review e testes

Definição

Ignorar as etapas de revisão de código e testes automatizados para código gerado por IA, tratando-o como isento de erros.

Por que acontece

A velocidade da IA cria a ilusão de que o código já passou por um "controle de qualidade implícito". O desenvolvedor assume que, se a IA gerou, está correto.

"Se você copia e cola sem ler, o erro deixa de ser da ferramenta. Passa a ser seu." — TechTudo

Essa falsa sensação de segurança é traiçoeira. O código gerado por IA ****não foi revisado por ninguém****. Ele é o equivalente a um primeiro rascunho escrito por alguém que nunca usou seu sistema.

A documentação do Claude Code é categórica:

"Give Claude a check it can run: tests, a build, a screenshot to compare. It's the difference between a session you watch and one you walk away from."

Sem uma verificação executável, "parece pronto" é o único sinal disponível. Você se torna o loop de verificação — cada erro espera ****você**** perceber.

Consequência

Dívida técnica acumulada, bugs não detectados, vulnerabilidades de segurança e código de difícil manutenção. O commit vai para o repositório com ****seu nome**** — a IA não assume responsabilidade.

Como corrigir

1. ****Code review obrigatório**** — revise código gerado por IA como revisaria de um colega. Pergunte: "Eu aceitaria isso num PR?" 2. ****Testes automatizados como verificação**** — antes de aceitar o código, peça para a IA gerar os testes também 3. ****CI pipeline**** — todo código, inclusive o gerado por IA, deve passar pelos mesmos checks automatizados

[!WARNING]

Pular code review em código gerado por IA não economiza tempo — ele **terceiriza o risco** para você. O bug vai aparecer, a pergunta vai ser "quem autorizou isso?", e a resposta será seu nome no commit.



5. Erro 4: Ignorar configuração do projeto (AGENTS.md / CLAUDE.md)

Definição

Não configurar arquivos de instrução persistente para os agentes de IA, deixando-os operar sem contexto do projeto.

Por que acontece

Arquivos como `AGENTS.md` (OpenCode) e `CLAUDE.md` (Claude Code) funcionam como a memória de longo prazo do agente. Eles contêm regras, padrões e convenções do projeto.

Sem eles, a IA opera com conhecimento genérico. Ela não sabe se o projeto usa React 18 ou Vue 3. Não sabe se prefere `const` ou `function`. Não sabe se os testes são com Vitest ou Jest. ****Ela chuta.****

"5 minutos de configuração economizam horas de retrabalho." — OpenCode Community

Consequência

Comportamento inconsistente do agente — o código gerado muda de estilo a cada interação, ignora padrões do projeto e força retrabalho manual.

Os números são contundentes:

Configuração	Taxa de erro	Melhoria
Sem regras	~40%	—
Com 4 regras básicas	~11%	~3,6x
Com 12 regras (pro pack)	~3%	~13,3x

Fonte: Claude Code Pro Pack Research (DEV.to)

Como corrigir

Crie um arquivo de instrução na raiz do projeto. A estrutura mínima inclui:

****Exemplo de `AGENTS.md` para um projeto front-end:****

Agente: Front-end

Stack

React 18 + TypeScript + Tailwind CSS

Regras

- Use arrow functions para componentes
- Prefira ``const`` sobre ``let``
- Testes com Vitest, não Jest
- Erros devem usar ``Result<T, E>`` (never throw)
- Nomes de arquivo em kebab-case
- Componentes em ``src/components/``, páginas em ``src/pages/``

Agente: Front-end

Stack

React 18 + TypeScript + Tailwind CSS

Regras

- Use arrow functions para componentes
- Prefira ``const`` sobre ``let``
- Testes com Vitest, não Jest
- Erros devem usar ``Result<T, E>`` (never throw)
- Nomes de arquivo em kebab-case
- Componentes em ``src/components/``, páginas em ``src/pages/``

[!TIP]

Invista 5 minutos agora para criar o ``AGENTS.md``. É o investimento com maior retorno por minuto no uso de IA para programar. A cada novo projeto, comece por ele.

****Pare e pense:**** seu projeto atual tem um arquivo de instruções para a IA? Se não, esse é o erro número 1 que você está cometendo sem perceber.

6. Erro 5: Tratar IA como resposta final

Definição

Usar a primeira resposta da IA como solução definitiva, sem refinamento ou iteração.

Por que acontece

A IA entrega respostas completas e aparentemente prontas. O desenvolvedor assume que a primeira tentativa é a melhor e encerra o ciclo ali. É o mesmo impulso de mandar um email sem reler — a gratificação imediata de "pronto" supera a disciplina de refinar.

"A primeira resposta raramente é a melhor. Ela é o ponto de partida, não o produto final." — TechTudo

Consequência

Resultados superficiais. O código funciona no caminho feliz, mas quebra nos casos extremos. O desenvolvedor perde a oportunidade de refinar, corrigir e adaptar a solução ao contexto real. Pior: nunca sabe o que perdeu.

Exemplo concreto

Comparação entre primeira resposta vs. versão refinada de um prompt de validação de email:

```
// Primeira resposta (rascunho)
function validateEmail(email) {
  return email.includes('@');
  // Aceita "@", "a@", "a@b"
}

// Versão refinada com feedback
function validateEmail(email) {
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return emailRegex.test(email);
  // Rejeita "@", "a@", "a@b", aceita "user@domain.com"
}
```

Como corrigir

Trate IA como **processo iterativo**, não como resposta final:

1. ****Obtenha uma primeira versão**** — o rascunho inicial 2. ****Revise e identifique pontos de melhoria**** — o que está faltando? 3. ****Refine com feedback direcionado**** — seja específico sobre o que mudar 4. ****Repita até atender aos critérios de qualidade**** — quando passar nos testes e na revisão

[!TIP]

Em vez de reescrever o prompt do zero, ****itere através de feedback****. Diga à IA especificamente o que ajustar: "Mude a nomenclatura para camelCase", "Extraia essa lógica para um hook separado" ou "Adicione tratamento para o caso de lista vazia."

A pesquisa SFEIR Institute confirma: desenvolvedores que iteram com feedback estruturado reduzem em ****~35% as iterações necessárias**** em comparação com quem reescreve prompts do zero.

7. Conclusão: como usar IA corretamente

Usar IA para programar não é sobre aceitar código — é sobre **colaboração inteligente**. A IA é uma ferramenta poderosa, mas sem direção, validação e contexto, ela produz resultados medíocres.

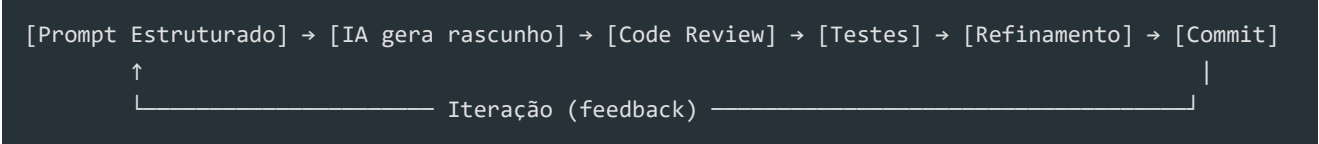
Os 5 mandamentos do uso correto de IA

#	Mandamento	Por quê
1	Valide toda saída da	IA erra com confiança, não
2	Estruture prompts com	Pedido fraco = resposta f
3	Revise e teste como q	Seu nome está no commit
4	Configure o projeto p	5 minutos economizam hora
5	Itere , não aceite a p	Refinamento separa o medi

[!NOTE]

Desenvolvedores que aplicam essas práticas reduzem em **~35%** as iterações necessárias (SFEIR Institute) e produzem código com **~97%** de acerto (Claude Code Pro Pack).

Diagrama conceitual: Fluxo ideal de uso de IA



Analogia principal

Usar IA para programar é como ter um **estagiário brilhante, mas inexperiente**. Ele trabalha rápido, escreve bem, mas precisa de supervisão, contexto e revisão constantes. Confiar cegamente nele é negligência; ignorá-lo é desperdício. O profissional sábio sabe exatamente quando delegar, quando revisar e como orientar.

Recursos didáticos sugeridos

****Exemplo prático para sala de aula:****

```
// Prompt vago: "Valide esse email"
function validateEmail(email) {
  return email.includes('@'); // Superficial, não valida domínio nem formato
}

// Prompt estruturado: "Valide email com regex básica de formato, retorne booleano"
function validateEmail(email) {
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return emailRegex.test(email); // Validação mais robusta
}
```

****Sugestão de diagrama:**** Mapa mental dos 5 erros com causas e correções lado a lado, em formato de tabela visual.

****Mini-exercício mental para reflexão:**** Pense no último código que você copiou de uma IA. Você revisou linha por linha? Escreveu testes? Se respondeu "não" para qualquer pergunta, identifique qual dos 5 erros você cometeu.

Exercício prático

****Título:**** Diagnosticando um prompt ruim

****Duração:**** 5 minutos

****Instruções:****

1. Leia o prompt abaixo: > "Faz aí uma função de login pra mim"
2. Liste ****3 problemas**** com este prompt (erro 2 — prompt vago)
3. Reescreva o prompt seguindo a estrutura ****Contexto + Intenção + Formato Esperado****
4. Se possível, execute o prompt reescrito em um assistente de IA e compare a qualidade da resposta

****Critérios de sucesso:**** - Identificou a falta de: linguagem/framework, definição de sucesso, tratamento de erros - Novo prompt inclui pelo menos: stack tecnológica, regras de negócio, formato da resposta esperada

****Gabarito (problemas identificados):**** | Problema | Explicação | |-----|-----| | Falta linguagem/framework | "Faz aí" não diz se é Node, Python, PHP, etc. | | Falta definição de sucesso | O que é "login"? JWT? Sessão? OAuth? | | Falta tratamento de erros | E se o usuário não existe? Senha errada? Taxa de limite? |

****Prompt reescrito (exemplo):****

"Crie uma função de login em Node.js com Express. O usuário envia email e senha no corpo da requisição. Valide o email com regex, compare a senha com bcrypt, e retorne um token JWT com expiração de 24h. Se o email não existir ou a senha estiver errada, retorne 401 com mensagem clara. Use `async/await` com `try-catch`."

Desafio final

****Título:** Auditoria de código gerado por IA**

****Duração:** 10 minutos**

****Cenário:** Você recebeu um Pull Request com código 100% gerado por IA. O desenvolvedor apenas copiou e colou sem revisar.**

****Tarefa:** Analise o trecho abaixo, identifique ****todos os erros**** e proponha correções:**

```
import os
import hashlib

def hash_password(password):
    # Gerar hash simples
    return hashlib.md5(password.encode()).hexdigest()

def save_user(username, password):
    hashed = hash_password(password)
    query = f"INSERT INTO users (username, password) VALUES ('{username}', '{hashed}')"
    os.system(f"mysql -e \"{query}\"")
```

****O que procurar:** - ****Erro 1:**** confiar cegamente — código tem vulnerabilidades graves - ****Erro 3:**** pular code review — falta de testes e validação - ****Erro 5:**** tratar como resposta final — código inseguro em produção**

****Resposta esperada:****

Erro identificado	Por que é grave	Correção
MD5 para senhas	MD5 é instantaneamente qu	Use `bcrypt` ou `argon2`
SQL injection	String formatada diretame	Use prepared statements /
`os.system`	Expõe o shell a comandos	Use biblioteca MySQL com
Falta validação de entrad	`username` pode conter ca	Valide e sanitize entrada

****Código corrigido (exemplo):****

```
import os
import bcrypt
import mysql.connector

def hash_password(password):
    return bcrypt.hashpw(password.encode(), bcrypt.gensalt())

def save_user(username, password):
    if not username or not password:
        raise ValueError("Campos obrigatórios")
```

```
hashed = hash_password(password)
conn = mysql.connector.connect(
    host=os.getenv("DB_HOST", "localhost"),
    user=os.getenv("DB_USER", "app"),
    password=os.getenv("DB_PASSWORD"),
    database=os.getenv("DB_NAME", "appdb")
)
cursor = conn.cursor()
cursor.execute(
    "INSERT INTO users (username, password) VALUES (%s, %s)",
    (username, hashed)
)
conn.commit()
cursor.close()
conn.close()
```

```
import os
import bcrypt
import mysql.connector

def hash_password(password):
    return bcrypt.hashpw(password.encode(), bcrypt.gensalt())

def save_user(username, password):
    if not username or not password:
        raise ValueError("Campos obrigatórios")
```


Leituras complementares

- GitHub Copilot Best Practices — <https://docs.github.com/en/copilot/get-started/best-practices>
- Claude Code Best Practices — <https://code.claude.com/docs/en/best-practices>
- OpenCode Official Docs — <https://opencode.ai/>
- "A Survey of Bugs in AI-Generated Code" (arXiv 2512.05239) — <https://arxiv.org/abs/2512.05239>
- 5 erros ao usar IA que sabotam suas respostas (TechTudo) — <https://www.techtudo.com.br/listas/2026/04/5-erros-ao-usar-ia-que-sabotam-suas-respostas-e-como-evita-los-edsoftwares.ghml>
- CLAUDE.md Rules: How to Cut AI Coding Mistakes from ~40% to ~3% (DEV.to) — <https://dev.to/rams901/claudemd-rules-how-to-cut-ai-coding-mistakes-from-40-to-3-in-2026-2j70>
- 10 Most Common Mistakes Using AI Coding Tools (Ryz Labs) — <https://www.ryzlabs.com/>